electronics-engineering-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `electronics_engineering_ai999x.about()` for the same answer.

Every capability electronics-engineering-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import electronics_engineering_ai999x

result = electronics_engineering_ai999x.solve('elx_thermal_voltage', T='300 K')
result.show()
# VT = 0.025852 V, VT_mV = 25.852
```

By topic

topic	capabilities
Active filters / approximations	Butterworth or Chebyshev low-pass filter order, Butterworth filter: Q of each second-order section, Chebyshev type I filter: section Q and frequencies
Active filters / first order	First-order active low-pass filter (inverting)
Active filters / multiple feedback	Multiple-feedback (MFB) low-pass filter, Multiple-feedback (MFB) band-pass filter
Active filters / Sallen-Key	Sallen-Key low-pass filter: f0, Q and gain, Sallen-Key high-pass filter: f0, Q and gain, Equal-component Sallen-Key low-pass design
BJT / amplifiers	Common-emitter amplifier: gain, input and output resistance, Common-collector (emitter-follower) amplifier, Common-base amplifier, BJT cascode: output resistance, transconductance and gain
BJT / current relations	BJT terminal currents from beta
BJT / current sources	BJT current mirror with finite beta, Wilson current mirror output current
BJT / DC bias	BJT fixed (base-resistor) bias: Q-point, BJT voltage-divider bias: exact Q-point, BJT dual-supply emitter bias: Q-point, BJT collector-feedback bias: Q-point
BJT / differential amplifiers	BJT differential pair: differential and common-mode gain, CMRR
BJT / frequency response	BJT beta cutoff and transition frequency fT, Common-emitter high-frequency cutoff: Miller effect and open-circuit time constants, Common-emitter low-frequency cutoff by short-circuit time constants
BJT / small-signal model	BJT small-signal (hybrid-pi) parameters
CMOS / inverter	CMOS inverter switching threshold VM
CMOS / timing	CMOS gate propagation delay (switch-resistor model)

topic	capabilities
Data converters / ADC	ADC output code, LSB and quantisation error, ADC quantisation SNR with oversampling, Effective number of bits (ENOB) from SINAD, Successive-approximation (SAR) ADC conversion, Dual-slope integrating ADC
Data converters / DAC	DAC output voltage (R-2R or any binary DAC), Binary-weighted resistor DAC
Data converters / sampling	SNR limit from sampling-clock (aperture) jitter, Sample-and-hold droop
Diodes / LEDs	LED series (current-limiting) resistor
Diodes / operating point	Operating point of a diode in series with a resistor (exact)
Diodes / rectifier filters	Capacitor-input filter: ripple voltage and DC output
Diodes / rectifiers	Rectifier output: average, RMS, ripple factor and PIV
Diodes / Shockley equation	Diode current from the Shockley equation, Diode voltage for a given current
Diodes / Zener regulators	Zener shunt regulator: load voltage and currents, Zener regulator design: series resistor and Zener rating
MOSFET / amplifiers	Common-source amplifier with source degeneration and ro, Source follower (common drain)
MOSFET / DC bias	MOSFET voltage-divider bias with source resistor: Q-point
MOSFET / drain current	MOSFET drain current (cutoff, triode, saturation)
MOSFET / small-signal model	MOSFET small-signal gm, ro and intrinsic gain
MOSFET / threshold voltage	MOSFET threshold voltage with body effect
Op-amps / basic amplifiers	Inverting op-amp amplifier, Non-inverting op-amp amplifier, Inverting summing amplifier
Op-amps / comparators	Inverting Schmitt trigger thresholds
Op-amps / difference amplifiers	Op-amp difference amplifier (with resistor mismatch), Three-op-amp instrumentation amplifier
Op-amps / feedback	Negative feedback: closed-loop gain and bandwidth extension
Op-amps / frequency limits	Closed-loop bandwidth from the gain-bandwidth product, Slew-rate limit: full-power bandwidth
Op-amps / integrator and differentiator	Op-amp integrator: output ramp and unity-gain frequency, Op-amp differentiator: gain and output amplitude
Op-amps / non-ideal	Inverting amplifier with finite open-loop gain, Op-amp output offset from input offset voltage and bias currents
Op-amps / stability	Phase margin of a two-pole amplifier with feedback
Oscillators / 555 timer	555 timer astable: frequency and duty cycle, 555 timer monostable pulse width
Oscillators / crystal	Quartz crystal: series and parallel resonance, Q
Oscillators / LC	Colpitts oscillator frequency and feedback ratio, Hartley oscillator frequency
Oscillators / phase-locked loops	Type-2 PLL with active PI loop filter: natural frequency and damping
Oscillators / RC	RC phase-shift oscillator frequency and gain, Wien-bridge oscillator frequency and required gain
Oscillators / relaxation	Op-amp relaxation (astable) oscillator
Power supplies / linear regulators	Linear regulator dissipation and efficiency, Adjustable three-terminal regulator output voltage, Zener-referenced series-pass (emitter-follower) regulator

topic	capabilities
Power supplies / thermal design	Heat-sink thermal path: junction, case and sink temperatures, Required heat-sink thermal resistance
Semiconductors / carrier statistics	Intrinsic carrier concentration, Electron and hole concentrations of doped (compensated) material
Semiconductors / conduction	Conductivity and resistivity of a semiconductor
Semiconductors / diffusion	Einstein relation and diffusion length
Semiconductors / drift	Drift velocity and drift current in a semiconductor bar
Semiconductors / pn junction	Built-in potential of a pn junction, Depletion width, field and capacitance of an abrupt pn junction, Junction (depletion) capacitance versus bias
Semiconductors / thermal voltage	Thermal voltage kT/q
Timing / interconnect	Elmore delay of an RC ladder (interconnect)
Timing / power	CMOS dynamic (switching) power
Timing / RC	RC delay, 10-90 % rise time and bandwidth
Timing / synchronous logic	Flip-flop setup and hold slack, maximum clock frequency, Synchronizer metastability MTBF

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on

field	meaning
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
elx_thermal_voltage	Thermal voltage kT/q	T	VT, VT_mV
elx_intrinsic_carrier_concentration	Intrinsic carrier concentration	Nc, Nv, Eg, T	ni, ni_cm3, Ei_minus_midgap_eV, Nc_T, Nv_T
elx_semiconductor_conductivity	Conductivity and resistivity of a semiconductor	n, p, mu_n, mu_p	sigma, rho, rho_ohm_cm
elx_drift_current	Drift velocity and drift current in a semiconductor bar	n, mu, V, L, A	E, v_d, J, I
elx_einstein_diffusion	Einstein relation and diffusion length	mu, T, tau	D, D_cm2, L_diff, L_um
elx_doped_carrier_concentrations	Electron and hole concentrations of doped (compensated) material	ND, NA, ni	n, p, n_cm3, p_cm3, type
elx_builtin_potential	Built-in potential of a pn junction	NA, ND, ni, T	Vbi
elx_depletion_width	Depletion width, field and capacitance of an abrupt pn junction	NA, ND, Vbi, VR, eps_r	W, W_um, xn, xp, Emax, Cj_area
elx_junction_capacitance	Junction (depletion) capacitance versus bias	Cj0, Vbi, VR, m	Cj, ratio
elx_shockley_diode_current	Diode current from the Shockley equation	Is, n, VD, T	ID, VT
elx_diode_voltage	Diode voltage for a given current	ID, Is, n, T	VD, VT
elx_diode_operating_point	Operating point of a diode in series with a resistor (exact)	VS, R, Is, n, T	ID, VD, P_D

id	operation	inputs	outputs
elx_rectifier_averages	Rectifier output: average, RMS, ripple factor and PIV	Vm, circuit, Von	Vp_out, Vdc, Vrms, ripple_factor, PIV, conduction_deg
elx_capacitor_filter_ripple	Capacitor-input filter: ripple voltage and DC output	Vp, f, RL, C, circuit	f_ripple, Vr, Vdc, Vr_rms, ripple_factor
elx_zener_regulator	Zener shunt regulator: load voltage and currents	Vin, Rs, Vz, RL, rz	Vout, I_s, I_L, I_z, P_z
elx_zener_series_resistor_design	Zener regulator design: series resistor and Zener rating	Vin_min, Vin_max, Vz, IL_min, IL_max, Iz_min	Rs_max, Iz_max, Pz_max, PRs_max
elx_led_series_resistor	LED series (current-limiting) resistor	Vs, Vf, If, n	R, P_R, P_LED
elx_bjt_current_relations	BJT terminal currents from beta	IB, beta	IC, IE, alpha
elx_bjt_fixed_bias	BJT fixed (base-resistor) bias: Q-point	VCC, RB, RC, beta, VBE	IB, IC, IE, VCE
elx_bjt_voltage_divider_bias	BJT voltage-divider bias: exact Q-point	VCC, R1, R2, RC, RE, beta, VBE	VTH, RTH, IB, IC, IE, VB, VE, VC, VCE
elx_bjt_emitter_bias	BJT dual-supply emitter bias: Q-point	VCC, VEE, RB, RC, RE, beta, VBE	IB, IC, IE, VB, VE, VC, VCE
elx_bjt_collector_feedback_bias	BJT collector-feedback bias: Q-point	VCC, RC, RF, beta, VBE, RE	IB, IC, IE, VC, VCE
elx_bjt_small_signal_parameters	BJT small-signal (hybrid-pi) parameters	IC, beta, VT, VA	gm, r_pi, r_e, r_o, A0
elx_common_emitter_amplifier	Common-emitter amplifier: gain, input and output resistance	IC, beta, VT, RC, RL, RE1, RB, Rsig	gm, r_pi, Av, Rin, Rout, Gv
elx_common_collector_amplifier	Common-collector (emitter-follower) amplifier	IC, beta, VT, RE, RL, RB, Rsig	Av, Rin, Rout, Gv
elx_common_base_amplifier	Common-base amplifier	IC, beta, VT, RC, RL, RE, Rsig	Av, Rin, Rout, Gv
elx_transition_frequency	BJT beta cutoff and transition frequency fT	gm, beta, Cpi, Cmu	r_pi, f_beta, f_T
elx_miller_high_cutoff	Common-emitter high-frequency cutoff: Miller effect and open-circuit time constants	gm, r_pi, Cpi, Cmu, RLp, Rsigp	A_mid, C_M, R_pi_eq, f_H_miller, tau_oc, f_H_octc
elx_low_frequency_cutoff	Common-emitter low-frequency cutoff by short-circuit time constants	Rsig, RB, r_pi, beta, RE, RC, RL, C1, C2, CE	R_C1, R_CE, R_C2, f_C1, f_CE, f_C2, f_L
elx_current_mirror	BJT current mirror with finite beta	VCC, R, VBE, beta	Iref, Iout, ratio
elx_wilson_current_mirror	Wilson current mirror output current	Iref, beta	Iout, ratio, error
elx_differential_pair	BJT differential pair: differential and common-mode gain, CMRR	I_tail, beta, VT, RC, REE	IC, gm, Ad_diff, Ad_se, Acm, CMRR, CMRR_dB
elx_bjt_cascode	BJT cascode: output resistance, transconductance and gain	IC, beta, VT, VA, RL	gm, r_o, Rout, Gm, Av

id	operation	inputs	outputs
elx_mosfet_drain_current	MOSFET drain current (cutoff, triode, saturation)	kn, Vt, VGS, VDS, lam	region, ID, VOV, VDS_sat
elx_mosfet_gm_ro	MOSFET small-signal gm, ro and intrinsic gain	kn, ID, lam, Vt	VOV, VGS, gm, r_o, A0
elx_mosfet_divider_bias	MOSFET voltage-divider bias with source resistor: Q-point	VDD, R1, R2, RD, RS, kn, Vt	VG, VGS, VOV, ID, VS, VD, VDS
elx_common_source_amplifier	Common-source amplifier with source degeneration and ro	gm, ro, RD, RL, RS, RG, Rsig	Av, Rin, Rout, Gv
elx_source_follower	Source follower (common drain)	gm, ro, RS, RL, RG, Rsig	Av, Rin, Rout, Gv
elx_mosfet_body_effect	MOSFET threshold voltage with body effect	Vt0, gamma, phi2F, VSB	Vt, dVt
elx_cmos_switching_threshold	CMOS inverter switching threshold VM	VDD, Vtn, Vtp, kn, kp	r, VM
elx_cmos_propagation_delay	CMOS gate propagation delay (switch-resistor model)	Rn, Rp, CL	tpHL, tpLH, tp
elx_inverting_amplifier	Inverting op-amp amplifier	Rf, Rin, Vin	G, Vout, Zin
elx_noninverting_amplifier	Non-inverting op-amp amplifier	Rf, Rg, Vin	G, Vout
elx_summing_amplifier	Inverting summing amplifier	V, R, Rf	Vout, I_f
elx_difference_amplifier	Op-amp difference amplifier (with resistor mismatch)	R1, R2, R3, R4, V1, V2	Vout, Ad, Acm
elx_instrumentation_amplifier	Three-op-amp instrumentation amplifier	R1, RG, R2, R3, V1, V2	G, G_stage1, Vout
elx_inverting_amplifier_finite_gain	Inverting amplifier with finite open-loop gain	Rf, R1, A0, Vin	G, G_ideal, Vout, error
elx_opamp_integrator	Op-amp integrator: output ramp and unity-gain frequency	R, C, Vin, t, V0	Vout, slope, f_unity
elx_opamp_differentiator	Op-amp differentiator: gain and output amplitude	R, C, A, f	gain, Vout_peak, gain_dB
elx_closed_loop_bandwidth	Closed-loop bandwidth from the gain-bandwidth product	GBW, config, Rf, R1	G0, noise_gain, f_3dB
elx_slew_rate_limit	Slew-rate limit: full-power bandwidth	SR, Vp, f	f_max, Vp_max, slope_needed, limited
elx_opamp_dc_offset	Op-amp output offset from input offset voltage and bias currents	Vos, IB, Ios, Rf, R1, Rp	Vout_Vos, Vout_bias, Vout_total, Rp_optimal
elx_schmitt_trigger	Inverting Schmitt trigger thresholds	Rf, R1, VsatH, VsatL, Vref	V_TH, V_TL, hysteresis, center
elx_negative_feedback	Negative feedback: closed-loop gain and bandwidth extension	A, beta, fH	Af, loop_gain, desensitivity, desensitivity_dB, fH_closed
elx_two_pole_phase_margin	Phase margin of a two-pole amplifier with feedback	A0, f1, f2, beta	f_c, PM_deg, Af
elx_first_order_active_lowpass	First-order active low-pass filter (inverting)	R1, Rf, C, f	H0, fc, gain_f, gain_dB
elx_sallen_key_lowpass	Sallen-Key low-pass filter: f0, Q and gain	R1, R2, C1, C2, K	f0, Q, H0, peak_gain

id	operation	inputs	outputs
elx_sallen_key_highpass	Sallen-Key high-pass filter: f0, Q and gain	C1, C2, R1, R2, K	f0, Q, H_inf
elx_mfb_lowpass	Multiple-feedback (MFB) low-pass filter	R1, R2, R3, C1, C2	f0, Q, H0
elx_mfb_bandpass	Multiple-feedback (MFB) band-pass filter	R1, R2, R3, C1, C2	f0, Q, BW, H_peak
elx_sallen_key_equal_component_design	Equal-component Sallen-Key low-pass design	fc, Q, C	R, K, Rf_over_Rg
elx_filter_order	Butterworth or Chebyshev low-pass filter order	ftype, fp, fs, Ap, As	n
elx_butterworth_section_q	Butterworth filter: Q of each second-order section	n	Q_max, Q, sections, first_order
elx_chebyshev_sections	Chebyshev type I filter: section Q and frequencies	n, ripple	Q_max, Q, f0_ratio, sections, real_pole
elx_rc_phase_shift_oscillator	RC phase-shift oscillator frequency and gain	R, C, ladder	f0, A_min, beta
elx_wien_bridge_oscillator	Wien-bridge oscillator frequency and required gain	R1, C1, R2, C2	f0, beta, A_min, Rf_over_Rg
elx_colpitts_oscillator	Colpitts oscillator frequency and feedback ratio	L, C1, C2	C_eq, f0, beta, A_min
elx_hartley_oscillator	Hartley oscillator frequency	L1, L2, M, C	L_eq, f0, A_min
elx_crystal_oscillator	Quartz crystal: series and parallel resonance, Q	Lm, Cm, Rm, C0	fs, fp, delta_f, Q
elx_opamp_relaxation_oscillator	Op-amp relaxation (astable) oscillator	R, C, R1, R2	beta, T, f
elx_555_astable	555 timer astable: frequency and duty cycle	RA, RB, C	t_high, t_low, T, f, duty
elx_555_monostable	555 timer monostable pulse width	R, C	T
elx_pll_loop_dynamics	Type-2 PLL with active PI loop filter: natural frequency and damping	Kd, Ko, tau1, tau2	wn, fn, zeta
elx_adc_code	ADC output code, LSB and quantisation error	Vin, Vref, n	code, bits, LSB, V_code, error
elx_adc_quantization_snr	ADC quantisation SNR with oversampling	n, OSR, A_frac	SNR_dB, noise_rms_lsb, SNR_full_scale_dB
elx_adc_enob	Effective number of bits (ENOB) from SINAD	SINAD, n	ENOB, bits_lost
elx_aperture_jitter_snr	SNR limit from sampling-clock (aperture) jitter	f, tj, n	SNR_jitter_dB, SNR_quant_dB, SNR_total_dB, ENOB_total
elx_dac_output	DAC output voltage (R-2R or any binary DAC)	n, code, Vref	Vout, LSB, V_full_scale
elx_dac_binary_weighted	Binary-weighted resistor DAC	n, code, Vref, R, Rf	Vout, I_total, R_LSB

id	operation	inputs	outputs
elx_sar_conversion	Successive-approximation (SAR) ADC conversion	Vin, Vref, n	code, bits, trials, V_dac, cycles
elx_dual_slope_adc	Dual-slope integrating ADC	Vin, Vref, N1, fclk, R, C	T1, T2, N2, V_peak
elx_sample_hold_droop	Sample-and-hold droop	I_leak, C, t_hold, Vref, n	droop_rate, droop, droop_lsb, t_max_half_lsb
elx_rc_delay_rise_time	RC delay, 10-90 % rise time and bandwidth	R, C	tau, t50, t_rise, f_3dB, tr_times_bw
elx_elmore_delay	Elmore delay of an RC ladder (interconnect)	R, C	tau_D, t50, delays
elx_cmos_dynamic_power	CMOS dynamic (switching) power	alpha, C, VDD, f	P, E_supply, E_stored, I_avg
elx_setup_hold_timing	Flip-flop setup and hold slack, maximum clock frequency	T, tcq, tpd, tsu, tskew, tccq, tcd, th	setup_slack, hold_slack, f_max, verdict
elx_metastability_mtbf	Synchronizer metastability MTBF	tr, tau, T0, fclk, fdata	MTBF_s, MTBF_years, log10_MTBF_s
elx_linear_regulator_efficiency	Linear regulator dissipation and efficiency	Vin, Vout, Iload, Iq	P_in, P_out, P_diss, efficiency, efficiency_pct
elx_adjustable_regulator_output	Adjustable three-terminal regulator output voltage	R1, R2, Vref, Iadj	Vout, I_R1
elx_series_pass_regulator	Zener-referenced series-pass (emitter-follower) regulator	Vin, Vz, Rs, RL, beta, VBE	Vout, I_L, I_B, I_z, I_Rs, P_Q
elx_heat_sink_temperatures	Heat-sink thermal path: junction, case and sink temperatures	P, Ta, theta_jc, theta_cs, theta_sa	Tj, Tc, Ts, Tj_C, theta_ja
elx_heat_sink_required	Required heat-sink thermal resistance	P, Ta, Tj_max, theta_jc, theta_cs	theta_ja_max, theta_sa_max

Thermal voltage kT/q, `elx_thermal_voltage`

The thermal voltage $VT = kT/q$ at an absolute temperature (about 25.85 mV at 300 K), the voltage scale of every diode and transistor equation.

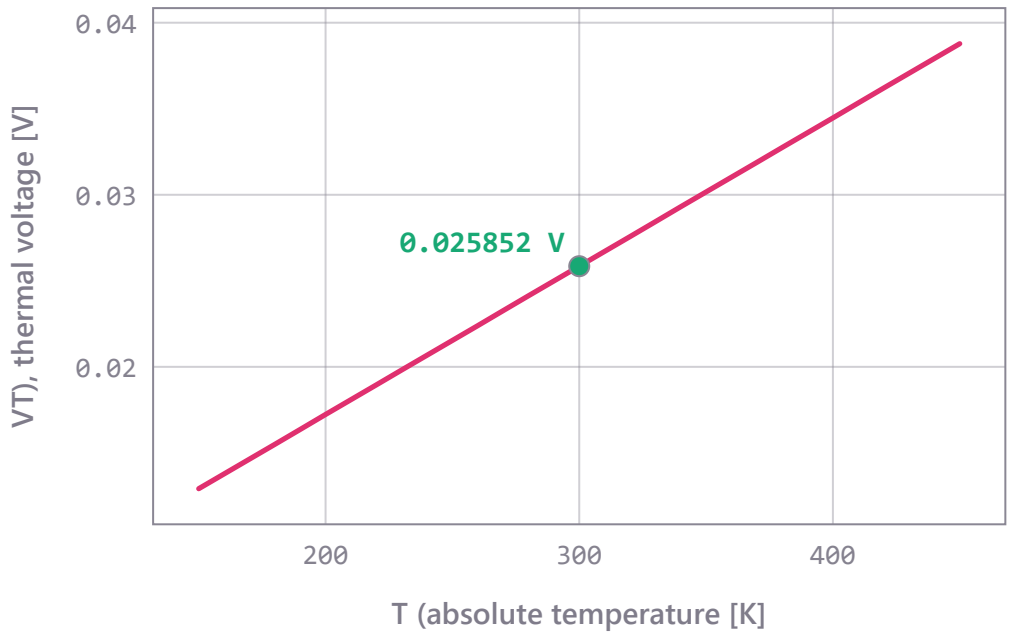
$$V_T = \frac{kT}{q}$$

input	meaning	condition
`T`	absolute temperature (degC is converted)	

output	meaning
`VT`	thermal voltage
`VT_mV`	thermal voltage in millivolts

- **Method:** $VT = k$ T / q
- **Assumes:** $T > 0$ K

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



VT (thermal voltage) against T (absolute temperature (degC is converted)) from 150 K to 450 K. 31 points computed by electronics-engineering-ai999x; marked: the worked example, T = 300 K, VT = 0.025852 V.

```
electronics_engineering_ai999x.solve('elx_thermal_voltage', T=300.0)
# -> VT = 0.025852 V, VT_mV = 25.852   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_thermal_voltage', T=350.0)
# -> VT = 0.0301607 V, VT_mV = 30.1607   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thermal voltage: T = 300 K	SOLVED	VT = 0.025852 V, VT_mV = 25.852
Thermal voltage: T = 350 K	SOLVED	VT = 0.0301607 V, VT_mV = 30.1607
Thermal voltage:	NEEDS_INPUT	elx_thermal_voltage needs T (absolute temperature (degC is converted)). Example: solve('elx_thermal_voltage', T='300 K')
Thermal voltage: T = 300 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te
Thermal voltage: T = 26.85 degC	SOLVED	VT = 0.025852 V, VT_mV = 25.852

Intrinsic carrier concentration, `elx_intrinsic_carrier_concentration`

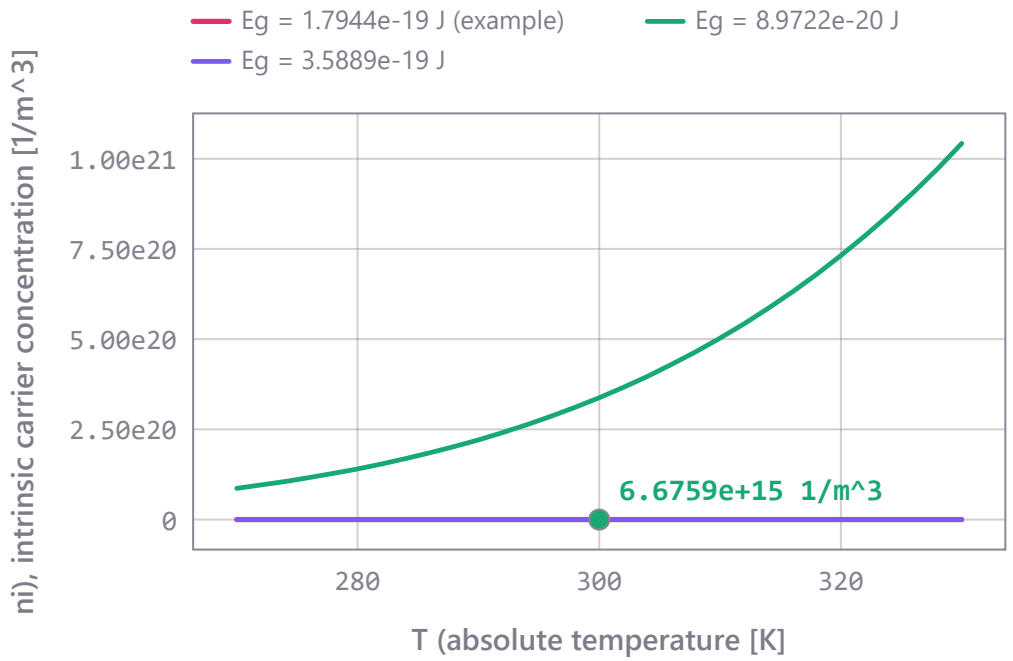
The intrinsic carrier concentration $ni = sqrt$(Nc Nv) exp(-Eg/2kT) at a temperature, from the effective densities of states at 300 K (scaled as T^1.5) and the band gap; also the intrinsic level's offset from midgap.

$$n_i = \sqrt{N_c N_v}\, e^{-E_g/2kT}, \quad N_{c,v}(T) = N_{c,v}(300)\left(\tfrac{T}{300}\right)^{3/2}$$

input	meaning	condition
`Nc`	effective density of states in the conduction band at 300 K	
`Nv`	effective density of states in the valence band at 300 K	
`Eg`	band gap energy	
`T`	absolute temperature	

output	meaning
`ni`	intrinsic carrier concentration
`ni_cm3`	ni in cm^{-3}
`Ei_minus_midgap_eV`	intrinsic Fermi level above midgap, eV
`Nc_T`	Nc at T
`Nv_T`	Nv at T

- **Method:** Nc(T) = Nc $(T/300)^{1.5}$, Nv likewise; $ni = sqrt$(Nc Nv) exp($-Eg/(2kT)$); Ei - Emid = (kT/2) ln(Nv/Nc)
- **Assumes:** Nc, Nv, Eg, $T > 0$; non-degenerate statistics, band gap independent of T
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ni (intrinsic carrier concentration) against T (absolute temperature) from 270 K to 330 K. One curve for each value of Eg (band gap energy): 1.7944e-19 J, 8.9722e-20 J, 3.5889e-19 J. Other inputs as in the worked example: Nc = 2.8e+25 1/m^3, Nv = 1.04e+25 1/m^3. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, T = 300 K, ni = 6.6759e+15 1/m^3.

```
electronics_engineering_ai999x.solve('elx_intrinsic_carrier_concentration', Nc=2.8e+25,
Nv=1.04e+25, Eg=1.7944378300800001e-19, T=300.0)
# -> ni = 6.6759e+15 1/m^3, ni_cm3 = 6.6759e+09, Ei_minus_midgap_eV = -0.0128019, Nc_T = 2.8e+25
1/m^3, Nv_T = 1.04e+25 1/m^3   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_intrinsic_carrier_concentration', Nc=4.7e+23, Nv=7e+24,
Eg=2.27509082028e-19, T=400.0)
# -> ni = 3.16519e+15 1/m^3, ni_cm3 = 3.16519e+09, Ei_minus_midgap_eV = 0.0465497, Nc_T =
7.23612e+23 1/m^3, Nv_T = 1.07772e+25 1/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Intrinsic carrier concentration: Nc = 2.8e+19 cm^-3, Nv = 1.04e+19 cm^-3, Eg = 1.12 eV, T = 300 K	SOLVED	ni = 6.6759e+15 1/m^3, ni_cm3 = 6.6759e+09, Ei_minus_midgap_eV = -0.0128019, Nc_T = 2.8e+25 1/m^3, Nv_T = 1.04e+25 1/m^3
Intrinsic carrier concentration: Nc = 4.7e+17 cm^-3, Nv = 7e+18 cm^-3, Eg = 1.42 eV, T = 400 K	SOLVED	ni = 3.16519e+15 1/m^3, ni_cm3 = 3.16519e+09, Ei_minus_midgap_eV = 0.0465497, Nc_T = 7.23612e+23 1/m^3, Nv_T = 1.07772e+25 1/m^3
Intrinsic carrier concentration: Nv = 1.04e+19 cm^-3, Eg = 1.12 eV, T = 300 K	NEEDS_INPUT	elx_intrinsic_carrier_concentration needs Nc (effective density of states in the conduction band at 300 K). Example: sol
Intrinsic carrier concentration: Nc = 2.8e+19 cm^-3, Nv = 1.04e+19 cm^-3, Eg = 1.12 eV, T = 300 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Conductivity and resistivity of a semiconductor, `elx_semiconductor_conductivity`

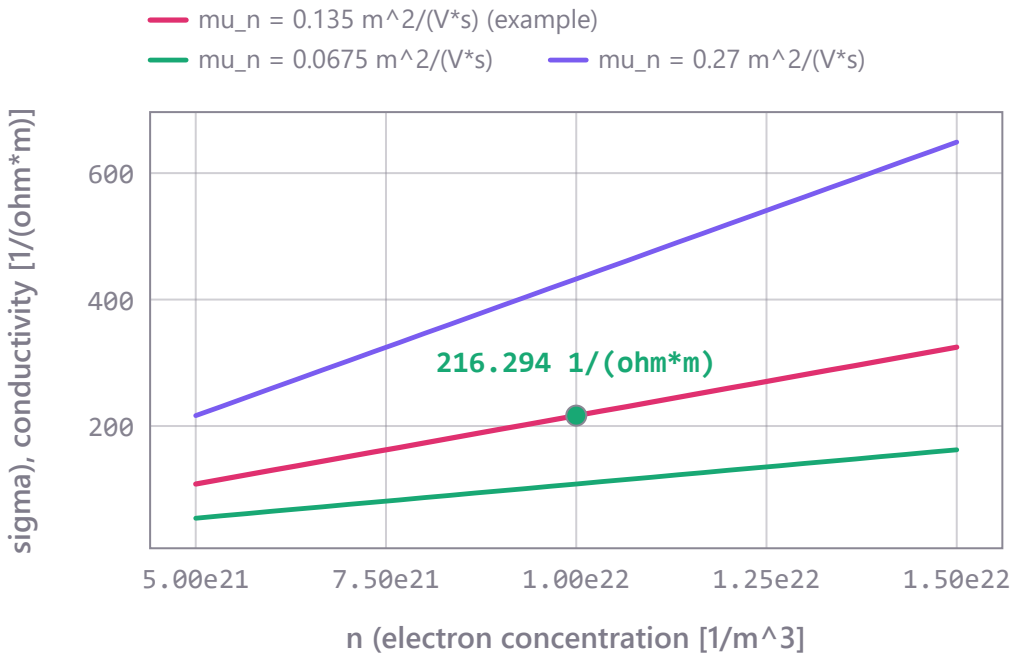
The conductivity $\sigma = q$ (n mu_n + p mu_p) and the resistivity of a semiconductor from its electron and hole concentrations and mobilities.

$$\sigma = q\,(n\mu_n + p\mu_p), \quad \rho = 1/\sigma$$

input	meaning	condition
`n`	electron concentration	
`p`	hole concentration	
`mu_n`	electron mobility	
`mu_p`	hole mobility	

output	meaning
`sigma`	conductivity
`rho`	resistivity
`rho_ohm_cm`	resistivity in ohm cm

- **Method:** $\sigma = q$ (n mu_n + p mu_p); $\rho = 1/\sigma$
- **Assumes:** n, $p \geq 0$ (not both zero); mobilities > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sigma (conductivity) against n (electron concentration) from 5.0000e+21 1/m^3 to 1.5000e+22 1/m^3. One curve for each value of mu_n (electron mobility): 0.135 m^2/(V*s), 0.0675 m^2/(V*s), 0.27 m^2/(V*s). Other inputs as in the worked example: p = 22500000000.0 1/m^3, mu_p = 0.048 m^2/(V*s). 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, n = 1.0000e+22 1/m^3, sigma = 216.294 1/(ohm*m).

```
electronics_engineering_ai999x.solve('elx_semiconductor_conductivity', n=1e+22, p=22500000000.0,
mu_n=0.135, mu_p=0.048)
# -> sigma = 216.294 1/(ohm*m), rho = 0.00462334 ohm*m, rho_ohm_cm = 0.462334   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_semiconductor_conductivity', n=1.5e+16, p=1.5e+16,
mu_n=0.135, mu_p=0.048)
# -> sigma = 0.000439797 1/(ohm*m), rho = 2273.77 ohm*m, rho_ohm_cm = 227377   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Semiconductor conductivity: n = 1e+16 cm^-3, p = 22500 cm^-3, mu_n = 1350 cm^2/(V*s), mu_p = 480 cm^2/(V*s)	SOLVED	sigma = 216.294 1/(ohm*m), rho = 0.00462334 ohm*m, rho_ohm_cm = 0.462334
Semiconductor conductivity: n = 1.5e+10 cm^-3, p = 1.5e+10 cm^-3, mu_n = 1350 cm^2/(V*s), mu_p = 480 cm^2/(V*s)	SOLVED	sigma = 0.000439797 1/(ohm*m), rho = 2273.77 ohm*m, rho_ohm_cm = 227377
Semiconductor conductivity: p = 22500 cm^-3, mu_n = 1350 cm^2/(V*s), mu_p = 480 cm^2/(V*s)	NEEDS_INPUT	elx_semiconductor_conductivity needs n (electron concentration). Example: solve('elx_semiconductor_conductivity', n='1e+
Semiconductor conductivity: n = 1e+16 cm^-3, p = 22500 cm^-3, mu_n = 1350 cm^2/(V*s), mu_p = 480 cm^2/(V*s), zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Drift velocity and drift current in a semiconductor bar, `elx_drift_current`

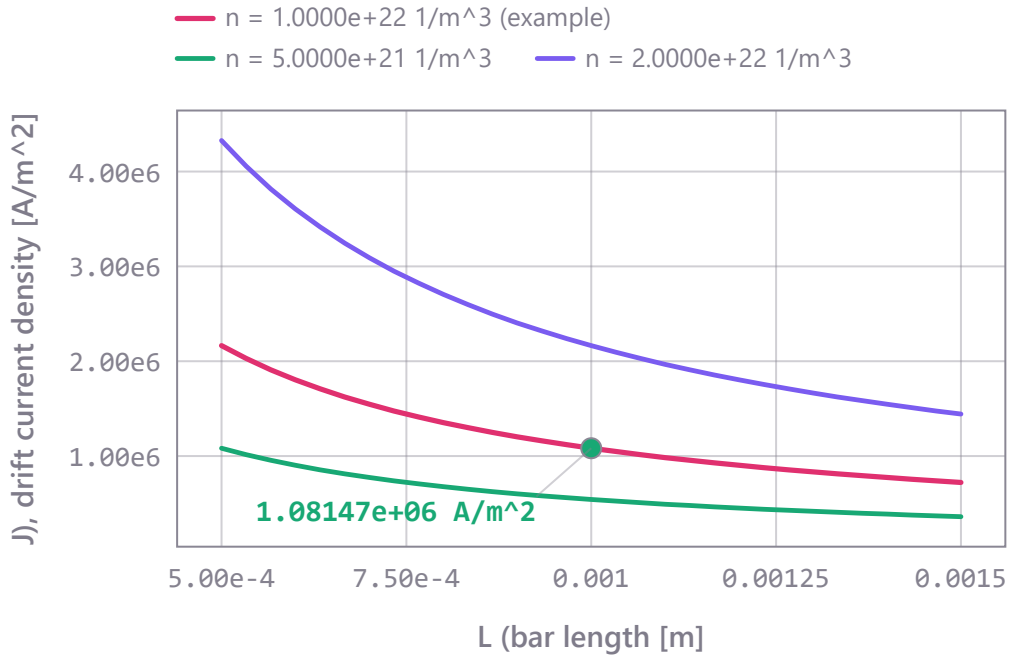
The electric field, the carrier drift velocity, the drift current density and the current when a voltage is applied along a uniformly doped bar.

$$J = q\,n\,\mu\,E, \quad E = V/L$$

input	meaning	condition
`n`	concentration of the carriers that drift	
`mu`	their mobility	
`V`	voltage applied along the bar	
`L`	bar length	
`A`	cross-sectional area	

output	meaning
`E`	electric field (V/m)
`v_d`	drift velocity
`J`	drift current density
`I`	current

- **Method:** $E = V/L$; $v_d = mu$ E; $J = q$ n v_d; $I = J$ A
- **Assumes:** n, mu, L, $A > 0$; low field (constant mobility)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



J (drift current density) against L (bar length) from 0.0005 m to 0.0015 m. One curve for each value of n (concentration of the carriers that drift): 1.0000e+22 1/m^3, 5.0000e+21 1/m^3, 2.0000e+22 1/m^3. Other inputs as in the worked example: mu = 0.135 m^2/(V*s), V = 5.0 V, A = 1e-08 m^2. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, L = 0.001 m, J = 1.08147e+06 A/m^2.

```
electronics_engineering_ai999x.solve('elx_drift_current', n=1e+22, mu=0.135, V=5.0, L=0.001, A=1e-08)
# -> E = 5000 N/C, v_d = 675 m/s, J = 1.08147e+06 A/m^2, I = 0.0108147 A   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_drift_current', n=5e+21, mu=0.048, V=2.0, L=0.0005, A=2.5e-09)
# -> E = 4000 N/C, v_d = 192 m/s, J = 153809 A/m^2, I = 0.000384522 A   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Drift current: n = 1e+16 cm^-3, mu = 1350 cm^2/(V*s), V = 5 V, L = 0.001 m, A = 1e-08 m^2	SOLVED	E = 5000 N/C, v_d = 675 m/s, J = 1.08147e+06 A/m^2, I = 0.0108147 A
Drift current: n = 5e+15 cm^-3, mu = 480 cm^2/(V*s), V = 2 V, L = 0.0005 m, A = 2.5e-09 m^2	SOLVED	E = 4000 N/C, v_d = 192 m/s, J = 153809 A/m^2, I = 0.000384522 A
Drift current: mu = 1350 cm^2/(V*s), V = 5 V, L = 0.001 m, A = 1e-08 m^2	NEEDS_INPUT	elx_drift_current needs n (concentration of the carriers that drift). Example: solve('elx_drift_current', n='1e+22 1/m^3
Drift current: n = 1e+16 cm^-3, mu = 1350 cm^2/(V*s), V = 5 V, L = 0.001 m, A = 1e-08 m^2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Einstein relation and diffusion length, `elx_einstein_diffusion`

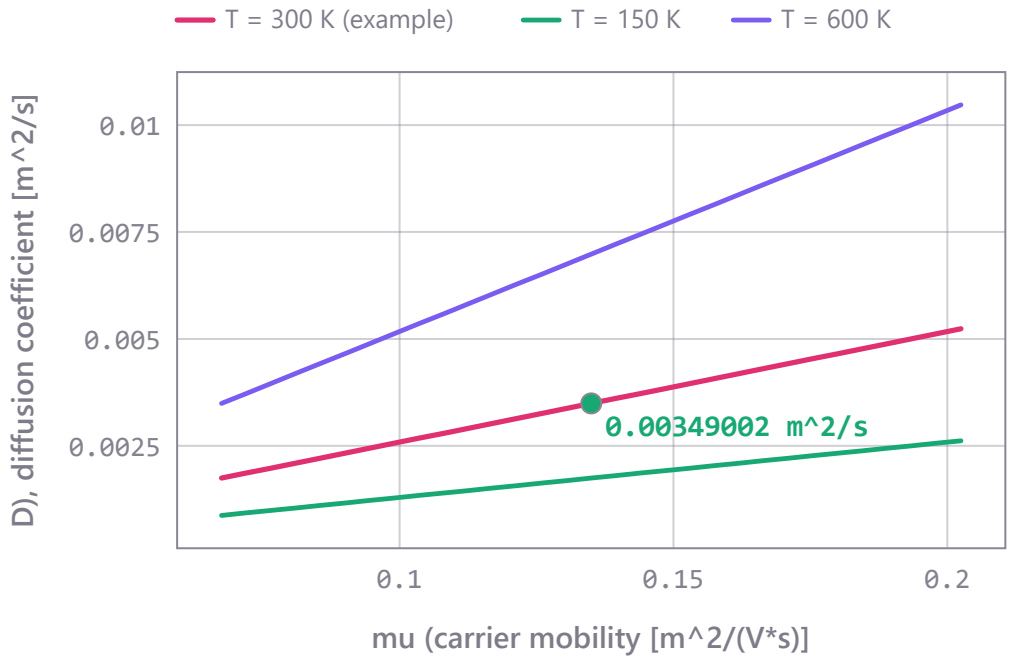
The diffusion coefficient $D = mu$ kT/q of carriers from their mobility (Einstein relation) and the diffusion length $L = sqrt$(D tau) from the minority-carrier lifetime.

$$D = \mu \frac{kT}{q}, \quad L = \sqrt{D\tau}$$

input	meaning	condition
`mu`	carrier mobility	
`T`	absolute temperature	
`tau`	minority-carrier lifetime	

output	meaning
`D`	diffusion coefficient
`D_cm2`	D in cm^2/s
`L_diff`	diffusion length
`L_um`	diffusion length in micrometres

- **Method:** $D = mu$ kT/q; $L = sqrt$(D tau)
- **Assumes:** mu, T, $\tau > 0$; non-degenerate material
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



D (diffusion coefficient) against mu (carrier mobility) from 0.0675 m^2/(V*s) to 0.2025 m^2/(V*s). One curve for each value of T (absolute temperature): 300 K, 150 K, 600 K. Other inputs as in the worked example: tau = 1e-06 s. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, mu = 0.135 m^2/(V*s), D = 0.00349002 m^2/s.

```
electronics_engineering_ai999x.solve('elx_einstein_diffusion', mu=0.135, T=300.0, tau=1e-06)
# -> D = 0.00349002 m^2/s, D_cm2 = 34.9002, L_diff = 5.90764e-05 m, L_um = 59.0764   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_einstein_diffusion', mu=0.048, T=300.0, tau=1e-05)
# -> D = 0.0012409 m^2/s, D_cm2 = 12.409, L_diff = 0.000111396 m, L_um = 111.396   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Einstein relation: mu = 1350 cm^2/(V*s), T = 300 K, tau = 1 us	SOLVED	D = 0.00349002 m^2/s, D_cm2 = 34.9002, L_diff = 5.90764e-05 m, L_um = 59.0764
Einstein relation: mu = 480 cm^2/(V*s), T = 300 K, tau = 10 us	SOLVED	D = 0.0012409 m^2/s, D_cm2 = 12.409, L_diff = 0.000111396 m, L_um = 111.396
Einstein relation: T = 300 K, tau = 1 us	NEEDS_INPUT	elx_einstein_diffusion needs mu (carrier mobility). Example: solve('elx_einstein_diffusion', mu='0.135 m^2/(V*s)', T='30
Einstein relation: mu = 1350 cm^2/(V*s), T = 300 K, tau = 1 us, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Electron and hole concentrations of doped (compensated) material,

`elx_doped_carrier_concentrations`

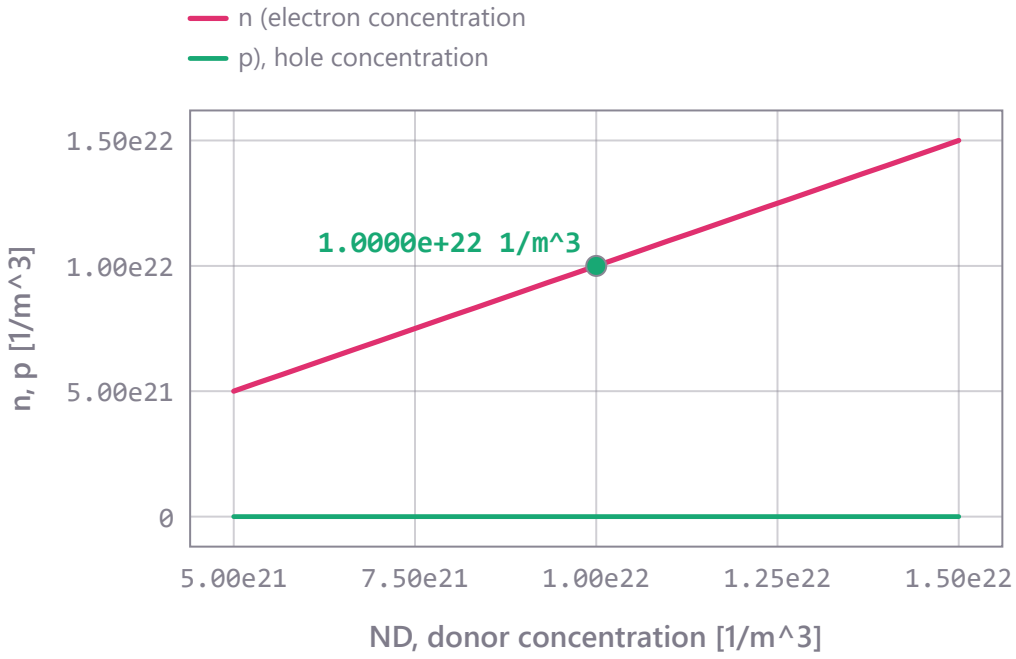
The equilibrium electron and hole concentrations of a semiconductor with donors and acceptors, from charge neutrality and the mass-action law (exact, so it also holds for light or compensated doping).

$$n = \frac{N_D - N_A}{2} + \sqrt{\left(\frac{N_D - N_A}{2}\right)^2 + n_i^2}, \quad p = \frac{n_i^2}{n}$$

input	meaning	condition
`ND`	donor concentration	
`NA`	acceptor concentration	
`ni`	intrinsic carrier concentration	

output	meaning
`n`	electron concentration
`p`	hole concentration
`n_cm3`	n in cm^{-3}
`p_cm3`	p in cm^{-3}
`type`	n-type, p-type or intrinsic

- **Method:** $n - p = ND - NA$ and n $p = ni^2$: majority = |ND-NA|/$2 + sqrt$((ND-NA)^2/4 + ni^2), minority = ni^2/majority
- **Assumes:** ND, $NA \geq 0$; $ni > 0$; complete ionisation, non-degenerate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



n (electron concentration) against ND (donor concentration) from 5.0000e+21 1/m^3 to 1.5000e+22 1/m^3. Outputs with the same unit drawn together: p (hole concentration). Other inputs as in the worked example: NA = 0.0 1/m^3, ni = 1.5e+16 1/m^3. 31 points computed by electronics-engineering-ai999x; marked: the worked example, ND = 1.0000e+22 1/m^3, n = 1.0000e+22 1/m^3.

```
electronics_engineering_ai999x.solve('elx_doped_carrier_concentrations', ND=1e+22, NA=0.0,
ni=1.5e+16)
# -> n = 1e+22 1/m^3, p = 2.25e+10 1/m^3, n_cm3 = 1e+16, p_cm3 = 22500, type = n-type   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_doped_carrier_concentrations', ND=3e+16, NA=5e+16,
ni=1.5e+16)
# -> n = 8.02776e+15 1/m^3, p = 2.80278e+16 1/m^3, n_cm3 = 8.02776e+09, p_cm3 = 2.80278e+10, type =
p-type   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Carrier concentrations of doped material: ND = 1e+16 cm^-3, NA = 0 cm^-3, ni = 1.5e+10 cm^-3	SOLVED	n = 1e+22 1/m^3, p = 2.25e+10 1/m^3, n_cm3 = 1e+16, p_cm3 = 22500, type = n-type
Carrier concentrations of doped material: ND = 3e+10 cm^-3, NA = 5e+10 cm^-3, ni = 1.5e+10 cm^-3	SOLVED	n = 8.02776e+15 1/m^3, p = 2.80278e+16 1/m^3, n_cm3 = 8.02776e+09, p_cm3 = 2.80278e+10, type = p-type
Carrier concentrations of doped material: NA = 0 cm^-3, ni = 1.5e+10 cm^-3	NEEDS_INPUT	elx_doped_carrier_concentrations needs ND (donor concentration). Example: solve('elx_doped_carrier_concentrations', ND='
Carrier concentrations of doped material: ND = 1e+16 cm^-3, NA = 0 cm^-3, ni = 1.5e+10 cm^-3, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Built-in potential of a pn junction, `elx_builtin_potential`

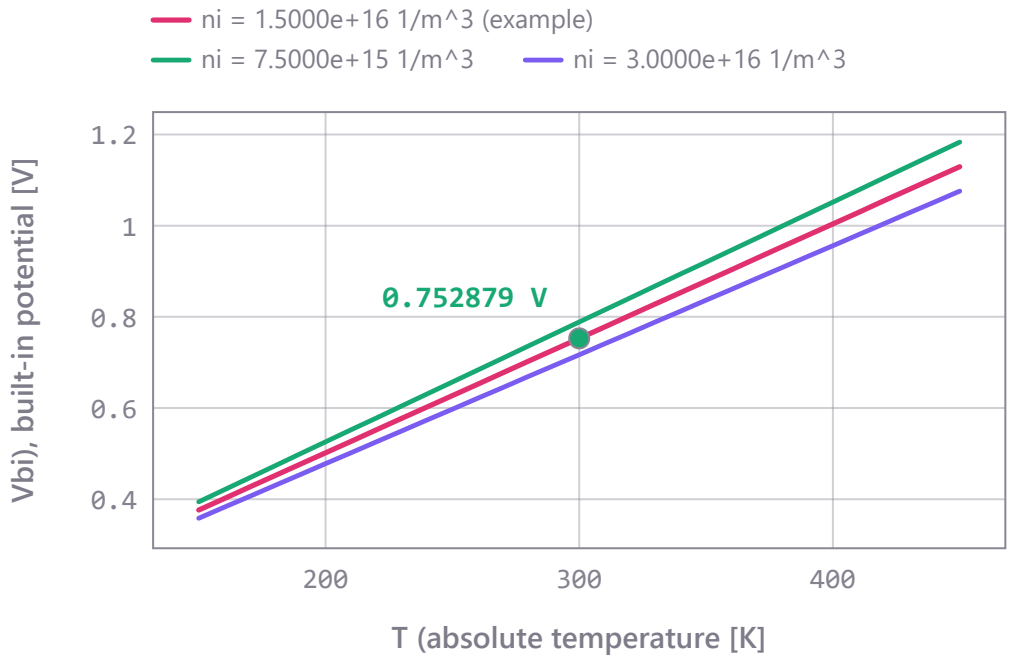
The built-in (contact) potential of an abrupt pn junction from the doping on each side, the intrinsic concentration and the temperature.

$$V_{bi} = \frac{kT}{q} \ln \frac{N_A N_D}{n_i^2}$$

input	meaning	condition
`NA`	acceptor concentration on the p side	
`ND`	donor concentration on the n side	
`ni`	intrinsic carrier concentration	
`T`	absolute temperature	

output	meaning
`Vbi`	built-in potential

- **Method:** Vbi = (kT/q) ln(NA ND / ni^2)
- **Assumes:** NA, $ND \geq 100$ ni; non-degenerate, complete ionisation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vbi (built-in potential) against T (absolute temperature) from 150 K to 450 K. One curve for each value of ni (intrinsic carrier concentration): 1.5000e+16 1/m^3, 7.5000e+15 1/m^3, 3.0000e+16 1/m^3. Other inputs as in the worked example: NA = 1e+23 1/m^3, ND = 1e+22 1/m^3. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, T = 300 K, Vbi = 0.752879 V.

```
electronics_engineering_ai999x.solve('elx_builtin_potential', NA=1e+23, ND=1e+22, ni=1.5e+16,
T=300.0)
# -> Vbi = 0.752879 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_builtin_potential', NA=1e+24, ND=1e+21, ni=1e+16,
T=300.0)
# -> Vbi = 0.773844 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Built-in potential: NA = 1e+17 cm^-3, ND = 1e+16 cm^-3, ni = 1.5e+10 cm^-3, T = 300 K	SOLVED	Vbi = 0.752879 V
Built-in potential: NA = 1e+18 cm^-3, ND = 1e+15 cm^-3, ni = 1e+10 cm^-3, T = 300 K	SOLVED	Vbi = 0.773844 V
Built-in potential: ND = 1e+16 cm^-3, ni = 1.5e+10 cm^-3, T = 300 K	NEEDS_INPUT	elx_builtin_potential needs NA (acceptor concentration on the p side). Example: solve('elx_builtin_potential', NA='1e+23
Built-in potential: NA = 1e+17 cm^-3, ND = 1e+16 cm^-3, ni = 1.5e+10 cm^-3, T = 300 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Depletion width, field and capacitance of an abrupt pn junction, `elx_depletion_width`

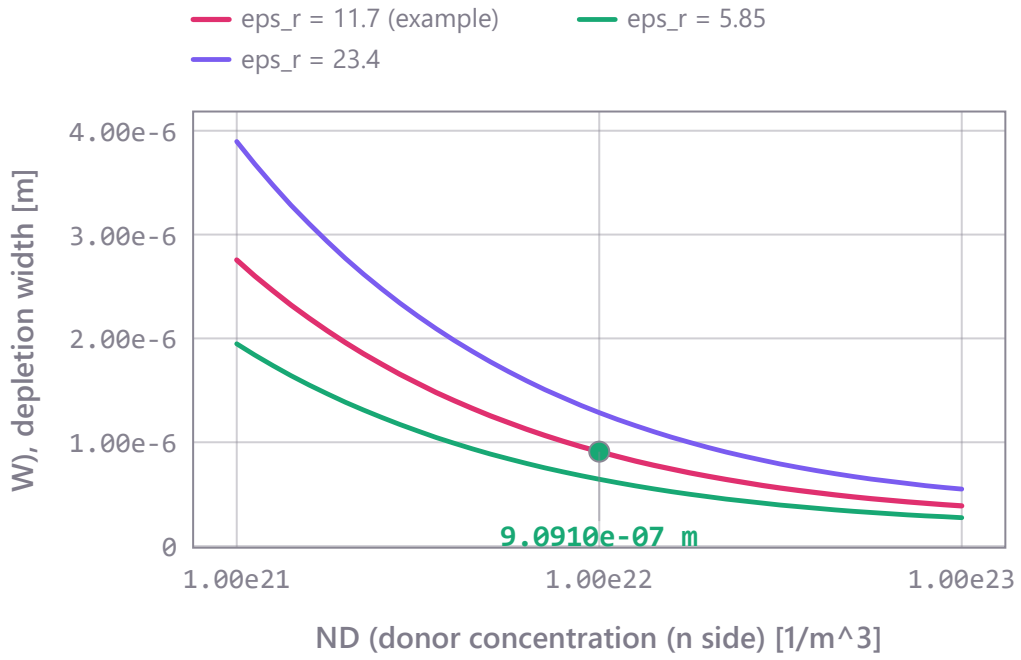
The depletion-region width (and its parts on each side), the peak electric field and the depletion capacitance per area of an abrupt pn junction at a reverse bias (negative VR for a forward bias below Vbi).

$$W = \sqrt{\frac{2\varepsilon(V_{bi} + V_R)}{q}\left(\frac{1}{N_A} + \frac{1}{N_D}\right)}$$

input	meaning	condition
`NA`	acceptor concentration (p side)	
`ND`	donor concentration (n side)	
`Vbi`	built-in potential	
`VR`	reverse bias (negative for forward)	
`eps_r`	relative permittivity of the semiconductor (11.7 for silicon)	

output	meaning
`W`	depletion width
`W_um`	depletion width in micrometres
`xn`	width on the n side
`xp`	width on the p side
`Emax`	peak electric field (V/m)
`Cj_area`	depletion capacitance per area

- **Method:** $W = sqrt$(2 eps (Vbi + VR)/q $(1/NA + 1/ND)$); $xn = W$ NA/(NA+ND); Emax = q ND xn/eps; C' = eps/W
- **Assumes:** Vbi + VR > 0; abrupt junction, depletion approximation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



W (depletion width) against ND (donor concentration (n side)) from 1.0000e+21 1/m^3 to 1.0000e+23 1/m^3 on a logarithmic axis. One curve for each value of eps_r (relative permittivity of the semiconductor (11.7 for silicon)): 11.7, 5.85, 23.4. Other inputs as in the worked example: NA = 1e+23 1/m^3, Vbi = 0.81 V, VR = 5.0 V. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, ND = 1.0000e+22 1/m^3, W = 9.0910e-07 m.

```
electronics_engineering_ai999x.solve('elx_depletion_width', NA=1e+23, ND=1e+22, Vbi=0.81, VR=5.0,
eps_r=11.7)
# -> W = 9.091e-07 m, W_um = 0.9091, xn = 8.26454e-07 m, xp = 8.26454e-08 m, Emax = 1.27819e+07
N/C, Cj_area = 0.000113952 F/m^2   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_depletion_width', NA=1e+22, ND=1e+22, Vbi=0.7, VR=0.0,
eps_r=11.7)
# -> W = 4.25492e-07 m, W_um = 0.425492, xn = 2.12746e-07 m, xp = 2.12746e-07 m, Emax = 3.29031e+06
N/C, Cj_area = 0.000243469 F/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Depletion width: NA = 1e+17 cm^-3, ND = 1e+16 cm^-3, Vbi = 0.81 V, VR = 5 V, eps_r = 11.7	SOLVED	W = 9.091e-07 m, W_um = 0.9091, xn = 8.26454e-07 m, xp = 8.26454e-08 m, Emax = 1.27819e+07 N/C, Cj_area = 0.000113952 F/m^2
Depletion width: NA = 1e+16 cm^-3, ND = 1e+16 cm^-3, Vbi = 0.7 V, VR = 0 V, eps_r = 11.7	SOLVED	W = 4.25492e-07 m, W_um = 0.425492, xn = 2.12746e-07 m, xp = 2.12746e-07 m, Emax = 3.29031e+06 N/C, Cj_area = 0.000243469 F/m^2
Depletion width: ND = 1e+16 cm^-3, Vbi = 0.81 V, VR = 5 V, eps_r = 11.7	NEEDS_INPUT	elx_depletion_width needs NA (acceptor concentration (p side)). Example: solve('elx_depletion_width', NA='1e+23 1/m^3',
Depletion width: NA = 1e+17 cm^-3, ND = 1e+16 cm^-3, Vbi = 0.81 V, VR = 5 V, eps_r = 11.7, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Junction (depletion) capacitance versus bias, `elx_junction_capacitance`

The depletion capacitance of a pn junction at a reverse bias from its zero-bias value, the built-in potential and the grading coefficient (1/2 abrupt, $1/3$ linearly graded), as used for varactors.

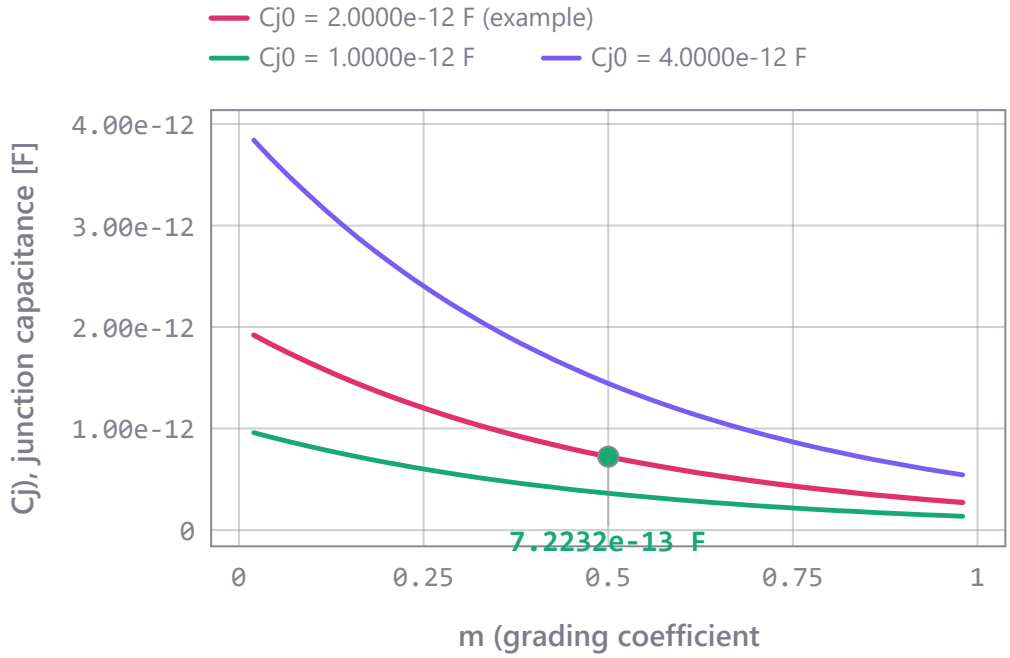
$$C_j = \frac{C_{j0}}{(1 + V_R/V_{bi})^m}$$

input	meaning	condition
`Cj0`	zero-bias junction capacitance	
`Vbi`	built-in potential	
`VR`	reverse bias (negative for forward)	
`m`	grading coefficient (0.5 abrupt, 0.33 linearly graded)	

output	meaning
`Cj`	junction capacitance
`ratio`	$Cj/Cj0$

- **Method:** Cj = Cj0 / (1 + VR/Vbi)^m
- **Assumes:** $0 < m < 1$; VR > -Vbi

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cj (junction capacitance) against m (grading coefficient (0.5 abrupt, 0.33 linearly graded)) from 0.02 to 0.98. One curve for each value of Cj0 (zero-bias junction capacitance): 2.0000e-12 F, 1.0000e-12 F, 4.0000e-12 F. Other inputs as in the worked example: Vbi = 0.75 V, VR = 5.0 V. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, m = 0.5, Cj = 7.2232e-13 F.

```
electronics_engineering_ai999x.solve('elx_junction_capacitance', Cj0=2e-12, Vbi=0.75, VR=5.0,
m=0.5)
# -> Cj = 7.22315e-13 F, ratio = 0.361158   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_junction_capacitance', Cj0=5e-11, Vbi=0.7, VR=2.0,
m=0.33)
# -> Cj = 3.2026e-11 F, ratio = 0.640519   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Junction capacitance: Cj0 = 2 pF, Vbi = 0.75 V, VR = 5 V, m = 0.5	SOLVED	Cj = 7.22315e-13 F, ratio = 0.361158
Junction capacitance: Cj0 = 50 pF, Vbi = 0.7 V, VR = 2 V, m = 0.33	SOLVED	Cj = 3.2026e-11 F, ratio = 0.640519
Junction capacitance: Vbi = 0.75 V, VR = 5 V, m = 0.5	NEEDS_INPUT	elx_junction_capacitance needs Cj0 (zero-bias junction capacitance). Example: solve('elx_junction_capacitance', Cj0='2e-
Junction capacitance: Cj0 = 2 pF, Vbi = 0.75 V, VR = 5 V, m = 0.5, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Diode current from the Shockley equation, `elx_shockley_diode_current`

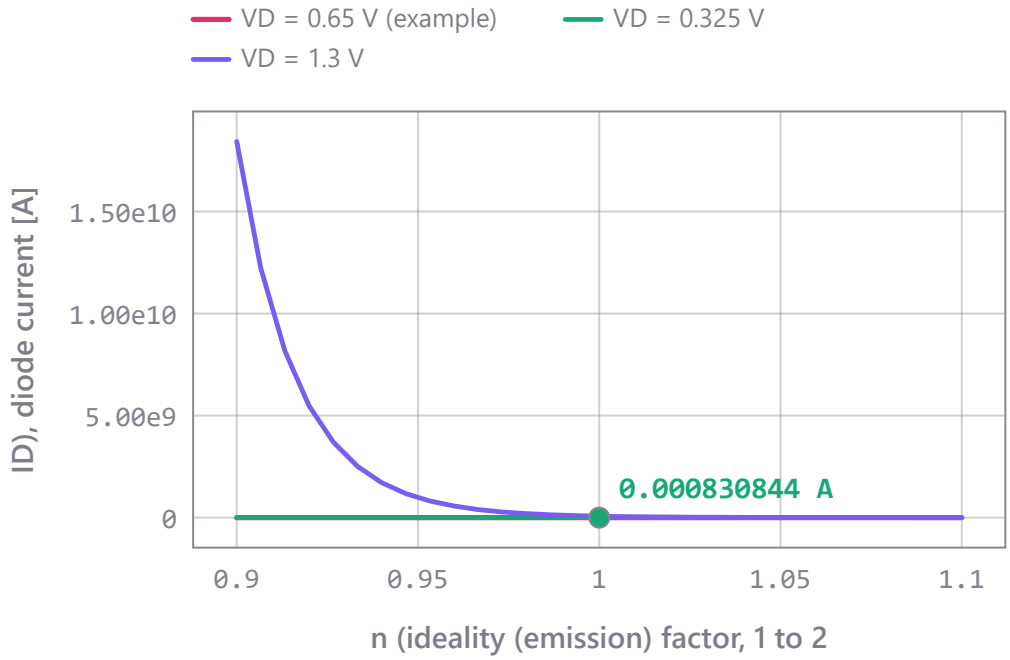
The current of a pn-junction diode at a terminal voltage from its saturation current, ideality factor and temperature (forward or reverse).

$$I_D = I_S \left(e^{V_D / n V_T} - 1 \right)$$

input	meaning	condition
`Is`	saturation current	
`n`	ideality (emission) factor, 1 to 2	
`VD`	diode voltage (negative for reverse bias)	
`T`	junction temperature	

output	meaning
`ID`	diode current
`VT`	thermal voltage

- **Method:** $ID = Is$ (exp(VD/(n VT)) - 1), $VT = kT/q$
- **Assumes:** Is, n, $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ID (diode current) against n (ideality (emission) factor, 1 to 2) from 0.9 to 1.1. One curve for each value of VD (diode voltage (negative for reverse bias)): 0.65 V, 0.325 V, 1.3 V. Other inputs as in the worked example: Is = 1e-14 A, T = 300.0 K. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, n = 1, ID = 0.000830844 A.

```
electronics_engineering_ai999x.solve('elx_shockley_diode_current', Is=1e-14, n=1.0, VD=0.65,
T=300.0)
# -> ID = 0.000830844 A, VT = 0.025852 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_shockley_diode_current', Is=2e-12, n=1.8, VD=-2.0,
T=300.0)
# -> ID = -2e-12 A, VT = 0.025852 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Shockley diode current: Is = 1e-14 A, n = 1, VD = 0.65 V, T = 300 K	SOLVED	ID = 0.000830844 A, VT = 0.025852 V
Shockley diode current: Is = 2e-12 A, n = 1.8, VD = -2 V, T = 300 K	SOLVED	ID = -2e-12 A, VT = 0.025852 V
Shockley diode current: n = 1, VD = 0.65 V, T = 300 K	NEEDS_INPUT	elx_shockley_diode_current needs Is (saturation current). Example: solve('elx_shockley_diode_current', Is='1e-14 A', n=1
Shockley diode current: Is = 1e-14 A, n = 1, VD = 0.65 V, T = 300 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Diode voltage for a given current, `elx_diode_voltage`

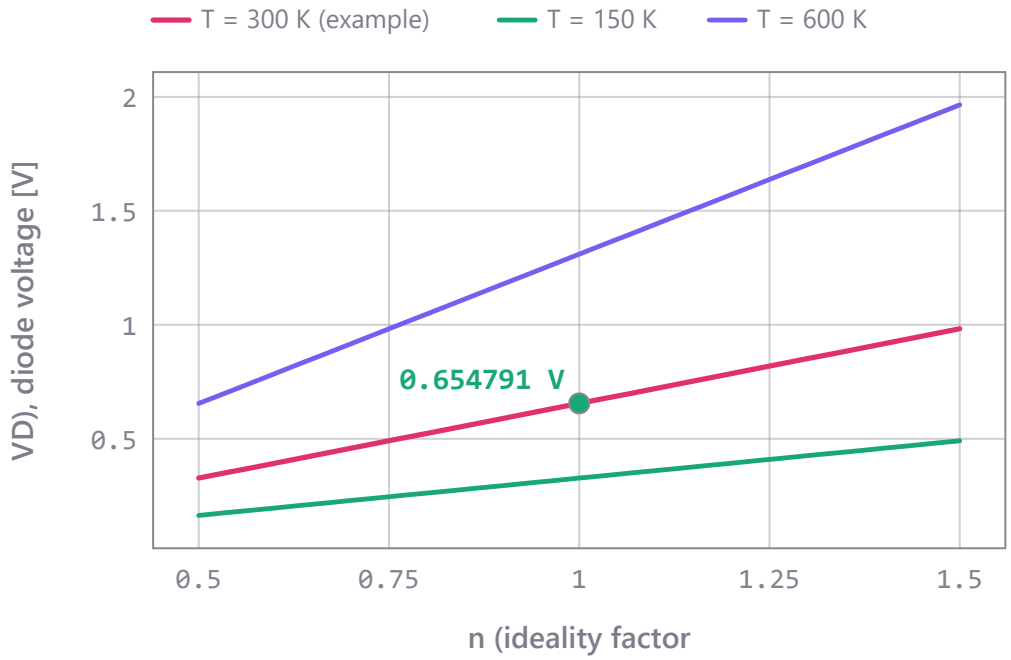
The voltage across a pn-junction diode carrying a given current (the inverse of the Shockley equation).

$$V_D = nV_T \ln\left(1 + \frac{I_D}{I_S}\right)$$

input	meaning	condition
`ID`	diode current	
`Is`	saturation current	
`n`	ideality factor	
`T`	junction temperature	

output	meaning
`VD`	diode voltage
`VT`	thermal voltage

- **Method:** $VD = n$ VT ln(1 + ID/Is)
- **Assumes:** $ID > -Is$; Is, n, $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



VD (diode voltage) against n (ideality factor) from 0.5 to 1.5. One curve for each value of T (junction temperature): 300 K, 150 K, 600 K. Other inputs as in the worked example: ID = 0.001 A, Is = 1e-14 A. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, n = 1, VD = 0.654791 V.

```
electronics_engineering_ai999x.solve('elx_diode_voltage', ID=0.001, Is=1e-14, n=1.0, T=300.0)
# -> VD = 0.654791 V, VT = 0.025852 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_diode_voltage', ID=0.02, Is=5e-12, n=1.7, T=320.0)
# -> VD = 1.03646 V, VT = 0.0275755 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Diode voltage for a current: ID = 0.001 A, Is = 1e-14 A, n = 1, T = 300 K	SOLVED	VD = 0.654791 V, VT = 0.025852 V
Diode voltage for a current: ID = 0.02 A, Is = 5e-12 A, n = 1.7, T = 320 K	SOLVED	VD = 1.03646 V, VT = 0.0275755 V
Diode voltage for a current: Is = 1e-14 A, n = 1, T = 300 K	NEEDS_INPUT	elx_diode_voltage needs ID (diode current). Example: solve('elx_diode_voltage', ID='0.001 A', Is='1e-14 A', n=1, T='300
Diode voltage for a current: ID = 0.001 A, Is = 1e-14 A, n = 1, T = 300 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Operating point of a diode in series with a resistor (exact), `elx_diode_operating_point`

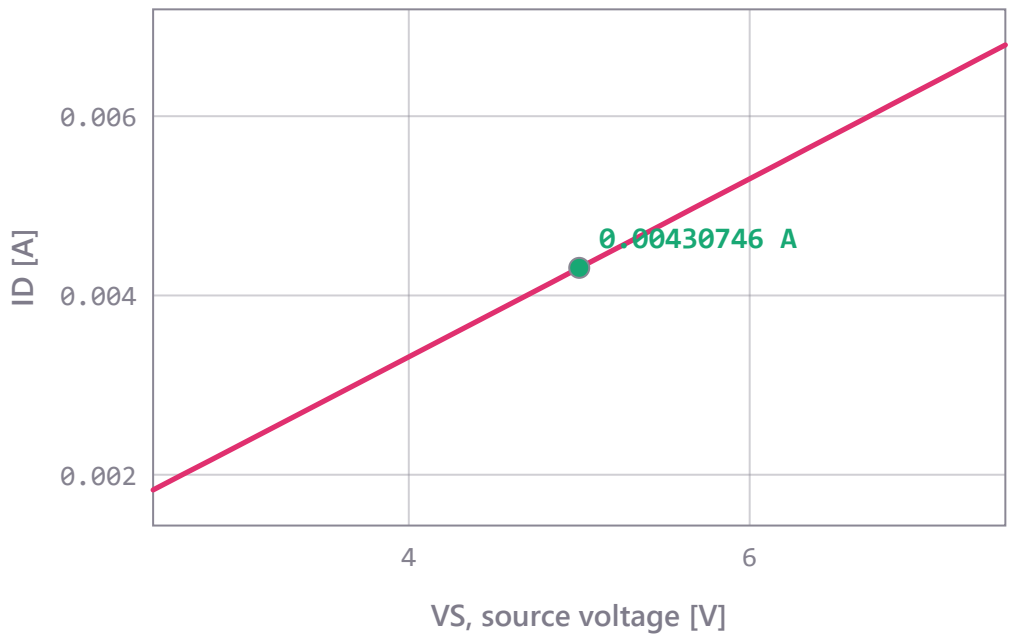
The current and voltage of a forward-biased diode fed from a DC source through a resistor, solving the Shockley equation and Kirchhoff's voltage law together (no constant-drop approximation).

$$V_S = I_D R + n V_T \ln\left(1 + \frac{I_D}{I_S}\right)$$

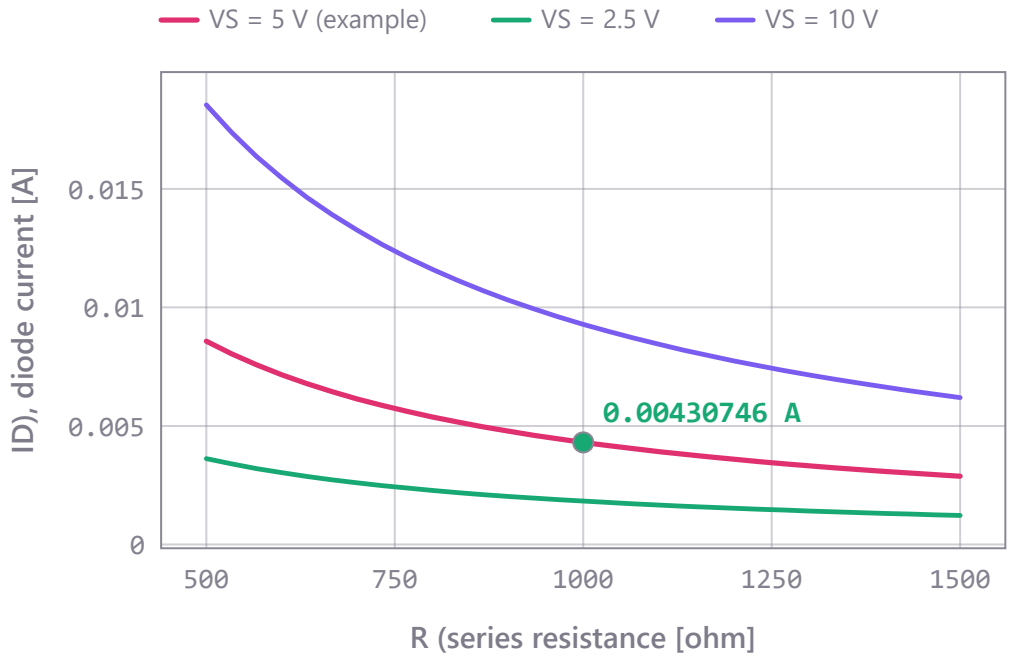
input	meaning	condition
`VS`	source voltage	
`R`	series resistance	
`Is`	saturation current	
`n`	ideality factor	
`T`	junction temperature	

output	meaning
`ID`	diode current
`VD`	diode voltage
`P_D`	power in the diode

- **Method:** solve $VS = ID$ R + n VT ln(1 + ID/Is) (bisection on VD)
- **Assumes:** $VS > 0$ (forward bias); R, Is, n, $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ID (diode current) against VS (source voltage), with Is = 1.00e-14 A, R = 1000 ohm, T = 300 K, n = 1. 31 points computed by electronics-engineering-ai999x; marked: VS = 5, ID = 0.00430746 A.



ID (diode current) against R (series resistance) from 500 ohm to 1500 ohm. One curve for each value of VS (source voltage): 5 V, 2.5 V, 10 V. Other inputs as in the worked example: Is = 1e-14 A, n = 1.0, T = 300.0 K. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R = 1000 ohm, ID = 0.00430746 A.

```
electronics_engineering_ai999x.solve('elx_diode_operating_point', VS=5.0, R=1000.0, Is=1e-14,
n=1.0, T=300.0)
# -> ID = 0.00430746 A, VD = 0.692544 V, P_D = 0.0029831 W   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_diode_operating_point', VS=12.0, R=2200.0, Is=3e-09,
n=1.9, T=300.0)
# -> ID = 0.00513409 A, VD = 0.704992 V, P_D = 0.0036195 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Diode operating point: VS = 5 V, R = 1000 ohm, Is = 1e-14 A, n = 1, T = 300 K	SOLVED	ID = 0.00430746 A, VD = 0.692544 V, P_D = 0.0029831 W
Diode operating point: VS = 12 V, R = 2200 ohm, Is = 3e-09 A, n = 1.9, T = 300 K	SOLVED	ID = 0.00513409 A, VD = 0.704992 V, P_D = 0.0036195 W
Diode operating point: R = 1000 ohm, Is = 1e-14 A, n = 1, T = 300 K	NEEDS_INPUT	elx_diode_operating_point needs VS (source voltage). Example: solve('elx_diode_operating_point', VS='5 V', R='1000 ohm',
Diode operating point: VS = 5 V, R = 1000 ohm, Is = 1e-14 A, n = 1, T = 300 K, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Rectifier output: average, RMS, ripple factor and PIV, `elx_rectifier_averages`

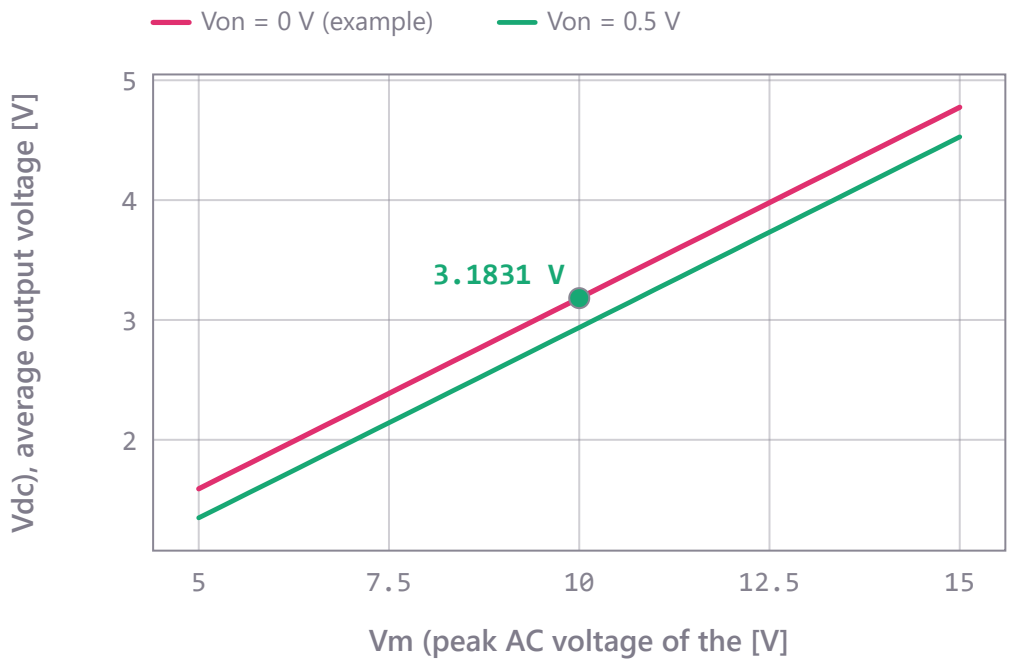
The DC (average) and RMS output voltage, ripple factor, peak inverse voltage and conduction angle of an unfiltered half-wave, bridge or centre-tapped full-wave rectifier with a constant diode drop (0 V for ideal diodes).

$$V_{dc} = \frac{k}{2\pi}[2V_m \cos\theta_1 - V(\pi - 2\theta_1)],\ \theta_1 = \arcsin\frac{V}{V_m}$$

input	meaning	condition
`Vm`	peak AC voltage of the (secondary) winding	
`circuit`	half-wave, bridge or center-tapped (Vm per half of the winding)	
`Von`	forward drop of one diode (0 V for ideal diodes)	

output	meaning
`Vp_out`	peak output voltage
`Vdc`	average output voltage
`Vrms`	RMS output voltage
`ripple_factor`	ripple factor (AC rms / DC)
`PIV`	peak inverse voltage of a diode
`conduction_deg`	conduction angle per half cycle, degrees

- **Method:** output max(0, Vm|sin| - V) with V = Von (2 Von for a bridge); averages integrated exactly; PIV Vm, Vm - Von, 2Vm - Von
- **Assumes:** diode drops below Vm; resistive load, no filter
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vdc (average output voltage) against Vm (peak AC voltage of the (secondary) winding) from 5 V to 15 V. One curve for each value of Von (forward drop of one diode (0 V for ideal diodes)): 0 V, 0.5 V. Other inputs as in the worked example: circuit = half-wave. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Vm = 10 V, Vdc = 3.1831 V.

```
electronics_engineering_ai999x.solve('elx_rectifier_averages', Vm=10.0, circuit='half-wave',
Von=0.0)
# -> Vp_out = 10 V, Vdc = 3.1831 V, Vrms = 5 V, ripple_factor = 1.21136, PIV = 10 V, conduction_deg
= 180   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_rectifier_averages', Vm=17.0, circuit='bridge', Von=0.7)
# -> Vp_out = 15.6 V, Vdc = 9.45926 V, Vrms = 10.776 V, ripple_factor = 0.545697, PIV = 16.3 V,
conduction_deg = 170.552   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rectifier output: Vm = 10 V, circuit = half-wave, Von = 0 V	SOLVED	Vp_out = 10 V, Vdc = 3.1831 V, Vrms = 5 V, ripple_factor = 1.21136, PIV = 10 V, conduction_deg = 180
Rectifier output: Vm = 17 V, circuit = bridge, Von = 0.7 V	SOLVED	Vp_out = 15.6 V, Vdc = 9.45926 V, Vrms = 10.776 V, ripple_factor = 0.545697, PIV = 16.3 V, conduction_deg = 170.552
Rectifier output: circuit = half-wave, Von = 0 V	NEEDS_INPUT	elx_rectifier_averages needs Vm (peak AC voltage of the (secondary) winding). Example: solve('elx_rectifier_averages', V
Rectifier output: Vm = 10 V, circuit = half-wave, Von = 0 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te
Rectifier output: Vm = 10 V, circuit = half wave, Von = 0 V	SOLVED	Vp_out = 10 V, Vdc = 3.1831 V, Vrms = 5 V, ripple_factor = 1.21136, PIV = 10 V, conduction_deg = 180

Capacitor-input filter: ripple voltage and DC output, `elx_capacitor_filter_ripple`

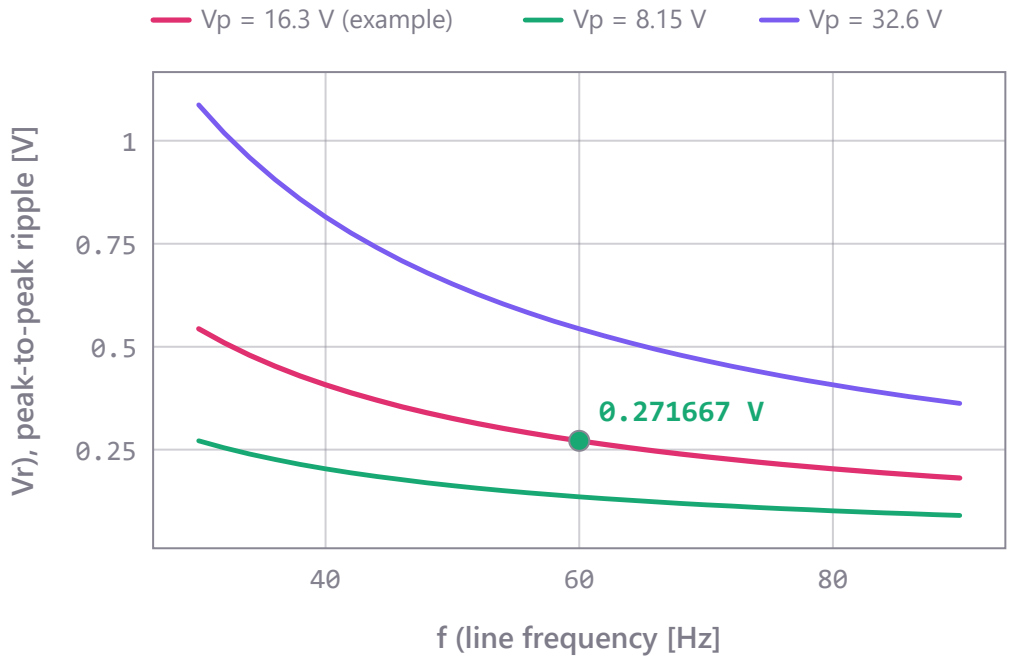
The peak-to-peak ripple, the DC output, the RMS ripple and the ripple factor of a rectifier with a smoothing capacitor and resistive load (small-ripple approximation).

$$V_r = \frac{V_p}{f_r R_L C}, \quad V_{dc} = V_p - \frac{V_r}{2}$$

input	meaning	condition
`Vp`	peak rectified voltage (after the diode drops)	
`f`	line frequency	
`RL`	load resistance	
`C`	filter capacitance	
`circuit`	half-wave or full-wave rectification	

output	meaning
`f_ripple`	ripple frequency
`Vr`	peak-to-peak ripple
`Vdc`	DC output voltage
`Vr_rms`	RMS ripple
`ripple_factor`	ripple factor

- **Method:** Vr = Vp/(f_r RL C); Vdc = Vp - Vr/2; Vr,rms = Vr/(2 sqrt 3)
- **Assumes:** $Vr < Vp$ (small ripple)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vr (peak-to-peak ripple) against f (line frequency) from 30 Hz to 90 Hz. One curve for each value of Vp (peak rectified voltage (after the diode drops)): 16.3 V, 8.15 V, 32.6 V. Other inputs as in the worked example: RL = 500.0 ohm, C = 0.001 F, circuit = full-wave. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, f = 60 Hz, Vr = 0.271667 V.

```
electronics_engineering_ai999x.solve('elx_capacitor_filter_ripple', Vp=16.3, f=60.0, RL=500.0,
C=0.001, circuit='full-wave')
# -> f_ripple = 120 Hz, Vr = 0.271667 V, Vdc = 16.1642 V, Vr_rms = 0.0784234 V, ripple_factor =
0.00485168   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_capacitor_filter_ripple', Vp=12.0, f=50.0, RL=1000.0,
C=0.00047, circuit='half-wave')
# -> f_ripple = 50 Hz, Vr = 0.510638 V, Vdc = 11.7447 V, Vr_rms = 0.147409 V, ripple_factor =
0.0125511   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Capacitor filter ripple: Vp = 16.3 V, f = 60 Hz, RL = 500 ohm, C = 1000 uF, circuit = full-wave	SOLVED	f_ripple = 120 Hz, Vr = 0.271667 V, Vdc = 16.1642 V, Vr_rms = 0.0784234 V, ripple_factor = 0.00485168
Capacitor filter ripple: Vp = 12 V, f = 50 Hz, RL = 1000 ohm, C = 470 uF, circuit = half-wave	SOLVED	f_ripple = 50 Hz, Vr = 0.510638 V, Vdc = 11.7447 V, Vr_rms = 0.147409 V, ripple_factor = 0.0125511
Capacitor filter ripple: f = 60 Hz, RL = 500 ohm, C = 1000 uF, circuit = full-wave	NEEDS_INPUT	elx_capacitor_filter_ripple needs Vp (peak rectified voltage (after the diode drops)). Example: solve('elx_capacitor_fil
Capacitor filter ripple: Vp = 16.3 V, f = 60 Hz, RL = 500 ohm, C = 1000 uF, circuit = full-wave, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Zener shunt regulator: load voltage and currents, `elx_zener_regulator`

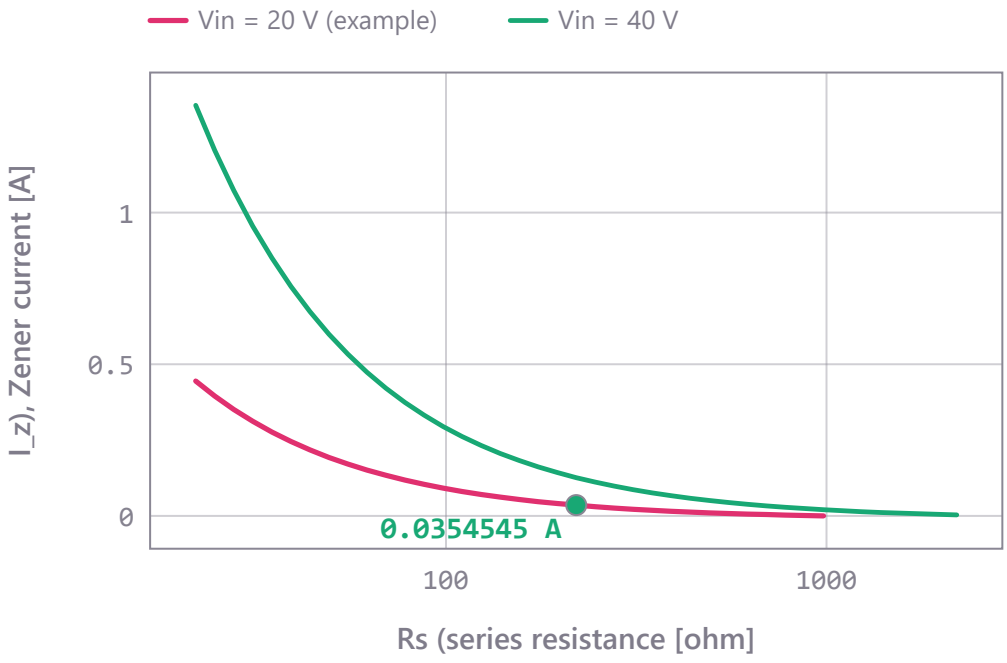
The output voltage and the source, load and Zener currents and the Zener power of a Zener shunt regulator (series resistor, Zener across the load), with an optional Zener resistance.

$$I_Z = \frac{V_{in} - V_Z}{R_S} - \frac{V_Z}{R_L}$$

input	meaning	condition
`Vin`	input voltage	
`Rs`	series resistance	
`Vz`	Zener (breakdown) voltage	
`RL`	load resistance	
`rz`	Zener resistance (0 for an ideal Zener)	

output	meaning
`Vout`	load voltage
`I_s`	current in the series resistor
`I_L`	load current
`I_z`	Zener current
`P_z`	Zener power

- **Method:** ideal: Vout = Vz, Iz = (Vin - Vz)/Rs - Vz/RL; with rz: node equation at the load
- **Assumes:** the Zener must be in breakdown: Vin $RL/(Rs + RL) > Vz$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



I_z (Zener current) against Rs (series resistance) from 22 ohm to 2200 ohm on a logarithmic axis. One curve for each value of Vin (input voltage): 20 V, 40 V. Other inputs as in the worked example: Vz = 10.0 V, RL = 1000.0 ohm. 34 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Rs = 220 ohm, I_z = 0.0354545 A.

```
electronics_engineering_ai999x.solve('elx_zener_regulator', Vin=20.0, Rs=220.0, Vz=10.0, RL=1000.0)
# -> Vout = 10 V, I_s = 0.0454545 A, I_L = 0.01 A, I_z = 0.0354545 A, P_z = 0.354545 W   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_zener_regulator', Vin=15.0, Rs=100.0, Vz=5.1, RL=470.0,
rz=7.0)
# -> Vout = 5.66876 V, I_s = 0.0933124 A, I_L = 0.0120612 A, I_z = 0.0812512 A, P_z = 0.460594 W
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Zener regulator: Vin = 20 V, Rs = 220 ohm, Vz = 10 V, RL = 1000 ohm	SOLVED	Vout = 10 V, I_s = 0.0454545 A, I_L = 0.01 A, I_z = 0.0354545 A, P_z = 0.354545 W
Zener regulator: Vin = 15 V, Rs = 100 ohm, Vz = 5.1 V, RL = 470 ohm, rz = 7 ohm	SOLVED	Vout = 5.66876 V, I_s = 0.0933124 A, I_L = 0.0120612 A, I_z = 0.0812512 A, P_z = 0.460594 W
Zener regulator: Rs = 220 ohm, Vz = 10 V, RL = 1000 ohm	NEEDS_INPUT	elx_zener_regulator needs Vin (input voltage). Example: solve('elx_zener_regulator', Vin='20 V', Rs='220 ohm', Vz='10 V'
Zener regulator: Vin = 20 V, Rs = 220 ohm, Vz = 10 V, RL = 1000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te
Zener regulator: Vin = 12 V, Rs = 1 kohm, Vz = 9 V, RL = 1 kohm	INVALID_INPUT	Zener shunt regulator: load voltage and currents: the Zener is not in breakdown: the open-circuit voltage across the loa

Zener regulator design: series resistor and Zener rating, `elx_zener_series_resistor_design`

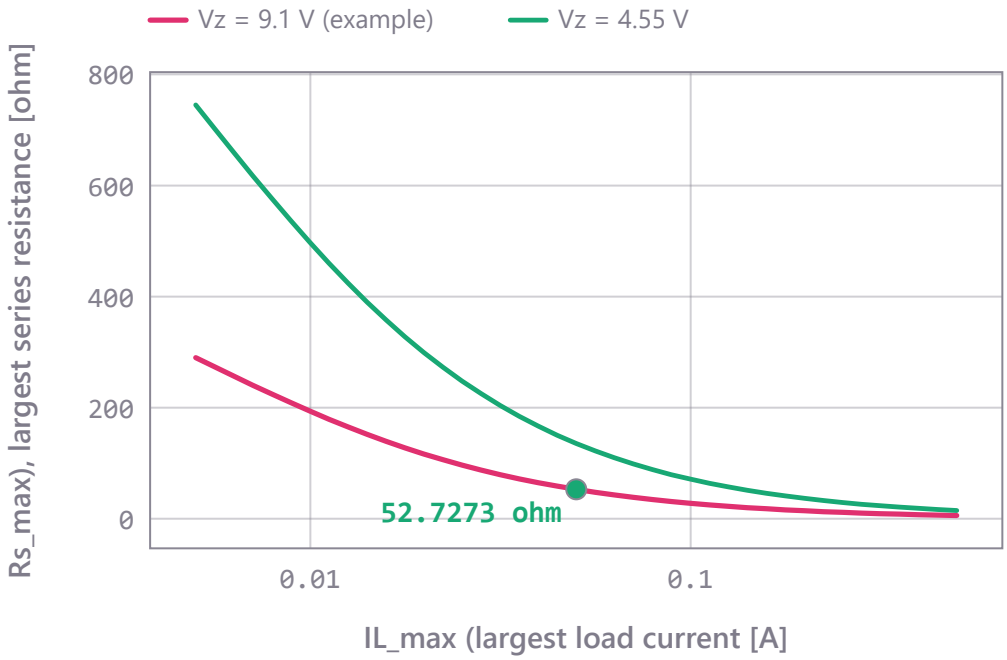
The largest series resistance that keeps a Zener regulator in regulation at the lowest input and largest load, and the resulting worst-case Zener current and power (highest input, smallest load) and resistor power.

$$R_S = \frac{V_{in,min} - V_Z}{I_{Z,min} + I_{L,max}}$$

input	meaning	condition
`Vin_min`	lowest input voltage	
`Vin_max`	highest input voltage	
`Vz`	Zener voltage	
`IL_min`	smallest load current	
`IL_max`	largest load current	
`Iz_min`	minimum Zener current for regulation	

output	meaning
`Rs_max`	largest series resistance
`Iz_max`	worst-case Zener current
`Pz_max`	worst-case Zener power
`PRs_max`	worst-case resistor power

- **Method:** Rs = (Vin,min - Vz)/(Iz,min + IL,max); Iz,max = (Vin,max - Vz)/Rs - IL,min; $Pz = Vz$ Iz,max
- **Assumes:** Vin_min > Vz; Vin_max >= Vin_min, IL_max >= IL_min >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Rs_max (largest series resistance) against IL_max (largest load current) from 0.005 A to 0.5 A on a logarithmic axis. One curve for each value of Vz (Zener voltage): 9.1 V, 4.55 V. Other inputs as in the worked example: Vin_min = 12.0 V, Vin_max = 15.0 V, IL_min = 0.0 A, Iz_min = 0.005 A. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, IL_max = 0.05 A, Rs_max = 52.7273 ohm.

```
electronics_engineering_ai999x.solve('elx_zener_series_resistor_design', Vin_min=12.0,
Vin_max=15.0, Vz=9.1, IL_min=0.0, IL_max=0.05, Iz_min=0.005)
# -> Rs_max = 52.7273 ohm, Iz_max = 0.111897 A, Pz_max = 1.01826 W, PRs_max = 0.66019 W   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_zener_series_resistor_design', Vin_min=9.0, Vin_max=11.0,
Vz=5.6, IL_min=0.01, IL_max=0.1, Iz_min=0.002)
# -> Rs_max = 33.3333 ohm, Iz_max = 0.152 A, Pz_max = 0.8512 W, PRs_max = 0.8748 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Zener regulator design: Vin_min = 12 V, Vin_max = 15 V, Vz = 9.1 V, IL_min = 0 mA, IL_max = 50 mA, Iz_min = 5 mA	SOLVED	Rs_max = 52.7273 ohm, Iz_max = 0.111897 A, Pz_max = 1.01826 W, PRs_max = 0.66019 W
Zener regulator design: Vin_min = 9 V, Vin_max = 11 V, Vz = 5.6 V, IL_min = 10 mA, IL_max = 100 mA, Iz_min = 2 mA	SOLVED	Rs_max = 33.3333 ohm, Iz_max = 0.152 A, Pz_max = 0.8512 W, PRs_max = 0.8748 W
Zener regulator design: Vin_max = 15 V, Vz = 9.1 V, IL_min = 0 mA, IL_max = 50 mA, Iz_min = 5 mA	NEEDS_INPUT	elx_zener_series_resistor_design needs Vin_min (lowest input voltage). Example: solve('elx_zener_series_resistor_design'
Zener regulator design: Vin_min = 12 V, Vin_max = 15 V, Vz = 9.1 V,	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form:

question	outcome	result
IL_min = 0 mA, IL_max = 50 mA, Iz_min = 5 mA, zz = 1		electronics_engineering_ai999x.te

LED series (current-limiting) resistor, `elx_led_series_resistor`

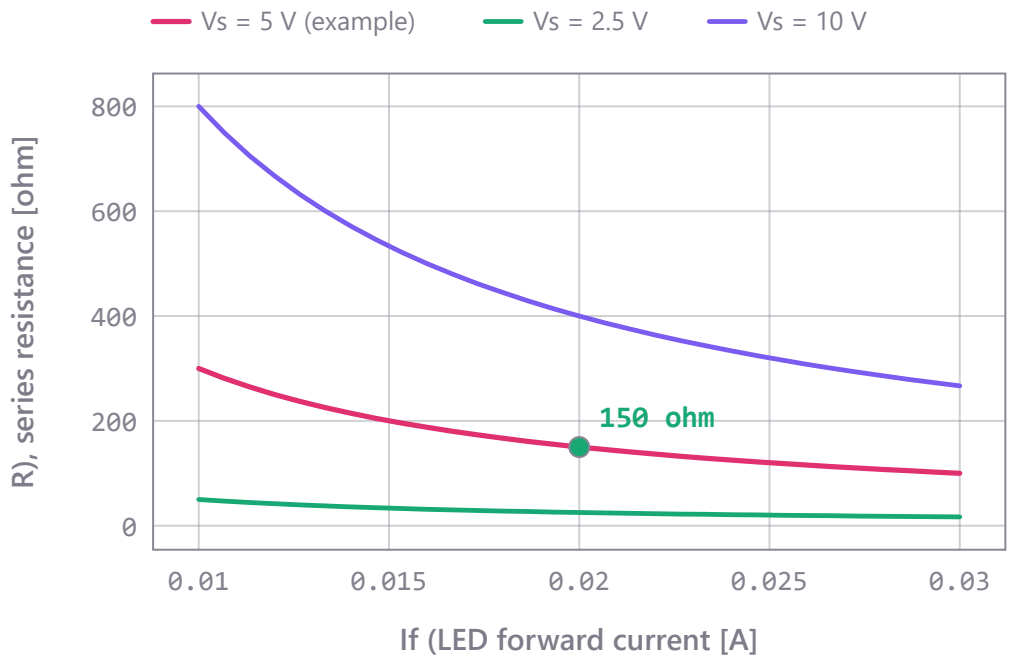
The current-limiting resistor for one LED or a string of LEDs in series at a chosen forward current, and the power in the resistor and in the LEDs.

$$R = \frac{V_S - nV_F}{I_F}$$

input	meaning	condition
`Vs`	supply voltage	
`Vf`	forward voltage of one LED	
`If`	LED forward current	
`n`	number of LEDs in series (default 1)	

output	meaning
`R`	series resistance
`P_R`	power in the resistor
`P_LED`	power in the LEDs

- **Method:** R = (Vs - n Vf)/If; $P_R = If^2$ R
- **Assumes:** $Vs > n$ Vf; $If > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



R (series resistance) against If (LED forward current) from 0.01 A to 0.03 A. One curve for each value of Vs (supply voltage): 5 V, 2.5 V, 10 V. Other inputs as in the worked example: Vf = 2.0 V. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, If = 0.02 A, R = 150 ohm.

```
electronics_engineering_ai999x.solve('elx_led_series_resistor', Vs=5.0, Vf=2.0, If=0.02)
# -> R = 150 ohm, P_R = 0.06 W, P_LED = 0.04 W   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_led_series_resistor', Vs=12.0, Vf=3.1, If=0.015, n=3)
# -> R = 180 ohm, P_R = 0.0405 W, P_LED = 0.1395 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
LED series resistor: Vs = 5 V, Vf = 2 V, If = 20 mA	SOLVED	R = 150 ohm, P_R = 0.06 W, P_LED = 0.04 W
LED series resistor: Vs = 12 V, Vf = 3.1 V, If = 15 mA, n = 3	SOLVED	R = 180 ohm, P_R = 0.0405 W, P_LED = 0.1395 W
LED series resistor: Vf = 2 V, If = 20 mA	NEEDS_INPUT	elx_led_series_resistor needs Vs (supply voltage). Example: solve('elx_led_series_resistor', Vs='5 V', Vf='2 V', If='0.0
LED series resistor: Vs = 5 V, Vf = 2 V, If = 20 mA, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

BJT terminal currents from beta, `elx_bjt_current_relations`

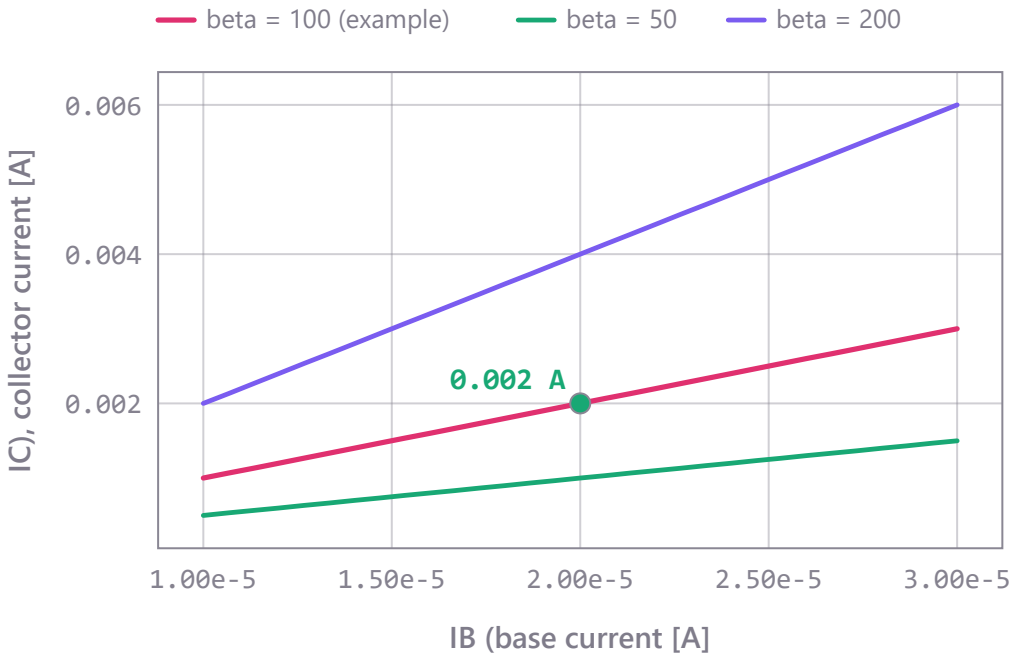
The collector and emitter currents and the common-base current gain alpha of a bipolar transistor in the active region from its base current and beta.

$$I_C = \beta I_B, \quad I_E = (\beta + 1) I_B, \quad \alpha = \frac{\beta}{\beta + 1}$$

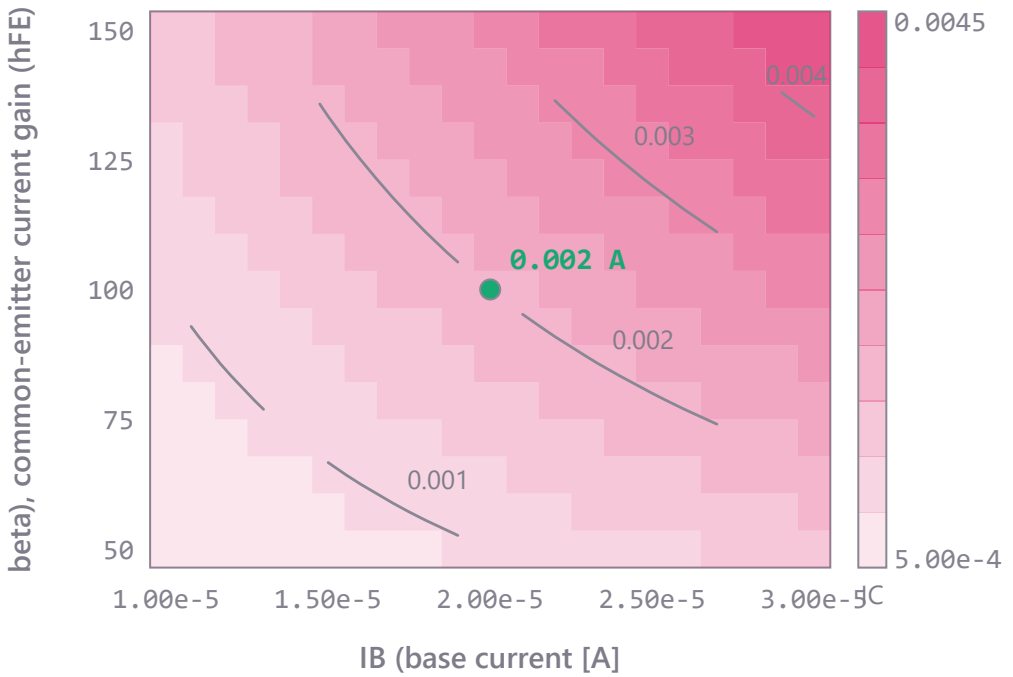
input	meaning	condition
`IB`	base current	
`beta`	common-emitter current gain (hFE)	

output	meaning
`IC`	collector current
`IE`	emitter current
`alpha`	$\alpha = \beta/(\beta+1)$

- **Method:** $IC = \beta$ IB; $IE = (\beta + 1)$ IB; $\alpha = \beta/(\beta + 1)$
- **Assumes:** $IB > 0$, $\beta > 0$; active region
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



IC (collector current) against IB (base current) from 1.0000e-05 A to 3.0000e-05 A. One curve for each value of beta (common-emitter current gain (hFE)): 100, 50, 200. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, IB = 2.0000e-05 A, IC = 0.002 A.



IC (collector current) over IB (base current) from 1.0000e-05 A to 3.0000e-05 A and beta (common-emitter current gain (hFE)) from 50 to 150: each of the 21 × 15 cells is one run of electronics-engineering-ai999x, shaded by its value (light 0.0005 A to dark 0.0045 A); iso-lines at IC = 0.001, 0.002, 0.003, 0.004 A. Marked: the worked example, IB = 2.0000e-05 A, beta = 100, IC = 0.002 A.

```
electronics_engineering_ai999x.solve('elx_bjt_current_relations', IB=2e-05, beta=100.0)
# -> IC = 0.002 A, IE = 0.00202 A, alpha = 0.990099   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_bjt_current_relations', IB=5e-05, beta=150.0)
# -> IC = 0.0075 A, IE = 0.00755 A, alpha = 0.993377   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BJT currents: IB = 20 uA, beta = 100	SOLVED	IC = 0.002 A, IE = 0.00202 A, alpha = 0.990099
BJT currents: IB = 50 uA, beta = 150	SOLVED	IC = 0.0075 A, IE = 0.00755 A, alpha = 0.993377
BJT currents: beta = 100	NEEDS_INPUT	elx_bjt_current_relations needs IB (base current). Example: solve('elx_bjt_current_relations', IB='2e-05 A', beta=100).
BJT currents: IB = 20 uA, beta = 100, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

BJT fixed (base-resistor) bias: Q-point, `elx_bjt_fixed_bias`

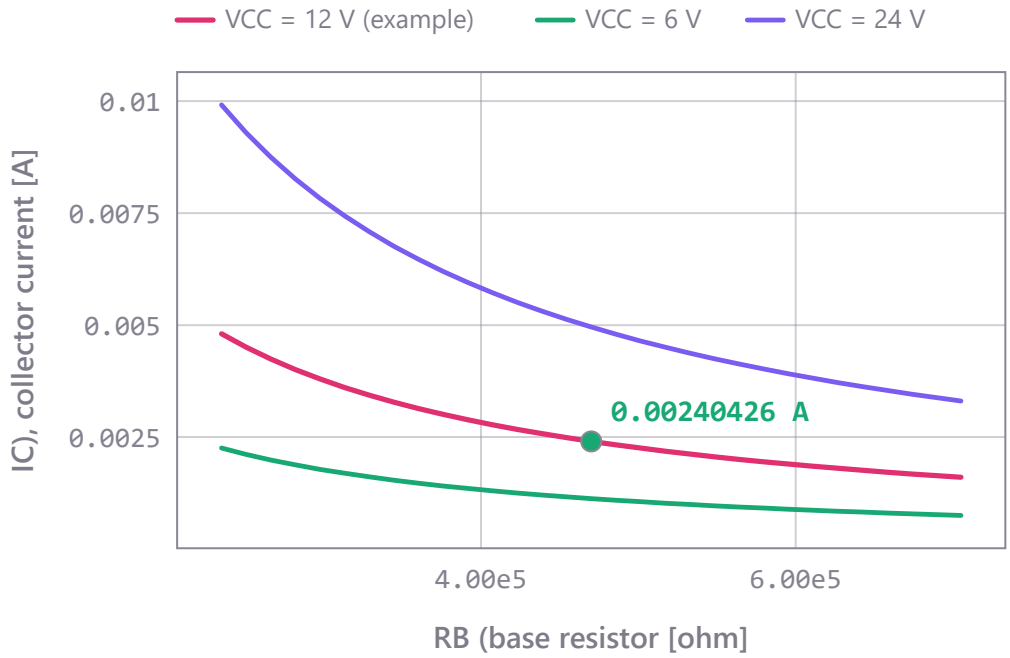
The quiescent base and collector currents and the collector-emitter voltage of a common-emitter stage biased by a single base resistor from the supply.

$$I_B = \frac{V_{CC} - V_{BE}}{R_B}, \quad V_{CE} = V_{CC} - \beta I_B R_C$$

input	meaning	condition
`VCC`	supply voltage	
`RB`	base resistor	
`RC`	collector resistor	
`beta`	current gain	
`VBE`	base-emitter voltage (about 0.7 V)	

output	meaning
`IB`	base current
`IC`	collector current
`IE`	emitter current
`VCE`	collector-emitter voltage

- **Method:** IB = (VCC - VBE)/RB; $IC = \beta$ IB; VCE = VCC - IC RC
- **Assumes:** active region: VCE >= 0.2 V (otherwise refused as saturated)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



IC (collector current) against RB (base resistor) from 235000 ohm to 705000 ohm. One curve for each value of VCC (supply voltage): 12 V, 6 V, 24 V. Other inputs as in the worked example: RC = 2200.0 ohm, beta = 100.0, VBE = 0.7 V. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, RB = 470000 ohm, IC = 0.00240426 A.

```
electronics_engineering_ai999x.solve('elx_bjt_fixed_bias', VCC=12.0, RB=470000.0, RC=2200.0,
beta=100.0, VBE=0.7)
# -> IB = 2.40426e-05 A, IC = 0.00240426 A, IE = 0.0024283 A, VCE = 6.71064 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_bjt_fixed_bias', VCC=9.0, RB=820000.0, RC=2200.0,
beta=200.0, VBE=0.65)
# -> IB = 1.01829e-05 A, IC = 0.00203659 A, IE = 0.00204677 A, VCE = 4.51951 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BJT fixed bias: VCC = 12 V, RB = 470000 ohm, RC = 2200 ohm, beta = 100, VBE = 0.7 V	SOLVED	IB = 2.40426e-05 A, IC = 0.00240426 A, IE = 0.0024283 A, VCE = 6.71064 V
BJT fixed bias: VCC = 9 V, RB = 820000 ohm, RC = 2200 ohm, beta = 200, VBE = 0.65 V	SOLVED	IB = 1.01829e-05 A, IC = 0.00203659 A, IE = 0.00204677 A, VCE = 4.51951 V
BJT fixed bias: RB = 470000 ohm, RC = 2200 ohm, beta = 100, VBE = 0.7 V	NEEDS_INPUT	elx_bjt_fixed_bias needs VCC (supply voltage). Example: solve('elx_bjt_fixed_bias', VCC='12 V', RB='470000 ohm', RC='220
BJT fixed bias: VCC = 12 V, RB = 470000 ohm, RC = 2200 ohm, beta = 100, VBE = 0.7 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te
BJT fixed bias: VCC = 12 V, RB = 100 kohm, RC = 4.7 kohm, beta = 100, VBE = 0.7 V	INVALID_INPUT	BJT fixed (base-resistor) bias: Q-point: the transistor is saturated (VCE below 0.2 V): the active-region bias equations

BJT voltage-divider bias: exact Q-point, `elx_bjt_voltage_divider_bias`

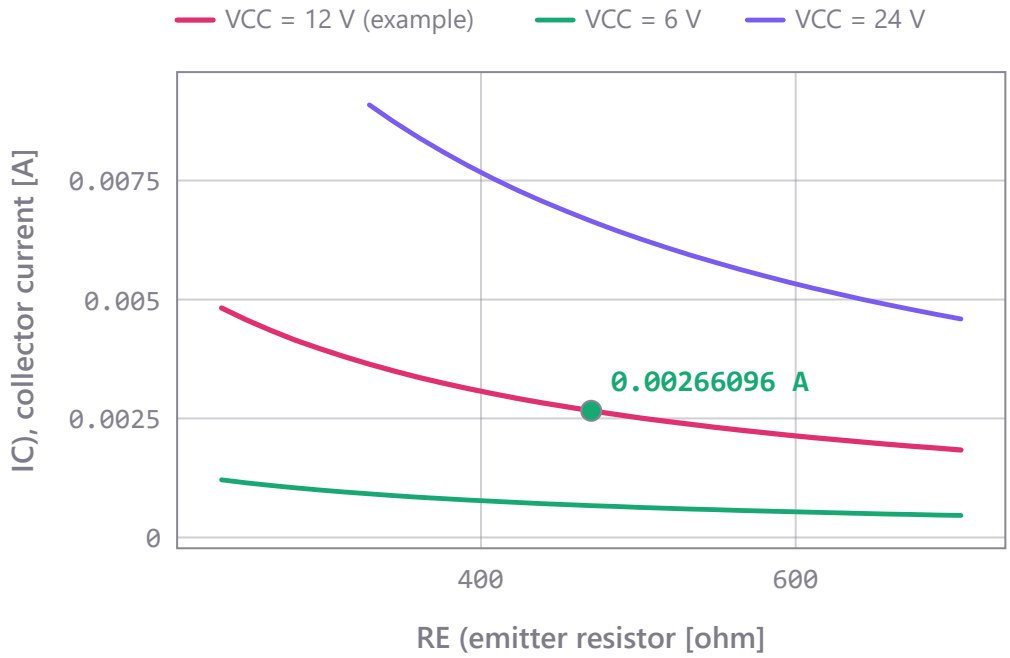
The Q-point (base, collector and emitter currents, node voltages and VCE) of a common-emitter stage with a base voltage divider and an emitter resistor, by the exact Thevenin analysis (base current included).

$$I_B = \frac{V_{TH} - V_{BE}}{R_{TH} + (\beta + 1) R_E}$$

input	meaning	condition
`VCC`	supply voltage	
`R1`	upper divider resistor (supply to base)	
`R2`	lower divider resistor (base to ground)	
`RC`	collector resistor	
`RE`	emitter resistor	
`beta`	current gain	
`VBE`	base-emitter voltage	

output	meaning
`VTH`	Thevenin voltage of the divider
`RTH`	Thevenin resistance of the divider
`IB`	base current
`IC`	collector current
`IE`	emitter current
`VB`	base voltage
`VE`	emitter voltage
`VC`	collector voltage
`VCE`	collector-emitter voltage

- **Method:** VTH = VCC $R_2/(R_1 + R_2)$; RTH = R1 || R2; IB = (VTH - VBE)/(RTH + (beta + 1) RE); VCE = VCC - IC RC - IE RE
- **Assumes:** VTH > VBE; active region: VCE >= 0.2 V
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



IC (collector current) against RE (emitter resistor) from 235 ohm to 705 ohm. One curve for each value of VCC (supply voltage): 12 V, 6 V, 24 V. Other inputs as in the worked example: R1 = 47000.0 ohm, R2 = 10000.0 ohm, RC = 2200.0 ohm, beta = 150.0, VBE = 0.7 V. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, RE = 470 ohm, IC = 0.00266096 A.

```
electronics_engineering_ai999x.solve('elx_bjt_voltage_divider_bias', VCC=12.0, R1=47000.0,
R2=10000.0, RC=2200.0, RE=470.0, beta=150.0, VBE=0.7)
# -> VTH = 2.10526 V, RTH = 8245.61 ohm, IB = 1.77397e-05 A, IC = 0.00266096 A, IE = 0.0026787 A,
VB = 1.95899 V, VE = 1.25899 V, VC = 6.14589 V, VCE = 4.8869 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_bjt_voltage_divider_bias', VCC=15.0, R1=33000.0,
R2=6800.0, RC=3300.0, RE=1000.0, beta=100.0, VBE=0.7)
# -> VTH = 2.56281 V, RTH = 5638.19 ohm, IB = 1.74685e-05 A, IC = 0.00174685 A, IE = 0.00176432 A,
VB = 2.46432 V, VE = 1.76432 V, VC = 9.23538 V, VCE = 7.47106 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BJT voltage-divider bias: VCC = 12 V, R1 = 47000 ohm, R2 = 10000 ohm, RC = 2200 ohm, RE = 470 ohm, beta = 150, VBE = 0.7 V	SOLVED	VTH = 2.10526 V, RTH = 8245.61 ohm, IB = 1.77397e-05 A, IC = 0.00266096 A, IE = 0.0026787 A, VB = 1.95899 V, VE = 1.25899 V, VC = 6.14589 V, VCE = 4.8869 V
BJT voltage-divider bias: VCC = 15 V, R1 = 33000 ohm, R2 = 6800 ohm, RC = 3300 ohm, RE = 1000 ohm, beta = 100, VBE = 0.7 V	SOLVED	VTH = 2.56281 V, RTH = 5638.19 ohm, IB = 1.74685e-05 A, IC = 0.00174685 A, IE = 0.00176432 A, VB = 2.46432 V, VE = 1.76432 V, VC = 9.23538 V, VCE = 7.47106 V
BJT voltage-divider bias: R1 = 47000 ohm, R2 = 10000 ohm, RC = 2200 ohm, RE = 470 ohm, beta = 150, VBE = 0.7 V	NEEDS_INPUT	elx_bjt_voltage_divider_bias needs VCC (supply voltage). Example: solve('elx_bjt_voltage_divider_bias', VCC='12 V', R1='
BJT voltage-divider bias: VCC = 12 V, R1 = 47000 ohm, R2 = 10000 ohm, RC = 2200 ohm, RE = 470	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

question	outcome	result
ohm, beta = 150, VBE = 0.7 V, zz = 1		

BJT dual-supply emitter bias: Q-point, `elx_bjt_emitter_bias`

The Q-point of a transistor with its base returned to ground through RB and its emitter returned to a negative supply -VEE through RE (emitter bias).

$$I_E = \frac{V_{EE} - V_{BE}}{R_E + R_B/(\beta + 1)}$$

input	meaning	condition
`VCC`	positive supply	
`VEE`	magnitude of the negative supply	
`RB`	base resistor to ground	
`RC`	collector resistor	
`RE`	emitter resistor to -VEE	
`beta`	current gain	
`VBE`	base-emitter voltage	

output	meaning
`IB`	base current
`IC`	collector current
`IE`	emitter current
`VB`	base voltage
`VE`	emitter voltage
`VC`	collector voltage
`VCE`	collector-emitter voltage

- **Method:** IE = (VEE - VBE)/(RE + RB/(beta + 1)); $VB = -IB$ RB; VE = VB - VBE; VC = VCC - IC RC
- **Assumes:** VEE > VBE; active region: VCE >= 0.2 V
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



IE (emitter current) against RE (emitter resistor to -VEE) from 5000 ohm to 15000 ohm. One curve for each value of VEE (magnitude of the negative supply): 10 V, 5 V, 20 V. Other inputs as in the worked example: VCC = 10.0 V, RB = 10000.0 ohm, RC = 4700.0 ohm, beta = 100.0, VBE = 0.7 V. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, RE = 10000 ohm, IE = 0.000920882 A.

```
electronics_engineering_ai999x.solve('elx_bjt_emitter_bias', VCC=10.0, VEE=10.0, RB=10000.0,
RC=4700.0, RE=10000.0, beta=100.0, VBE=0.7)
# -> IB = 9.11765e-06 A, IC = 0.000911765 A, IE = 0.000920882 A, VB = -0.0911765 V, VE = -0.791176
V, VC = 5.71471 V, VCE = 6.50588 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_bjt_emitter_bias', VCC=12.0, VEE=9.0, RB=47000.0,
RC=5600.0, RE=8200.0, beta=200.0, VBE=0.65)
# -> IB = 4.92567e-06 A, IC = 0.000985134 A, IE = 0.00099006 A, VB = -0.231507 V, VE = -0.881507 V,
VC = 6.48325 V, VCE = 7.36475 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BJT emitter bias: VCC = 10 V, VEE = 10 V, RB = 10000 ohm, RC = 4700 ohm, RE = 10000 ohm, beta = 100, VBE = 0.7 V	SOLVED	IB = 9.11765e-06 A, IC = 0.000911765 A, IE = 0.000920882 A, VB = -0.0911765 V, VE = -0.791176 V, VC = 5.71471 V, VCE = 6.50588 V
BJT emitter bias: VCC = 12 V, VEE = 9 V, RB = 47000 ohm, RC = 5600 ohm, RE = 8200 ohm, beta = 200, VBE = 0.65 V	SOLVED	IB = 4.92567e-06 A, IC = 0.000985134 A, IE = 0.00099006 A, VB = -0.231507 V, VE = -0.881507 V, VC = 6.48325 V, VCE = 7.36475 V
BJT emitter bias: VEE = 10 V, RB = 10000 ohm, RC = 4700 ohm, RE = 10000 ohm, beta = 100, VBE = 0.7 V	NEEDS_INPUT	elx_bjt_emitter_bias needs VCC (positive supply). Example: solve('elx_bjt_emitter_bias', VCC='10 V', VEE='10 V', RB='100
BJT emitter bias: VCC = 10 V, VEE = 10 V, RB = 10000 ohm, RC = 4700 ohm, RE = 10000 ohm, beta = 100, VBE = 0.7 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

BJT collector-feedback bias: Q-point, `elx_bjt_collector_feedback_bias`

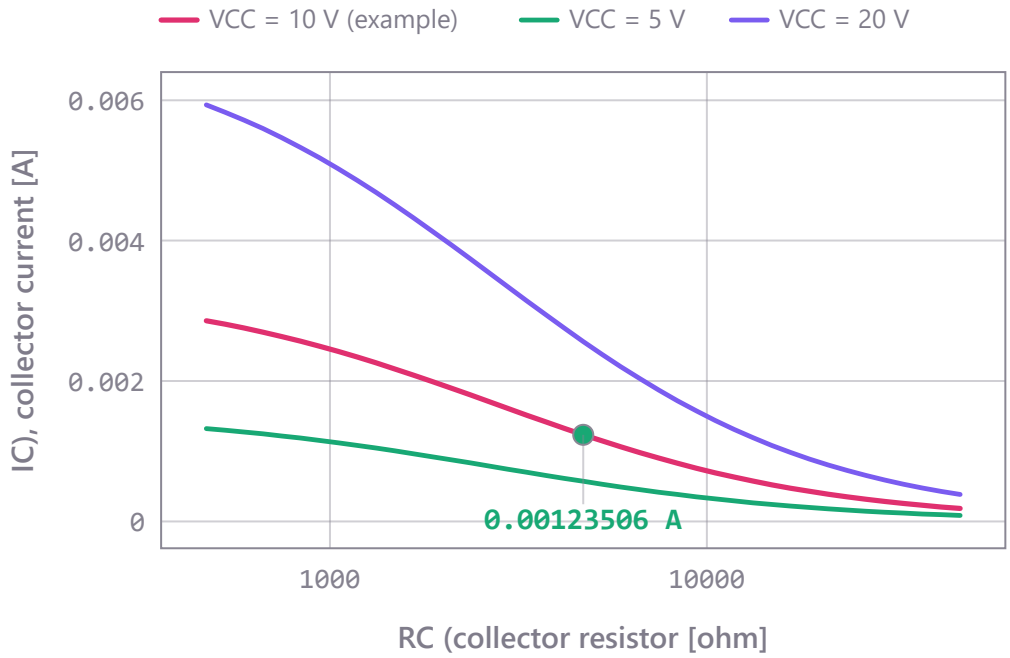
The Q-point of a transistor biased by a resistor from collector to base (collector feedback), with an optional emitter resistor.

$$I_B = \frac{V_{CC} - V_{BE}}{R_F + (\beta + 1)(R_C + R_E)}$$

input	meaning	condition
`VCC`	supply voltage	
`RC`	collector resistor (carries IC + IB)	
`RF`	feedback resistor, collector to base	
`beta`	current gain	
`VBE`	base-emitter voltage	
`RE`	emitter resistor (default 0)	

output	meaning
`IB`	base current
`IC`	collector current
`IE`	emitter current
`VC`	collector voltage
`VCE`	collector-emitter voltage

- **Method:** IB = (VCC - VBE)/(RF + (beta + 1)(RC + RE)); VC = VCC - IE RC
- **Assumes:** active region: VCE >= 0.2 V
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



IC (collector current) against RC (collector resistor (carries IC + IB)) from 470 ohm to 47000 ohm on a logarithmic axis. One curve for each value of VCC (supply voltage): 10 V, 5 V, 20 V. Other inputs as in the worked example: RF = 250000.0 ohm, beta = 90.0, VBE = 0.7 V. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, RC = 4700 ohm, IC = 0.00123506 A.

```
electronics_engineering_ai999x.solve('elx_bjt_collector_feedback_bias', VCC=10.0, RC=4700.0,
RF=250000.0, beta=90.0, VBE=0.7)
# -> IB = 1.37229e-05 A, IC = 0.00123506 A, IE = 0.00124878 A, VC = 4.13072 V, VCE = 4.13072 V
(SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_bjt_collector_feedback_bias', VCC=12.0, RC=3300.0,
RF=330000.0, beta=120.0, VBE=0.7, RE=470.0)
# -> IB = 1.43735e-05 A, IC = 0.00172482 A, IE = 0.00173919 A, VC = 6.26067 V, VCE = 5.44325 V
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BJT collector-feedback bias: VCC = 10 V, RC = 4700 ohm, RF = 250000 ohm, beta = 90, VBE = 0.7 V	SOLVED	IB = 1.37229e-05 A, IC = 0.00123506 A, IE = 0.00124878 A, VC = 4.13072 V, VCE = 4.13072 V
BJT collector-feedback bias: VCC = 12 V, RC = 3300 ohm, RF = 330000 ohm, beta = 120, VBE = 0.7 V, RE = 470 ohm	SOLVED	IB = 1.43735e-05 A, IC = 0.00172482 A, IE = 0.00173919 A, VC = 6.26067 V, VCE = 5.44325 V
BJT collector-feedback bias: RC = 4700 ohm, RF = 250000 ohm, beta = 90, VBE = 0.7 V	NEEDS_INPUT	elx_bjt_collector_feedback_bias needs VCC (supply voltage). Example: solve('elx_bjt_collector_feedback_bias', VCC='10 V'
BJT collector-feedback bias: VCC = 10 V, RC = 4700 ohm, RF = 250000 ohm, beta = 90, VBE = 0.7 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

BJT small-signal (hybrid-pi) parameters, `elx_bjt_small_signal_parameters`

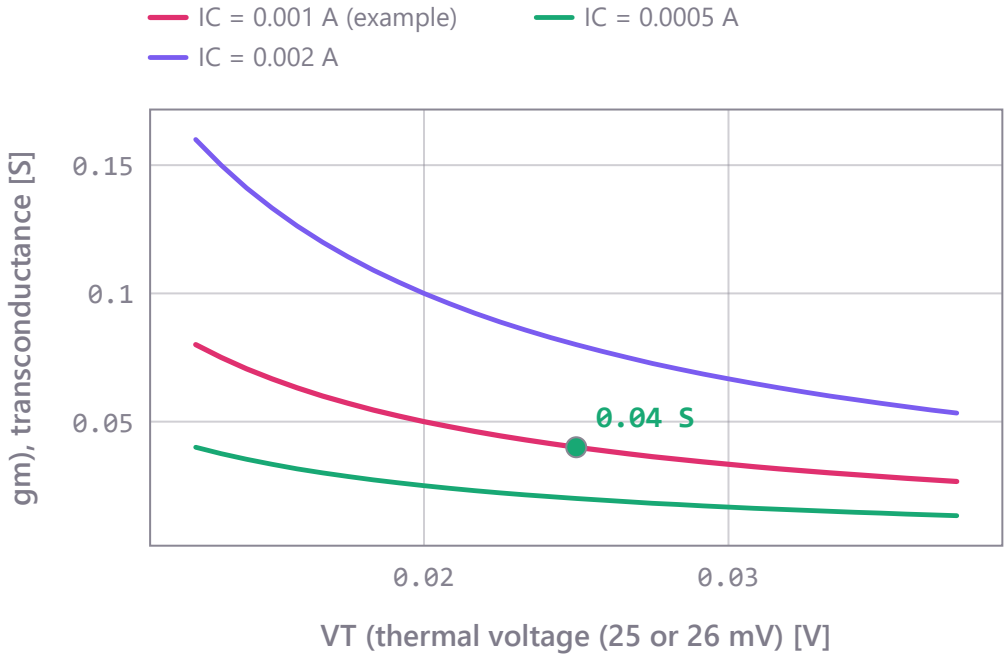
The transconductance gm, the input resistance r_pi, the emitter resistance r_e, the output resistance r_o (Early voltage) and the intrinsic gain of a bipolar transistor at its bias current.

$$g_m = \frac{I_C}{V_T}, \quad r_\pi = \frac{\beta}{g_m}, \quad r_e = \frac{\alpha}{g_m}, \quad r_o = \frac{V_A}{I_C}$$

input	meaning	condition
`IC`	collector bias current	
`beta`	current gain	
`VT`	thermal voltage (25 or 26 mV)	
`VA`	Early voltage	

output	meaning
`gm`	transconductance
`r_pi`	base input resistance r_pi
`r_e`	emitter resistance r_e
`r_o`	output resistance
`A0`	intrinsic voltage gain gm r_o

- **Method:** $gm = IC/VT$; $r_pi = \beta/gm$; $r_e = \alpha/gm$; $r_o = VA/IC$; $A_0 = VA/VT$
- **Assumes:** IC, beta, VT, $VA > 0$; active region
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



gm (transconductance) against VT (thermal voltage (25 or 26 mV)) from 0.0125 V to 0.0375 V. One curve for each value of IC (collector bias current): 0.001 A, 0.0005 A, 0.002 A. Other inputs as in the worked example: beta = 100.0, VA = 100.0 V. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, VT = 0.025 V, gm = 0.04 S.

```
electronics_engineering_ai999x.solve('elx_bjt_small_signal_parameters', IC=0.001, beta=100.0,
VT=0.025, VA=100.0)
# -> gm = 0.04 S, r_pi = 2500 ohm, r_e = 24.7525 ohm, r_o = 100000 ohm, A0 = 4000   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_bjt_small_signal_parameters', IC=0.0025, beta=150.0,
VT=0.026, VA=75.0)
# -> gm = 0.0961538 S, r_pi = 1560 ohm, r_e = 10.3311 ohm, r_o = 30000 ohm, A0 = 2884.62   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BJT small-signal parameters: IC = 1 mA, beta = 100, VT = 25 mV, VA = 100 V	SOLVED	gm = 0.04 S, r_pi = 2500 ohm, r_e = 24.7525 ohm, r_o = 100000 ohm, A0 = 4000
BJT small-signal parameters: IC = 2.5 mA, beta = 150, VT = 26 mV, VA = 75 V	SOLVED	gm = 0.0961538 S, r_pi = 1560 ohm, r_e = 10.3311 ohm, r_o = 30000 ohm, A0 = 2884.62
BJT small-signal parameters: beta = 100, VT = 25 mV, VA = 100 V	NEEDS_INPUT	elx_bjt_small_signal_parameters needs IC (collector bias current). Example: solve('elx_bjt_small_signal_parameters', IC=
BJT small-signal parameters: IC = 1 mA, beta = 100, VT = 25 mV, VA = 100 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Common-emitter amplifier: gain, input and output resistance, `elx_common_emitter_amplifier`

The voltage gain (base to collector and source to load), the input and output resistances of a common-emitter stage with an optional unbypassed emitter resistance, from the bias current (hybrid-pi model, r_o neglected).

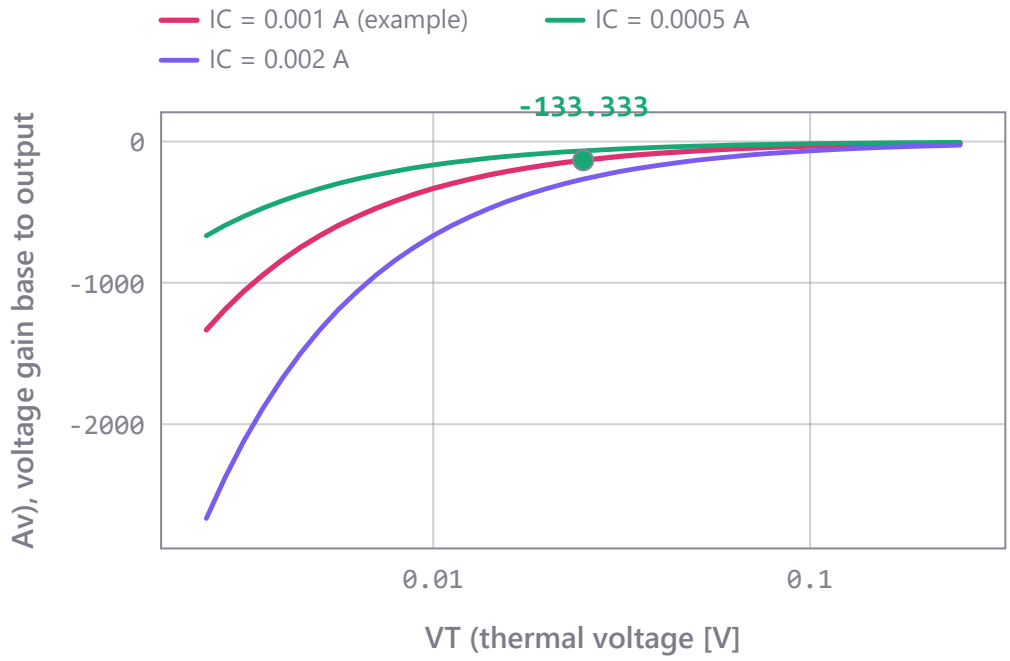
$$A_v = -\frac{\beta\,(R_C \parallel R_L)}{r_\pi + (\beta+1)R_{E1}}$$

input	meaning	condition
`IC`	collector bias current	
`beta`	current gain	
`VT`	thermal voltage	
`RC`	collector resistor	
`RL`	load resistor	
`RE1`	unbypassed emitter resistance (0 if fully bypassed)	
`RB`	bias resistance seen at the base (R1 || R2)	
`Rsig`	source resistance	

output	meaning
`gm`	transconductance
`r_pi`	r_pi
`Av`	voltage gain base to output

output	meaning
`Rin`	input resistance (including RB)
`Rout`	output resistance (RC)
`Gv`	overall gain from the source voltage

- **Method:** $Av = -\beta$ (RC || RL)/($r_pi + (\beta + 1)$ RE1); Rin = RB || (r_pi + (beta + 1) RE1); $Gv = Av$ Rin/(Rin + Rsig)
- **Assumes:** r_o neglected; coupling and bypass capacitors short at signal frequency
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Av (voltage gain base to output) against VT (thermal voltage) from 0.0025 V to 0.25 V on a logarithmic axis. One curve for each value of IC (collector bias current): 0.001 A, 0.0005 A, 0.002 A. Other inputs as in the worked example: beta = 100.0, RC = 5000.0 ohm, RL = 10000.0 ohm, RE1 = 0.0 ohm, RB = 20000.0 ohm, Rsig = 1000.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, VT = 0.025 V, Av = -133.333.

```
electronics_engineering_ai999x.solve('elx_common_emitter_amplifier', IC=0.001, beta=100.0,
VT=0.025, RC=5000.0, RL=10000.0, RE1=0.0, RB=20000.0, Rsig=1000.0)
# -> gm = 0.04 S, r_pi = 2500 ohm, Av = -133.333, Rin = 2222.22 ohm, Rout = 5000 ohm, Gv = -91.954
(SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_common_emitter_amplifier', IC=0.002, beta=150.0,
VT=0.025, RC=3300.0, RL=4700.0, RE1=220.0, RB=30000.0, Rsig=600.0)
# -> gm = 0.08 S, r_pi = 1875 ohm, Av = -8.28644, Rin = 16174.1 ohm, Rout = 3300 ohm, Gv = -7.99003
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Common-emitter amplifier: IC = 1 mA, beta = 100, VT = 25 mV, RC = 5000 ohm, RL = 10000 ohm, RE1 = 0 ohm, RB = 20000 ohm, Rsig = 1000 ohm	SOLVED	gm = 0.04 S, r_pi = 2500 ohm, Av = -133.333, Rin = 2222.22 ohm, Rout = 5000 ohm, Gv = -91.954

question	outcome	result
Common-emitter amplifier: IC = 2 mA, beta = 150, VT = 25 mV, RC = 3300 ohm, RL = 4700 ohm, RE1 = 220 ohm, RB = 30000 ohm, Rsig = 600 ohm	SOLVED	gm = 0.08 S, r_pi = 1875 ohm, Av = -8.28644, Rin = 16174.1 ohm, Rout = 3300 ohm, Gv = -7.99003
Common-emitter amplifier: beta = 100, VT = 25 mV, RC = 5000 ohm, RL = 10000 ohm, RE1 = 0 ohm, RB = 20000 ohm, Rsig = 1000 ohm	NEEDS_INPUT	elx_common_emitter_amplifier needs IC (collector bias current). Example: solve('elx_common_emitter_amplifier', IC='0.001
Common-emitter amplifier: IC = 1 mA, beta = 100, VT = 25 mV, RC = 5000 ohm, RL = 10000 ohm, RE1 = 0 ohm, RB = 20000 ohm, Rsig = 1000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Common-collector (emitter-follower) amplifier, `elx_common_collector_amplifier`

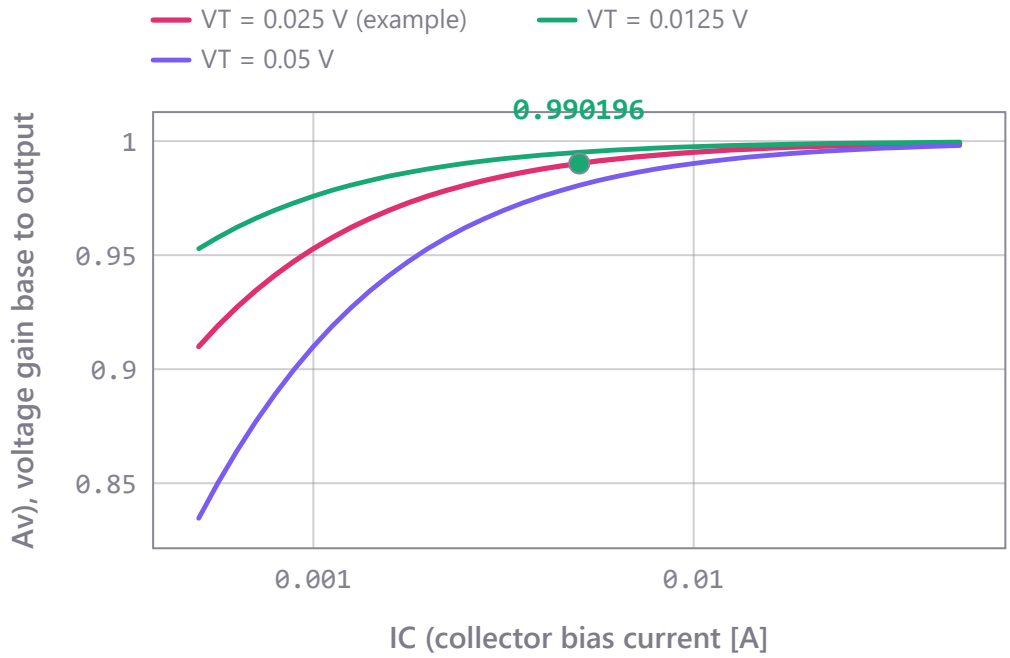
The voltage gain, input resistance and output resistance of an emitter follower, including the source resistance reflected into the emitter (hybrid-pi model, r_o neglected).

$$A_v = \frac{(\beta+1)R'_E}{r_\pi + (\beta+1)R'_E}, \quad R_{out} = R_E \parallel \frac{r_\pi + R_{sig} \parallel R_B}{\beta+1}$$

input	meaning	condition
`IC`	collector bias current	
`beta`	current gain	
`VT`	thermal voltage	
`RE`	emitter resistor	
`RL`	load resistor	
`RB`	bias resistance at the base	
`Rsig`	source resistance	

output	meaning
`Av`	voltage gain base to output
`Rin`	input resistance
`Rout`	output resistance
`Gv`	overall gain from the source

- **Method:** $Av = (\beta+1)$(RE || RL)$/(r_pi + (\beta+1)$(RE || RL)); Rout = RE || (r_pi + Rsig || RB)/(beta + 1)
- **Assumes:** r_o neglected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Av (voltage gain base to output) against IC (collector bias current) from 0.0005 A to 0.05 A on a logarithmic axis. One curve for each value of VT (thermal voltage): 0.025 V, 0.0125 V, 0.05 V. Other inputs as in the worked example: beta = 100.0, RE = 1000.0 ohm, RL = 1000.0 ohm, RB = 100000.0 ohm, Rsig = 10000.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, IC = 0.005 A, Av = 0.990196.

```
electronics_engineering_ai999x.solve('elx_common_collector_amplifier', IC=0.005, beta=100.0,
VT=0.025, RE=1000.0, RL=1000.0, RB=100000.0, Rsig=10000.0)
# -> Av = 0.990196, Rin = 33774.8 ohm, Rout = 86.7242 ohm, Gv = 0.763994   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_common_collector_amplifier', IC=0.001, beta=200.0,
VT=0.026, RE=4700.0, RL=2000.0, RB=47000.0, Rsig=5000.0)
# -> Av = 0.981894, Rin = 40390.2 ohm, Rout = 47.862 ohm, Gv = 0.873733   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Common-collector amplifier: IC = 5 mA, beta = 100, VT = 25 mV, RE = 1000 ohm, RL = 1000 ohm, RB = 100000 ohm, Rsig = 10000 ohm	SOLVED	Av = 0.990196, Rin = 33774.8 ohm, Rout = 86.7242 ohm, Gv = 0.763994
Common-collector amplifier: IC = 1 mA, beta = 200, VT = 26 mV, RE = 4700 ohm, RL = 2000 ohm, RB = 47000 ohm, Rsig = 5000 ohm	SOLVED	Av = 0.981894, Rin = 40390.2 ohm, Rout = 47.862 ohm, Gv = 0.873733
Common-collector amplifier: beta = 100, VT = 25 mV, RE = 1000 ohm, RL = 1000 ohm, RB = 100000 ohm, Rsig = 10000 ohm	NEEDS_INPUT	elx_common_collector_amplifier needs IC (collector bias current). Example: solve('elx_common_collector_amplifier', IC='0
Common-collector amplifier: IC = 5 mA, beta = 100, VT = 25 mV, RE = 1000 ohm, RL = 1000 ohm, RB = 100000 ohm, Rsig = 10000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Common-base amplifier, `elx_common_base_amplifier`

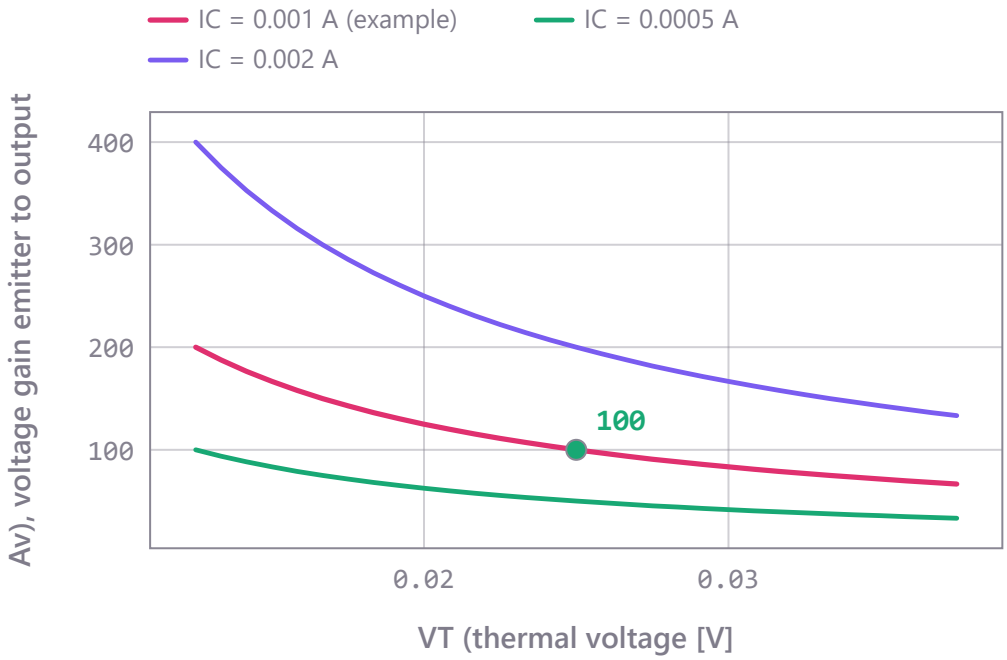
The voltage gain (emitter to collector and source to load), the low input resistance and the output resistance of a common-base stage (hybrid-pi model, r_o neglected).

$$A_v = g_m(R_C \parallel R_L), \quad R_{in} = R_E \parallel r_e$$

input	meaning	condition
`IC`	collector bias current	
`beta`	current gain	
`VT`	thermal voltage	
`RC`	collector resistor	
`RL`	load resistor	
`RE`	emitter bias resistor	
`Rsig`	source resistance	

output	meaning
`Av`	voltage gain emitter to output
`Rin`	input resistance (RE || r_e)
`Rout`	output resistance (RC)
`Gv`	overall gain from the source

- **Method:** $Av = gm$ (RC || RL); Rin = RE || r_e with $r_e = \alpha / gm$; $Gv = Av$ Rin/(Rin + Rsig)
- **Assumes:** r_o neglected; base at signal ground
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Av (voltage gain emitter to output) against VT (thermal voltage) from 0.0125 V to 0.0375 V. One curve for each value of IC (collector bias current): 0.001 A, 0.0005 A, 0.002 A. Other inputs as in the worked example: beta = 100.0, RC = 5000.0 ohm, RL = 5000.0 ohm, RE = 10000.0 ohm, Rsig = 50.0 ohm. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, VT = 0.025 V, Av = 100.

```
electronics_engineering_ai999x.solve('elx_common_base_amplifier', IC=0.001, beta=100.0, VT=0.025,
RC=5000.0, RL=5000.0, RE=10000.0, Rsig=50.0)
# -> Av = 100, Rin = 24.6914 ohm, Rout = 5000 ohm, Gv = 33.0579   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_common_base_amplifier', IC=0.0005, beta=120.0, VT=0.026,
RC=10000.0, RL=20000.0, RE=4700.0, Rsig=100.0)
# -> Av = 128.205, Rin = 51.0105 ohm, Rout = 10000 ohm, Gv = 43.307   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Common-base amplifier: IC = 1 mA, beta = 100, VT = 25 mV, RC = 5000 ohm, RL = 5000 ohm, RE = 10000 ohm, Rsig = 50 ohm	SOLVED	Av = 100, Rin = 24.6914 ohm, Rout = 5000 ohm, Gv = 33.0579
Common-base amplifier: IC = 0.5 mA, beta = 120, VT = 26 mV, RC = 10000 ohm, RL = 20000 ohm, RE = 4700 ohm, Rsig = 100 ohm	SOLVED	Av = 128.205, Rin = 51.0105 ohm, Rout = 10000 ohm, Gv = 43.307
Common-base amplifier: beta = 100, VT = 25 mV, RC = 5000 ohm, RL = 5000 ohm, RE = 10000 ohm, Rsig = 50 ohm	NEEDS_INPUT	elx_common_base_amplifier needs IC (collector bias current). Example: solve('elx_common_base_amplifier', IC='0.001 A', b
Common-base amplifier: IC = 1 mA, beta = 100, VT = 25 mV, RC = 5000 ohm, RL = 5000 ohm, RE = 10000 ohm, Rsig = 50 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

BJT beta cutoff and transition frequency fT, `elx_transition_frequency`

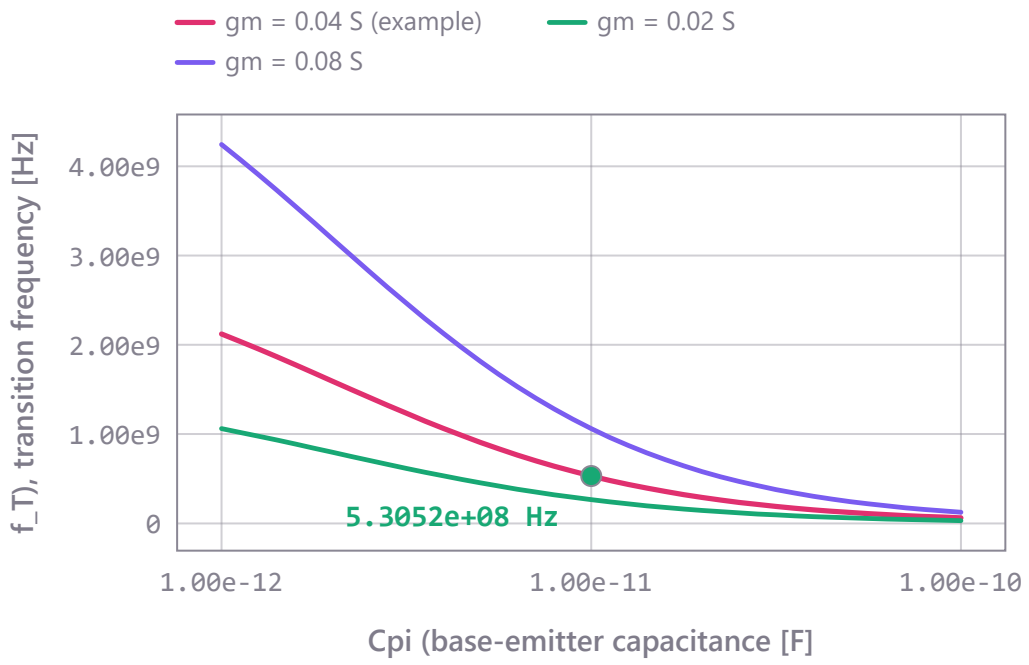
The beta cutoff frequency f_beta and the unity-current-gain (transition) frequency fT = gm/(2 pi (Cpi + Cmu)) of a bipolar transistor from its hybrid-pi capacitances.

$$f_T = \frac{g_m}{2\pi(C_\pi + C_\mu)}, \quad f_\beta = \frac{f_T}{\beta}$$

input	meaning	condition
`gm`	transconductance	
`beta`	low-frequency current gain	
`Cpi`	base-emitter capacitance	
`Cmu`	base-collector capacitance	

output	meaning
`r_pi`	$r_pi = \beta/gm$
`f_beta`	beta cutoff frequency
`f_T`	transition frequency

- **Method:** f_beta = 1/(2 pi r_pi (Cpi + Cmu)); $fT = \beta$ f_beta = gm/(2 pi (Cpi + Cmu))
- **Assumes:** single-pole h_fe (the feed-forward zero through Cmu neglected)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f_T (transition frequency) against Cpi (base-emitter capacitance) from 1.0000e-12 F to 1.0000e-10 F on a logarithmic axis. One curve for each value of gm (transconductance): 0.04 S, 0.02 S, 0.08 S. Other inputs as in the worked example: beta = 100.0, Cmu = 2e-12 F. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Cpi = 1.0000e-11 F, f_T = 5.3052e+08 Hz.

```
electronics_engineering_ai999x.solve('elx_transition_frequency', gm=0.04, beta=100.0, Cpi=1e-11,
Cmu=2e-12)
# -> r_pi = 2500 ohm, f_beta = 5.30516e+06 Hz, f_T = 5.30516e+08 Hz   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_transition_frequency', gm=0.1, beta=150.0, Cpi=2e-11,
Cmu=1e-12)
# -> r_pi = 1500 ohm, f_beta = 5.05254e+06 Hz, f_T = 7.57881e+08 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transition frequency: gm = 40 mS, beta = 100, Cpi = 10 pF, Cmu = 2 pF	SOLVED	r_pi = 2500 ohm, f_beta = 5.30516e+06 Hz, f_T = 5.30516e+08 Hz
Transition frequency: gm = 100 mS, beta = 150, Cpi = 20 pF, Cmu = 1 pF	SOLVED	r_pi = 1500 ohm, f_beta = 5.05254e+06 Hz, f_T = 7.57881e+08 Hz
Transition frequency: beta = 100, Cpi = 10 pF, Cmu = 2 pF	NEEDS_INPUT	elx_transition_frequency needs gm (transconductance). Example: solve('elx_transition_frequency', gm='0.04 S', beta=100,
Transition frequency: gm = 40 mS, beta = 100, Cpi = 10 pF, Cmu = 2 pF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Common-emitter high-frequency cutoff: Miller effect and open-circuit time constants, `elx_miller_high_cutoff`

The Miller capacitance, the upper -3 dB frequency of a common-emitter (or common-source) stage by the Miller approximation, and the open-circuit time-constant estimate that also counts the output side of Cmu.

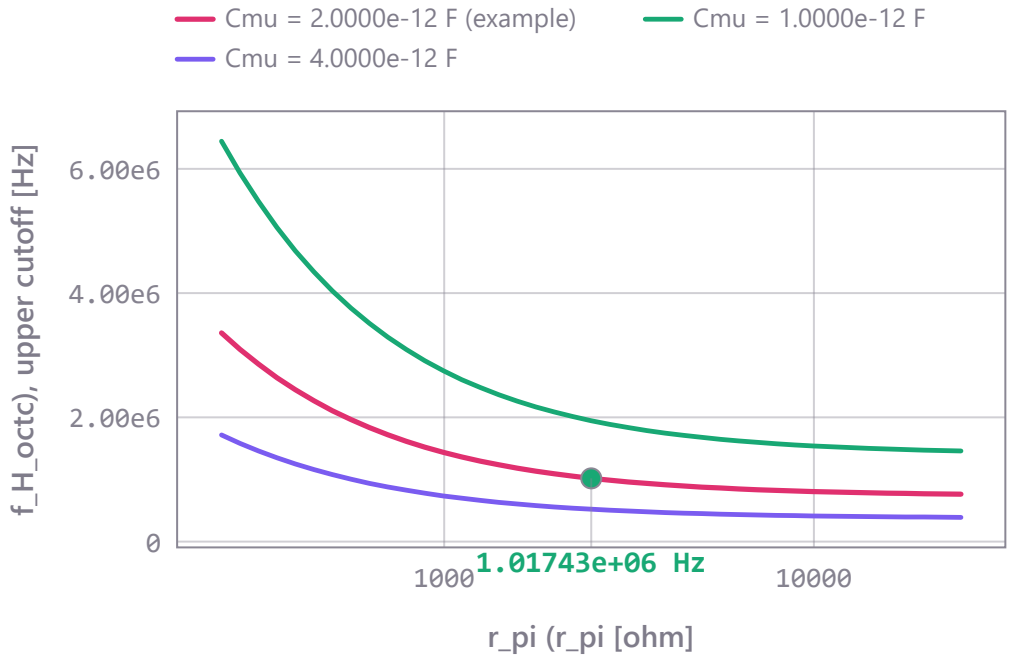
$$C_M = C_\mu(1 + g_m R'_L), \quad \tau_{oc} = C_\pi R'_\pi + C_\mu \left(R'_\pi + R'_L + g_m R'_\pi R'_L\right)$$

input	meaning	condition
`gm`	transconductance	
`r_pi`	r_pi (use a very large value for a MOSFET)	
`Cpi`	input capacitance Cpi (Cgs)	
`Cmu`	feedback capacitance Cmu (Cgd)	
`RLp`	total load at the collector (RC || RL || r_o)	
`Rsigp`	source resistance seen at the base (Rsig || RB)	

output	meaning
`A_mid`	gain across Cmu, -gm RL'
`C_M`	Miller capacitance
`R_pi_eq`	resistance seen by Cpi (r_pi || Rsig')
`f_H_miller`	upper cutoff, Miller approximation
`tau_oc`	sum of open-circuit time constants

output	meaning
`f_H_octc`	upper cutoff, open-circuit time constants

- **Method:** C_M = Cmu (1 + gm RL'); f_H = 1/(2 pi (r_pi || Rsig')(Cpi + C_M)); tau = Cpi R' + Cmu (R' + RL' + gm R' RL')
- **Assumes:** mid-band gain unchanged by the capacitors; dominant-pole estimate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f_H_octc (upper cutoff, open-circuit time constants) against r_pi (r_pi (use a very large value for a MOSFET)) from 250 ohm to 25000 ohm on a logarithmic axis. One curve for each value of Cmu (feedback capacitance Cmu (Cgd)): 2.0000e-12 F, 1.0000e-12 F, 4.0000e-12 F. Other inputs as in the worked example: gm = 0.04 S, Cpi = 1e-11 F, RLp = 2500.0 ohm, Rsigp = 1000.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, r_pi = 2500 ohm, f_H_octc = 1.01743e+06 Hz.

```
electronics_engineering_ai999x.solve('elx_miller_high_cutoff', gm=0.04, r_pi=2500.0, Cpi=1e-11,
Cmu=2e-12, RLp=2500.0, Rsigp=1000.0)
# -> A_mid = -100, C_M = 2.02e-10 F, R_pi_eq = 714.286 ohm, f_H_miller = 1.05102e+06 Hz, tau_oc =
1.56429e-07 s, f_H_octc = 1.01743e+06 Hz   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_miller_high_cutoff', gm=0.1, r_pi=1500.0, Cpi=2e-11,
Cmu=1e-12, RLp=1000.0, Rsigp=500.0)
# -> A_mid = -100, C_M = 1.01e-10 F, R_pi_eq = 375 ohm, f_H_miller = 3.50755e+06 Hz, tau_oc =
4.6375e-08 s, f_H_octc = 3.43191e+06 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Miller high-frequency cutoff: gm = 40 mS, r_pi = 2500 ohm, Cpi = 10 pF, Cmu = 2 pF, RLp = 2500 ohm, Rsigp = 1000 ohm	SOLVED	A_mid = -100, C_M = 2.02e-10 F, R_pi_eq = 714.286 ohm, f_H_miller = 1.05102e+06 Hz, tau_oc = 1.56429e-07 s, f_H_octc = 1.01743e+06 Hz
Miller high-frequency cutoff: gm = 100 mS, r_pi = 1500 ohm, Cpi = 20	SOLVED	A_mid = -100, C_M = 1.01e-10 F, R_pi_eq = 375 ohm, f_H_miller = 3.50755e+06 Hz, tau_oc = 4.6375e-08 s,

question	outcome	result
pF, Cmu = 1 pF, RLp = 1000 ohm, Rsigp = 500 ohm		f_H_octc = 3.43191e+06 Hz
Miller high-frequency cutoff: r_pi = 2500 ohm, Cpi = 10 pF, Cmu = 2 pF, RLp = 2500 ohm, Rsigp = 1000 ohm	NEEDS_INPUT	elx_miller_high_cutoff needs gm (transconductance). Example: solve('elx_miller_high_cutoff', gm='0.04 S', r_pi='2500 ohm
Miller high-frequency cutoff: gm = 40 mS, r_pi = 2500 ohm, Cpi = 10 pF, Cmu = 2 pF, RLp = 2500 ohm, Rsigp = 1000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Common-emitter low-frequency cutoff by short-circuit time constants,

`elx_low_frequency_cutoff`

The resistance seen by each coupling and bypass capacitor of a common-emitter amplifier and the lower -3 dB frequency estimated as the sum of the short-circuit pole frequencies (r_o neglected).

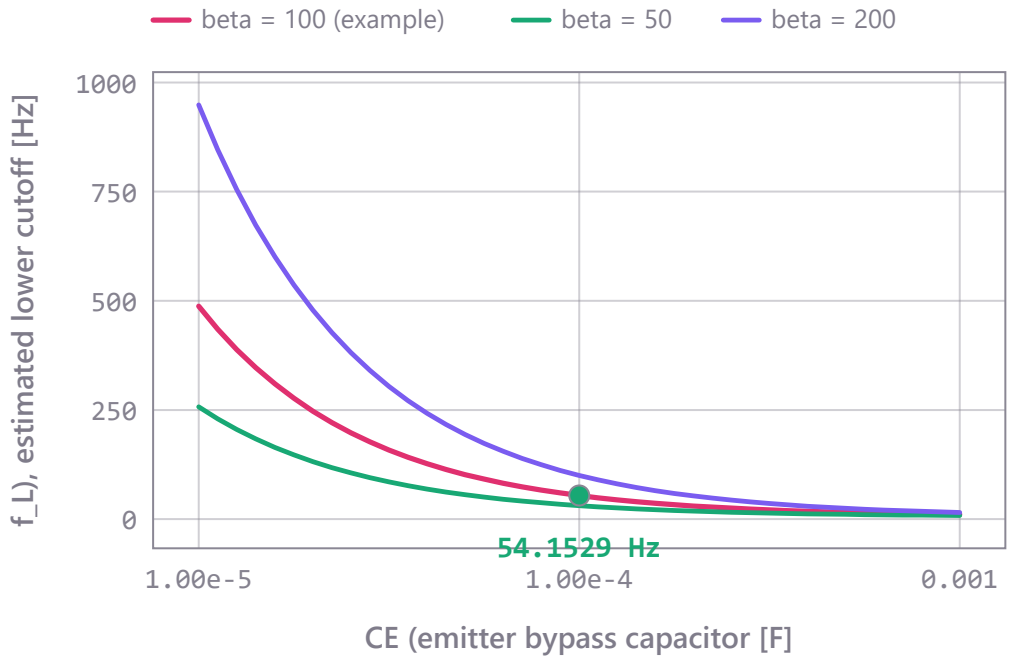
$$f_L \approx \sum_i \frac{1}{2\pi R_i C_i}$$

input	meaning	condition
`Rsig`	source resistance	
`RB`	bias resistance at the base (R1 || R2)	
`r_pi`	r_pi	
`beta`	current gain	
`RE`	emitter resistor (bypassed by CE)	
`RC`	collector resistor	
`RL`	load resistor	
`C1`	input coupling capacitor	
`C2`	output coupling capacitor	
`CE`	emitter bypass capacitor	

output	meaning
`R_C1`	resistance seen by C1
`R_CE`	resistance seen by CE
`R_C2`	resistance seen by C2
`f_C1`	pole of C1
`f_CE`	pole of CE
`f_C2`	pole of C2
`f_L`	estimated lower cutoff

- **Method:** R_C1 = Rsig + RB || r_pi; $R_C E = RE$ || (r_pi + Rsig || RB)/(beta + 1); $R_C 2 = RC + RL$; f_L = sum 1/(2 pi R C)
- **Assumes:** short-circuit time-constant estimate; r_o neglected

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f_L (estimated lower cutoff) against CE (emitter bypass capacitor) from 1.0000e-05 F to 0.001 F on a logarithmic axis. One curve for each value of beta (current gain): 100, 50, 200. Other inputs as in the worked example: Rsig = 1000.0 ohm, RB = 20000.0 ohm, r_pi = 2500.0 ohm, RE = 1000.0 ohm, RC = 5000.0 ohm, RL = 10000.0 ohm, C1 = 1e-05 F, C2 = 1e-05 F. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, CE = 0.0001 F, f_L = 54.1529 Hz.

```
electronics_engineering_ai999x.solve('elx_low_frequency_cutoff', Rsig=1000.0, RB=20000.0,
r_pi=2500.0, beta=100.0, RE=1000.0, RC=5000.0, RL=10000.0, C1=1e-05, C2=1e-05, CE=0.0001)
# -> R_C1 = 3222.22 ohm, R_CE = 33.0522 ohm, R_C2 = 15000 ohm, f_C1 = 4.93929 Hz, f_CE = 48.1526
Hz, f_C2 = 1.06103 Hz, f_L = 54.1529 Hz   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_low_frequency_cutoff', Rsig=600.0, RB=30000.0,
r_pi=1875.0, beta=150.0, RE=470.0, RC=3300.0, RL=4700.0, C1=4.7e-06, C2=1e-06, CE=0.00022)
# -> R_C1 = 2364.71 ohm, R_CE = 15.7656 ohm, R_C2 = 8000 ohm, f_C1 = 14.3201 Hz, f_CE = 45.8867 Hz,
f_C2 = 19.8944 Hz, f_L = 80.1011 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Low-frequency cutoff: Rsig = 1000 ohm, RB = 20000 ohm, r_pi = 2500 ohm, beta = 100, RE = 1000 ohm, RC = 5000 ohm, RL = 10000 ohm, C1 = 10 uF, C2 = 10 uF, CE = 100 uF	SOLVED	R_C1 = 3222.22 ohm, R_CE = 33.0522 ohm, R_C2 = 15000 ohm, f_C1 = 4.93929 Hz, f_CE = 48.1526 Hz, f_C2 = 1.06103 Hz, f_L = 54.1529 Hz
Low-frequency cutoff: Rsig = 600 ohm, RB = 30000 ohm, r_pi = 1875 ohm, beta = 150, RE = 470 ohm, RC = 3300 ohm, RL = 4700 ohm, C1 = 4.7 uF, C2 = 1 uF, CE = 220 uF	SOLVED	R_C1 = 2364.71 ohm, R_CE = 15.7656 ohm, R_C2 = 8000 ohm, f_C1 = 14.3201 Hz, f_CE = 45.8867 Hz, f_C2 = 19.8944 Hz, f_L = 80.1011 Hz
Low-frequency cutoff: RB = 20000 ohm, r_pi = 2500 ohm, beta = 100, RE = 1000	NEEDS_INPUT	elx_low_frequency_cutoff needs Rsig (source resistance). Example:

question	outcome	result
ohm, RC = 5000 ohm, RL = 10000 ohm, C1 = 10 uF, C2 = 10 uF, CE = 100 uF		solve('elx_low_frequency_cutoff', Rsig='1000 ohm', RB=
Low-frequency cutoff: Rsig = 1000 ohm, RB = 20000 ohm, r_pi = 2500 ohm, beta = 100, RE = 1000 ohm, RC = 5000 ohm, RL = 10000 ohm, C1 = 10 uF, C2 = 10 uF, CE = 100 uF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

BJT current mirror with finite beta, `elx_current_mirror`

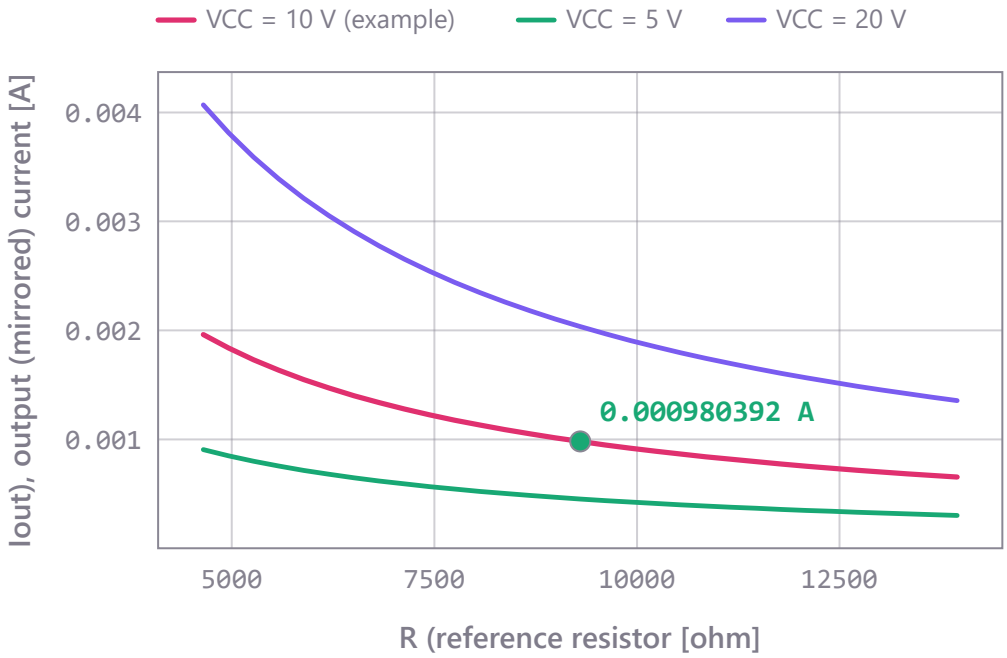
The reference current and the output current of a basic two-transistor current mirror with a resistor to the supply, including the base-current error Iout = Iref $\beta/(\beta+2)$.

$$I_{out} = \frac{\beta}{\beta+2} I_{ref}$$

input	meaning	condition
`VCC`	supply voltage	
`R`	reference resistor	
`VBE`	base-emitter voltage	
`beta`	current gain of each (matched) transistor	

output	meaning
`Iref`	current in the reference resistor
`Iout`	output (mirrored) current
`ratio`	Iout/Iref

- **Method:** Iref = (VCC - VBE)/R; Iout = Iref $\beta/(\beta+2)$
- **Assumes:** matched transistors, Early effect neglected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Iout (output (mirrored) current) against R (reference resistor) from 4650 ohm to 13950 ohm. One curve for each value of VCC (supply voltage): 10 V, 5 V, 20 V. Other inputs as in the worked example: VBE = 0.7 V, beta = 100.0. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R = 9300 ohm, Iout = 0.000980392 A.

```
electronics_engineering_ai999x.solve('elx_current_mirror', VCC=10.0, R=9300.0, VBE=0.7, beta=100.0)
# -> Iref = 0.001 A, Iout = 0.000980392 A, ratio = 0.980392   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_current_mirror', VCC=5.0, R=4300.0, VBE=0.7, beta=50.0)
# -> Iref = 0.001 A, Iout = 0.000961538 A, ratio = 0.961538   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Current mirror: VCC = 10 V, R = 9300 ohm, VBE = 0.7 V, beta = 100	SOLVED	Iref = 0.001 A, Iout = 0.000980392 A, ratio = 0.980392
Current mirror: VCC = 5 V, R = 4300 ohm, VBE = 0.7 V, beta = 50	SOLVED	Iref = 0.001 A, Iout = 0.000961538 A, ratio = 0.961538
Current mirror: R = 9300 ohm, VBE = 0.7 V, beta = 100	NEEDS_INPUT	elx_current_mirror needs VCC (supply voltage). Example: solve('elx_current_mirror', VCC='10 V', R='9300 ohm', VBE='0.7 V
Current mirror: VCC = 10 V, R = 9300 ohm, VBE = 0.7 V, beta = 100, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Wilson current mirror output current, `elx_wilson_current_mirror`

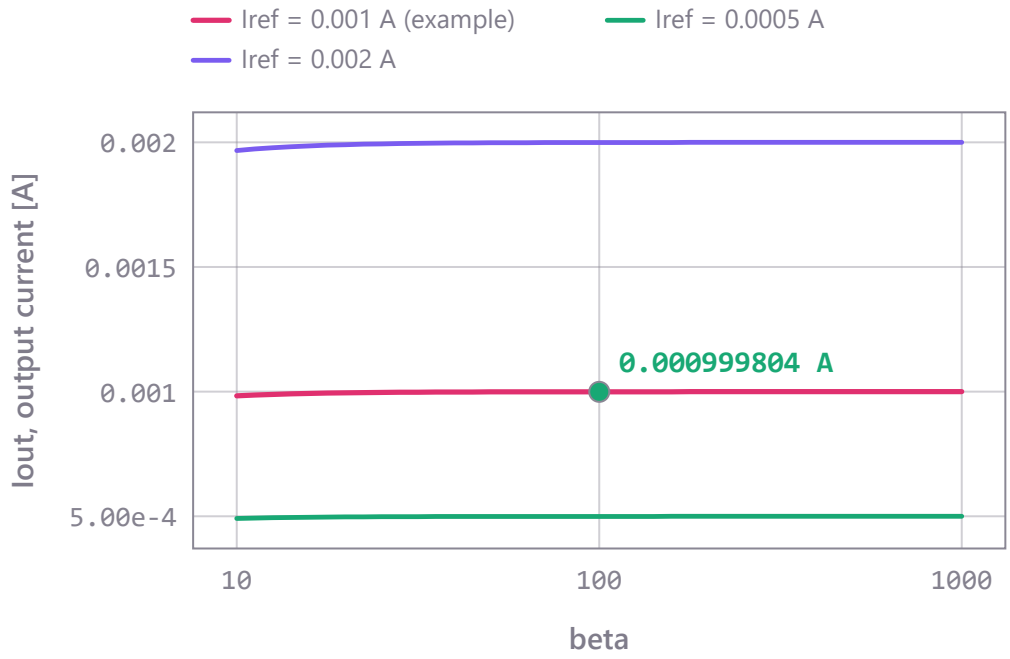
The output current of a three-transistor Wilson current mirror and its small base-current error, 2/(beta^2 + 2 beta + 2).

$$I_{out} = I_{ref}\left(1 - \frac{2}{\beta^2 + 2\beta + 2}\right)$$

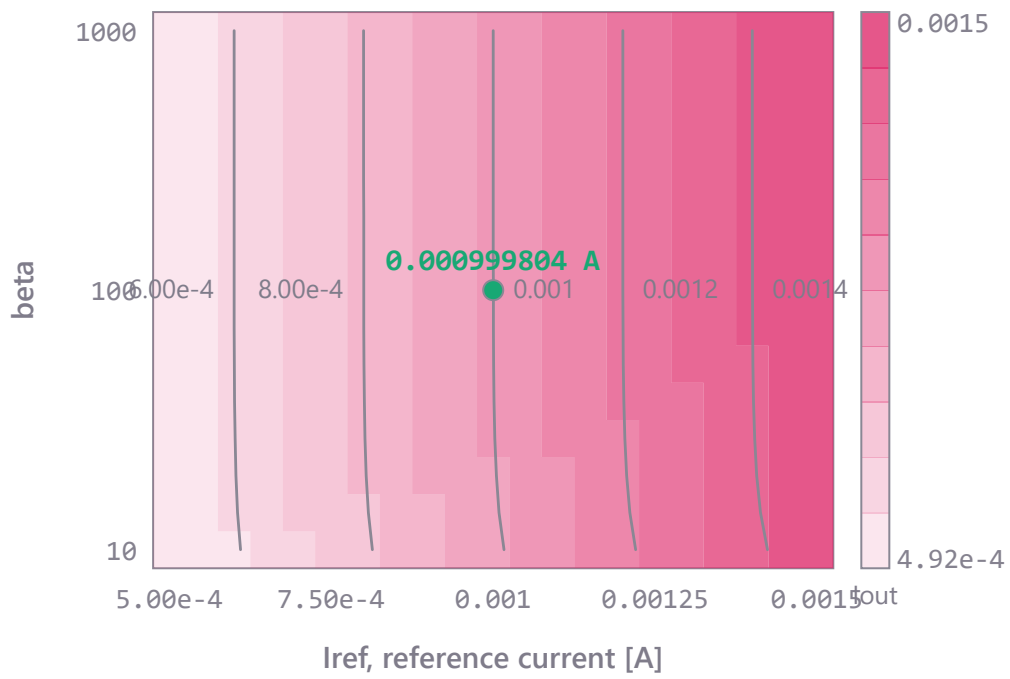
input	meaning	condition
`Iref`	reference current	
`beta`	current gain of each matched transistor	

output	meaning
`Iout`	output current
`ratio`	Iout/Iref
`error`	relative error 1 - Iout/Iref

- **Method:** Iout = Iref (1 - 2/(beta^2 + 2 beta + 2))
- **Assumes:** matched transistors, Early effect neglected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Iout (output current) against beta (current gain of each matched transistor) from 10 to 1000 on a logarithmic axis. One curve for each value of Iref (reference current): 0.001 A, 0.0005 A, 0.002 A. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, beta = 100, Iout = 0.000999804 A.



Iout (output current) over Iref (reference current) from 0.0005 A to 0.0015 A and beta (current gain of each matched transistor) from 10 to 1000 (logarithmic vertical axis): each of the 21 × 15 cells is one run of electronics-engineering-ai999x, shaded by its value (light 0.000491803 A to dark 0.0015 A); iso-lines at Iout = 6.00e-4, 8.00e-4, 0.001, 0.0012, 0.0014 A. Marked: the worked example, Iref = 0.001 A, beta = 100, Iout = 0.000999804 A.

```
electronics_engineering_ai999x.solve('elx_wilson_current_mirror', Iref=0.001, beta=100.0)
# -> Iout = 0.000999804 A, ratio = 0.999804, error = 0.00019604   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_wilson_current_mirror', Iref=0.0002, beta=20.0)
# -> Iout = 0.000199095 A, ratio = 0.995475, error = 0.00452489   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wilson current mirror: Iref = 1 mA, beta = 100	SOLVED	Iout = 0.000999804 A, ratio = 0.999804, error = 0.00019604
Wilson current mirror: Iref = 0.2 mA, beta = 20	SOLVED	Iout = 0.000199095 A, ratio = 0.995475, error = 0.00452489
Wilson current mirror: beta = 100	NEEDS_INPUT	elx_wilson_current_mirror needs Iref (reference current). Example: solve('elx_wilson_current_mirror', Iref='0.001 A', be
Wilson current mirror: Iref = 1 mA, beta = 100, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

BJT differential pair: differential and common-mode gain, CMRR, `elx_differential_pair`

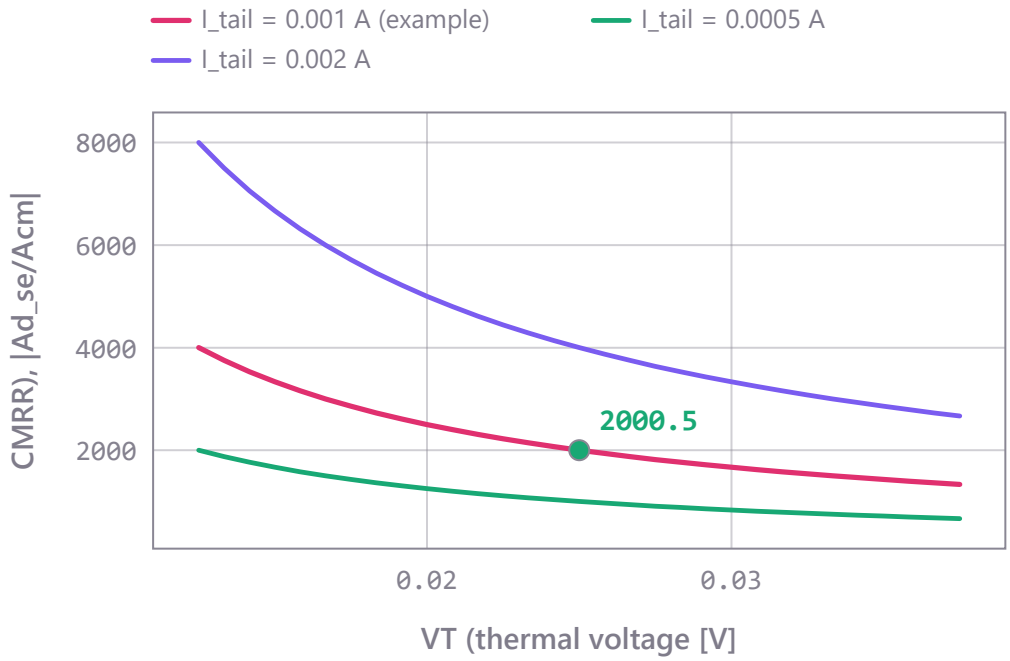
The differential gain (differential and single-ended output), the common-mode gain and the common-mode rejection ratio of a resistively loaded BJT differential pair with a tail current source of finite resistance.

$$A_{cm} = -\frac{g_m R_C}{1 + 2R_{EE}(\beta+1)/r_\pi}, \quad \mathrm{CMRR} = \frac{1 + 2R_{EE}(\beta+1)/r_\pi}{2}$$

input	meaning	condition
`I_tail`	tail (bias) current	
`beta`	current gain	
`VT`	thermal voltage	
`RC`	collector resistor of each side	
`REE`	output resistance of the tail current source	

output	meaning
`IC`	collector current of each transistor
`gm`	transconductance of each
`Ad_diff`	differential gain, differential output
`Ad_se`	differential gain, single-ended output
`Acm`	common-mode gain, single-ended output
`CMRR`	|Ad_se/Acm|
`CMRR_dB`	CMRR in dB

- **Method:** $IC = \alpha\, I/2$; $gm = IC/VT$; $Ad = -gm$ RC (/2 single-ended); Acm = -gm RC/(1 + 2 REE (beta + 1)/r_pi); CMRR = |Ad,se/Acm|
- **Assumes:** matched transistors, r_o neglected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



CMRR (|Ad_se/Acm|) against VT (thermal voltage) from 0.0125 V to 0.0375 V. One curve for each value of I_tail (tail (bias) current): 0.001 A, 0.0005 A, 0.002 A. Other inputs as in the worked example: beta = 100.0, RC = 10000.0 ohm, REE = 100000.0 ohm. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, VT = 0.025 V, CMRR = 2000.5.

```
electronics_engineering_ai999x.solve('elx_differential_pair', I_tail=0.001, beta=100.0, VT=0.025,
RC=10000.0, REE=100000.0)
# -> IC = 0.00049505 A, gm = 0.019802 S, Ad_diff = -198.02, Ad_se = -99.0099, Acm = -0.0494926,
CMRR = 2000.5, CMRR_dB = 66.0228   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_differential_pair', I_tail=0.002, beta=200.0, VT=0.026,
RC=5000.0, REE=50000.0)
# -> IC = 0.000995025 A, gm = 0.0382702 S, Ad_diff = -191.351, Ad_se = -95.6755, Acm = -0.0497383,
CMRR = 1923.58, CMRR_dB = 65.6822   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Differential pair: I_tail = 1 mA, beta = 100, VT = 25 mV, RC = 10000 ohm, REE = 100000 ohm	SOLVED	IC = 0.00049505 A, gm = 0.019802 S, Ad_diff = -198.02, Ad_se = -99.0099, Acm = -0.0494926, CMRR = 2000.5, CMRR_dB = 66.0228
Differential pair: I_tail = 2 mA, beta = 200, VT = 26 mV, RC = 5000 ohm, REE = 50000 ohm	SOLVED	IC = 0.000995025 A, gm = 0.0382702 S, Ad_diff = -191.351, Ad_se = -95.6755, Acm = -0.0497383, CMRR = 1923.58, CMRR_dB = 65.6822
Differential pair: beta = 100, VT = 25 mV, RC = 10000 ohm, REE = 100000 ohm	NEEDS_INPUT	elx_differential_pair needs I_tail (tail (bias) current). Example: solve('elx_differential_pair', I_tail='0.001 A', beta
Differential pair: I_tail = 1 mA, beta = 100, VT = 25 mV, RC = 10000 ohm, REE = 100000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

BJT cascode: output resistance, transconductance and gain, `elx_bjt_cascode`

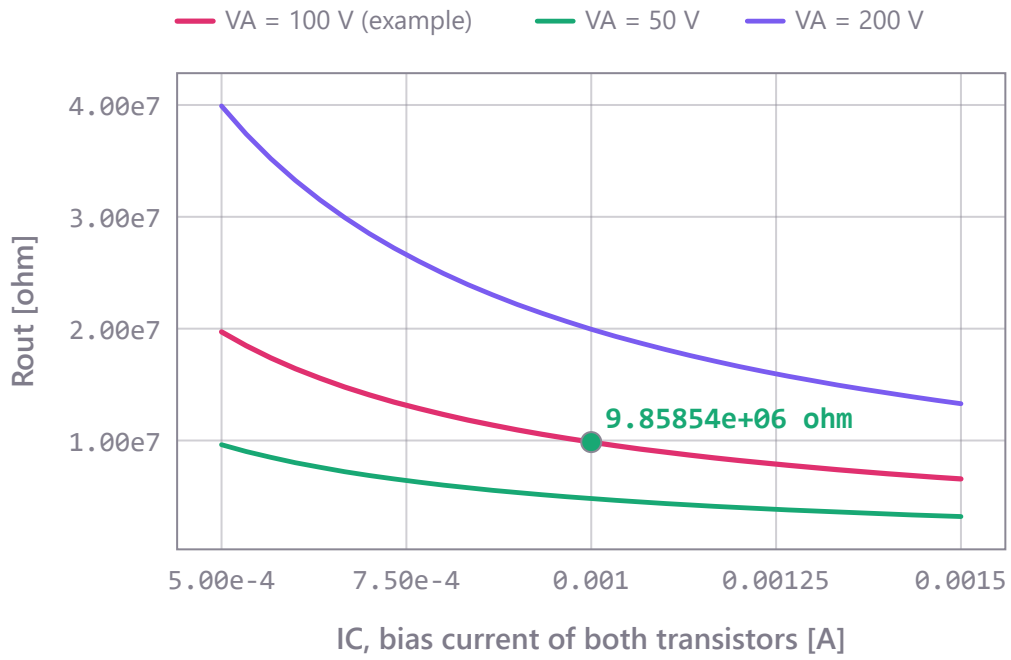
The output resistance, the short-circuit transconductance and the voltage gain into a load of a cascode (common emitter driving common base) of two identical transistors at the same bias current.

$$R_{out} = r_o + (1 + g_m r_o)(r_o \parallel r_\pi), \quad A_v = -G_m(R_{out} \parallel R_L)$$

input	meaning	condition
`IC`	bias current of both transistors	
`beta`	current gain	
`VT`	thermal voltage	
`VA`	Early voltage	
`RL`	load resistance	

output	meaning
`gm`	transconductance of each transistor
`r_o`	output resistance of each
`Rout`	output resistance of the cascode
`Gm`	short-circuit transconductance
`Av`	voltage gain into RL

- **Method:** Rout = r_o + (1 + gm r_o)(r_o || r_pi); $Gm = gm\ (gm + 1/r_o)/(gm + 2/r_o + 1/r_pi)$; $Av = -Gm$ (Rout || RL)
- **Assumes:** identical transistors, base of the upper one at signal ground
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Rout (output resistance of the cascode) against IC (bias current of both transistors) from 0.0005 A to 0.0015 A. One curve for each value of VA (Early voltage): 100 V, 50 V, 200 V. Other inputs as in the worked example: beta = 100.0, VT = 0.025 V, RL = 1000000.0 ohm. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, IC = 0.001 A, Rout = 9.85854e+06 ohm.

```
electronics_engineering_ai999x.solve('elx_bjt_cascode', IC=0.001, beta=100.0, VT=0.025, VA=100.0, RL=1000000.0)
# -> gm = 0.04 S, r_o = 100000 ohm, Rout = 9.85854e+06 ohm, Gm = 0.0395943 S, Av = -35947.9 (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_bjt_cascode', IC=0.0001, beta=200.0, VT=0.026, VA=50.0, RL=10000000.0)
# -> gm = 0.00384615 S, r_o = 500000 ohm, Rout = 9.11268e+07 ohm, Gm = 0.00382505 S, Av = -34468.1 (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BJT cascode: IC = 1 mA, beta = 100, VT = 25 mV, VA = 100 V, RL = 1000000 ohm	SOLVED	gm = 0.04 S, r_o = 100000 ohm, Rout = 9.85854e+06 ohm, Gm = 0.0395943 S, Av = -35947.9
BJT cascode: IC = 0.1 mA, beta = 200, VT = 26 mV, VA = 50 V, RL = 10000000 ohm	SOLVED	gm = 0.00384615 S, r_o = 500000 ohm, Rout = 9.11268e+07 ohm, Gm = 0.00382505 S, Av = -34468.1
BJT cascode: beta = 100, VT = 25 mV, VA = 100 V, RL = 1000000 ohm	NEEDS_INPUT	elx_bjt_cascode needs IC (bias current of both transistors). Example: solve('elx_bjt_cascode', IC='0.001 A', beta=100, V

question	outcome	result
BJT cascode: IC = 1 mA, beta = 100, VT = 25 mV, VA = 100 V, RL = 1000000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

MOSFET drain current (cutoff, triode, saturation), `elx_mosfet_drain_current`

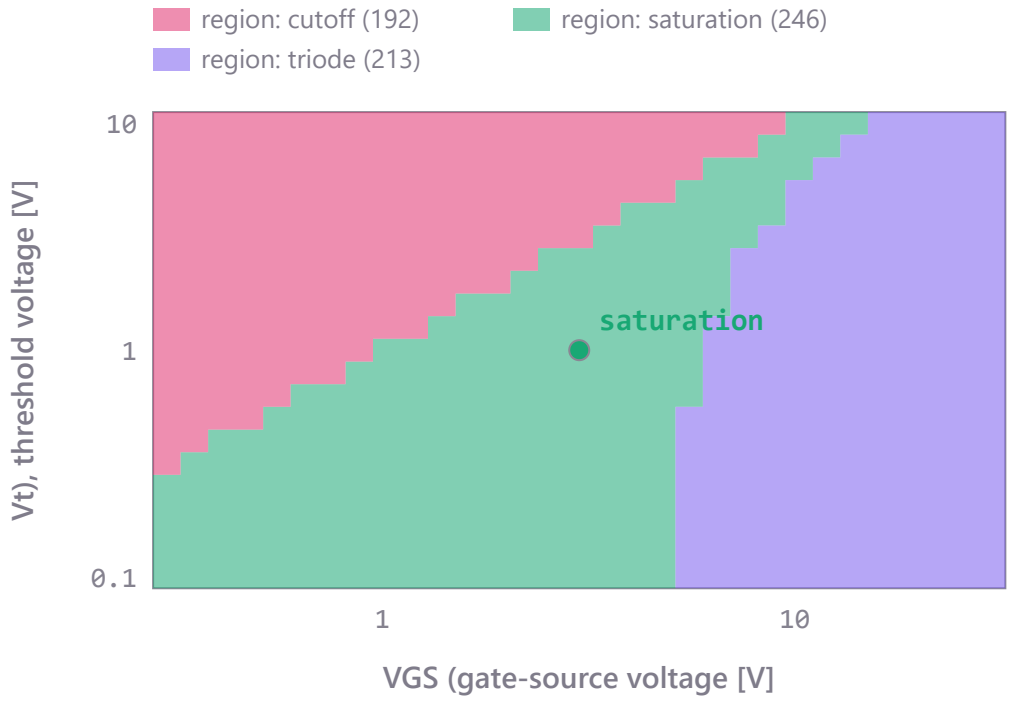
The operating region and drain current of an enhancement NMOS transistor by the square law: cutoff, triode or saturation with optional channel-length modulation (use magnitudes for PMOS).

$$I_D = \begin{cases} k_n\left(V_{OV}V_{DS} - \frac{1}{2}V_{DS}^2\right) & V_{DS} < V_{OV} \\ \frac{1}{2}k_n V_{OV}^2(1 + \lambda V_{DS}) & V_{DS} \geq V_{OV} \end{cases}$$

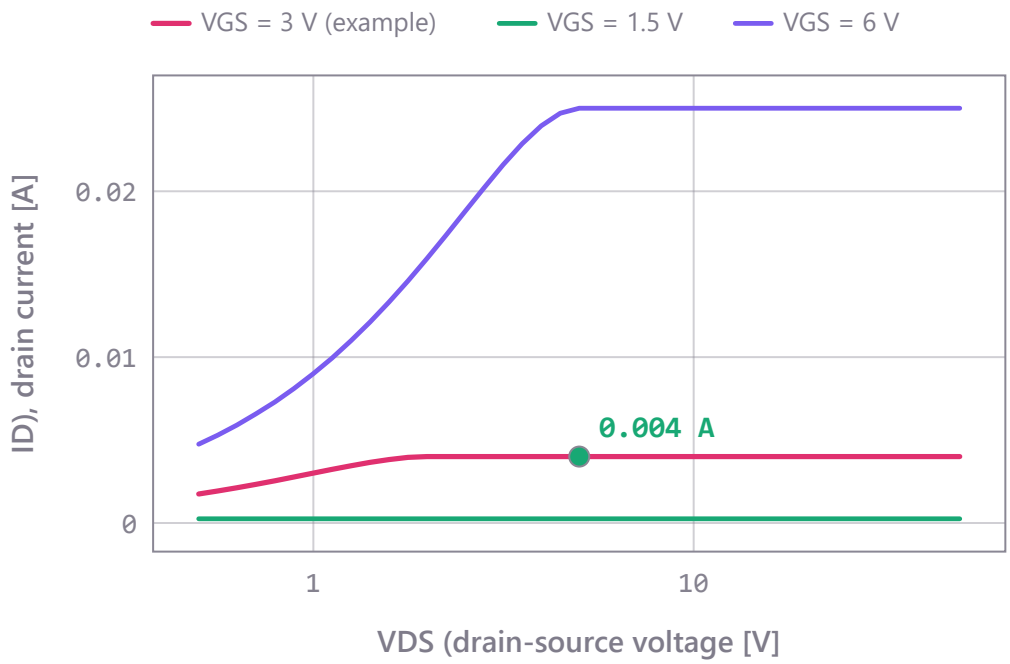
input	meaning	condition
`kn`	process transconductance times aspect ratio, mu Cox W/L	
`Vt`	threshold voltage	
`VGS`	gate-source voltage	
`VDS`	drain-source voltage	
`lam`	channel-length modulation lambda (default 0)	

output	meaning
`region`	cutoff, triode or saturation
`ID`	drain current
`VOV`	overdrive VGS - Vt
`VDS_sat`	saturation voltage

- **Method:** cutoff VOV <= 0; triode VDS < VOV: $ID = kn$ (VOV VDS - VDS^2/2); saturation: $ID = (kn/2)$ VOV^2 (1 + lambda VDS)
- **Assumes:** kn, $Vt > 0$; VDS >= 0; long-channel square law
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cutoff, triode or saturation (region) over VGS (gate-source voltage) from 0.3 V to 30 V and Vt (threshold voltage) from 0.1 V to 10 V (logarithmic axes): each of the 31 × 21 cells is one run of electronics-engineering-ai999x, coloured by the verdict it returned, cutoff: 192 cells, saturation: 246 cells, triode: 213 cells. Other inputs as in the worked example: kn = 0.002 A/V^2, VDS = 5.0 V. Marked: the worked example, VGS = 3 V, Vt = 1 V → saturation.



ID (drain current) against VDS (drain-source voltage) from 0.5 V to 50 V on a logarithmic axis. One curve for each value of VGS (gate-source voltage): 3 V, 1.5 V, 6 V. Other inputs as in the worked example: kn = 0.002 A/V^2, Vt = 1.0 V. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, VDS = 5 V, ID = 0.004 A.

```
electronics_engineering_ai999x.solve('elx_mosfet_drain_current', kn=0.002, Vt=1.0, VGS=3.0,
VDS=5.0)
# -> region = saturation, ID = 0.004 A, VOV = 2 V, VDS_sat = 2 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_mosfet_drain_current', kn=0.001, Vt=0.7, VGS=2.0,
VDS=0.5, lam=0.02)
# -> region = triode, ID = 0.000525 A, VOV = 1.3 V, VDS_sat = 1.3 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MOSFET drain current: kn = 2 mA/V^2, Vt = 1 V, VGS = 3 V, VDS = 5 V	SOLVED	region = saturation, ID = 0.004 A, VOV = 2 V, VDS_sat = 2 V
MOSFET drain current: kn = 1 mA/V^2, Vt = 0.7 V, VGS = 2 V, VDS = 0.5 V, lam = 0.02 1/V	SOLVED	region = triode, ID = 0.000525 A, VOV = 1.3 V, VDS_sat = 1.3 V
MOSFET drain current: Vt = 1 V, VGS = 3 V, VDS = 5 V	NEEDS_INPUT	elx_mosfet_drain_current needs kn (process transconductance times aspect ratio, mu Cox W/L). Example: solve('elx_mosfet_
MOSFET drain current: kn = 2 mA/V^2, Vt = 1 V, VGS = 3 V, VDS = 5 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

MOSFET small-signal gm, ro and intrinsic gain, `elx_mosfet_gm_ro`

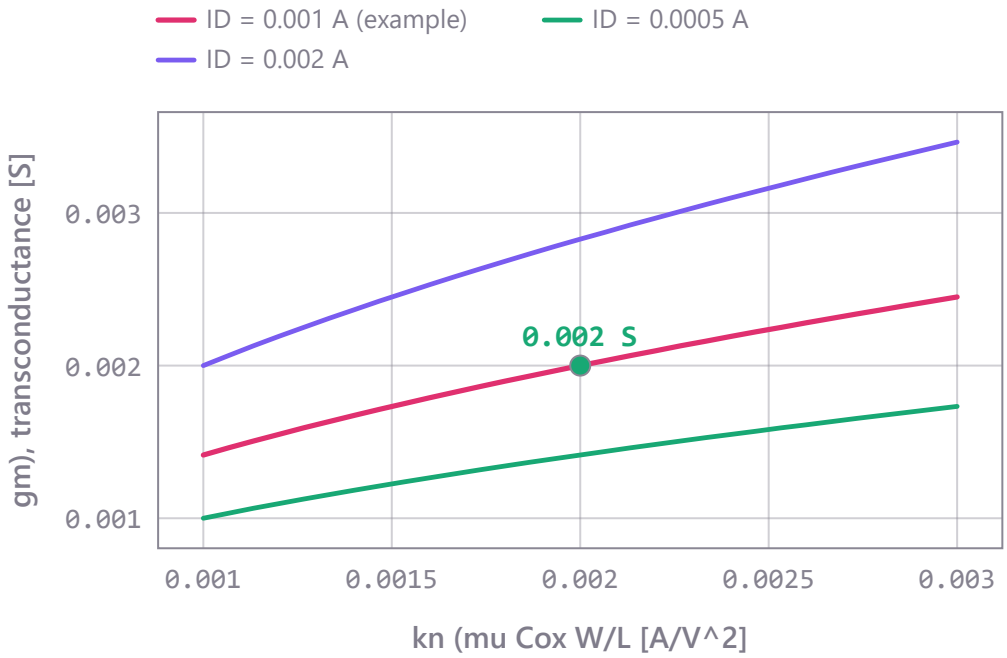
The overdrive voltage, gate-source voltage, transconductance $gm = sqrt(2\ kn\ ID)$, output resistance ro = 1/(lambda ID) and intrinsic gain of a MOSFET in saturation at a drain current.

$$g_m = \sqrt{2k_n I_D} = \frac{2I_D}{V_{OV}}, \quad r_o = \frac{1}{\lambda I_D}$$

input	meaning	condition
`kn`	mu Cox W/L	
`ID`	drain bias current	
`lam`	channel-length modulation lambda	
`Vt`	threshold voltage	

output	meaning
`VOV`	overdrive voltage
`VGS`	gate-source voltage
`gm`	transconductance
`r_o`	output resistance
`A0`	intrinsic gain gm r_o

- **Method:** VOV = sqrt(2 ID/kn); $gm = sqrt(2\ kn\ ID)$ = 2 ID/VOV; ro = 1/(lambda ID)
- **Assumes:** saturation; ID taken without the (1 + lambda VDS) factor
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



gm (transconductance) against kn (mu Cox W/L) from 0.001 A/V^2 to 0.003 A/V^2. One curve for each value of ID (drain bias current): 0.001 A, 0.0005 A, 0.002 A. Other inputs as in the worked example: lam = 0.01 1/V, Vt = 0.7 V. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, kn = 0.002 A/V^2, gm = 0.002 S.

```
electronics_engineering_ai999x.solve('elx_mosfet_gm_ro', kn=0.002, ID=0.001, lam=0.01, Vt=0.7)
# -> VOV = 1 V, VGS = 1.7 V, gm = 0.002 S, r_o = 100000 ohm, A0 = 200   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_mosfet_gm_ro', kn=0.005, ID=0.0004, lam=0.05, Vt=0.5)
# -> VOV = 0.4 V, VGS = 0.9 V, gm = 0.002 S, r_o = 50000 ohm, A0 = 100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MOSFET small-signal parameters: kn = 2 mA/V^2, ID = 1 mA, lam = 0.01 1/V, Vt = 0.7 V	SOLVED	VOV = 1 V, VGS = 1.7 V, gm = 0.002 S, r_o = 100000 ohm, A0 = 200
MOSFET small-signal parameters: kn = 5 mA/V^2, ID = 0.4 mA, lam = 0.05 1/V, Vt = 0.5 V	SOLVED	VOV = 0.4 V, VGS = 0.9 V, gm = 0.002 S, r_o = 50000 ohm, A0 = 100
MOSFET small-signal parameters: ID = 1 mA, lam = 0.01 1/V, Vt = 0.7 V	NEEDS_INPUT	elx_mosfet_gm_ro needs kn (mu Cox W/L). Example: solve('elx_mosfet_gm_ro', kn='0.002 A/V^2', ID='0.001 A', lam='0.01 1/V
MOSFET small-signal parameters: kn = 2 mA/V^2, ID = 1 mA, lam = 0.01 1/V, Vt = 0.7 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

MOSFET voltage-divider bias with source resistor: Q-point, `elx_mosfet_divider_bias`

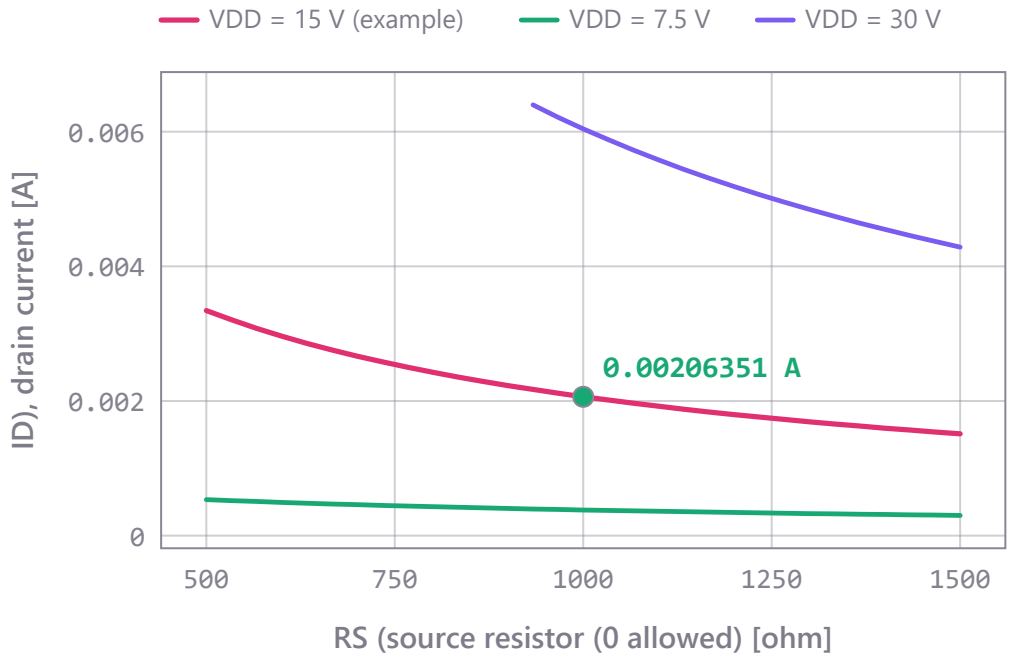
The Q-point (gate voltage, VGS, drain current, node voltages and VDS) of an NMOS stage with a gate divider and a source resistor, solving the square law and the gate loop (the physical root), with a saturation check.

$$V_{OV} = \frac{2(V_G - V_t)}{1 + \sqrt{1 + 2k_n R_S (V_G - V_t)}}, \quad I_D = \tfrac{1}{2} k_n V_{OV}^2$$

input	meaning	condition
VDD	supply voltage	
R1	upper gate resistor	
R2	lower gate resistor	
RD	drain resistor	
RS	source resistor (0 allowed)	
kn	mu Cox W/L	
Vt	threshold voltage	

output	meaning
VG	gate voltage
VGS	gate-source voltage
VOV	overdrive
ID	drain current
VS	source voltage
VD	drain voltage
VDS	drain-source voltage

- **Method:** VG = VDD $R_2/(R_1 + R_2)$; (kn RS/2) VOV^2 + VOV - (VG - Vt) = 0, positive root; $ID = (kn/2)$ VOV^2
- **Assumes:** $VG > Vt$; saturation: VDS >= VOV (otherwise refused); lambda neglected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ID (drain current) against RS (source resistor (0 allowed)) from 500 ohm to 1500 ohm. One curve for each value of VDD (supply voltage): 15 V, 7.5 V, 30 V. Other inputs as in the worked example: R1 = 1000000.0 ohm, R2 = 500000.0 ohm, RD = 3300.0 ohm, kn = 0.002 A/V^2, Vt = 1.5 V. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, RS = 1000 ohm, ID = 0.00206351 A.

```
electronics_engineering_ai999x.solve('elx_mosfet_divider_bias', VDD=15.0, R1=1000000.0,
R2=500000.0, RD=3300.0, RS=1000.0, kn=0.002, Vt=1.5)
# -> VG = 5 V, VGS = 2.93649 V, VOV = 1.43649 V, ID = 0.00206351 A, VS = 2.06351 V, VD = 8.19042 V,
VDS = 6.12691 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_mosfet_divider_bias', VDD=12.0, R1=680000.0, R2=320000.0,
RD=2200.0, RS=0.0, kn=0.001, Vt=1.0)
# -> VG = 3.84 V, VGS = 3.84 V, VOV = 2.84 V, ID = 0.0040328 A, VS = 0 V, VD = 3.12784 V, VDS =
3.12784 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MOSFET divider bias: VDD = 15 V, R1 = 1000000 ohm, R2 = 500000 ohm, RD = 3300 ohm, RS = 1000 ohm, kn = 2 mA/V^2, Vt = 1.5 V	SOLVED	VG = 5 V, VGS = 2.93649 V, VOV = 1.43649 V, ID = 0.00206351 A, VS = 2.06351 V, VD = 8.19042 V, VDS = 6.12691 V
MOSFET divider bias: VDD = 12 V, R1 = 680000 ohm, R2 = 320000 ohm, RD = 2200 ohm, RS = 0 ohm, kn = 1 mA/V^2, Vt = 1 V	SOLVED	VG = 3.84 V, VGS = 3.84 V, VOV = 2.84 V, ID = 0.0040328 A, VS = 0 V, VD = 3.12784 V, VDS = 3.12784 V
MOSFET divider bias: R1 = 1000000 ohm, R2 = 500000 ohm, RD = 3300 ohm, RS = 1000 ohm, kn = 2 mA/V^2, Vt = 1.5 V	NEEDS_INPUT	elx_mosfet_divider_bias needs VDD (supply voltage). Example: solve('elx_mosfet_divider_bias', VDD='15 V', R1='1e+06 ohm'
MOSFET divider bias: VDD = 15 V, R1 = 1000000 ohm, R2 = 500000 ohm, RD = 3300 ohm, RS = 1000 ohm, kn = 2 mA/V^2, Vt = 1.5 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Common-source amplifier with source degeneration and ro, `elx_common_source_amplifier`

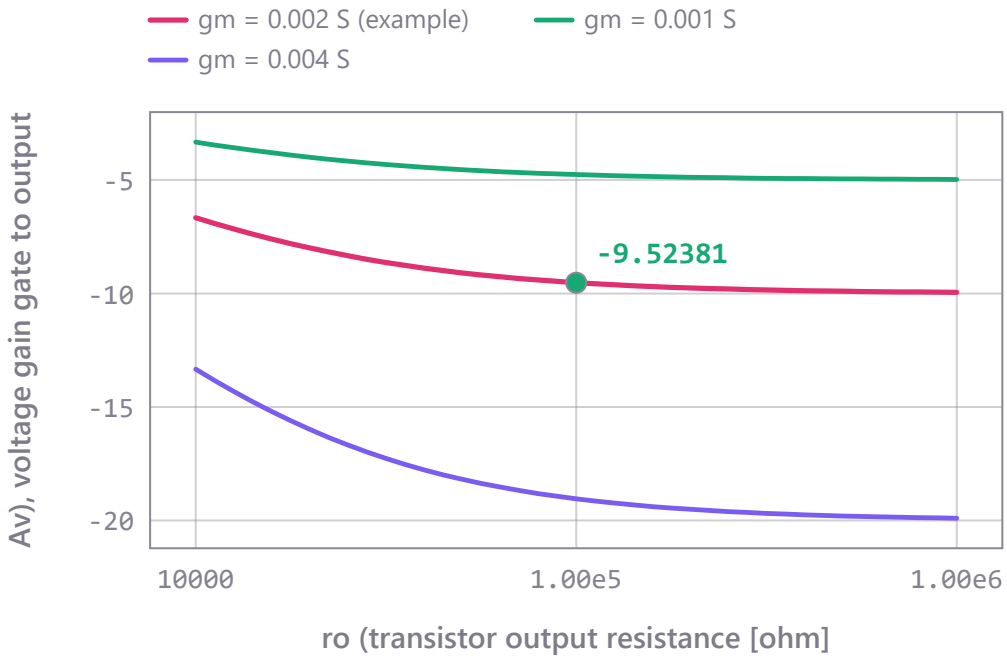
The voltage gain, input and output resistances and overall gain of a common-source stage, exact with the transistor's output resistance ro and an optional unbypassed source resistor.

$$A_v = -\frac{g_m r_o R'_D}{R'_D + r_o + R_S(1 + g_m r_o)}$$

input	meaning	condition
`gm`	transconductance	
`ro`	transistor output resistance	
`RD`	drain resistor	
`RL`	load resistor	
`RS`	unbypassed source resistance (0 if bypassed)	
`RG`	gate bias resistance	
`Rsig`	source resistance	

output	meaning
`Av`	voltage gain gate to output
`Rin`	input resistance
`Rout`	output resistance (excluding RL)
`Gv`	overall gain from the source

- **Method:** $Av = -gm$ ro RD'/(RD' + ro + RS (1 + gm ro)) with RD' = RD || RL; Rout = RD || (ro + RS($1 + gm$ ro))
- **Assumes:** gm, ro, RD, RL, RG, Rsig > 0; $RS \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Av (voltage gain gate to output) against ro (transistor output resistance) from 10000 ohm to 1e+06 ohm on a logarithmic axis. One curve for each value of gm (transconductance): 0.002 S, 0.001 S, 0.004 S. Other inputs as in the worked example: RD = 10000.0 ohm, RL = 10000.0 ohm, RS = 0.0 ohm, RG = 1000000.0 ohm, Rsig = 10000.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, ro = 100000 ohm, Av = -9.52381.

```
electronics_engineering_ai999x.solve('elx_common_source_amplifier', gm=0.002, ro=100000.0,
RD=10000.0, RL=10000.0, RS=0.0, RG=1000000.0, Rsig=10000.0)
# -> Av = -9.52381, Rin = 1e+06 ohm, Rout = 9090.91 ohm, Gv = -9.42951   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_common_source_amplifier', gm=0.004, ro=50000.0,
RD=4700.0, RL=22000.0, RS=500.0, RG=2200000.0, Rsig=50000.0)
# -> Av = -5.01729, Rin = 2.2e+06 ohm, Rout = 4557.67 ohm, Gv = -4.90579   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Common-source amplifier: gm = 2 mS, ro = 100000 ohm, RD = 10000 ohm, RL = 10000 ohm, RS = 0 ohm, RG = 1000000 ohm, Rsig = 10000 ohm	SOLVED	Av = -9.52381, Rin = 1e+06 ohm, Rout = 9090.91 ohm, Gv = -9.42951
Common-source amplifier: gm = 4 mS, ro = 50000 ohm, RD = 4700 ohm, RL = 22000 ohm, RS = 500 ohm, RG = 2200000 ohm, Rsig = 50000 ohm	SOLVED	Av = -5.01729, Rin = 2.2e+06 ohm, Rout = 4557.67 ohm, Gv = -4.90579
Common-source amplifier: ro = 100000 ohm, RD = 10000 ohm, RL = 10000 ohm, RS = 0 ohm, RG = 1000000 ohm, Rsig = 10000 ohm	NEEDS_INPUT	elx_common_source_amplifier needs gm (transconductance). Example: solve('elx_common_source_amplifier', gm='0.002 S', ro=
Common-source amplifier: gm = 2 mS, ro = 100000 ohm, RD = 10000 ohm, RL = 10000 ohm, RS = 0 ohm, RG = 1000000 ohm, Rsig = 10000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Source follower (common drain), `elx_source_follower`

The voltage gain, input resistance, output resistance and overall gain of a MOSFET source follower, including ro.

$$A_v = \frac{g_m R}{1 + g_m R}, \quad R_{out} = R_S \parallel r_o \parallel \frac{1}{g_m}$$

input	meaning	condition
`gm`	transconductance	
`ro`	transistor output resistance	
`RS`	source resistor	
`RL`	load resistor	
`RG`	gate bias resistance	
`Rsig`	source resistance	

output	meaning
`Av`	voltage gain gate to output
`Rin`	input resistance
`Rout`	output resistance
`Gv`	overall gain

- **Method:** $Av = gm$ R/(1 + gm R) with $R = RS$ || RL || ro; Rout = RS || ro || $1/gm$
- **Assumes:** drain at signal ground
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Av (voltage gain gate to output) against ro (transistor output resistance) from 5000 ohm to 500000 ohm on a logarithmic axis. One curve for each value of gm (transconductance): 0.002 S, 0.001 S, 0.004 S. Other inputs as in the worked example: RS = 2000.0 ohm, RL = 2000.0 ohm, RG = 1000000.0 ohm, Rsig = 10000.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, ro = 50000 ohm, Av = 0.662252.

```
electronics_engineering_ai999x.solve('elx_source_follower', gm=0.002, ro=50000.0, RS=2000.0,
RL=2000.0, RG=1000000.0, Rsig=10000.0)
# -> Av = 0.662252, Rin = 1e+06 ohm, Rout = 396.825 ohm, Gv = 0.655695   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_source_follower', gm=0.005, ro=20000.0, RS=1000.0,
RL=10000.0, RG=470000.0, Rsig=1000.0)
# -> Av = 0.813008, Rin = 470000 ohm, Rout = 165.289 ohm, Gv = 0.811282   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Source follower: gm = 2 mS, ro = 50000 ohm, RS = 2000 ohm, RL = 2000 ohm, RG = 1000000 ohm, Rsig = 10000 ohm	SOLVED	Av = 0.662252, Rin = 1e+06 ohm, Rout = 396.825 ohm, Gv = 0.655695
Source follower: gm = 5 mS, ro = 20000 ohm, RS = 1000 ohm, RL = 10000 ohm, RG = 470000 ohm, Rsig = 1000 ohm	SOLVED	Av = 0.813008, Rin = 470000 ohm, Rout = 165.289 ohm, Gv = 0.811282
Source follower: ro = 50000 ohm, RS = 2000 ohm, RL = 2000 ohm, RG = 1000000 ohm, Rsig = 10000 ohm	NEEDS_INPUT	elx_source_follower needs gm (transconductance). Example: solve('elx_source_follower', gm='0.002 S', ro='50000 ohm', RS=
Source follower: gm = 2 mS, ro = 50000 ohm, RS = 2000 ohm, RL = 2000 ohm, RG = 1000000 ohm, Rsig = 10000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

MOSFET threshold voltage with body effect, `elx_mosfet_body_effect`

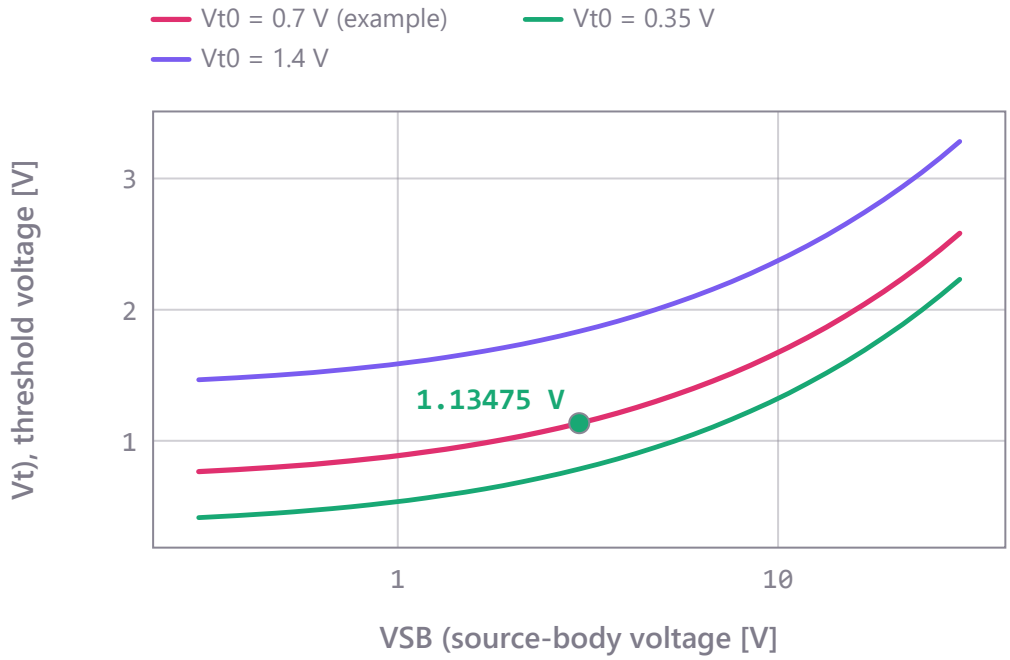
The threshold voltage of a MOSFET whose source is above its body, $Vt = Vt0 + \gamma$ (sqrt(2 phiF + VSB) - sqrt(2 phiF)).

$$V_t = V_{t0} + \gamma\left(\sqrt{2\phi_F + V_{SB}} - \sqrt{2\phi_F}\right)$$

input	meaning	condition
`Vt0`	zero-bias threshold voltage	
`gamma`	body-effect coefficient in $V^{0.5}$	
`phi2F`	surface potential 2 phi_F (about 0.6 to 0.9 V)	
`VSB`	source-body voltage	

output	meaning
`Vt`	threshold voltage
`dVt`	threshold increase

- **Method:** $Vt = Vt0 + \gamma$ (sqrt(2 phiF + VSB) - sqrt(2 phiF))
- **Assumes:** VSB >= 0; uniform substrate doping
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vt (threshold voltage) against VSB (source-body voltage) from 0.3 V to 30 V on a logarithmic axis. One curve for each value of Vt0 (zero-bias threshold voltage): 0.7 V, 0.35 V, 1.4 V. Other inputs as in the worked example: gamma = 0.4, phi2F = 0.7 V. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, VSB = 3 V, Vt = 1.13475 V.

```
electronics_engineering_ai999x.solve('elx_mosfet_body_effect', Vt0=0.7, gamma=0.4, phi2F=0.7,
VSB=3.0)
# -> Vt = 1.13475 V, dVt = 0.434751 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_mosfet_body_effect', Vt0=0.5, gamma=0.5, phi2F=0.8,
VSB=1.0)
# -> Vt = 0.723607 V, dVt = 0.223607 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Body effect threshold voltage: Vt0 = 0.7 V, gamma = 0.4 V^0.5, phi2F = 0.7 V, VSB = 3 V	SOLVED	Vt = 1.13475 V, dVt = 0.434751 V
Body effect threshold voltage: Vt0 = 0.5 V, gamma = 0.5 V^0.5, phi2F = 0.8 V, VSB = 1 V	SOLVED	Vt = 0.723607 V, dVt = 0.223607 V
Body effect threshold voltage: gamma = 0.4 V^0.5, phi2F = 0.7 V, VSB = 3 V	NEEDS_INPUT	elx_mosfet_body_effect needs Vt0 (zero-bias threshold voltage). Example: solve('elx_mosfet_body_effect', Vt0='0.7 V', ga
Body effect threshold voltage: Vt0 = 0.7 V, gamma = 0.4 V^0.5, phi2F = 0.7 V, VSB = 3 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te
Body effect threshold voltage: Vt0 = 0.7 V, gamma = 0.4 V^0.5, phi2F = 0.7 V, VSB = 3 V	SOLVED	Vt = 1.13475 V, dVt = 0.434751 V

CMOS inverter switching threshold VM, `elx_cmos_switching_threshold`

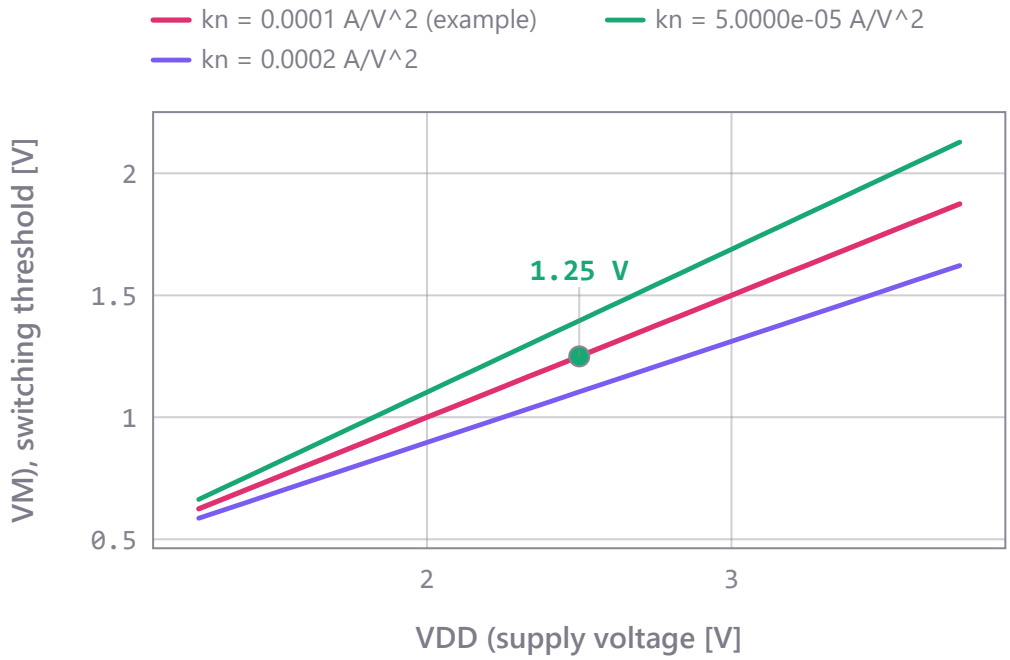
The switching threshold of a CMOS inverter (input = output, both transistors saturated) from the supply, the two threshold voltages and the two transconductance parameters.

$$V_M = \frac{V_{tn} + r\,(V_{DD} - |V_{tp}|)}{1 + r}, \quad r = \sqrt{k_p / k_n}$$

input	meaning	condition
`VDD`	supply voltage	
`Vtn`	NMOS threshold	
`Vtp`	magnitude of the PMOS threshold	
`kn`	NMOS mu_n Cox W/L	
`kp`	PMOS mu_p Cox W/L	

output	meaning
`r`	sqrt(kp/kn)
`VM`	switching threshold

- **Method:** VM = (Vtn + r (VDD - |Vtp|))/$(1 + r)$, $r = \sqrt{\frac{kp}{kn}}$
- **Assumes:** Vtn + |Vtp| < VDD; lambda neglected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



VM (switching threshold) against VDD (supply voltage) from 1.25 V to 3.75 V. One curve for each value of kn (NMOS mu_n Cox W/L): 0.0001 A/V^2, 5.0000e-05 A/V^2, 0.0002 A/V^2. Other inputs as in the worked example: Vtn = 0.4 V, Vtp = 0.4 V, kp = 0.0001 A/V^2. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, VDD = 2.5 V, VM = 1.25 V.

```
electronics_engineering_ai999x.solve('elx_cmos_switching_threshold', VDD=2.5, Vtn=0.4, Vtp=0.4,
kn=0.0001, kp=0.0001)
# -> r = 1, VM = 1.25 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_cmos_switching_threshold', VDD=3.3, Vtn=0.5, Vtp=0.6,
kn=0.0002, kp=8e-05)
# -> r = 0.632456, VM = 1.35234 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
CMOS switching threshold: VDD = 2.5 V, Vtn = 0.4 V, Vtp = 0.4 V, kn = 0.1 mA/V^2, kp = 0.1 mA/V^2	SOLVED	r = 1, VM = 1.25 V
CMOS switching threshold: VDD = 3.3 V, Vtn = 0.5 V, Vtp = 0.6 V, kn = 0.2 mA/V^2, kp = 0.08 mA/V^2	SOLVED	r = 0.632456, VM = 1.35234 V
CMOS switching threshold: Vtn = 0.4 V, Vtp = 0.4 V, kn = 0.1 mA/V^2, kp = 0.1 mA/V^2	NEEDS_INPUT	elx_cmos_switching_threshold needs VDD (supply voltage). Example: solve('elx_cmos_switching_threshold', VDD='2.5 V', Vtn
CMOS switching threshold: VDD = 2.5 V, Vtn = 0.4 V, Vtp = 0.4 V, kn = 0.1 mA/V^2, kp = 0.1 mA/V^2, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

CMOS gate propagation delay (switch-resistor model), `elx_cmos_propagation_delay`

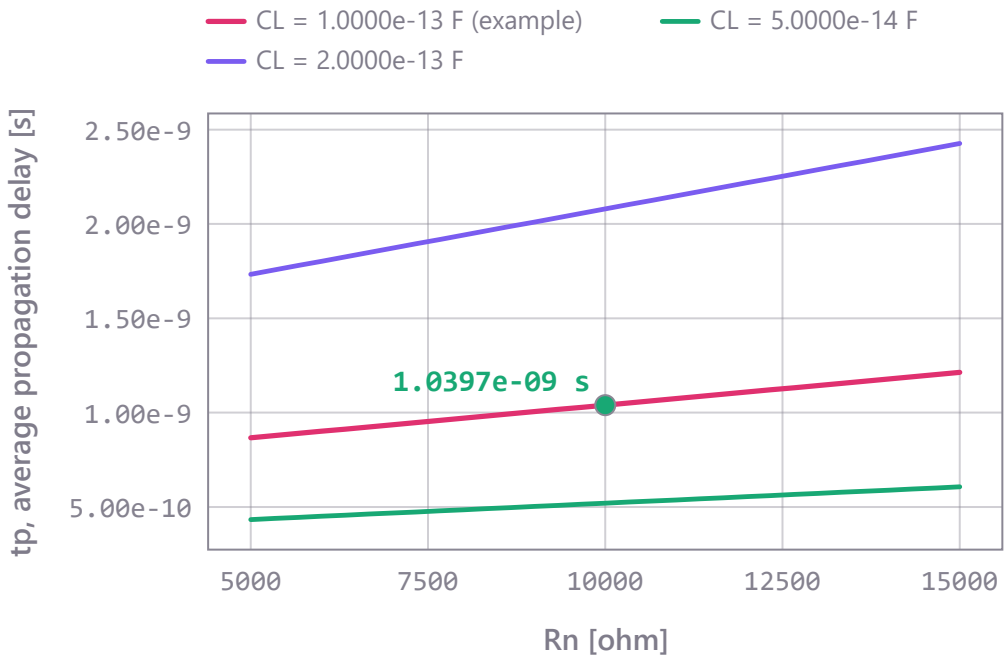
The high-to-low and low-to-high propagation delays (50 % points) and their average for a CMOS gate modelled as switched equivalent resistances driving a load capacitance.

$$t_{pHL} = \ln 2\, R_n C_L, \quad t_{pLH} = \ln 2\, R_p C_L$$

input	meaning	condition
`Rn`	equivalent on-resistance of the pull-down network	
`Rp`	equivalent on-resistance of the pull-up network	
`CL`	load capacitance	

output	meaning
`tpHL`	high-to-low delay
`tpLH`	low-to-high delay
`tp`	average propagation delay

- **Method:** tpHL = ln 2 Rn CL; tpLH = ln 2 Rp CL; tp = (tpHL + tpLH)/2
- **Assumes:** step input, RC switch model
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



tp (average propagation delay) against Rn (equivalent on-resistance of the pull-down network) from 5000 ohm to 15000 ohm. One curve for each value of CL (load capacitance): 1.0000e-13 F, 5.0000e-14 F, 2.0000e-13 F. Other inputs as in the worked example: Rp = 20000.0 ohm. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Rn = 10000 ohm, tp = 1.0397e-09 s.

```
electronics_engineering_ai999x.solve('elx_cmos_propagation_delay', Rn=10000.0, Rp=20000.0, CL=1e-13)
# -> tpHL = 6.93147e-10 s, tpLH = 1.38629e-09 s, tp = 1.03972e-09 s   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_cmos_propagation_delay', Rn=5000.0, Rp=12000.0, CL=5e-14)
# -> tpHL = 1.73287e-10 s, tpLH = 4.15888e-10 s, tp = 2.94588e-10 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
CMOS propagation delay: Rn = 10000 ohm, Rp = 20000 ohm, CL = 100 fF	SOLVED	tpHL = 6.93147e-10 s, tpLH = 1.38629e-09 s, tp = 1.03972e-09 s
CMOS propagation delay: Rn = 5000 ohm, Rp = 12000 ohm, CL = 50 fF	SOLVED	tpHL = 1.73287e-10 s, tpLH = 4.15888e-10 s, tp = 2.94588e-10 s
CMOS propagation delay: Rp = 20000 ohm, CL = 100 fF	NEEDS_INPUT	elx_cmos_propagation_delay needs Rn (equivalent on-resistance of the pull-down network). Example: solve('elx_cmos_propag
CMOS propagation delay: Rn = 10000 ohm, Rp = 20000 ohm, CL = 100 fF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Inverting op-amp amplifier, `elx_inverting_amplifier`

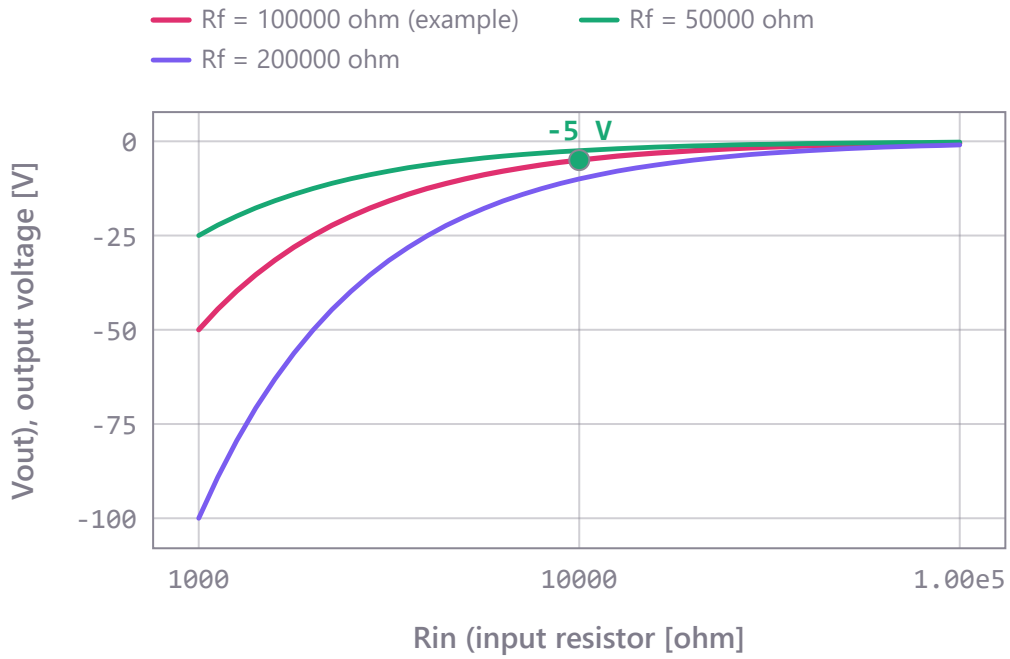
The closed-loop gain, output voltage and input resistance of an ideal inverting amplifier.

$$V_{out} = -\frac{R_f}{R_{in}} V_{in}$$

input	meaning	condition
`Rf`	feedback resistor	
`Rin`	input resistor	
`Vin`	input voltage	

output	meaning
`G`	closed-loop gain -Rf/Rin
`Vout`	output voltage
`Zin`	input resistance

- **Method:** G = -Rf/Rin; Vout = G Vin; Zin = Rin
- **Assumes:** ideal op-amp, output not saturated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout (output voltage) against Rin (input resistor) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of Rf (feedback resistor): 100000 ohm, 50000 ohm, 200000 ohm. Other inputs as in the worked example: Vin = 0.5 V. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Rin = 10000 ohm, Vout = -5 V.

```
electronics_engineering_ai999x.solve('elx_inverting_amplifier', Rf=100000.0, Rin=10000.0, Vin=0.5)
# -> G = -10, Vout = -5 V, Zin = 10000 ohm   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_inverting_amplifier', Rf=47000.0, Rin=4700.0, Vin=-0.2)
# -> G = -10, Vout = 2 V, Zin = 4700 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Inverting amplifier: Rf = 100000 ohm, Rin = 10000 ohm, Vin = 0.5 V	SOLVED	G = -10, Vout = -5 V, Zin = 10000 ohm
Inverting amplifier: Rf = 47000 ohm, Rin = 4700 ohm, Vin = -0.2 V	SOLVED	G = -10, Vout = 2 V, Zin = 4700 ohm
Inverting amplifier: Rin = 10000 ohm, Vin = 0.5 V	NEEDS_INPUT	elx_inverting_amplifier needs Rf (feedback resistor). Example: solve('elx_inverting_amplifier', Rf='100000 ohm', Rin='10
Inverting amplifier: Rf = 100000 ohm, Rin = 10000 ohm, Vin = 0.5 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Non-inverting op-amp amplifier, `elx_noninverting_amplifier`

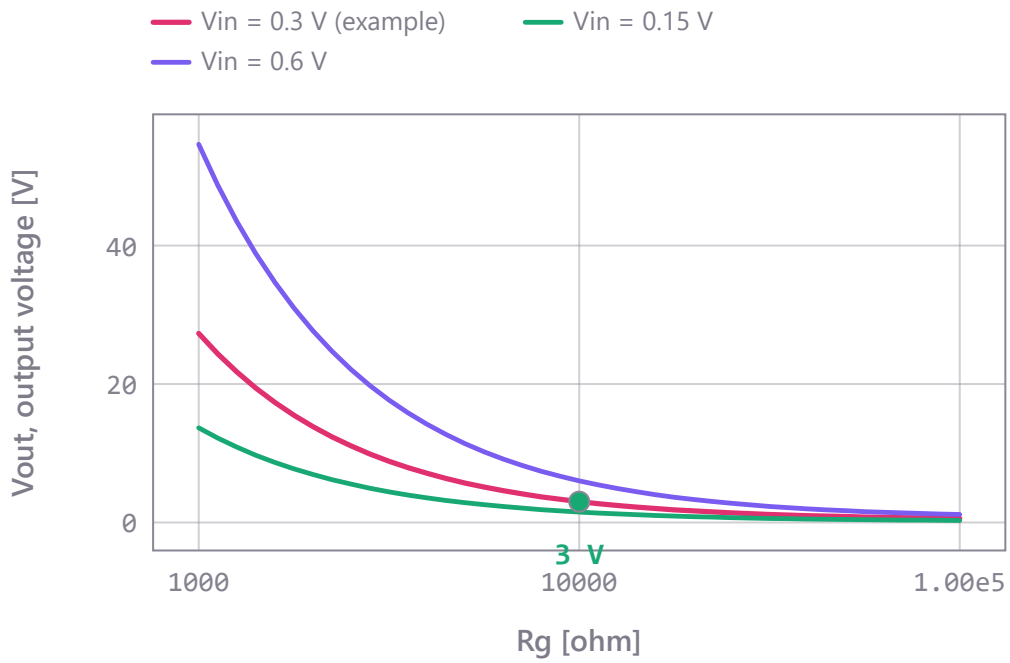
The closed-loop gain $1 + Rf/Rg$ and the output voltage of an ideal non-inverting amplifier.

$$V_{out} = \left(1 + \frac{R_f}{R_g}\right) V_{in}$$

input	meaning	condition
`Rf`	feedback resistor	
`Rg`	resistor from the inverting input to ground	
`Vin`	input voltage	

output	meaning
`G`	closed-loop gain
`Vout`	output voltage

- **Method:** $G = 1 + Rf/Rg$; Vout = G Vin
- **Assumes:** ideal op-amp
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout (output voltage) against Rg (resistor from the inverting input to ground) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of Vin (input voltage): 0.3 V, 0.15 V, 0.6 V. Other inputs as in the worked example: Rf = 90000.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Rg = 10000 ohm, Vout = 3 V.

```
electronics_engineering_ai999x.solve('elx_noninverting_amplifier', Rf=90000.0, Rg=10000.0, Vin=0.3)
# -> G = 10, Vout = 3 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_noninverting_amplifier', Rf=22000.0, Rg=4700.0, Vin=1.2)
# -> G = 5.68085, Vout = 6.81702 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Non-inverting amplifier: Rf = 90000 ohm, Rg = 10000 ohm, Vin = 0.3 V	SOLVED	G = 10, Vout = 3 V

question	outcome	result
Non-inverting amplifier: Rf = 22000 ohm, Rg = 4700 ohm, Vin = 1.2 V	SOLVED	G = 5.68085, Vout = 6.81702 V
Non-inverting amplifier: Rg = 10000 ohm, Vin = 0.3 V	NEEDS_INPUT	elx_noninverting_amplifier needs Rf (feedback resistor). Example: solve('elx_noninverting_amplifier', Rf='90000 ohm', Rg
Non-inverting amplifier: Rf = 90000 ohm, Rg = 10000 ohm, Vin = 0.3 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Inverting summing amplifier, `elx_summing_amplifier`

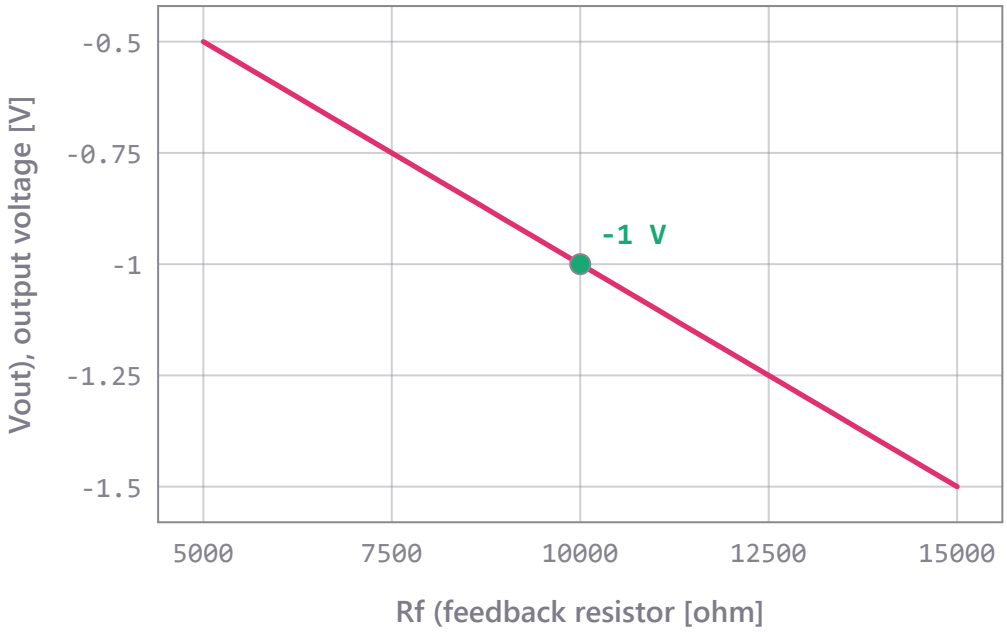
The output of an ideal inverting summing amplifier with several inputs, each through its own resistor, and the feedback current.

$$V_{out} = -R_f \sum_i \frac{V_i}{R_i}$$

input	meaning	condition
`V`	input voltages in volts	
`R`	input resistors in kilohms, one per input	
`Rf`	feedback resistor	

output	meaning
`Vout`	output voltage
`I_f`	current through the feedback resistor

- **Method:** Vout = -Rf sum(Vi/Ri)
- **Assumes:** one resistor per input; ideal op-amp
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout (output voltage) against Rf (feedback resistor) from 5000 ohm to 15000 ohm. Other inputs as in the worked example: V = [1.0, 2.0, -0.5], R = [10.0, 20.0, 5.0]. 31 points computed by electronics-engineering-ai999x; marked: the worked example, Rf = 10000 ohm, Vout = -1 V.

```
electronics_engineering_ai999x.solve('elx_summing_amplifier', V=[1.0, 2.0, -0.5], R=[10.0, 20.0,
5.0], Rf=10000.0)
# -> Vout = -1 V, I_f = 0.0001 A   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_summing_amplifier', V=[0.2, 0.4], R=[1.0, 1.0],
Rf=4700.0)
# -> Vout = -2.82 V, I_f = 0.0006 A   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Summing amplifier: V = [1, 2, -0.5], R = [10, 20, 5], Rf = 10000 ohm	SOLVED	Vout = -1 V, I_f = 0.0001 A
Summing amplifier: V = [0.2, 0.4], R = [1, 1], Rf = 4700 ohm	SOLVED	Vout = -2.82 V, I_f = 0.0006 A
Summing amplifier: R = [10, 20, 5], Rf = 10000 ohm	NEEDS_INPUT	elx_summing_amplifier needs V (input voltages in volts). Example: solve('elx_summing_amplifier', V=[1.0, 2.0, -0.5], R=[
Summing amplifier: V = [1, 2, -0.5], R = [10, 20, 5], Rf = 10000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Op-amp difference amplifier (with resistor mismatch), `elx_difference_amplifier`

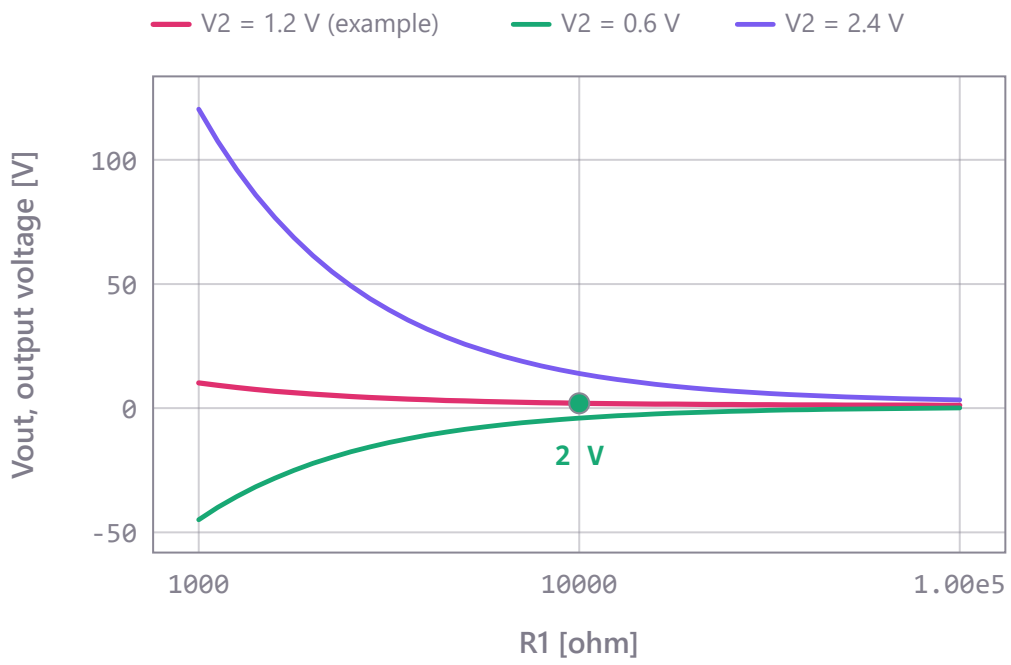
The output of a one-op-amp difference amplifier with any four resistors, and its differential and common-mode gains (the common-mode gain is zero only when R4/R3 = R2/R1).

$$V_{out} = \frac{R_4}{R_3 + R_4}\left(1 + \frac{R_2}{R_1}\right)V_2 - \frac{R_2}{R_1}V_1$$

input	meaning	condition
`R1`	resistor from V1 to the inverting input	
`R2`	feedback resistor	
`R3`	resistor from V2 to the non-inverting input	
`R4`	resistor from the non-inverting input to ground	
`V1`	voltage at the inverting side	
`V2`	voltage at the non-inverting side	

output	meaning
`Vout`	output voltage
`Ad`	differential gain (per volt of V2 - V1)
`Acm`	common-mode gain

- **Method:** Vout = -(R_2/R_1) V1 + (R4/(R3 + R4))$(1 + R_2/R_1)$ V2; $Ad = (a_2 - a_1)/2$; Acm = a1 + a2
- **Assumes:** ideal op-amp
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout (output voltage) against R1 (resistor from V1 to the inverting input) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of V2 (voltage at the non-inverting side): 1.2 V, 0.6 V, 2.4 V. Other inputs as in the worked example: R2 = 100000.0 ohm, R3 = 10000.0 ohm, R4 = 100000.0 ohm, V1 = 1.0 V. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R1 = 10000 ohm, Vout = 2 V.

```
electronics_engineering_ai999x.solve('elx_difference_amplifier', R1=10000.0, R2=100000.0,
R3=10000.0, R4=100000.0, V1=1.0, V2=1.2)
# -> Vout = 2 V, Ad = 10, Acm = 0   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_difference_amplifier', R1=10000.0, R2=50000.0,
R3=10500.0, R4=49000.0, V1=2.0, V2=2.1)
# -> Vout = 0.376471 V, Ad = 4.97059, Acm = -0.0588235   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Difference amplifier: R1 = 10000 ohm, R2 = 100000 ohm, R3 = 10000 ohm, R4 = 100000 ohm, V1 = 1 V, V2 = 1.2 V	SOLVED	Vout = 2 V, Ad = 10, Acm = 0
Difference amplifier: R1 = 10000 ohm, R2 = 50000 ohm, R3 = 10500 ohm, R4 = 49000 ohm, V1 = 2 V, V2 = 2.1 V	SOLVED	Vout = 0.376471 V, Ad = 4.97059, Acm = -0.0588235
Difference amplifier: R2 = 100000 ohm, R3 = 10000 ohm, R4 = 100000 ohm, V1 = 1 V, V2 = 1.2 V	NEEDS_INPUT	elx_difference_amplifier needs R1 (resistor from V1 to the inverting input). Example: solve('elx_difference_amplifier',
Difference amplifier: R1 = 10000 ohm, R2 = 100000 ohm, R3 = 10000 ohm, R4 = 100000 ohm, V1 = 1 V, V2 = 1.2 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Three-op-amp instrumentation amplifier, `elx_instrumentation_amplifier`

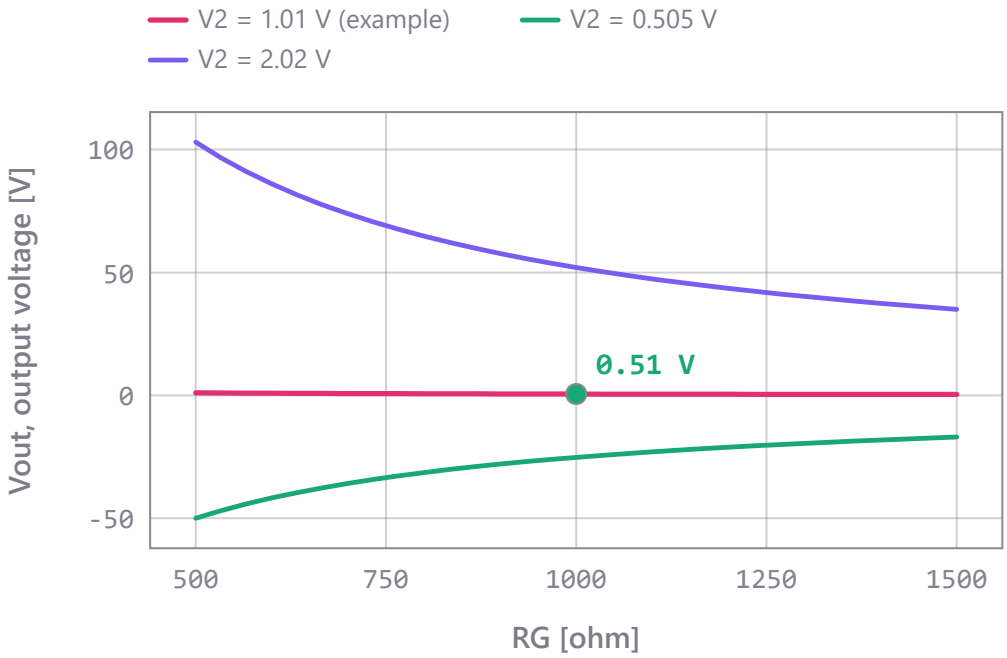
The gain and output of the classic three-op-amp instrumentation amplifier: input stage gain $1 + 2\,R_1/RG$ times the difference stage gain R_3/R_2.

$$V_{out} = \left(1 + \frac{2R_1}{R_G}\right)\frac{R_3}{R_2}(V_2 - V_1)$$

input	meaning	condition
`R1`	feedback resistor of each input buffer	
`RG`	gain-setting resistor between the buffers	
`R2`	input resistors of the difference stage	
`R3`	feedback and ground resistors of the difference stage	
`V1`	input at the inverting side	
`V2`	input at the non-inverting side	

output	meaning
`G`	differential gain
`G_stage1`	gain of the input stage
`Vout`	output voltage

- **Method:** G = (1 + 2 R1/RG)(R_3/R_2); Vout = G $(V_2 - V_1)$
- **Assumes:** ideal op-amps, matched resistors
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout (output voltage) against RG (gain-setting resistor between the buffers) from 500 ohm to 1500 ohm. One curve for each value of V2 (input at the non-inverting side): 1.01 V, 0.505 V, 2.02 V. Other inputs as in the worked example: R1 = 25000.0 ohm, R2 = 10000.0 ohm, R3 = 10000.0 ohm, V1 = 1.0 V. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, RG = 1000 ohm, Vout = 0.51 V.

```
electronics_engineering_ai999x.solve('elx_instrumentation_amplifier', R1=25000.0, RG=1000.0,
R2=10000.0, R3=10000.0, V1=1.0, V2=1.01)
# -> G = 51, G_stage1 = 51, Vout = 0.51 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_instrumentation_amplifier', R1=49400.0, RG=494.0,
R2=10000.0, R3=20000.0, V1=0.5, V2=0.502)
# -> G = 402, G_stage1 = 201, Vout = 0.804 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Instrumentation amplifier: R1 = 25000 ohm, RG = 1000 ohm, R2 = 10000 ohm, R3 = 10000 ohm, V1 = 1 V, V2 = 1.01 V	SOLVED	G = 51, G_stage1 = 51, Vout = 0.51 V
Instrumentation amplifier: R1 = 49400 ohm, RG = 494 ohm, R2 = 10000 ohm, R3 = 20000 ohm, V1 = 0.5 V, V2 = 0.502 V	SOLVED	G = 402, G_stage1 = 201, Vout = 0.804 V
Instrumentation amplifier: RG = 1000 ohm, R2 = 10000 ohm, R3 = 10000 ohm, V1 = 1 V, V2 = 1.01 V	NEEDS_INPUT	elx_instrumentation_amplifier needs R1 (feedback resistor of each input buffer). Example: solve('elx_instrumentation_amp
Instrumentation amplifier: R1 = 25000 ohm, RG = 1000 ohm, R2 = 10000 ohm, R3 = 10000 ohm, V1 = 1 V, V2 = 1.01 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Inverting amplifier with finite open-loop gain, `elx_inverting_amplifier_finite_gain`

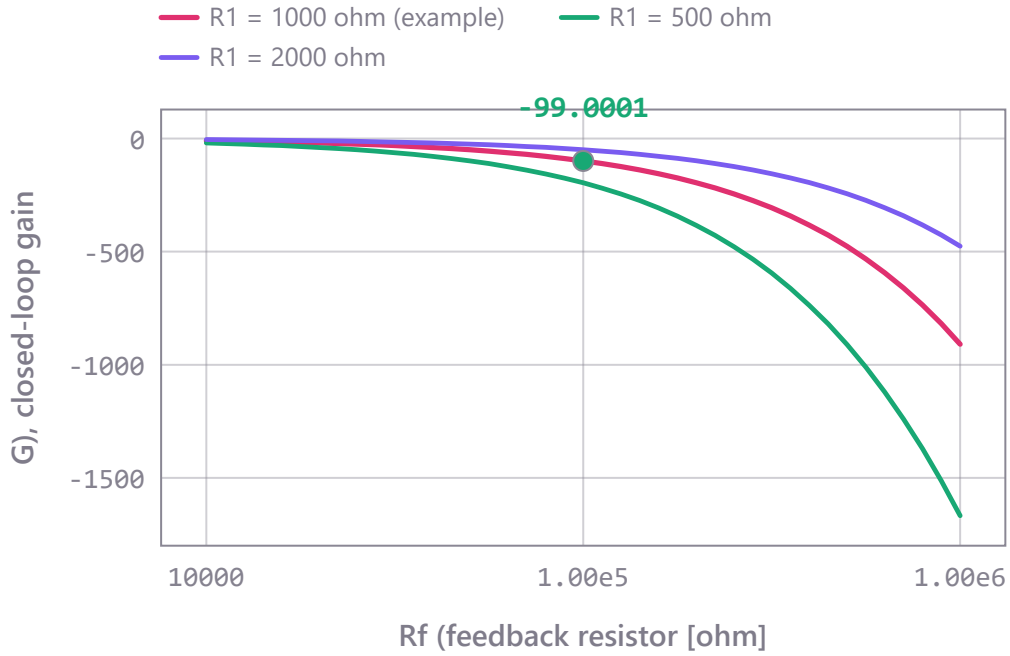
The exact closed-loop gain of an inverting amplifier built with an op-amp of finite open-loop gain A0, its relative gain error and output.

$$G = \frac{-R_f/R_1}{1 + \dfrac{1 + R_f/R_1}{A_0}}$$

input	meaning	condition
`Rf`	feedback resistor	
`R1`	input resistor	
`A0`	open-loop voltage gain (a ratio, not dB)	
`Vin`	input voltage	

output	meaning
`G`	closed-loop gain
`G_ideal`	ideal gain -Rf/R1
`Vout`	output voltage
`error`	relative gain error 1 - G/G_ideal

- **Method:** G = -$(Rf/R_1)/(1 + (1 + Rf/R_1)/A_0)$
- **Assumes:** $A_0 > 0$ (ratio); infinite input resistance, zero output resistance
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



G (closed-loop gain) against Rf (feedback resistor) from 10000 ohm to 1e+06 ohm on a logarithmic axis. One curve for each value of R1 (input resistor): 1000 ohm, 500 ohm, 2000 ohm. Other inputs as in the worked example: A0 = 10000.0, Vin = 0.01 V. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Rf = 100000 ohm, G = -99.0001.

```
electronics_engineering_ai999x.solve('elx_inverting_amplifier_finite_gain', Rf=100000.0, R1=1000.0, A0=10000.0, Vin=0.01)
# -> G = -99.0001, G_ideal = -100, Vout = -0.990001 V, error = 0.00999901   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_inverting_amplifier_finite_gain', Rf=10000.0, R1=1000.0, A0=1000.0, Vin=0.1)
# -> G = -9.8912, G_ideal = -10, Vout = -0.98912 V, error = 0.0108803   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Inverting amplifier with finite gain: Rf = 100000 ohm, R1 = 1000 ohm, A0 = 10000, Vin = 0.01 V	SOLVED	G = -99.0001, G_ideal = -100, Vout = -0.990001 V, error = 0.00999901
Inverting amplifier with finite gain: Rf = 10000 ohm, R1 = 1000 ohm, A0 = 1000, Vin = 0.1 V	SOLVED	G = -9.8912, G_ideal = -10, Vout = -0.98912 V, error = 0.0108803
Inverting amplifier with finite gain: R1 = 1000 ohm, A0 = 10000, Vin = 0.01 V	NEEDS_INPUT	elx_inverting_amplifier_finite_gain needs Rf (feedback resistor). Example: solve('elx_inverting_amplifier_finite_gain',
Inverting amplifier with finite gain: Rf = 100000 ohm, R1 = 1000 ohm, A0 = 10000, Vin = 0.01 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Op-amp integrator: output ramp and unity-gain frequency, `elx_opamp_integrator`

The output of an ideal inverting integrator after a time with a constant input, its ramp rate, and the frequency at which its gain is one.

$$V_{out}(t) = V_0 - \frac{1}{RC}\int_0^t V_{in}\,dt$$

input	meaning	condition
`R`	input resistor	
`C`	feedback capacitor	
`Vin`	constant input voltage	
`t`	time since the start	
`V0`	initial output voltage (default 0)	

output	meaning
`Vout`	output voltage at t
`slope`	output ramp rate
`f_unity`	unity-gain frequency 1/(2 pi R C)

- **Method:** Vout = V0 - Vin t/(R C); f = 1/(2 pi R C)
- **Assumes:** ideal op-amp, output not saturated

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout (output voltage at t) against R (input resistor) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of C (feedback capacitor): 1.0000e-06 F, 5.0000e-07 F, 2.0000e-06 F. Other inputs as in the worked example: Vin = 0.5 V, t = 0.002 s. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R = 10000 ohm, Vout = -0.1 V.

```
electronics_engineering_ai999x.solve('elx_opamp_integrator', R=10000.0, C=1e-06, Vin=0.5, t=0.002)
# -> Vout = -0.1 V, slope = -50 V/s, f_unity = 15.9155 Hz   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_opamp_integrator', R=100000.0, C=1e-07, Vin=-1.0,
t=0.005, V0=-2.0)
# -> Vout = -1.5 V, slope = 100 V/s, f_unity = 15.9155 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Op-amp integrator: R = 10000 ohm, C = 1 uF, Vin = 0.5 V, t = 2 ms	SOLVED	Vout = -0.1 V, slope = -50 V/s, f_unity = 15.9155 Hz
Op-amp integrator: R = 100000 ohm, C = 0.1 uF, Vin = -1 V, t = 5 ms, V0 = -2 V	SOLVED	Vout = -1.5 V, slope = 100 V/s, f_unity = 15.9155 Hz
Op-amp integrator: C = 1 uF, Vin = 0.5 V, t = 2 ms	NEEDS_INPUT	elx_opamp_integrator needs R (input resistor). Example: solve('elx_opamp_integrator', R='10000 ohm', C='1e-06 F', Vin='0
Op-amp integrator: R = 10000 ohm, C = 1 uF, Vin = 0.5 V, t = 2 ms, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Op-amp differentiator: gain and output amplitude, `elx_opamp_differentiator`

The gain |H| = 2 pi f R C of an ideal inverting differentiator at a frequency and the peak output for a sinusoidal input of given amplitude.

$$|H(j\omega)| = \omega RC$$

input	meaning	condition
`R`	feedback resistor	
`C`	input capacitor	
`A`	peak input amplitude	
`f`	input frequency	

output	meaning
`gain`	gain magnitude
`Vout_peak`	peak output voltage
`gain_dB`	gain in dB

- **Method:** |H| = 2 pi f R C; Vout,peak = |H| A
- **Assumes:** ideal op-amp (no high-frequency limiting)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout_peak (peak output voltage) against R (feedback resistor) from 5000 ohm to 15000 ohm. One curve for each value of C (input capacitor): 1.0000e-08 F, 5.0000e-09 F, 2.0000e-08 F. Other inputs as in the worked example: A = 1.0 V, f = 1000.0 Hz. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R = 10000 ohm, Vout_peak = 0.628319 V.

```
electronics_engineering_ai999x.solve('elx_opamp_differentiator', R=10000.0, C=1e-08, A=1.0,
f=1000.0)
# -> gain = 0.628319, Vout_peak = 0.628319 V, gain_dB = -4.0364   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_opamp_differentiator', R=100000.0, C=1e-09, A=0.5,
f=500.0)
# -> gain = 0.314159, Vout_peak = 0.15708 V, gain_dB = -10.057   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Op-amp differentiator: R = 10000 ohm, C = 10 nF, A = 1 V, f = 1000 Hz	SOLVED	gain = 0.628319, Vout_peak = 0.628319 V, gain_dB = -4.0364
Op-amp differentiator: R = 100000 ohm, C = 1 nF, A = 0.5 V, f = 500 Hz	SOLVED	gain = 0.314159, Vout_peak = 0.15708 V, gain_dB = -10.057
Op-amp differentiator: C = 10 nF, A = 1 V, f = 1000 Hz	NEEDS_INPUT	elx_opamp_differentiator needs R (feedback resistor). Example: solve('elx_opamp_differentiator', R='10000 ohm', C='1e-08
Op-amp differentiator: R = 10000 ohm, C = 10 nF, A = 1 V, f = 1000 Hz, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Closed-loop bandwidth from the gain-bandwidth product, `elx_closed_loop_bandwidth`

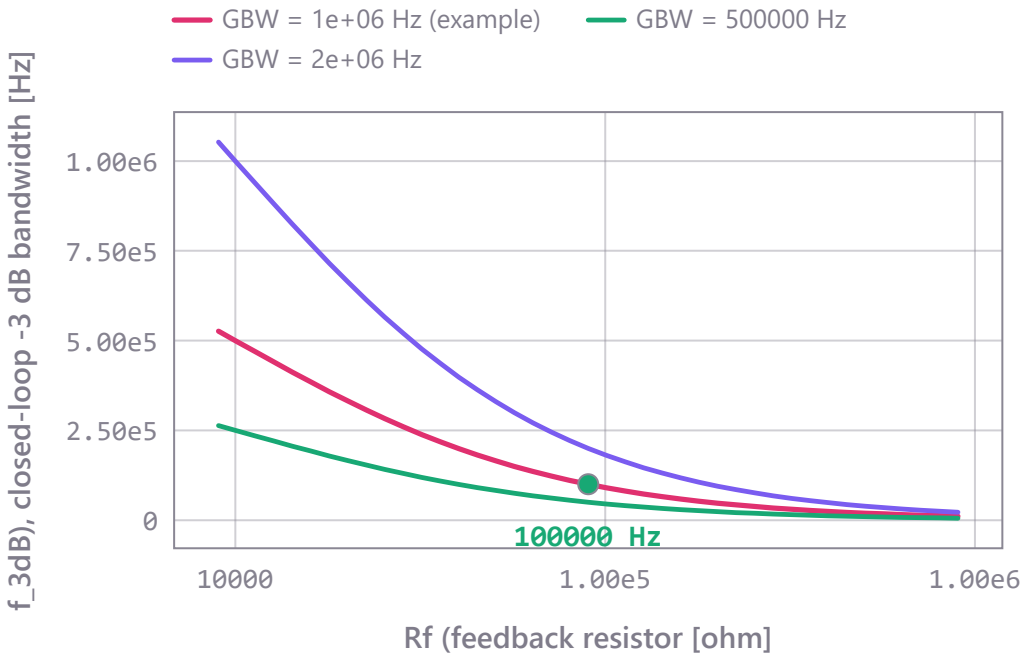
The closed-loop gain, the noise gain $1 + Rf/R_1$ and the -3 dB bandwidth GBW/(noise gain) of an inverting or non-inverting amplifier built with a single-pole op-amp.

$$f_{3dB} = \frac{\mathrm{GBW}}{1 + R_f/R_1}$$

input	meaning	condition
`GBW`	gain-bandwidth product (unity-gain frequency)	
`config`	inverting or non-inverting amplifier	
`Rf`	feedback resistor	
`R1`	input (or ground) resistor	

output	meaning
`G0`	low-frequency closed-loop gain
`noise_gain`	noise gain $1 + Rf/R_1$
`f_3dB`	closed-loop -3 dB bandwidth

- **Method:** f_3dB = GBW/(1 + Rf/R1) for both configurations
- **Assumes:** single-pole op-amp with open-loop gain much larger than the closed-loop gain
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f_3dB (closed-loop -3 dB bandwidth) against Rf (feedback resistor) from 9000 ohm to 900000 ohm on a logarithmic axis. One curve for each value of GBW (gain-bandwidth product (unity-gain frequency)): 1e+06 Hz, 500000 Hz, 2e+06 Hz. Other inputs as in the worked example: config = non-inverting, R1 = 10000.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Rf = 90000 ohm, f_3dB = 100000 Hz.

```
electronics_engineering_ai999x.solve('elx_closed_loop_bandwidth', GBW=1000000.0, config='non-inverting', Rf=90000.0, R1=10000.0)
# -> G0 = 10, noise_gain = 10, f_3dB = 100000 Hz   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_closed_loop_bandwidth', GBW=10000000.0, config='inverting', Rf=100000.0, R1=10000.0)
# -> G0 = -10, noise_gain = 11, f_3dB = 909091 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Closed-loop bandwidth: GBW = 1000000 Hz, config = non-inverting, Rf = 90000 ohm, R1 = 10000 ohm	SOLVED	G0 = 10, noise_gain = 10, f_3dB = 100000 Hz
Closed-loop bandwidth: GBW = 10000000 Hz, config = inverting, Rf = 100000 ohm, R1 = 10000 ohm	SOLVED	G0 = -10, noise_gain = 11, f_3dB = 909091 Hz
Closed-loop bandwidth: config = non-inverting, Rf = 90000 ohm, R1 = 10000 ohm	NEEDS_INPUT	elx_closed_loop_bandwidth needs GBW (gain-bandwidth product (unity-gain frequency)). Example: solve('elx_closed_loop_ban
Closed-loop bandwidth: GBW = 1000000 Hz, config = non-inverting, Rf = 90000 ohm, R1 = 10000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te
Closed-loop bandwidth: GBW = 1 MHz, config = non-inverting, Rf = 90 kohm, R1 = 10 kohm	SOLVED	G0 = 10, noise_gain = 10, f_3dB = 100000 Hz

Slew-rate limit: full-power bandwidth, `elx_slew_rate_limit`

The full-power bandwidth SR/(2 pi Vp) of an op-amp, the largest undistorted amplitude at a frequency, the slope a sine wave needs and whether it is slew-limited.

$$f_{max} = \frac{SR}{2\pi V_p}$$

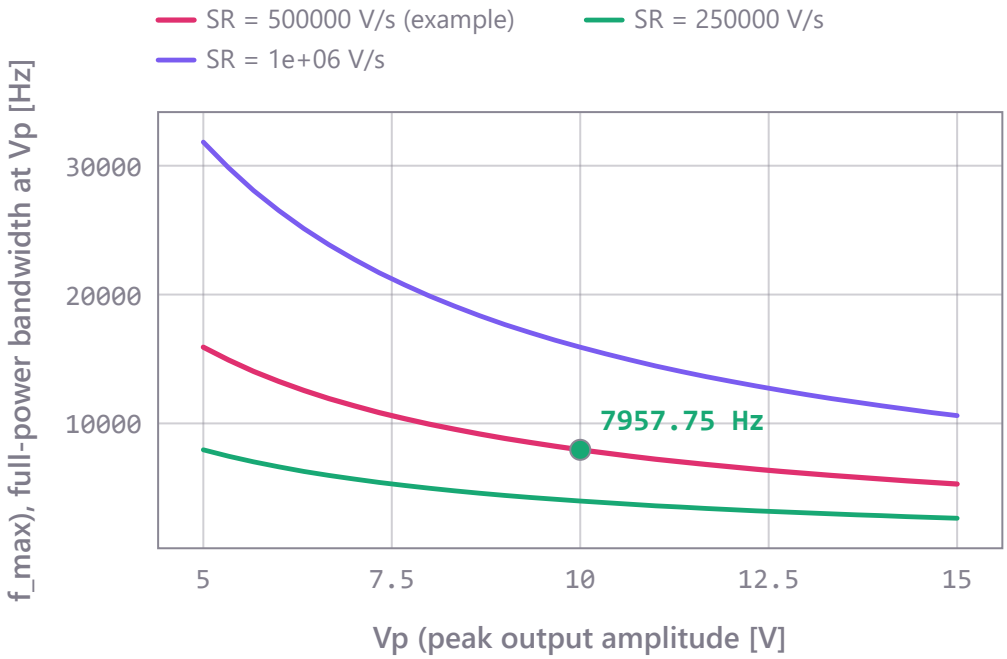
input	meaning	condition
`SR`	slew rate	
`Vp`	peak output amplitude	
`f`	signal frequency	

output	meaning
`f_max`	full-power bandwidth at Vp
`Vp_max`	largest undistorted amplitude at f
`slope_needed`	maximum slope of the sine, 2 pi f Vp
`limited`	slew-limited or not slew-limited

- **Method:** f_max = SR/(2 pi Vp); Vp,max = SR/(2 pi f); slope = 2 pi f Vp
- **Assumes:** sinusoidal output
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



slew-limited or not slew-limited (limited) over SR (slew rate) from 50000 V/s to 5e+06 V/s and Vp (peak output amplitude) from 1 V to 100 V (logarithmic axes): each of the 31 × 21 cells is one run of electronics-engineering-ai999x, coloured by the verdict it returned, not slew-limited: 208 cells, slew-limited: 443 cells. Other inputs as in the worked example: f = 20000.0 Hz. Marked: the worked example, SR = 500000 V/s, Vp = 10 V → slew-limited.



f_max (full-power bandwidth at Vp) against Vp (peak output amplitude) from 5 V to 15 V. One curve for each value of SR (slew rate): 500000 V/s, 250000 V/s, 1e+06 V/s. Other inputs as in the worked example: f = 20000.0 Hz. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Vp = 10 V, f_max = 7957.75 Hz.

```
electronics_engineering_ai999x.solve('elx_slew_rate_limit', SR=500000.0, Vp=10.0, f=20000.0)
# -> f_max = 7957.75 Hz, Vp_max = 3.97887 V, slope_needed = 1.25664e+06 V/s, limited = slew-limited
(SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_slew_rate_limit', SR=13000000.0, Vp=5.0, f=100000.0)
# -> f_max = 413803 Hz, Vp_max = 20.6901 V, slope_needed = 3.14159e+06 V/s, limited = not slew-
limited   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Slew-rate limit: SR = 0.5 V/us, Vp = 10 V, f = 20000 Hz	SOLVED	f_max = 7957.75 Hz, Vp_max = 3.97887 V, slope_needed = 1.25664e+06 V/s, limited = slew-limited
Slew-rate limit: SR = 13 V/us, Vp = 5 V, f = 100000 Hz	SOLVED	f_max = 413803 Hz, Vp_max = 20.6901 V, slope_needed = 3.14159e+06 V/s, limited = not slew-limited
Slew-rate limit: Vp = 10 V, f = 20000 Hz	NEEDS_INPUT	elx_slew_rate_limit needs SR (slew rate). Example: solve('elx_slew_rate_limit', SR='500000 V/s', Vp='10 V', f='20000 Hz'
Slew-rate limit: SR = 0.5 V/us, Vp = 10 V, f = 20000 Hz, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Op-amp output offset from input offset voltage and bias currents, `elx_opamp_dc_offset`

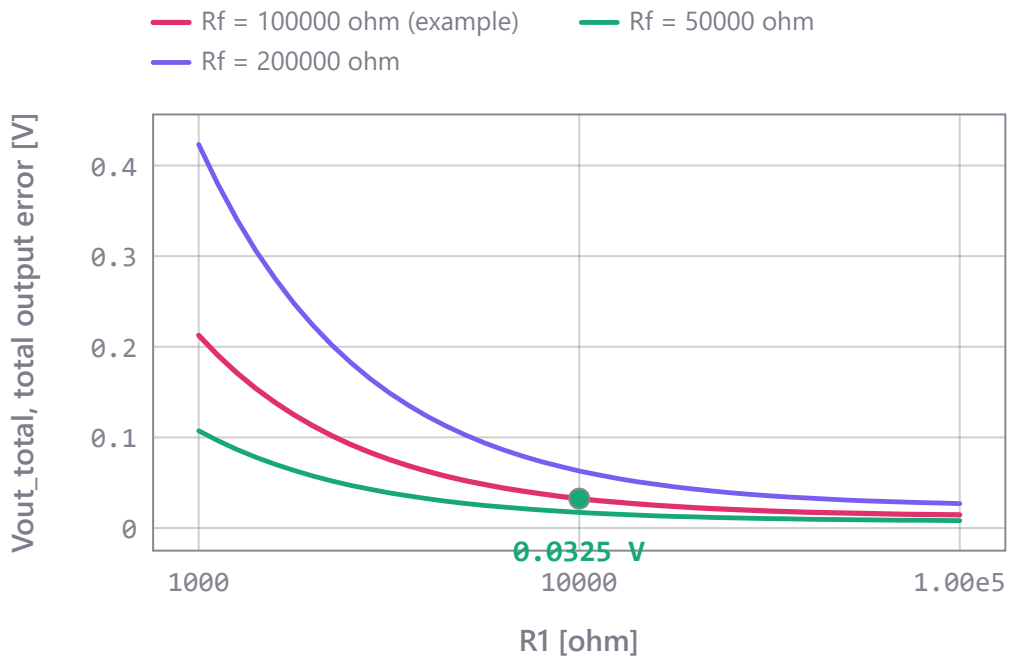
The DC output error of an inverting or non-inverting amplifier (input grounded) caused by the input offset voltage and the input bias and offset currents, and the compensating resistor that cancels the bias-current error.

$$V_{out} = (V_{os} - I_{B+}R_p)\left(1 + \frac{R_f}{R_1}\right) + I_{B-}R_f$$

input	meaning	condition
`Vos`	input offset voltage (in series with the + input)	
`IB`	average input bias current	
`Ios`	input offset current IB- minus IB+	
`Rf`	feedback resistor	
`R1`	resistor from the inverting input to ground	
`Rp`	resistor from the + input to ground (0 if none)	

output	meaning
`Vout_Vos`	output error from Vos
`Vout_bias`	output error from the bias currents
`Vout_total`	total output error
`Rp_optimal`	bias-cancelling $Rp = R_1$ || Rf

- **Method:** Vout = Vos $(1 + Rf/R_1) + IB - Rf - IB + Rp\ (1 + Rf/R_1)$
- **Assumes:** currents flow into the inputs; IB- = IB + Ios/2, IB+ = IB - Ios/2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout_total (total output error) against R1 (resistor from the inverting input to ground) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of Rf (feedback resistor): 100000 ohm, 50000 ohm, 200000 ohm. Other inputs as in the worked example: Vos = 0.002 V, IB = 1e-07 A, Ios = 1e-08 A, Rp = 0.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R1 = 10000 ohm, Vout_total = 0.0325 V.

```
electronics_engineering_ai999x.solve('elx_opamp_dc_offset', Vos=0.002, IB=1e-07, Ios=1e-08,
Rf=100000.0, R1=10000.0, Rp=0.0)
# -> Vout_Vos = 0.022 V, Vout_bias = 0.0105 V, Vout_total = 0.0325 V, Rp_optimal = 9090.91 ohm
(SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_opamp_dc_offset', Vos=0.001, IB=8e-08, Ios=2e-08,
Rf=100000.0, R1=10000.0, Rp=9100.0)
# -> Vout_Vos = 0.011 V, Vout_bias = 0.001993 V, Vout_total = 0.012993 V, Rp_optimal = 9090.91 ohm
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Op-amp output offset: Vos = 2 mV, IB = 100 nA, Ios = 10 nA, Rf = 100000 ohm, R1 = 10000 ohm, Rp = 0 ohm	SOLVED	Vout_Vos = 0.022 V, Vout_bias = 0.0105 V, Vout_total = 0.0325 V, Rp_optimal = 9090.91 ohm
Op-amp output offset: Vos = 1 mV, IB = 80 nA, Ios = 20 nA, Rf = 100000 ohm, R1 = 10000 ohm, Rp = 9100 ohm	SOLVED	Vout_Vos = 0.011 V, Vout_bias = 0.001993 V, Vout_total = 0.012993 V, Rp_optimal = 9090.91 ohm
Op-amp output offset: IB = 100 nA, Ios = 10 nA, Rf = 100000 ohm, R1 = 10000 ohm, Rp = 0 ohm	NEEDS_INPUT	elx_opamp_dc_offset needs Vos (input offset voltage (in series with the + input)). Example: solve('elx_opamp_dc_offset',
Op-amp output offset: Vos = 2 mV, IB = 100 nA, Ios = 10 nA, Rf = 100000 ohm, R1 = 10000 ohm, Rp = 0 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Inverting Schmitt trigger thresholds, `elx_schmitt_trigger`

The upper and lower switching thresholds, the hysteresis width and the centre of an inverting op-amp Schmitt trigger whose positive feedback divider (Rf from the output, R1 to a reference) sets the + input.

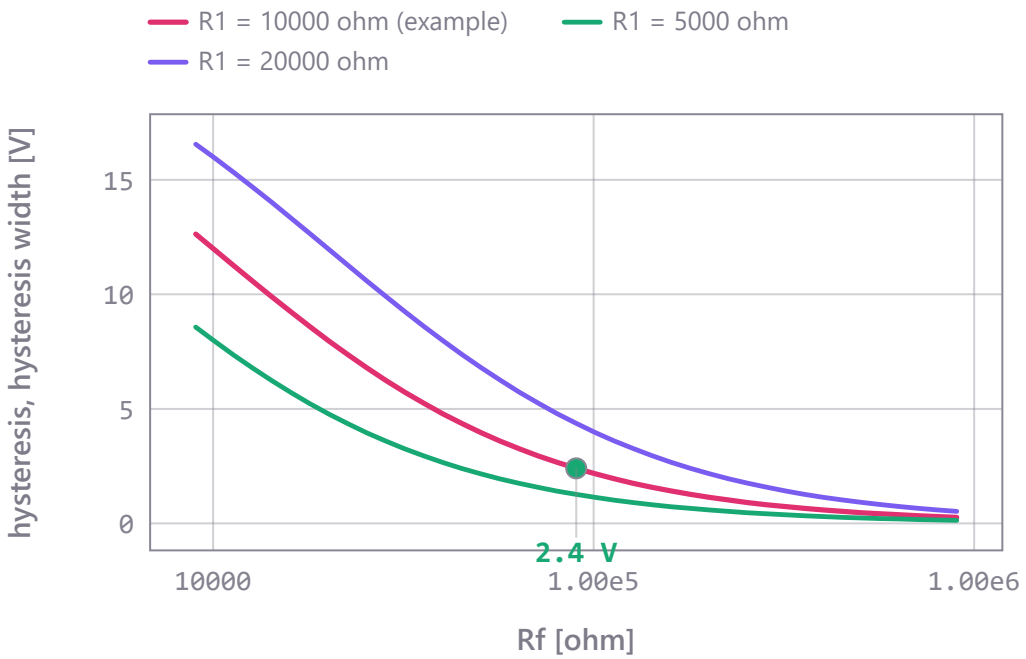
$$V_{TH,TL} = \frac{V_{sat} R_1 + V_{ref} R_f}{R_1 + R_f}$$

input	meaning	condition
`Rf`	resistor from the output to the + input	
`R1`	resistor from the + input to Vref	
`VsatH`	high output saturation voltage	
`VsatL`	low output saturation voltage (negative for a split supply)	
`Vref`	reference voltage (0 V for ground)	

output	meaning
`V_TH`	upper threshold
`V_TL`	lower threshold
`hysteresis`	hysteresis width

output	meaning
`center`	centre of the hysteresis band

- **Method:** V_T = (Vsat R1 + Vref Rf)/(R1 + Rf); hysteresis = (VsatH - VsatL) $R_1/(R_1 + Rf)$
- **Assumes:** VsatH > VsatL; input on the inverting terminal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



hysteresis (hysteresis width) against Rf (resistor from the output to the + input) from 9000 ohm to 900000 ohm on a logarithmic axis. One curve for each value of R1 (resistor from the + input to Vref): 10000 ohm, 5000 ohm, 20000 ohm. Other inputs as in the worked example: VsatH = 12.0 V, VsatL = -12.0 V, Vref = 0.0 V. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Rf = 90000 ohm, hysteresis = 2.4 V.

```
electronics_engineering_ai999x.solve('elx_schmitt_trigger', Rf=90000.0, R1=10000.0, VsatH=12.0,
VsatL=-12.0, Vref=0.0)
# -> V_TH = 1.2 V, V_TL = -1.2 V, hysteresis = 2.4 V, center = 0 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_schmitt_trigger', Rf=100000.0, R1=22000.0, VsatH=4.8,
VsatL=0.1, Vref=2.5)
# -> V_TH = 2.91475 V, V_TL = 2.06721 V, hysteresis = 0.847541 V, center = 2.49098 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Schmitt trigger thresholds: Rf = 90000 ohm, R1 = 10000 ohm, VsatH = 12 V, VsatL = -12 V, Vref = 0 V	SOLVED	V_TH = 1.2 V, V_TL = -1.2 V, hysteresis = 2.4 V, center = 0 V
Schmitt trigger thresholds: Rf = 100000 ohm, R1 = 22000 ohm, VsatH = 4.8 V, VsatL = 0.1 V, Vref = 2.5 V	SOLVED	V_TH = 2.91475 V, V_TL = 2.06721 V, hysteresis = 0.847541 V, center = 2.49098 V
Schmitt trigger thresholds: R1 = 10000 ohm, VsatH = 12 V, VsatL = -12 V, Vref = 0 V	NEEDS_INPUT	elx_schmitt_trigger needs Rf (resistor from the output to the + input). Example: solve('elx_schmitt_trigger', Rf='90000

question	outcome	result
Schmitt trigger thresholds: Rf = 90000 ohm, R1 = 10000 ohm, VsatH = 12 V, VsatL = -12 V, Vref = 0 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Negative feedback: closed-loop gain and bandwidth extension, `elx_negative_feedback`

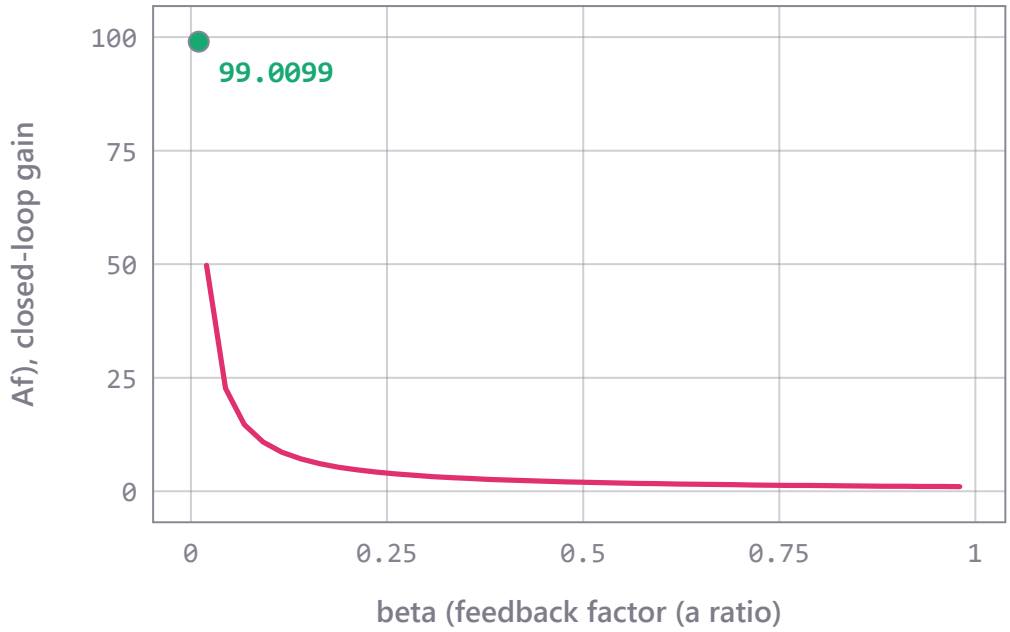
The closed-loop gain A/(1 + A beta), the loop gain, the desensitivity factor (also in dB) and the extended bandwidth of a single-pole amplifier with negative feedback.

$$A_f = \frac{A}{1 + A\beta}, \quad f_{H,f} = f_H(1 + A\beta)$$

input	meaning	condition
`A`	open-loop gain (a ratio)	
`beta`	feedback factor (a ratio)	
`fH`	open-loop -3 dB bandwidth	

output	meaning
`Af`	closed-loop gain
`loop_gain`	loop gain A beta
`desensitivity`	$1 + A$ beta
`desensitivity_dB`	amount of feedback in dB
`fH_closed`	closed-loop bandwidth

- **Method:** Af = A/(1 + A beta); fH, $f = fH$ (1 + A beta)
- **Assumes:** single-pole open-loop response; frequency-independent beta
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Af (closed-loop gain) against beta (feedback factor (a ratio)) from 0.02 to 0.98. Other inputs as in the worked example: A = 10000.0, fH = 1000.0 Hz. 41 points computed by electronics-engineering-ai999x; marked: the worked example, beta = 0.01, Af = 99.0099.

```
electronics_engineering_ai999x.solve('elx_negative_feedback', A=10000.0, beta=0.01, fH=1000.0)
# -> Af = 99.0099, loop_gain = 100, desensitivity = 101, desensitivity_dB = 40.0864, fH_closed =
101000 Hz   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_negative_feedback', A=200000.0, beta=0.1, fH=10.0)
# -> Af = 9.9995, loop_gain = 20000, desensitivity = 20001, desensitivity_dB = 86.021, fH_closed =
200010 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Negative feedback amplifier: A = 10000, beta = 0.01, fH = 1000 Hz	SOLVED	Af = 99.0099, loop_gain = 100, desensitivity = 101, desensitivity_dB = 40.0864, fH_closed = 101000 Hz
Negative feedback amplifier: A = 200000, beta = 0.1, fH = 10 Hz	SOLVED	Af = 9.9995, loop_gain = 20000, desensitivity = 20001, desensitivity_dB = 86.021, fH_closed = 200010 Hz
Negative feedback amplifier: beta = 0.01, fH = 1000 Hz	NEEDS_INPUT	elx_negative_feedback needs A (open-loop gain (a ratio)). Example: solve('elx_negative_feedback', A=10000, beta=0.01, fH
Negative feedback amplifier: A = 10000, beta = 0.01, fH = 1000 Hz, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Phase margin of a two-pole amplifier with feedback, `elx_two_pole_phase_margin`

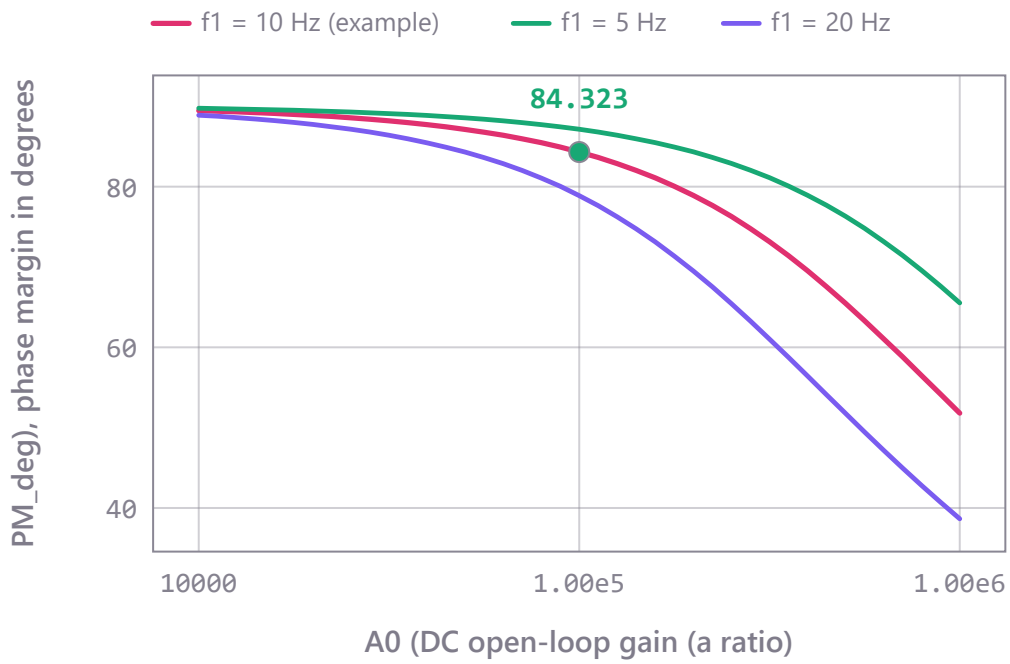
The loop-gain crossover frequency and the phase margin of an amplifier with two real poles and a frequency-independent feedback factor, and its closed-loop DC gain.

$$PM = 180° - \arctan\frac{f_c}{f_1} - \arctan\frac{f_c}{f_2}$$

input	meaning	condition
`A0`	DC open-loop gain (a ratio)	
`f1`	first pole frequency	
`f2`	second pole frequency	
`beta`	feedback factor	

output	meaning
`f_c`	loop-gain crossover frequency
`PM_deg`	phase margin in degrees
`Af`	closed-loop DC gain

- **Method:** |A0 beta|^$2 = (1 + (fc/f_1)^2)(1 + (fc/f_2)^2)$ solved for fc^2; PM = 180 - atan(fc/f1) - atan(fc/f2)
- **Assumes:** A0 $\beta > 1$; two real left-half-plane poles
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



PM_deg (phase margin in degrees) against A0 (DC open-loop gain (a ratio)) from 10000 to 1e+06 on a logarithmic axis. One curve for each value of f1 (first pole frequency): 10 Hz, 5 Hz, 20 Hz. Other inputs as in the worked example: f2 = 1000000.0 Hz, beta = 0.1. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, A0 = 100000, PM_deg = 84.323.

```
electronics_engineering_ai999x.solve('elx_two_pole_phase_margin', A0=100000.0, f1=10.0,
f2=1000000.0, beta=0.1)
# -> f_c = 99508.5 Hz, PM_deg = 84.323, Af = 9.999   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_two_pole_phase_margin', A0=20000.0, f1=100.0,
f2=200000.0, beta=1.0)
# -> f_c = 616847 Hz, PM_deg = 17.9735, Af = 0.99995   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-pole phase margin: A0 = 100000, f1 = 10 Hz, f2 = 1000000 Hz, beta = 0.1	SOLVED	f_c = 99508.5 Hz, PM_deg = 84.323, Af = 9.999
Two-pole phase margin: A0 = 20000, f1 = 100 Hz, f2 = 200000 Hz, beta = 1	SOLVED	f_c = 616847 Hz, PM_deg = 17.9735, Af = 0.99995
Two-pole phase margin: f1 = 10 Hz, f2 = 1000000 Hz, beta = 0.1	NEEDS_INPUT	elx_two_pole_phase_margin needs A0 (DC open-loop gain (a ratio)). Example: solve('elx_two_pole_phase_margin', A0=100000,
Two-pole phase margin: A0 = 100000, f1 = 10 Hz, f2 = 1000000 Hz, beta = 0.1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

First-order active low-pass filter (inverting), `elx_first_order_active_lowpass`

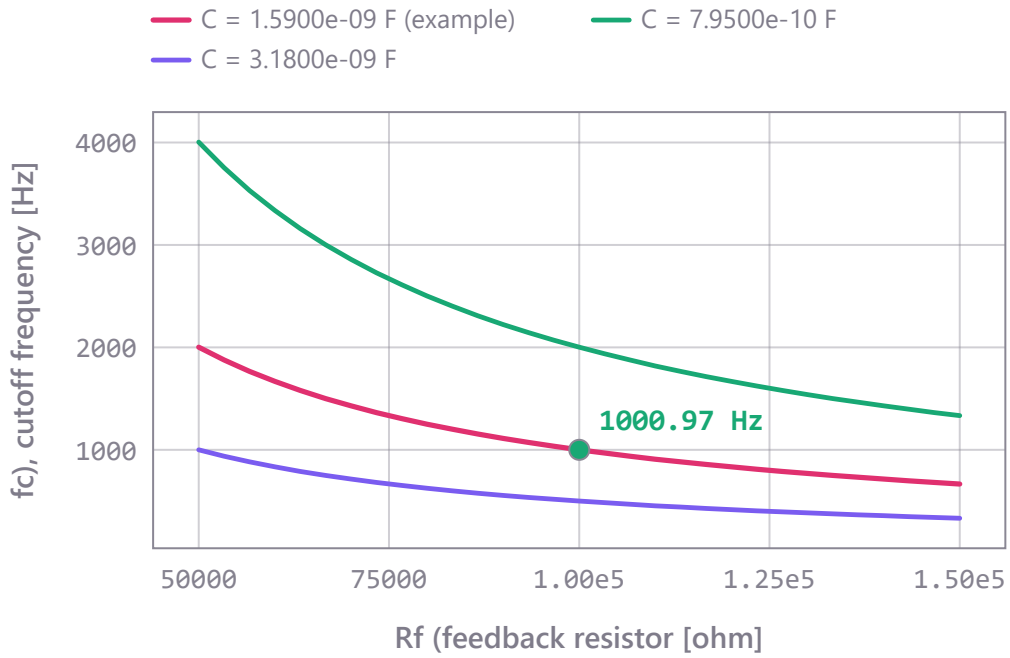
The passband gain -Rf/R1, the cutoff frequency 1/(2 pi Rf C) and the gain at a frequency of an inverting op-amp low-pass filter with C across the feedback resistor.

$$H(j\omega) = -\frac{R_f/R_1}{1 + j\omega R_f C}, \quad f_c = \frac{1}{2\pi R_f C}$$

input	meaning	condition
`R1`	input resistor	
`Rf`	feedback resistor	
`C`	capacitor across Rf	
`f`	frequency of interest	

output	meaning
`H0`	passband gain
`fc`	cutoff frequency
`gain_f`	gain magnitude at f
`gain_dB`	gain at f in dB

- **Method:** H(s) = -(Rf/R1)/(1 + s Rf C); fc = 1/(2 pi Rf C)
- **Assumes:** ideal op-amp
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



fc (cutoff frequency) against Rf (feedback resistor) from 50000 ohm to 150000 ohm. One curve for each value of C (capacitor across Rf): 1.5900e-09 F, 7.9500e-10 F, 3.1800e-09 F. Other inputs as in the worked example: R1 = 10000.0 ohm, f = 1000.0 Hz. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Rf = 100000 ohm, fc = 1000.97 Hz.

```
electronics_engineering_ai999x.solve('elx_first_order_active_lowpass', R1=10000.0, Rf=100000.0, C=1.59e-09, f=1000.0)
# -> H0 = -10, fc = 1000.97 Hz, gain_f = 7.07451, gain_dB = 16.9939   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_first_order_active_lowpass', R1=4700.0, Rf=47000.0, C=1e-08, f=500.0)
# -> H0 = -10, fc = 338.628 Hz, gain_f = 5.60755, gain_dB = 14.9755   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
First-order active low-pass filter: R1 = 10000 ohm, Rf = 100000 ohm, C = 1.59 nF, f = 1000 Hz	SOLVED	H0 = -10, fc = 1000.97 Hz, gain_f = 7.07451, gain_dB = 16.9939
First-order active low-pass filter: R1 = 4700 ohm, Rf = 47000 ohm, C = 10 nF, f = 500 Hz	SOLVED	H0 = -10, fc = 338.628 Hz, gain_f = 5.60755, gain_dB = 14.9755
First-order active low-pass filter: Rf = 100000 ohm, C = 1.59 nF, f = 1000 Hz	NEEDS_INPUT	elx_first_order_active_lowpass needs R1 (input resistor). Example: solve('elx_first_order_active_lowpass', R1='10000 ohm
First-order active low-pass filter: R1 = 10000 ohm, Rf = 100000 ohm, C = 1.59 nF, f = 1000 Hz, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Sallen-Key low-pass filter: f0, Q and gain, `elx_sallen_key_lowpass`

The natural frequency, quality factor, passband gain and the gain at f0 (K Q) of a second-order Sallen-Key low-pass filter with any components and amplifier gain K.

$$\omega_0 = \frac{1}{\sqrt{R_1 R_2 C_1 C_2}}, \quad Q = \frac{\sqrt{R_1 R_2 C_1 C_2}}{R_1 C_2 + R_2 C_2 + R_1 C_1 (1 - K)}$$

input	meaning	condition
`R1`	first series resistor	
`R2`	second series resistor	
`C1`	capacitor from the middle node to the output (feedback)	
`C2`	capacitor from the amplifier input to ground	
`K`	amplifier gain (1 for a unity-gain buffer)	

output	meaning
`f0`	natural frequency
`Q`	quality factor
`H0`	passband gain
`peak_gain`	gain at f0, K Q

- **Method:** $w_0 = 1/sqrt$(R1 R2 C1 C2); $Q = sqrt$(R1 R2 C1 C_2)/(R_1 C_2 + R_2 C_2 + R_1 C1 $(1 - K)$)
- **Assumes:** stable: R1 C_2 + R_2 C_2 + R_1 C1 $(1 - K) > 0$; ideal amplifier
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (quality factor) against R1 (first series resistor), with C1 = 2.00e-8 F, C2 = 1.00e-8 F, K = 1, R2 = 10000 ohm. 31 points computed by electronics-engineering-ai999x; marked: R1 = 10000, Q = 0.707107.



Q (quality factor) against R1 (first series resistor) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of K (amplifier gain (1 for a unity-gain buffer)): 1, 0.5, 2. Other inputs as in the worked example: R2 = 10000.0 ohm, C1 = 2e-08 F, C2 = 1e-08 F. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R1 = 10000 ohm, Q = 0.707107.

```
electronics_engineering_ai999x.solve('elx_sallen_key_lowpass', R1=10000.0, R2=10000.0, C1=2e-08, C2=1e-08, K=1.0)
# -> f0 = 1125.4 Hz, Q = 0.707107, H0 = 1, peak_gain = 0.707107   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_sallen_key_lowpass', R1=15000.0, R2=15000.0, C1=1e-08, C2=1e-08, K=1.586)
# -> f0 = 1061.03 Hz, Q = 0.707214, H0 = 1.586, peak_gain = 1.12164   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sallen-Key low-pass filter: R1 = 10000 ohm, R2 = 10000 ohm, C1 = 20 nF, C2 = 10 nF, K = 1	SOLVED	f0 = 1125.4 Hz, Q = 0.707107, H0 = 1, peak_gain = 0.707107
Sallen-Key low-pass filter: R1 = 15000 ohm, R2 = 15000 ohm, C1 = 10 nF, C2 = 10 nF, K = 1.586	SOLVED	f0 = 1061.03 Hz, Q = 0.707214, H0 = 1.586, peak_gain = 1.12164
Sallen-Key low-pass filter: R2 = 10000 ohm, C1 = 20 nF, C2 = 10 nF, K = 1	NEEDS_INPUT	elx_sallen_key_lowpass needs R1 (first series resistor). Example: solve('elx_sallen_key_lowpass', R1='10000 ohm', R2='10
Sallen-Key low-pass filter: R1 = 10000 ohm, R2 = 10000 ohm, C1 = 20 nF, C2 = 10 nF, K = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Sallen-Key high-pass filter: f0, Q and gain, `elx_sallen_key_highpass`

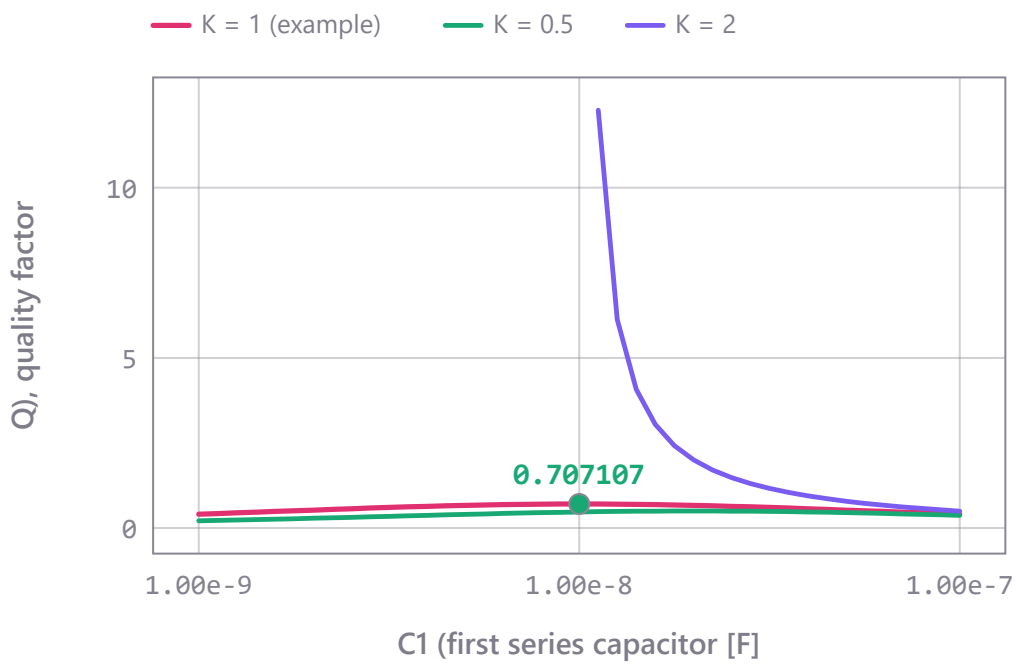
The natural frequency, quality factor and high-frequency gain of a second-order Sallen-Key high-pass filter with any components and amplifier gain K.

$$Q = \frac{\sqrt{R_1 R_2 C_1 C_2}}{R_1(C_1 + C_2) + R_2 C_2 (1 - K)}$$

input	meaning	condition
`C1`	first series capacitor	
`C2`	second series capacitor	
`R1`	resistor from the middle node to the output (feedback)	
`R2`	resistor from the amplifier input to ground	
`K`	amplifier gain	

output	meaning
`f0`	natural frequency
`Q`	quality factor
`H_inf`	high-frequency gain

- **Method:** $w_0 = 1/sqrt$(R1 R2 C1 C2); $Q = sqrt$(R1 R2 C1 $C_2)/(R_1\ (C_1 + C_2) + R_2$ C2 $(1 - K)$)
- **Assumes:** stable: R1 $(C_1 + C_2) + R_2$ C2 $(1 - K) > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (quality factor) against C1 (first series capacitor) from 1.0000e-09 F to 1.0000e-07 F on a logarithmic axis. One curve for each value of K (amplifier gain): 1, 0.5, 2. Other inputs as in the worked example: C2 = 1e-08 F, R1 = 11250.0 ohm, R2 = 22500.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, C1 = 1.0000e-08 F, Q = 0.707107.

```
electronics_engineering_ai999x.solve('elx_sallen_key_highpass', C1=1e-08, C2=1e-08, R1=11250.0,
R2=22500.0, K=1.0)
# -> f0 = 1000.35 Hz, Q = 0.707107, H_inf = 1   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_sallen_key_highpass', C1=2.2e-08, C2=2.2e-08, R1=10000.0,
R2=10000.0, K=2.0)
# -> f0 = 723.432 Hz, Q = 1, H_inf = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sallen-Key high-pass filter: C1 = 10 nF, C2 = 10 nF, R1 = 11250 ohm, R2 = 22500 ohm, K = 1	SOLVED	f0 = 1000.35 Hz, Q = 0.707107, H_inf = 1
Sallen-Key high-pass filter: C1 = 22 nF, C2 = 22 nF, R1 = 10000 ohm, R2 = 10000 ohm, K = 2	SOLVED	f0 = 723.432 Hz, Q = 1, H_inf = 2
Sallen-Key high-pass filter: C2 = 10 nF, R1 = 11250 ohm, R2 = 22500 ohm, K = 1	NEEDS_INPUT	elx_sallen_key_highpass needs C1 (first series capacitor). Example: solve('elx_sallen_key_highpass', C1='1e-08 F', C2='1
Sallen-Key high-pass filter: C1 = 10 nF, C2 = 10 nF, R1 = 11250 ohm, R2 = 22500 ohm, K = 1, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Multiple-feedback (MFB) low-pass filter, `elx_mfb_lowpass`

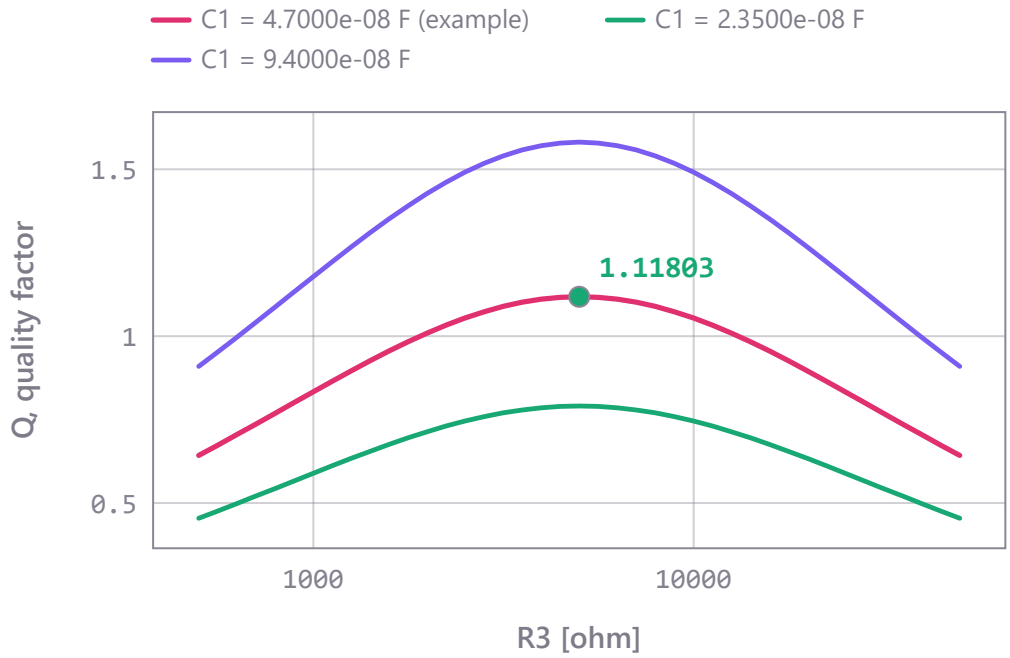
The natural frequency, quality factor and inverting passband gain -R2/R1 of a second-order multiple-feedback low-pass filter.

$$Q = \frac{\sqrt{R_2 R_3 C_1 C_2}}{C_2\left(R_2 + R_3 + R_2 R_3 / R_1\right)}, \quad H_0 = -\frac{R_2}{R_1}$$

input	meaning	condition
`R1`	input resistor	
`R2`	feedback resistor (middle node to output)	
`R3`	resistor from the middle node to the inverting input	
`C1`	capacitor from the middle node to ground	
`C2`	capacitor from the inverting input to the output	

output	meaning
`f0`	natural frequency
`Q`	quality factor
`H0`	passband gain

- **Method:** $w_0 = 1/sqrt$(R2 R3 C1 C2); $Q = sqrt$(R2 R3 C1 C_2)/(C_2 (R2 + R3 + R2 R3/R1)); $H_0 = -R_2/R_1$
- **Assumes:** ideal op-amp
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (quality factor) against R3 (resistor from the middle node to the inverting input) from 500 ohm to 50000 ohm on a logarithmic axis. One curve for each value of C1 (capacitor from the middle node to ground): 4.7000e-08 F, 2.3500e-08 F, 9.4000e-08 F. Other inputs as in the worked example: R1 = 10000.0 ohm, R2 = 10000.0 ohm, C2 = 4.7e-09 F. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R3 = 5000 ohm, Q = 1.11803.

```
electronics_engineering_ai999x.solve('elx_mfb_lowpass', R1=10000.0, R2=10000.0, R3=5000.0, C1=4.7e-08, C2=4.7e-09)
# -> f0 = 1514.39 Hz, Q = 1.11803, H0 = -1   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_mfb_lowpass', R1=4700.0, R2=47000.0, R3=10000.0, C1=1e-07, C2=1e-09)
# -> f0 = 734.127 Hz, Q = 1.38086, H0 = -10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MFB low-pass filter: R1 = 10000 ohm, R2 = 10000 ohm, R3 = 5000 ohm, C1 = 47 nF, C2 = 4.7 nF	SOLVED	f0 = 1514.39 Hz, Q = 1.11803, H0 = -1
MFB low-pass filter: R1 = 4700 ohm, R2 = 47000 ohm, R3 = 10000 ohm, C1 = 100 nF, C2 = 1 nF	SOLVED	f0 = 734.127 Hz, Q = 1.38086, H0 = -10
MFB low-pass filter: R2 = 10000 ohm, R3 = 5000 ohm, C1 = 47 nF, C2 = 4.7 nF	NEEDS_INPUT	elx_mfb_lowpass needs R1 (input resistor). Example: solve('elx_mfb_lowpass', R1='10000 ohm', R2='10000 ohm', R3='5000 oh
MFB low-pass filter: R1 = 10000 ohm, R2 = 10000 ohm, R3 = 5000 ohm, C1 = 47 nF, C2 = 4.7 nF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Multiple-feedback (MFB) band-pass filter, `elx_mfb_bandpass`

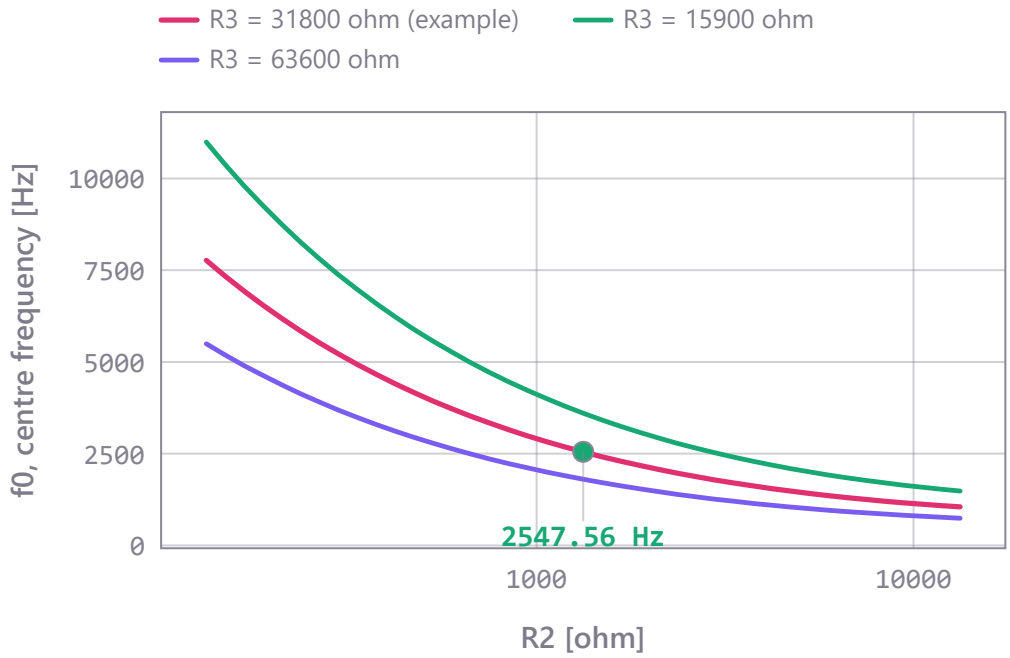
The centre frequency, quality factor, -3 dB bandwidth and peak (inverting) gain of a multiple-feedback band-pass filter.

$$\omega_0 = \sqrt{\frac{R_1 + R_2}{R_1 R_2 R_3 C_1 C_2}}, \quad Q = \frac{\omega_0 C_1 C_2 R_3}{C_1 + C_2}$$

input	meaning	condition
`R1`	input resistor	
`R2`	resistor from the middle node to ground	
`R3`	feedback resistor (inverting input to output)	
`C1`	capacitor from the middle node to the inverting input	
`C2`	capacitor from the middle node to the output	

output	meaning
`f0`	centre frequency
`Q`	quality factor
`BW`	-3 dB bandwidth
`H_peak`	gain at the centre frequency

- **Method:** $w_0 = sqrt$((R1 + R2)/(R1 R2 R3 C1 C2)); $Q = w_0$ C1 C2 $R_3/(C_1 + C_2)$; $H = -C_1$ R3/(R1 $(C_1 + C_2)$)
- **Assumes:** ideal op-amp
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f0 (centre frequency) against R2 (resistor from the middle node to ground) from 133 ohm to 13300 ohm on a logarithmic axis. One curve for each value of R3 (feedback resistor (inverting input to output)): 31800 ohm, 15900 ohm, 63600 ohm. Other inputs as in the worked example: R1 = 15900.0 ohm, C1 = 1e-08 F, C2 = 1e-08 F. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R2 = 1330 ohm, f0 = 2547.56 Hz.

```
electronics_engineering_ai999x.solve('elx_mfb_bandpass', R1=15900.0, R2=1330.0, R3=31800.0, C1=1e-08, C2=1e-08)
# -> f0 = 2547.56 Hz, Q = 2.54508, BW = 1000.97 Hz, H_peak = -1   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_mfb_bandpass', R1=10000.0, R2=470.0, R3=100000.0, C1=2.2e-08, C2=2.2e-08)
# -> f0 = 1079.75 Hz, Q = 7.46267, BW = 144.686 Hz, H_peak = -5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MFB band-pass filter: R1 = 15900 ohm, R2 = 1330 ohm, R3 = 31800 ohm, C1 = 10 nF, C2 = 10 nF	SOLVED	f0 = 2547.56 Hz, Q = 2.54508, BW = 1000.97 Hz, H_peak = -1
MFB band-pass filter: R1 = 10000 ohm, R2 = 470 ohm, R3 = 100000 ohm, C1 = 22 nF, C2 = 22 nF	SOLVED	f0 = 1079.75 Hz, Q = 7.46267, BW = 144.686 Hz, H_peak = -5
MFB band-pass filter: R2 = 1330 ohm, R3 = 31800 ohm, C1 = 10 nF, C2 = 10 nF	NEEDS_INPUT	elx_mfb_bandpass needs R1 (input resistor). Example: solve('elx_mfb_bandpass', R1='15900 ohm', R2='1330 ohm', R3='31800
MFB band-pass filter: R1 = 15900 ohm, R2 = 1330 ohm, R3 = 31800 ohm, C1 = 10 nF, C2 = 10 nF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Equal-component Sallen-Key low-pass design, `elx_sallen_key_equal_component_design`

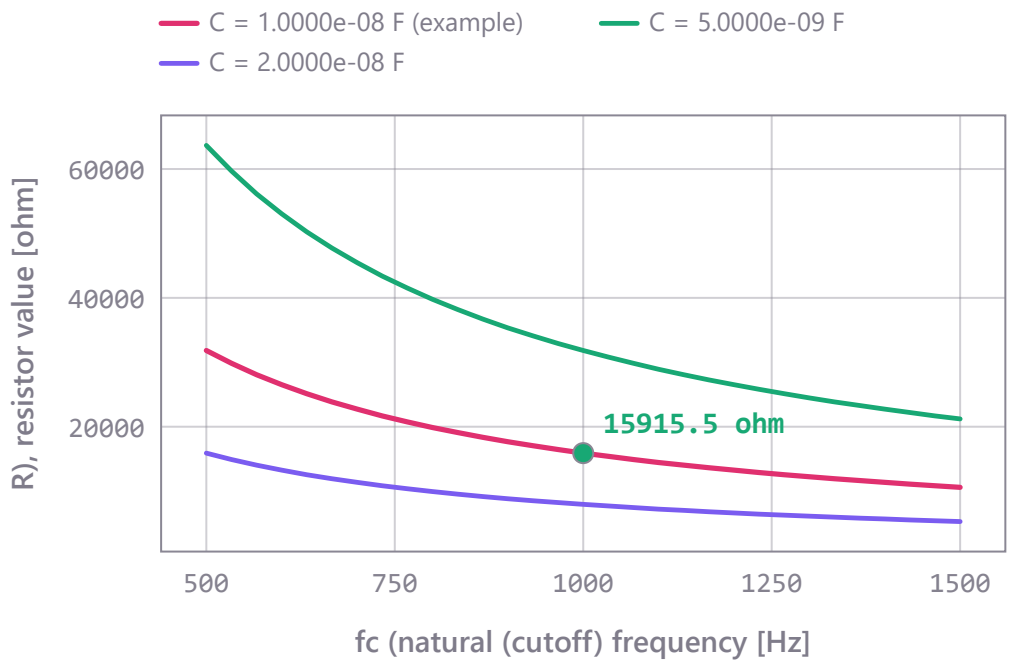
The resistor value and amplifier gain of an equal-component (R1 = R2 = R, C1 = C2 = C) Sallen-Key low-pass filter for a cutoff frequency and Q, with the non-inverting gain ratio $Rf/Rg = K - 1$.

$$R = \frac{1}{2\pi f_c C}, \quad K = 3 - \frac{1}{Q}$$

input	meaning	condition
`fc`	natural (cutoff) frequency	
`Q`	quality factor (0.7071 for Butterworth)	
`C`	chosen capacitor value	

output	meaning
`R`	resistor value
`K`	amplifier gain
`Rf_over_Rg`	gain resistor ratio Rf/Rg

- **Method:** R = 1/(2 pi fc C); $K = 3 - 1/Q$; $Rf/Rg = K - 1$
- **Assumes:** $0.5 \leq Q \leq 20$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



R (resistor value) against fc (natural (cutoff) frequency) from 500 Hz to 1500 Hz. One curve for each value of C (chosen capacitor value): 1.0000e-08 F, 5.0000e-09 F, 2.0000e-08 F. Other inputs as in the worked example: Q = 0.7071. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, fc = 1000 Hz, R = 15915.5 ohm.

```
electronics_engineering_ai999x.solve('elx_sallen_key_equal_component_design', fc=1000.0, Q=0.7071,
C=1e-08)
# -> R = 15915.5 ohm, K = 1.58577, Rf_over_Rg = 0.585773   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_sallen_key_equal_component_design', fc=5000.0, Q=2.0,
C=2.2e-09)
# -> R = 14468.6 ohm, K = 2.5, Rf_over_Rg = 1.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Equal-component Sallen-Key design: fc = 1000 Hz, Q = 0.7071, C = 10 nF	SOLVED	R = 15915.5 ohm, K = 1.58577, Rf_over_Rg = 0.585773
Equal-component Sallen-Key design: fc = 5000 Hz, Q = 2, C = 2.2 nF	SOLVED	R = 14468.6 ohm, K = 2.5, Rf_over_Rg = 1.5
Equal-component Sallen-Key design: Q = 0.7071, C = 10 nF	NEEDS_INPUT	elx_sallen_key_equal_component_design needs fc (natural (cutoff) frequency). Example: solve('elx_sallen_key_equal_compon
Equal-component Sallen-Key design: fc = 1000 Hz, Q = 0.7071, C = 10 nF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Butterworth or Chebyshev low-pass filter order, `elx_filter_order`

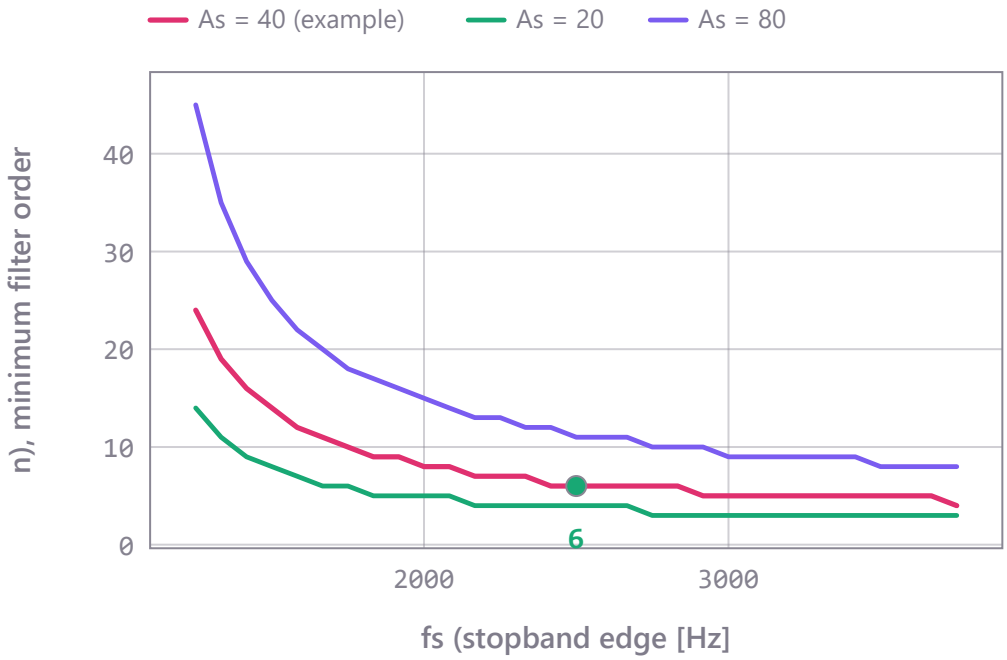
The minimum order of a Butterworth or Chebyshev type I low-pass filter meeting a passband edge with at most Ap dB attenuation and a stopband edge with at least As dB.

$$n \geq \frac{\log_{10} D}{2 \log_{10}(f_s/f_p)}, \quad n \geq \frac{\cosh^{-1}\sqrt{D}}{\cosh^{-1}(f_s/f_p)}$$

input	meaning	condition
`ftype`	butterworth or chebyshev (type I)	
`fp`	passband edge	
`fs`	stopband edge	
`Ap`	maximum passband attenuation (ripple) in dB	
`As`	minimum stopband attenuation in dB	

output	meaning
`n`	minimum filter order

- **Method:** Butterworth: $n \geq log10(D)/(2$ log10(fs/fp)); Chebyshev: n >= acosh(sqrt D)/acosh(fs/fp), $D = (10^{As/10} - 1)/(10^{Ap/10} - 1)$
- **Assumes:** $fs > fp$; $As > Ap > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



n (minimum filter order) against fs (stopband edge) from 1250 Hz to 3750 Hz. One curve for each value of As (minimum stopband attenuation in dB): 40, 20, 80. Other inputs as in the worked example: ftype = Butterworth, fp = 1000.0 Hz, Ap = 1.0. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, fs = 2500 Hz, n = 6.

```
electronics_engineering_ai999x.solve('elx_filter_order', ftype='Butterworth', fp=1000.0, fs=2500.0,
Ap=1.0, As=40.0)
# -> n = 6   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_filter_order', ftype='Chebyshev', fp=1000.0, fs=2500.0,
Ap=1.0, As=40.0)
# -> n = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Filter order: ftype = Butterworth, fp = 1000 Hz, fs = 2500 Hz, Ap = 1 dB, As = 40 dB	SOLVED	n = 6
Filter order: ftype = Chebyshev, fp = 1000 Hz, fs = 2500 Hz, Ap = 1 dB, As = 40 dB	SOLVED	n = 4
Filter order: fp = 1000 Hz, fs = 2500 Hz, Ap = 1 dB, As = 40 dB	NEEDS_INPUT	elx_filter_order needs ftype (butterworth or chebyshev (type I)). Example: solve('elx_filter_order', ftype='Butterworth'
Filter order: ftype = Butterworth, fp = 1000 Hz, fs = 2500 Hz, Ap = 1 dB, As = 40 dB, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te
Filter order: type = Butterworth, fp = 1 kHz, fs = 2.5 kHz, Ap = 1 dB, As = 40 dB	SOLVED	n = 6
Filter order: ftype = butterworth, fp = 1 kHz, fs = 2.5 kHz, Ap = 1, As = 40 dB	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Butterworth filter: Q of each second-order section, `elx_butterworth_section_q`

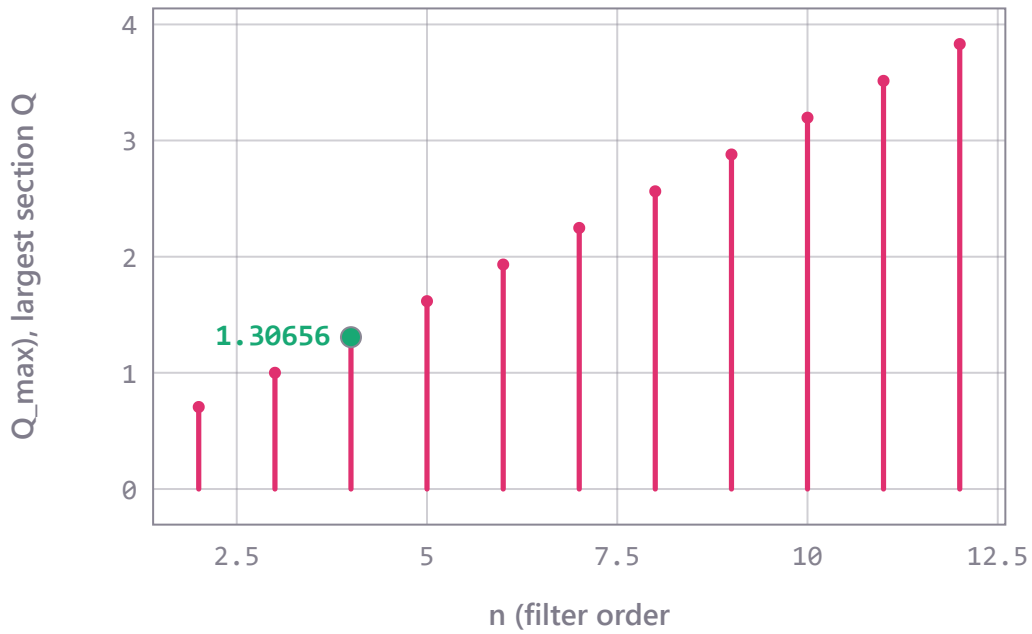
The quality factors of the cascaded second-order sections of an n-th order Butterworth filter (all at the cutoff frequency), and whether a first-order section is needed.

$$Q_k = \frac{1}{2\sin\left(\frac{(2k-1)\pi}{2n}\right)}$$

input	meaning	condition
`n`	filter order	

output	meaning
`Q_max`	largest section Q
`Q`	section Q values, ascending
`sections`	number of second-order sections
`first_order`	1 if a first-order section is needed

- **Method:** Q_k = 1/(2 sin((2k − 1) pi/(2n))), $k = 1$ .. floor(n/2)
- **Assumes:** $2 \le n \le 20$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q_max (largest section Q) against n (filter order) from 1 to 12. 11 points computed by electronics-engineering-ai999x; marked: the worked example, n = 4, Q_max = 1.30656.

```
electronics_engineering_ai999x.solve('elx_butterworth_section_q', n=4)
# -> Q_max = 1.30656, Q = [0.541196100146, 1.30656296488], sections = 2, first_order = 0   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_butterworth_section_q', n=5)
# -> Q_max = 1.61803, Q = [0.61803398875, 1.61803398875], sections = 2, first_order = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Butterworth section Q: n = 4	SOLVED	Q_max = 1.30656, Q = [0.541196100146, 1.30656296488], sections = 2, first_order = 0
Butterworth section Q: n = 5	SOLVED	Q_max = 1.61803, Q = [0.61803398875, 1.61803398875], sections = 2, first_order = 1
Butterworth section Q:	NEEDS_INPUT	elx_butterworth_section_q needs n (filter order). Example: solve('elx_butterworth_section_q', n=4).
Butterworth section Q: n = 4, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Chebyshev type I filter: section Q and frequencies, `elx_chebyshev_sections`

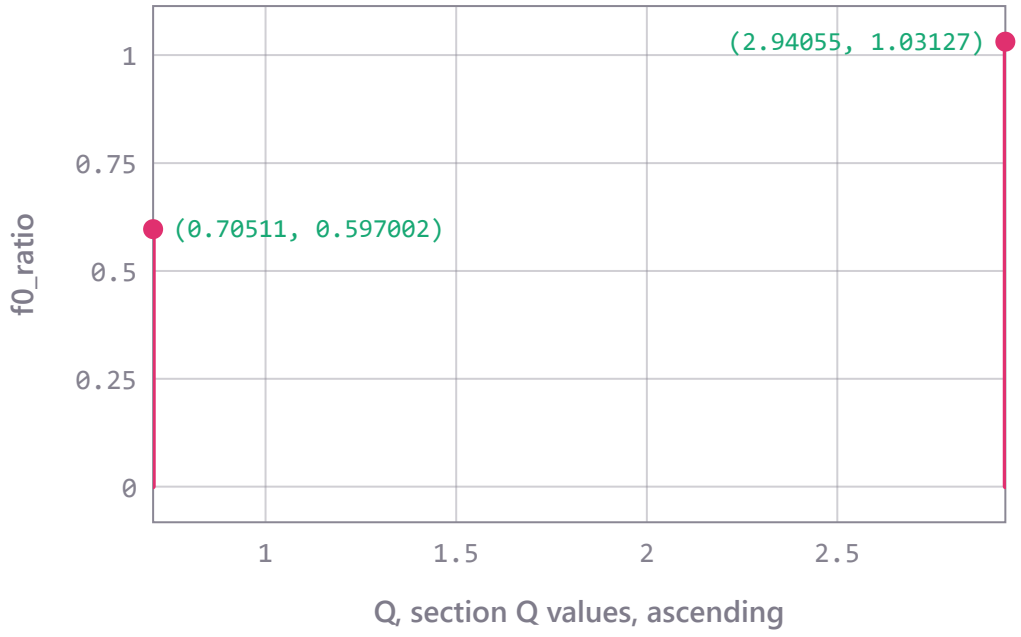
The quality factors and natural frequencies (relative to the passband edge) of the second-order sections of an n-th order Chebyshev type I low-pass filter with a given passband ripple, and the real pole of an odd order.

$$p_k = -\sinh a \sin\theta_k + j \cosh a \cos\theta_k,\ a = \frac{1}{n}\sinh^{-1}\frac{1}{\varepsilon},\ Q_k = \frac{|p_k|}{2|\Re p_k|}$$

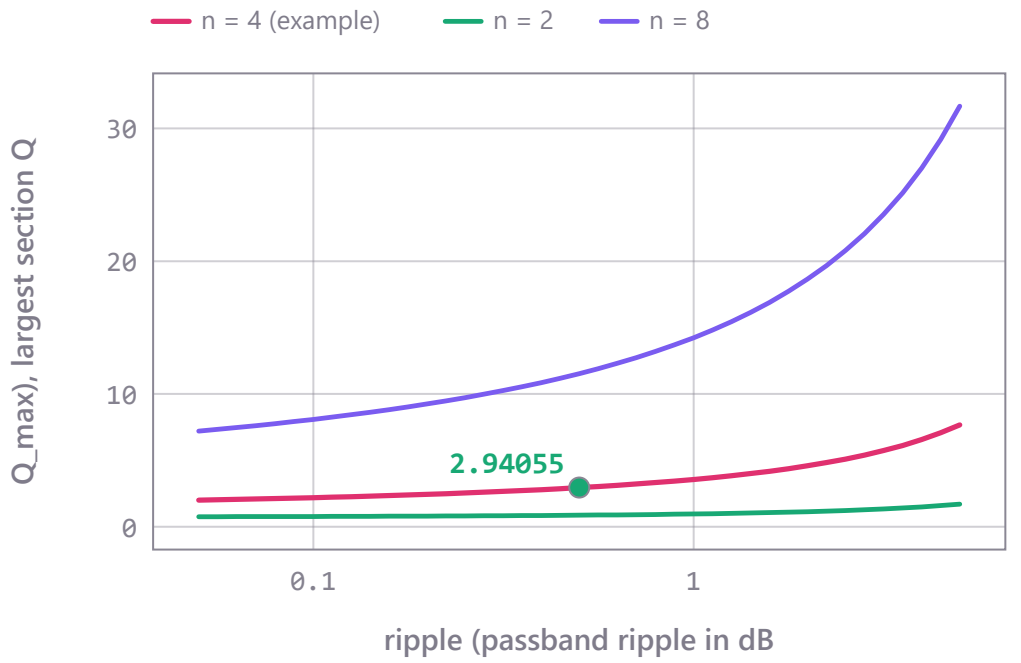
input	meaning	condition
`n`	filter order	
`ripple`	passband ripple in dB	

output	meaning
`Q_max`	largest section Q
`Q`	section Q values, ascending
`f0_ratio`	section natural frequencies / passband edge, in the same order
`sections`	number of second-order sections
`real_pole`	real pole / passband edge (0 for even n)

- **Method:** $eps = \sqrt{10^{\frac{r}{10}} - 1}$; a = asinh(1/eps)/n; poles -sinh a sin th_k + j cosh a cos th_k; Q = |p|/(2 |Re p|)
- **Assumes:** $2 \le n \le 16$; 0.001 dB <= ripple <= 6 dB
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



section natural frequencies / passband edge, in the same order (f0_ratio) at each section Q values, ascending (Q), both returned by electronics-engineering-ai999x for the worked example (n = 4, ripple = 0.5): 0.70511 → 0.597002; 2.94055 → 1.03127.



Q_max (largest section Q) against ripple (passband ripple in dB) from 0.05 to 5 on a logarithmic axis. One curve for each value of n (filter order): 4, 2, 8. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, ripple = 0.5, Q_max = 2.94055.

```
electronics_engineering_ai999x.solve('elx_chebyshev_sections', n=4, ripple=0.5)
# -> Q_max = 2.94055, Q = [0.705110236803, 2.94055417439], f0_ratio = [0.59700239512,
1.03127040148], sections = 2, real_pole = 0   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_chebyshev_sections', n=3, ripple=1.0)
# -> Q_max = 2.01772, Q = [2.01772034366], f0_ratio = [0.997098082833], sections = 1, real_pole =
0.494171   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Chebyshev filter sections: n = 4, ripple = 0.5 dB	SOLVED	Q_max = 2.94055, Q = [0.705110236803, 2.94055417439], f0_ratio = [0.59700239512, 1.03127040148], sections = 2, real_pole = 0
Chebyshev filter sections: n = 3, ripple = 1 dB	SOLVED	Q_max = 2.01772, Q = [2.01772034366], f0_ratio = [0.997098082833], sections = 1, real_pole = 0.494171
Chebyshev filter sections: ripple = 0.5 dB	NEEDS_INPUT	elx_chebyshev_sections needs n (filter order). Example: solve('elx_chebyshev_sections', n=4, ripple=0.5).
Chebyshev filter sections: n = 4, ripple = 0.5 dB, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

RC phase-shift oscillator frequency and gain, `elx_rc_phase_shift_oscillator`

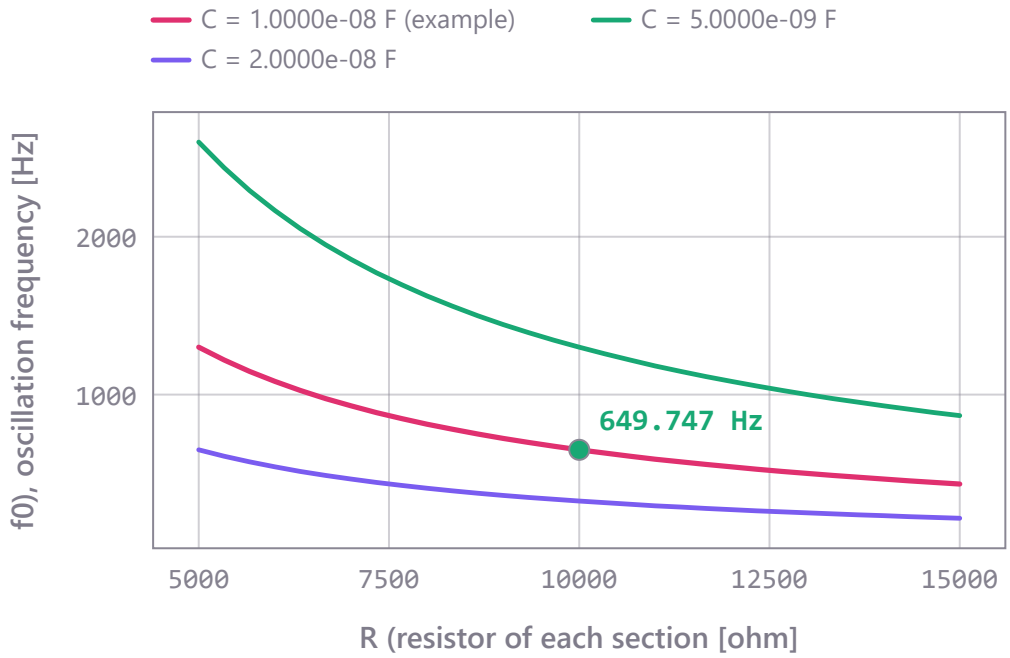
The oscillation frequency of a three-section RC phase-shift oscillator (high-pass ladder: series C, shunt R; or low-pass ladder) and the amplifier gain of 29 it needs.

$$f_0 = \frac{1}{2\pi\sqrt{6}\,RC}, \quad |A| \geq 29$$

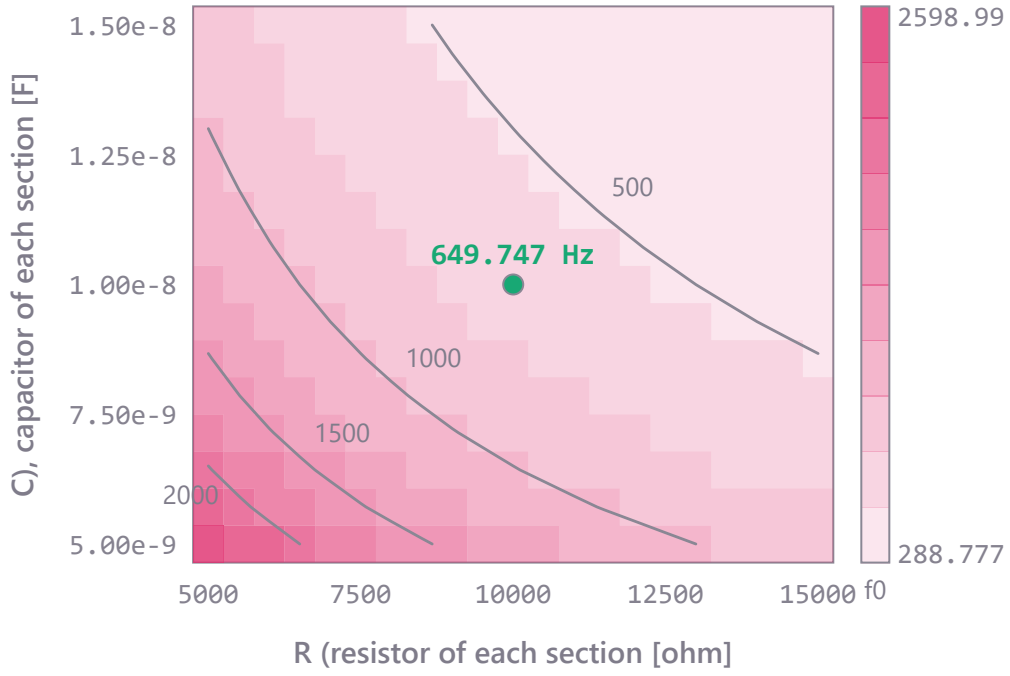
input	meaning	condition
`R`	resistor of each section	
`C`	capacitor of each section	
`ladder`	highpass (series C, shunt R) or lowpass (series R, shunt C)	

output	meaning
`f0`	oscillation frequency
`A_min`	minimum inverting amplifier gain
`beta`	feedback network attenuation at f0

- **Method:** high-pass ladder f = 1/(2 pi sqrt(6) R C); low-pass ladder $f = sqrt(6)/(2$ pi R C); |beta| = $1/29$
- **Assumes:** unloaded ladder (driven by a voltage source, feeding a high impedance)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f0 (oscillation frequency) against R (resistor of each section) from 5000 ohm to 15000 ohm. One curve for each value of C (capacitor of each section): 1.0000e-08 F, 5.0000e-09 F, 2.0000e-08 F. Other inputs as in the worked example: ladder = high-pass. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R = 10000 ohm, f0 = 649.747 Hz.



f0 (oscillation frequency) over R (resistor of each section) from 5000 ohm to 15000 ohm and C (capacitor of each section) from 5.0000e-09 F to 1.5000e-08 F: each of the 21 × 15 cells is one run of electronics-engineering-ai999x, shaded by its value (light 288.777 Hz to dark 2598.99 Hz); iso-lines at f0 = 500, 1000, 1500, 2000 Hz. Other inputs as in the worked example: ladder = high-pass. Marked: the worked example, R = 10000 ohm, C = 1.0000e-08 F, f0 = 649.747 Hz.

```
electronics_engineering_ai999x.solve('elx_rc_phase_shift_oscillator', R=10000.0, C=1e-08,
ladder='high-pass')
# -> f0 = 649.747 Hz, A_min = 29, beta = 0.0344828   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_rc_phase_shift_oscillator', R=4700.0, C=4.7e-08,
ladder='low-pass')
# -> f0 = 1764.82 Hz, A_min = 29, beta = 0.0344828   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
RC phase-shift oscillator: R = 10000 ohm, C = 10 nF, ladder = high-pass	SOLVED	f0 = 649.747 Hz, A_min = 29, beta = 0.0344828
RC phase-shift oscillator: R = 4700 ohm, C = 47 nF, ladder = low-pass	SOLVED	f0 = 1764.82 Hz, A_min = 29, beta = 0.0344828
RC phase-shift oscillator: C = 10 nF, ladder = high-pass	NEEDS_INPUT	elx_rc_phase_shift_oscillator needs R (resistor of each section). Example: solve('elx_rc_phase_shift_oscillator', R='100
RC phase-shift oscillator: R = 10000 ohm, C = 10 nF, ladder = high-pass, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Wien-bridge oscillator frequency and required gain, `elx_wien_bridge_oscillator`

The oscillation frequency of a Wien-bridge oscillator (series R1 C1, parallel R2 C2) and the non-inverting gain it needs (3 for equal components).

$$f_0 = \frac{1}{2\pi\sqrt{R_1 R_2 C_1 C_2}}, \quad A = 1 + \frac{R_1}{R_2} + \frac{C_2}{C_1}$$

input	meaning	condition
`R1`	series-arm resistor	
`C1`	series-arm capacitor	
`R2`	parallel-arm resistor	
`C2`	parallel-arm capacitor	

output	meaning
`f0`	oscillation frequency
`beta`	feedback fraction at f0
`A_min`	required amplifier gain
`Rf_over_Rg`	gain resistor ratio

- **Method:** f0 = 1/(2 pi sqrt(R1 R2 C1 C2)); $A = 1 + R_1/R_2 + C_2/C_1$
- **Assumes:** ideal amplifier
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f0 (oscillation frequency) against R1 (series-arm resistor) from 5000 ohm to 15000 ohm. One curve for each value of C1 (series-arm capacitor): 1.6000e-08 F, 8.0000e-09 F, 3.2000e-08 F. Other inputs as in the worked example: R2 = 10000.0 ohm, C2 = 1.6e-08 F. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R1 = 10000 ohm, f0 = 994.718 Hz.

```
electronics_engineering_ai999x.solve('elx_wien_bridge_oscillator', R1=10000.0, C1=1.6e-08,
R2=10000.0, C2=1.6e-08)
# -> f0 = 994.718 Hz, beta = 0.333333, A_min = 3, Rf_over_Rg = 2   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_wien_bridge_oscillator', R1=22000.0, C1=1e-08,
R2=10000.0, C2=2.2e-08)
# -> f0 = 723.432 Hz, beta = 0.185185, A_min = 5.4, Rf_over_Rg = 4.4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wien-bridge oscillator: R1 = 10000 ohm, C1 = 16 nF, R2 = 10000 ohm, C2 = 16 nF	SOLVED	f0 = 994.718 Hz, beta = 0.333333, A_min = 3, Rf_over_Rg = 2
Wien-bridge oscillator: R1 = 22000 ohm, C1 = 10 nF, R2 = 10000 ohm, C2 = 22 nF	SOLVED	f0 = 723.432 Hz, beta = 0.185185, A_min = 5.4, Rf_over_Rg = 4.4
Wien-bridge oscillator: C1 = 16 nF, R2 = 10000 ohm, C2 = 16 nF	NEEDS_INPUT	elx_wien_bridge_oscillator needs R1 (series-arm resistor). Example: solve('elx_wien_bridge_oscillator', R1='10000 ohm',
Wien-bridge oscillator: R1 = 10000 ohm, C1 = 16 nF, R2 = 10000 ohm, C2 = 16 nF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Colpitts oscillator frequency and feedback ratio, `elx_colpitts_oscillator`

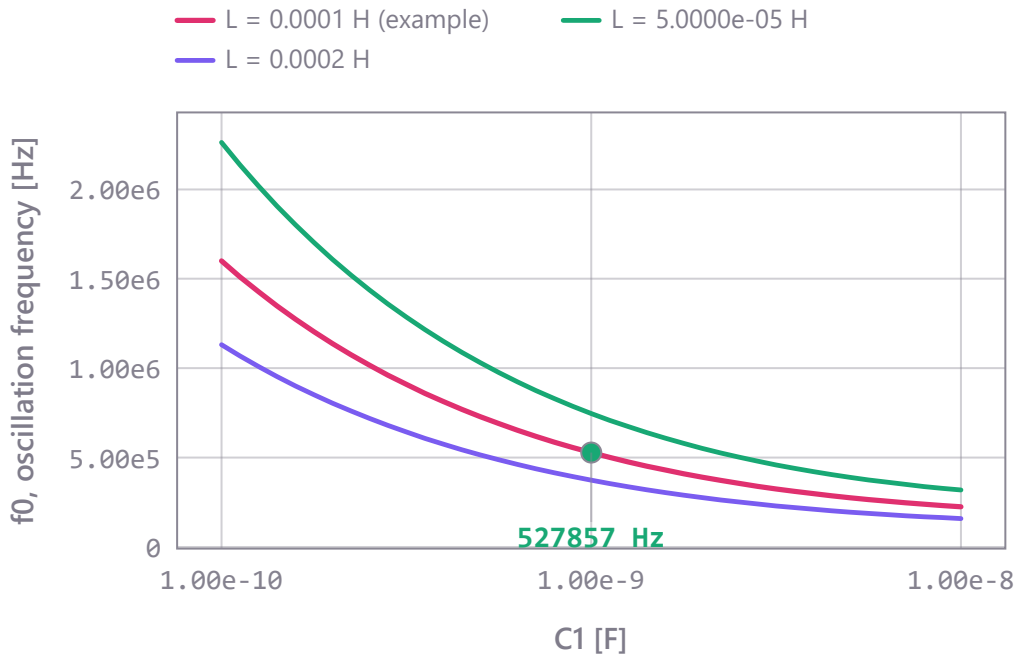
The oscillation frequency of a Colpitts oscillator from its inductor and the series combination of its two tank capacitors, the feedback fraction C_1/C_2 and the minimum amplifier gain C_2/C_1.

$$f_0 = \frac{1}{2\pi\sqrt{L\,\frac{C_1 C_2}{C_1+C_2}}}, \quad A_{min} = \frac{C_2}{C_1}$$

input	meaning	condition
`L`	tank inductance	
`C1`	capacitor across the amplifier output	
`C2`	capacitor across the amplifier input (feedback side)	

output	meaning
`C_eq`	series equivalent capacitance
`f0`	oscillation frequency
`beta`	feedback fraction C_1/C_2
`A_min`	minimum amplifier gain C_2/C_1

- **Method:** $C_eq = C_1\,C_2/(C_1 + C_2)$; f0 = 1/(2 pi sqrt(L C_eq)); $\beta = C_1/C_2$
- **Assumes:** lossless tank, amplifier does not load the tank
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f0 (oscillation frequency) against C1 (capacitor across the amplifier output) from 1.0000e-10 F to 1.0000e-08 F on a logarithmic axis. One curve for each value of L (tank inductance): 0.0001 H, 5.0000e-05 H, 0.0002 H. Other inputs as in the worked example: C2 = 1e-08 F. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, C1 = 1.0000e-09 F, f0 = 527857 Hz.

```
electronics_engineering_ai999x.solve('elx_colpitts_oscillator', L=0.0001, C1=1e-09, C2=1e-08)
# -> C_eq = 9.09091e-10 F, f0 = 527857 Hz, beta = 0.1, A_min = 10   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_colpitts_oscillator', L=1e-05, C1=4.7e-10, C2=4.7e-09)
# -> C_eq = 4.27273e-10 F, f0 = 2.43482e+06 Hz, beta = 0.1, A_min = 10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Colpitts oscillator: L = 100 uH, C1 = 1 nF, C2 = 10 nF	SOLVED	C_eq = 9.09091e-10 F, f0 = 527857 Hz, beta = 0.1, A_min = 10
Colpitts oscillator: L = 10 uH, C1 = 0.47 nF, C2 = 4.7 nF	SOLVED	C_eq = 4.27273e-10 F, f0 = 2.43482e+06 Hz, beta = 0.1, A_min = 10
Colpitts oscillator: C1 = 1 nF, C2 = 10 nF	NEEDS_INPUT	elx_colpitts_oscillator needs L (tank inductance). Example: solve('elx_colpitts_oscillator', L='0.0001 H', C1='1e-09 F',
Colpitts oscillator: L = 100 uH, C1 = 1 nF, C2 = 10 nF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Hartley oscillator frequency, `elx_hartley_oscillator`

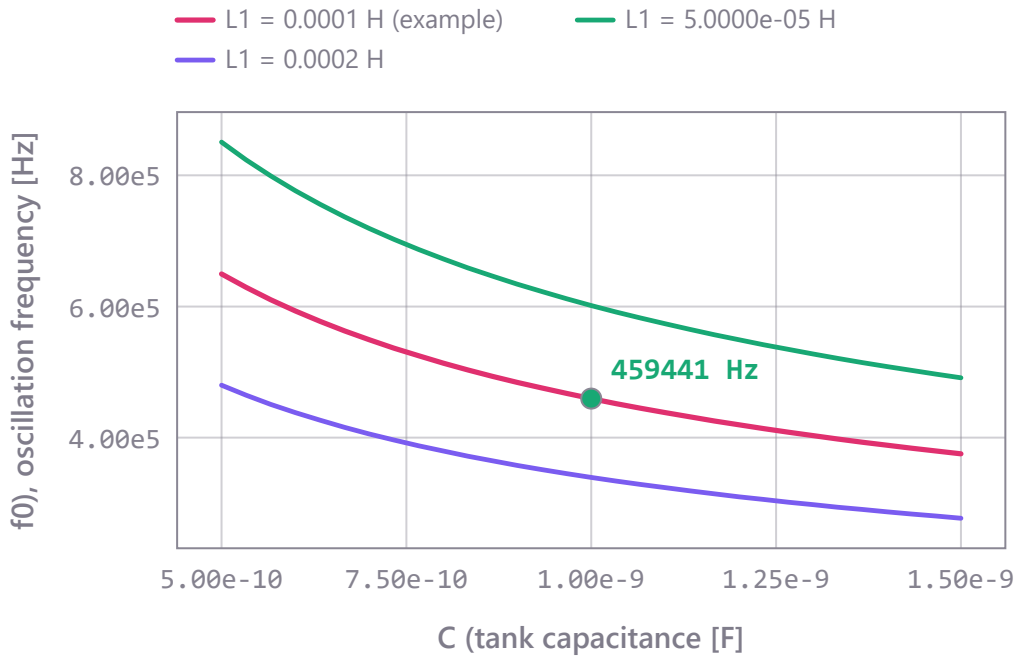
The oscillation frequency of a Hartley oscillator with a tapped (or two coupled) inductors and a tank capacitor, and the gain ratio $(L_1 + M)/(L_2 + M)$.

$$f_0 = \frac{1}{2\pi\sqrt{(L_1 + L_2 + 2M)C}}$$

input	meaning	condition
`L1`	inductance of the output-side coil	
`L2`	inductance of the feedback-side coil	
`M`	mutual inductance (0 for uncoupled coils)	
`C`	tank capacitance	

output	meaning
`L_eq`	total inductance $L_1 + L_2 + 2M$
`f0`	oscillation frequency
`A_min`	minimum amplifier gain $(L_1 + M)/(L_2 + M)$

- **Method:** $L_eq = L_1 + L_2 + 2M$; f0 = 1/(2 pi sqrt(L_eq C))
- **Assumes:** series-aiding coils, lossless tank; $M \leq sqrt$(L1 L2)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f0 (oscillation frequency) against C (tank capacitance) from 5.0000e-10 F to 1.5000e-09 F. One curve for each value of L1 (inductance of the output-side coil): 0.0001 H, 5.0000e-05 H, 0.0002 H. Other inputs as in the worked example: L2 = 2e-05 H, M = 0.0 H. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, C = 1.0000e-09 F, f0 = 459441 Hz.

```
electronics_engineering_ai999x.solve('elx_hartley_oscillator', L1=0.0001, L2=2e-05, M=0.0, C=1e-09)
# -> L_eq = 0.00012 H, f0 = 459441 Hz, A_min = 5   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_hartley_oscillator', L1=5e-05, L2=1e-05, M=5e-06, C=4.7e-10)
# -> L_eq = 7e-05 H, f0 = 877450 Hz, A_min = 3.66667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hartley oscillator: L1 = 100 uH, L2 = 20 uH, M = 0 uH, C = 1 nF	SOLVED	L_eq = 0.00012 H, f0 = 459441 Hz, A_min = 5
Hartley oscillator: L1 = 50 uH, L2 = 10 uH, M = 5 uH, C = 0.47 nF	SOLVED	L_eq = 7e-05 H, f0 = 877450 Hz, A_min = 3.66667
Hartley oscillator: L2 = 20 uH, M = 0 uH, C = 1 nF	NEEDS_INPUT	elx_hartley_oscillator needs L1 (inductance of the output-side coil). Example: solve('elx_hartley_oscillator', L1='0.000
Hartley oscillator: L1 = 100 uH, L2 = 20 uH, M = 0 uH, C = 1 nF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Quartz crystal: series and parallel resonance, Q, `elx_crystal_oscillator`

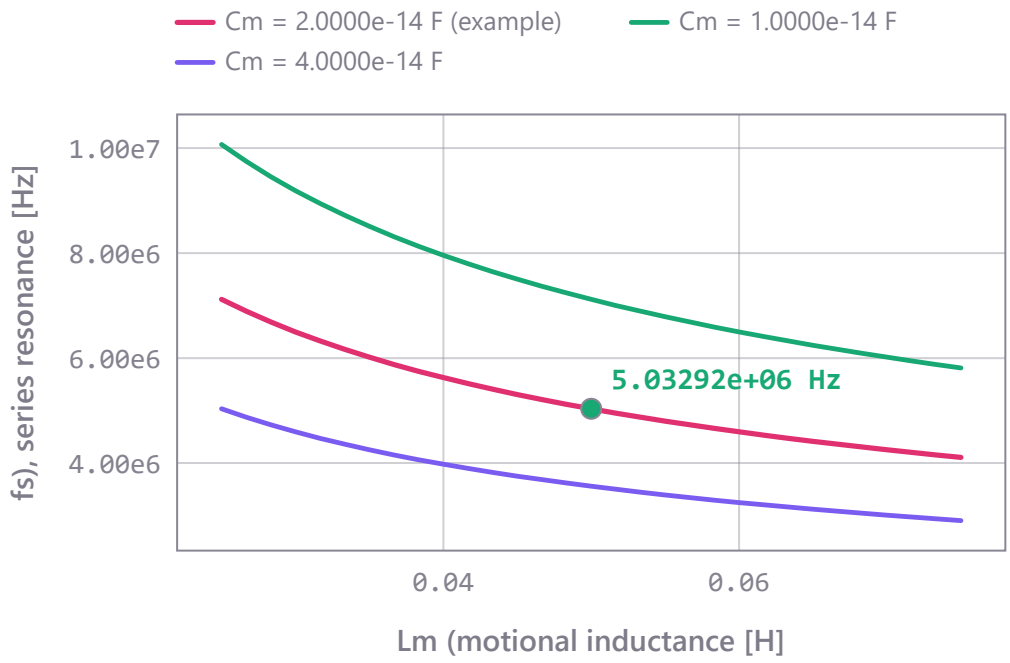
The series and parallel (anti-) resonant frequencies, their spacing and the quality factor of a quartz crystal from its Butterworth-Van Dyke equivalent circuit.

$$f_s = \frac{1}{2\pi\sqrt{L_m C_m}}, \quad f_p = f_s\sqrt{1 + \frac{C_m}{C_0}}$$

input	meaning	condition
`Lm`	motional inductance	
`Cm`	motional capacitance	
`Rm`	motional resistance	
`C0`	shunt (holder) capacitance	

output	meaning
`fs`	series resonance
`fp`	parallel resonance
`delta_f`	fp - fs
`Q`	quality factor

- **Method:** fs = 1/(2 pi sqrt(Lm Cm)); $fp = fs$ sqrt(1 + Cm/C0); $Q = \frac{\sqrt{\frac{Lm}{Cm}}}{Rm}$
- **Assumes:** resonances of the lossless circuit; Q from Rm
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



fs (series resonance) against Lm (motional inductance) from 0.025 H to 0.075 H. One curve for each value of Cm (motional capacitance): 2.0000e-14 F, 1.0000e-14 F, 4.0000e-14 F. Other inputs as in the worked example: Rm = 20.0 ohm, C0 = 5e-12 F. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Lm = 0.05 H, fs = 5.03292e+06 Hz.

```
electronics_engineering_ai999x.solve('elx_crystal_oscillator', Lm=0.05, Cm=2e-14, Rm=20.0, C0=5e-12)
# -> fs = 5.03292e+06 Hz, fp = 5.04298e+06 Hz, delta_f = 10055.8 Hz, Q = 79056.9   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_crystal_oscillator', Lm=0.26, Cm=2.5e-14, Rm=50.0, C0=6e-12)
# -> fs = 1.97407e+06 Hz, fp = 1.97818e+06 Hz, delta_f = 4108.38 Hz, Q = 64498.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Crystal resonance: Lm = 0.05 H, Cm = 20 fF, Rm = 20 ohm, C0 = 5 pF	SOLVED	fs = 5.03292e+06 Hz, fp = 5.04298e+06 Hz, delta_f = 10055.8 Hz, Q = 79056.9
Crystal resonance: Lm = 0.26 H, Cm = 25 fF, Rm = 50 ohm, C0 = 6 pF	SOLVED	fs = 1.97407e+06 Hz, fp = 1.97818e+06 Hz, delta_f = 4108.38 Hz, Q = 64498.1
Crystal resonance: Cm = 20 fF, Rm = 20 ohm, C0 = 5 pF	NEEDS_INPUT	elx_crystal_oscillator needs Lm (motional inductance). Example: solve('elx_crystal_oscillator', Lm='0.05 H', Cm='2e-14 F
Crystal resonance: Lm = 0.05 H, Cm = 20 fF, Rm = 20 ohm, C0 = 5 pF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Op-amp relaxation (astable) oscillator, `elx_opamp_relaxation_oscillator`

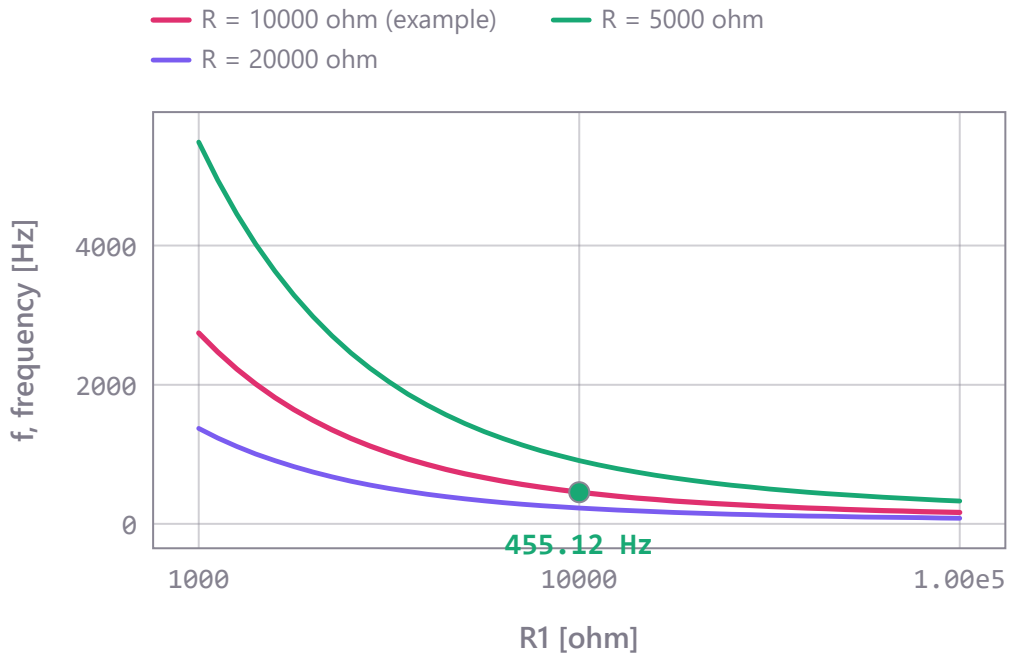
The period and frequency of a square-wave op-amp astable: an RC timing network on the inverting input and positive feedback $R_1/(R_1+R_2)$ to the non-inverting input, symmetric supplies.

$$T = 2RC\ln\frac{1+\beta}{1-\beta}, \quad \beta = \frac{R_1}{R_1+R_2}$$

input	meaning	condition
`R`	timing resistor (output to inverting input)	
`C`	timing capacitor	
`R1`	resistor from the + input to ground	
`R2`	resistor from the output to the + input	

output	meaning
`beta`	feedback fraction $R_1/(R_1+R_2)$
`T`	period
`f`	frequency

- **Method:** $T = 2$ R C ln$((1+\beta)/(1-\beta))$ = 2 R C ln$(1+2$ R1/R2)
- **Assumes:** symmetric output levels; ideal comparator action
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f (frequency) against R1 (resistor from the + input to ground) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of R (timing resistor (output to inverting input)): 10000 ohm, 5000 ohm, 20000 ohm. Other inputs as in the worked example: C = 1e-07 F, R2 = 10000.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R1 = 10000 ohm, f = 455.12 Hz.

```
electronics_engineering_ai999x.solve('elx_opamp_relaxation_oscillator', R=10000.0, C=1e-07,
R1=10000.0, R2=10000.0)
# -> beta = 0.5, T = 0.00219722 s, f = 455.12 Hz   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_opamp_relaxation_oscillator', R=47000.0, C=1e-08,
R1=8600.0, R2=10000.0)
# -> beta = 0.462366, T = 0.000940594 s, f = 1063.16 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Op-amp relaxation oscillator: R = 10000 ohm, C = 100 nF, R1 = 10000 ohm, R2 = 10000 ohm	SOLVED	beta = 0.5, T = 0.00219722 s, f = 455.12 Hz
Op-amp relaxation oscillator: R = 47000 ohm, C = 10 nF, R1 = 8600 ohm, R2 = 10000 ohm	SOLVED	beta = 0.462366, T = 0.000940594 s, f = 1063.16 Hz
Op-amp relaxation oscillator: C = 100 nF, R1 = 10000 ohm, R2 = 10000 ohm	NEEDS_INPUT	elx_opamp_relaxation_oscillator needs R (timing resistor (output to inverting input)). Example: solve('elx_opamp_relaxat
Op-amp relaxation oscillator: R = 10000 ohm, C = 100 nF, R1 = 10000 ohm, R2 = 10000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

555 timer astable: frequency and duty cycle, `elx_555_astable`

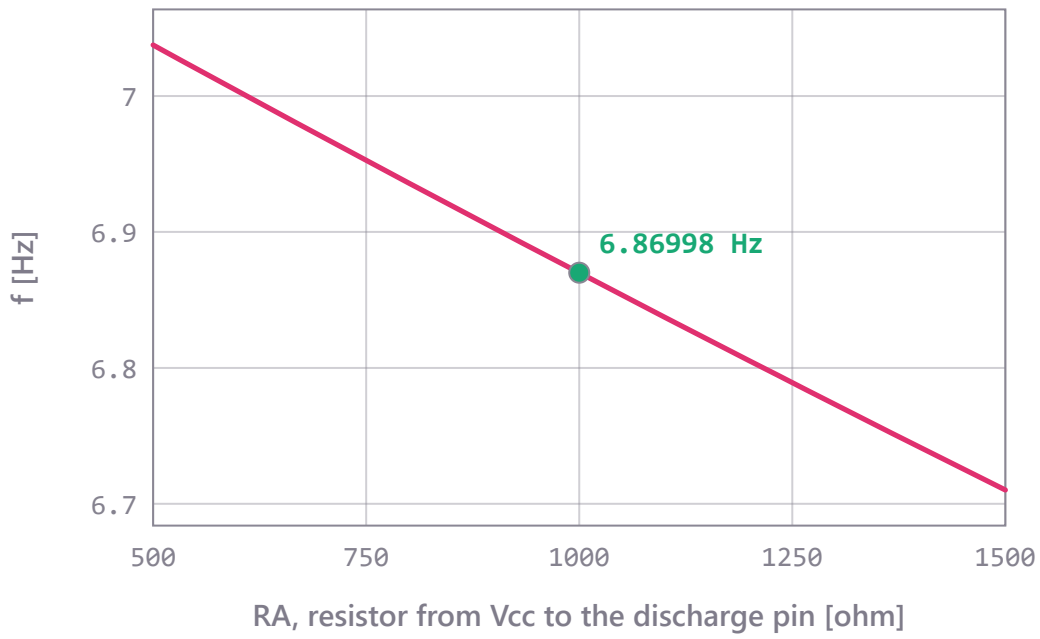
The high and low times, period, frequency and duty cycle of a 555 timer in astable mode with resistors RA and RB and timing capacitor C.

$$t_H = \ln 2\,(R_A + R_B)C, \quad t_L = \ln 2\,R_B C, \quad f = \frac{1}{\ln 2\,(R_A + 2R_B)C}$$

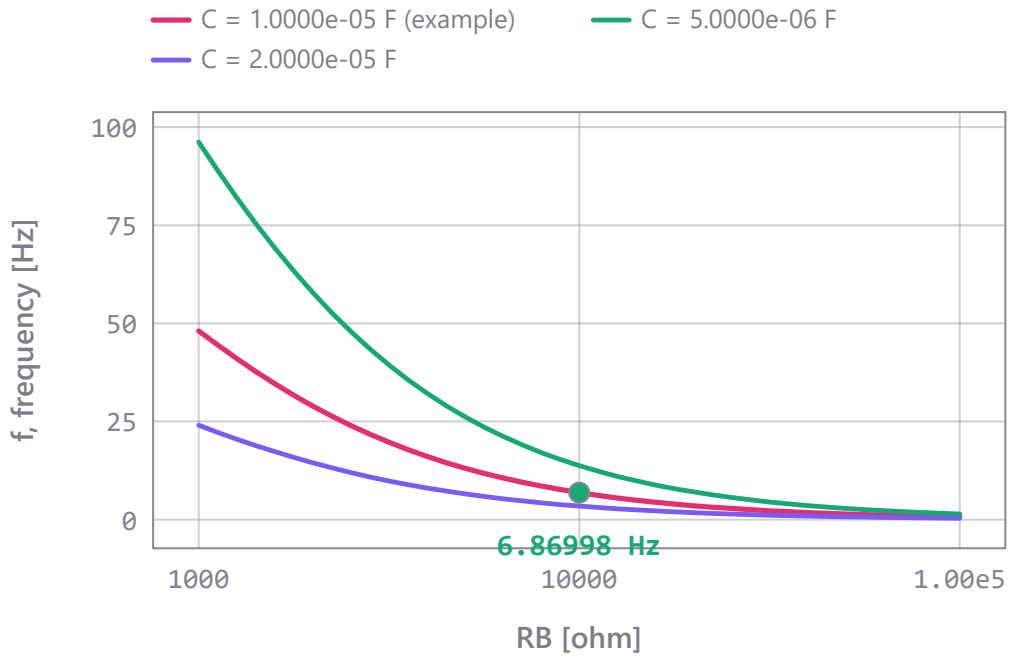
input	meaning	condition
`RA`	resistor from Vcc to the discharge pin	
`RB`	resistor from discharge to threshold/trigger	
`C`	timing capacitor	

output	meaning
`t_high`	output high time
`t_low`	output low time
`T`	period
`f`	frequency
`duty`	duty cycle (fraction high)

- **Method:** $t_H = ln2$ (RA + RB) C; $t_L = ln2$ RB C; $f = 1/(t_H + t_L)$
- **Assumes:** thresholds Vcc/3 and 2Vcc/3
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f (frequency) against RA (resistor from Vcc to the discharge pin), with C = 1.00e-5 F, RB = 10000 ohm. 31 points computed by electronics-engineering-ai999x; marked: RA = 1000, f = 6.86998 Hz.



f (frequency) against RB (resistor from discharge to threshold/trigger) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of C (timing capacitor): 1.0000e-05 F, 5.0000e-06 F, 2.0000e-05 F. Other inputs as in the worked example: RA = 1000.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, RB = 10000 ohm, f = 6.86998 Hz.

```
electronics_engineering_ai999x.solve('elx_555_astable', RA=1000.0, RB=10000.0, C=1e-05)
# -> t_high = 0.0762462 s, t_low = 0.0693147 s, T = 0.145561 s, f = 6.86998 Hz, duty = 0.52381
(SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_555_astable', RA=4700.0, RB=4700.0, C=1e-07)
# -> t_high = 0.000651558 s, t_low = 0.000325779 s, T = 0.000977338 s, f = 1023.19 Hz, duty =
0.666667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
555 astable: RA = 1000 ohm, RB = 10000 ohm, C = 10 uF	SOLVED	t_high = 0.0762462 s, t_low = 0.0693147 s, T = 0.145561 s, f = 6.86998 Hz, duty = 0.52381
555 astable: RA = 4700 ohm, RB = 4700 ohm, C = 0.1 uF	SOLVED	t_high = 0.000651558 s, t_low = 0.000325779 s, T = 0.000977338 s, f = 1023.19 Hz, duty = 0.666667
555 astable: RB = 10000 ohm, C = 10 uF	NEEDS_INPUT	elx_555_astable needs RA (resistor from Vcc to the discharge pin). Example: solve('elx_555_astable', RA='1000 ohm', RB='
555 astable: RA = 1000 ohm, RB = 10000 ohm, C = 10 uF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

555 timer monostable pulse width, `elx_555_monostable`

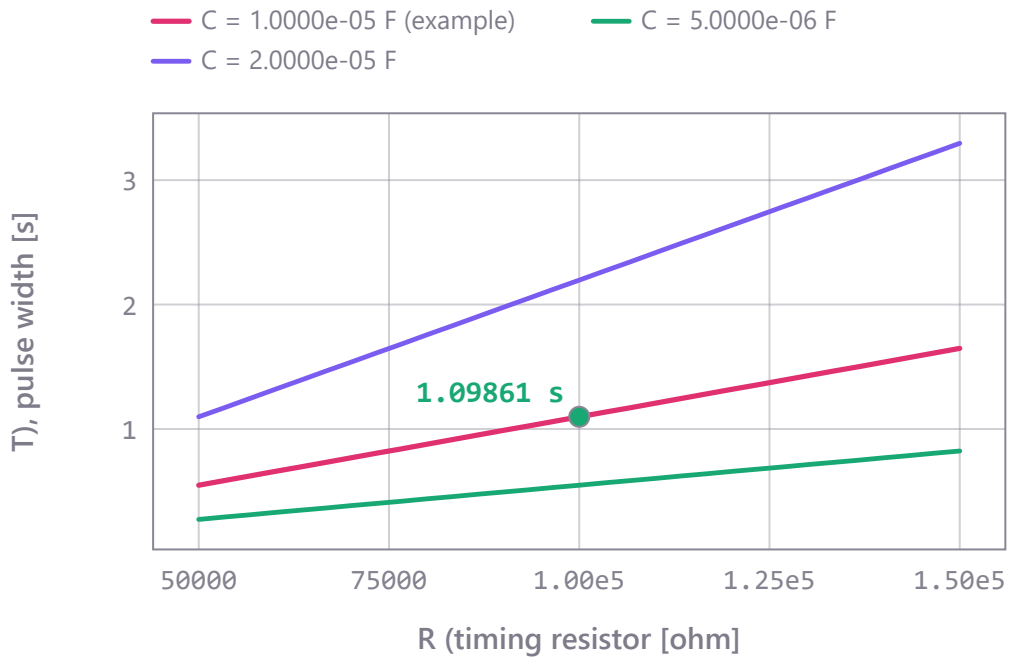
The output pulse width $T = ln$ 3 R C (about 1.1 R C) of a 555 timer in monostable (one-shot) mode.

$$T = \ln 3 \, RC \approx 1.1 \, RC$$

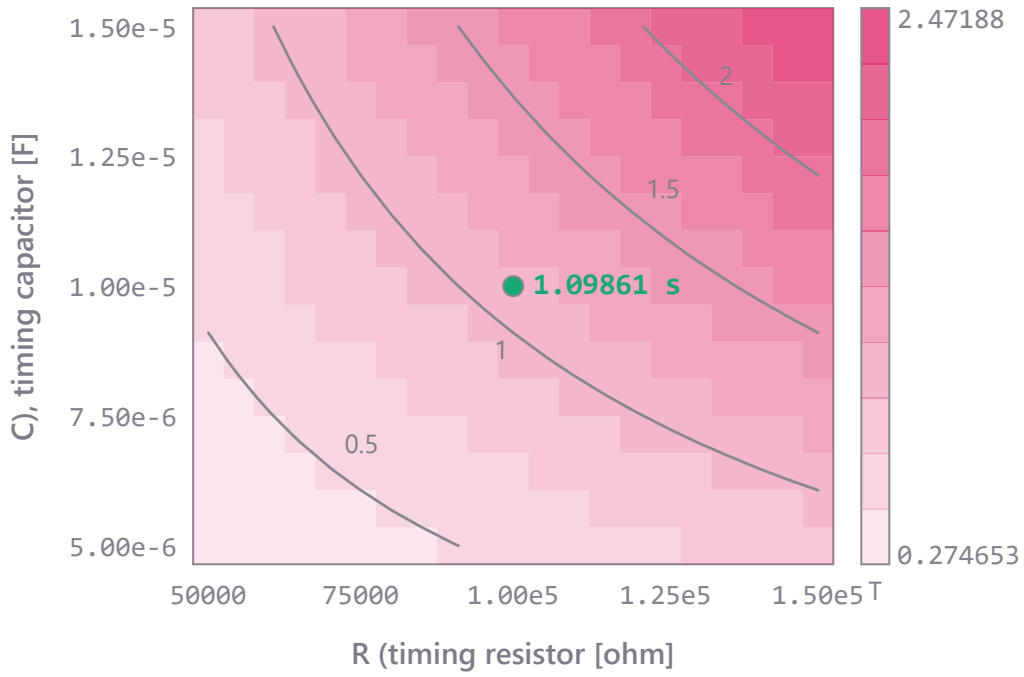
input	meaning	condition
`R`	timing resistor	
`C`	timing capacitor	

output	meaning
`T`	pulse width

- **Method:** $T = ln$ 3 R C
- **Assumes:** threshold at 2Vcc/3; trigger pulse shorter than T
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



T (pulse width) against R (timing resistor) from 50000 ohm to 150000 ohm. One curve for each value of C (timing capacitor): 1.0000e-05 F, 5.0000e-06 F, 2.0000e-05 F. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R = 100000 ohm, T = 1.09861 s.



T (pulse width) over R (timing resistor) from 50000 ohm to 150000 ohm and C (timing capacitor) from 5.0000e-06 F to 1.5000e-05 F: each of the 21 × 15 cells is one run of electronics-engineering-ai999x, shaded by its value (light 0.274653 s to dark 2.47188 s); iso-lines at T = 0.5, 1, 1.5, 2 s. Marked: the worked example, R = 100000 ohm, C = 1.0000e-05 F, T = 1.09861 s.

```
electronics_engineering_ai999x.solve('elx_555_monostable', R=100000.0, C=1e-05)
# -> T = 1.09861 s   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_555_monostable', R=47000.0, C=1e-06)
# -> T = 0.0516348 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
555 monostable: R = 100000 ohm, C = 10 uF	SOLVED	T = 1.09861 s
555 monostable: R = 47000 ohm, C = 1 uF	SOLVED	T = 0.0516348 s
555 monostable: C = 10 uF	NEEDS_INPUT	elx_555_monostable needs R (timing resistor). Example: solve('elx_555_monostable', R='100000 ohm', C='1e-05 F').
555 monostable: R = 100000 ohm, C = 10 uF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Type-2 PLL with active PI loop filter: natural frequency and damping, `elx_pll_loop_dynamics`

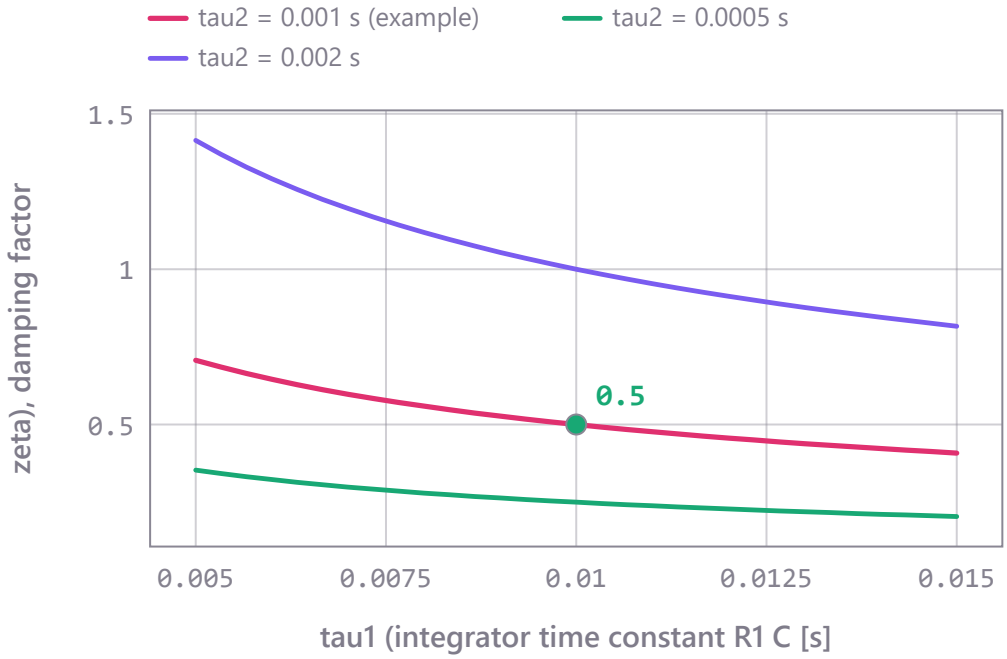
The natural frequency and damping factor of a second-order (type-2) phase-locked loop with phase-detector gain Kd, VCO gain Ko and an active PI loop filter F(s) = $(1 + s\,tau2)/(s$ tau1).

$$\omega_n = \sqrt{\frac{K_d K_o}{\tau_1}}, \quad \zeta = \frac{\tau_2 \omega_n}{2}$$

input	meaning	condition
`Kd`	phase-detector gain in V/rad	
`Ko`	VCO gain in rad/(s V)	
`tau1`	integrator time constant R1 C	
`tau2`	zero time constant R2 C	

output	meaning
`wn`	natural frequency
`fn`	natural frequency in Hz
`zeta`	damping factor

- **Method:** H(s) = K (1 + s $tau2$)/$tau1$/(s^2 + s K tau2/tau1 + K/tau1), $K = Kd$ Ko; $wn = \sqrt{\frac{K}{\tau_1}}$; $\zeta = tau2\ wn/2$
- **Assumes:** linear phase model, ideal integrator
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



zeta (damping factor) against tau1 (integrator time constant R1 C) from 0.005 s to 0.015 s. One curve for each value of tau2 (zero time constant R2 C): 0.001 s, 0.0005 s, 0.002 s. Other inputs as in the worked example: Kd = 0.5, Ko = 20000.0. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, tau1 = 0.01 s, zeta = 0.5.

```
electronics_engineering_ai999x.solve('elx_pll_loop_dynamics', Kd=0.5, Ko=20000.0, tau1=0.01,
tau2=0.001)
# -> wn = 1000 rad/s, fn = 159.155 Hz, zeta = 0.5   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_pll_loop_dynamics', Kd=1.0, Ko=100000.0, tau1=0.1,
tau2=0.002)
# -> wn = 1000 rad/s, fn = 159.155 Hz, zeta = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PLL loop dynamics: Kd = 0.5 V/rad, Ko = 20000 rad/(s*V), tau1 = 10 ms, tau2 = 1 ms	SOLVED	wn = 1000 rad/s, fn = 159.155 Hz, zeta = 0.5
PLL loop dynamics: Kd = 1 V/rad, Ko = 100000 rad/(s*V), tau1 = 100 ms, tau2 = 2 ms	SOLVED	wn = 1000 rad/s, fn = 159.155 Hz, zeta = 1
PLL loop dynamics: Ko = 20000 rad/(s*V), tau1 = 10 ms, tau2 = 1 ms	NEEDS_INPUT	elx_pll_loop_dynamics needs Kd (phase-detector gain in V/rad). Example: solve('elx_pll_loop_dynamics', Kd=0.5, Ko=20000,
PLL loop dynamics: Kd = 0.5 V/rad, Ko = 20000 rad/(s*V), tau1 = 10 ms, tau2 = 1 ms, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

ADC output code, LSB and quantisation error, `elx_adc_code`

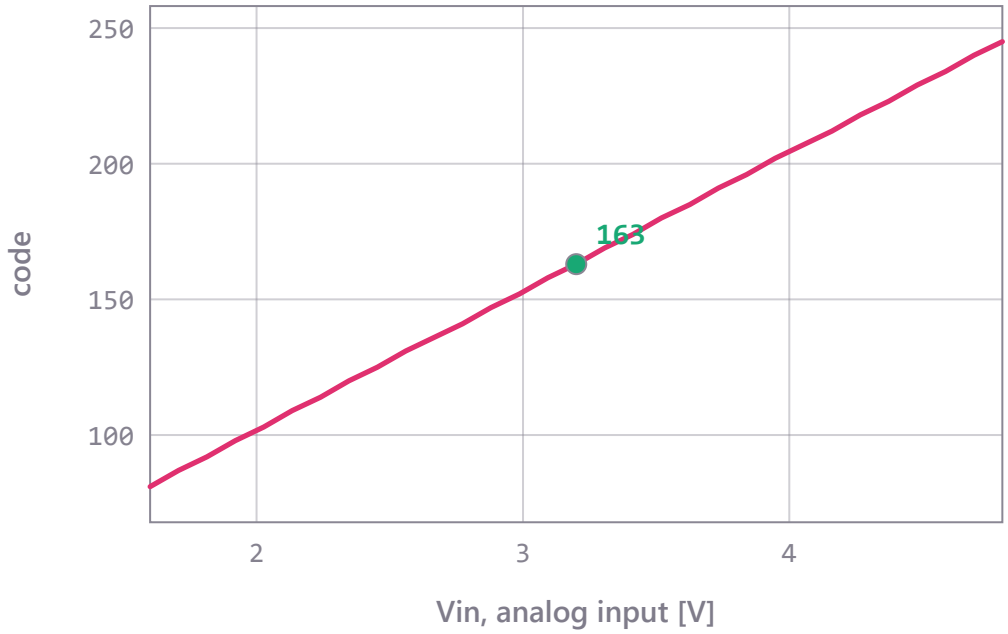
The output code (decimal and binary) of an ideal unipolar truncating n-bit ADC for an input voltage, the LSB size, the voltage the code represents and the quantisation error.

$$D = \left\lfloor \frac{V_{in}}{V_{ref}} 2^n \right\rfloor, \quad \mathrm{LSB} = \frac{V_{ref}}{2^n}$$

input	meaning	condition
`Vin`	analog input	
`Vref`	reference (full-scale) voltage	
`n`	resolution in bits	

output	meaning
`code`	output code (decimal)
`bits`	output code in binary
`LSB`	LSB = Vref/2^n
`V_code`	voltage of the code
`error`	quantisation error Vin - code LSB

- **Method:** code = floor(Vin 2^n/Vref), at most $2^n - 1$; LSB = Vref/2^n
- **Assumes:** 0 <= Vin <= Vref; $1 \le n \le 16$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



code (output code (decimal)) against Vin (analog input), with Vref = 5 V, n = 8. 31 points computed by electronics-engineering-ai999x; marked: Vin = 3.2, code = 163.



code (output code (decimal)) against n (resolution in bits) from 1 to 16, logarithmic output axis. Other inputs as in the worked example: Vin = 3.2 V, Vref = 5.0 V. 16 points computed by electronics-engineering-ai999x; marked: the worked example, n = 8, code = 163.

```
electronics_engineering_ai999x.solve('elx_adc_code', Vin=3.2, Vref=5.0, n=8)
# -> code = 163, bits = 10100011, LSB = 0.0195312 V, V_code = 3.18359 V, error = 0.0164063 V
(SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_adc_code', Vin=1.234, Vref=2.5, n=12)
# -> code = 2021, bits = 011111100101, LSB = 0.000610352 V, V_code = 1.23352 V, error = 0.000479492
V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ADC output code: Vin = 3.2 V, Vref = 5 V, n = 8	SOLVED	code = 163, bits = 10100011, LSB = 0.0195312 V, V_code = 3.18359 V, error = 0.0164063 V
ADC output code: Vin = 1.234 V, Vref = 2.5 V, n = 12	SOLVED	code = 2021, bits = 011111100101, LSB = 0.000610352 V, V_code = 1.23352 V, error = 0.000479492 V
ADC output code: Vref = 5 V, n = 8	NEEDS_INPUT	elx_adc_code needs Vin (analog input). Example: solve('elx_adc_code', Vin='3.2 V', Vref='5 V', n=8).
ADC output code: Vin = 3.2 V, Vref = 5 V, n = 8, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

ADC quantisation SNR with oversampling, `elx_adc_quantization_snr`

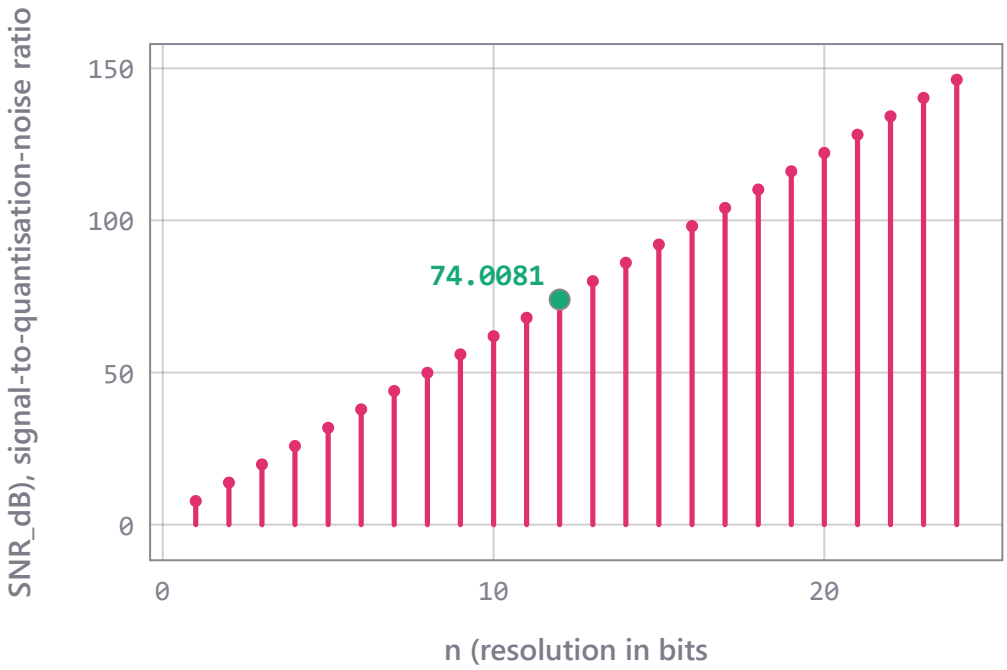
The signal-to-quantisation-noise ratio of an ideal n-bit converter for a sine input (6.02 $n + 1.76$ dB at full scale), with an oversampling ratio and a reduced amplitude, and the in-band noise in LSB.

$$\mathrm{SNR} = 6.02\,n + 1.76 + 10\log_{10}\mathrm{OSR}\ \mathrm{dB}$$

input	meaning	condition
`n`	resolution in bits	
`OSR`	oversampling ratio (default 1)	
`A_frac`	sine amplitude as a fraction of full scale (default 1)	

output	meaning
`SNR_dB`	signal-to-quantisation-noise ratio, dB
`noise_rms_lsb`	in-band noise rms in LSB
`SNR_full_scale_dB`	full-scale SNR without oversampling, dB

- **Method:** SNR = 20 log10(A 2^n sqrt(1.5)) + 10 log10(OSR) = 6.02 $n + 1.76 + 20$ log10 $A + 10$ log10 OSR
- **Assumes:** uniform quantisation noise, white over Nyquist; OSR >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



SNR_dB (signal-to-quantisation-noise ratio, dB) against n (resolution in bits) from 1 to 24. 24 points computed by electronics-engineering-ai999x; marked: the worked example, n = 12, SNR_dB = 74.0081.

```
electronics_engineering_ai999x.solve('elx_adc_quantization_snr', n=12)
# -> SNR_dB = 74.0081, noise_rms_lsb = 0.288675, SNR_full_scale_dB = 74.0081   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_adc_quantization_snr', n=10, OSR=16.0, A_frac=0.5)
# -> SNR_dB = 67.9875, noise_rms_lsb = 0.0721688, SNR_full_scale_dB = 61.9669   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ADC quantization SNR: n = 12	SOLVED	SNR_dB = 74.0081, noise_rms_lsb = 0.288675, SNR_full_scale_dB = 74.0081
ADC quantization SNR: n = 10, OSR = 16, A_frac = 0.5	SOLVED	SNR_dB = 67.9875, noise_rms_lsb = 0.0721688, SNR_full_scale_dB = 61.9669
ADC quantization SNR:	NEEDS_INPUT	elx_adc_quantization_snr needs n (resolution in bits). Example: solve('elx_adc_quantization_snr', n=12).
ADC quantization SNR: n = 12, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Effective number of bits (ENOB) from SINAD, `elx_adc_enob`

The effective number of bits of a converter from its measured SINAD (IEEE 1241: ENOB = (SINAD - 1.76)/6.02) and the bits lost against its nominal resolution.

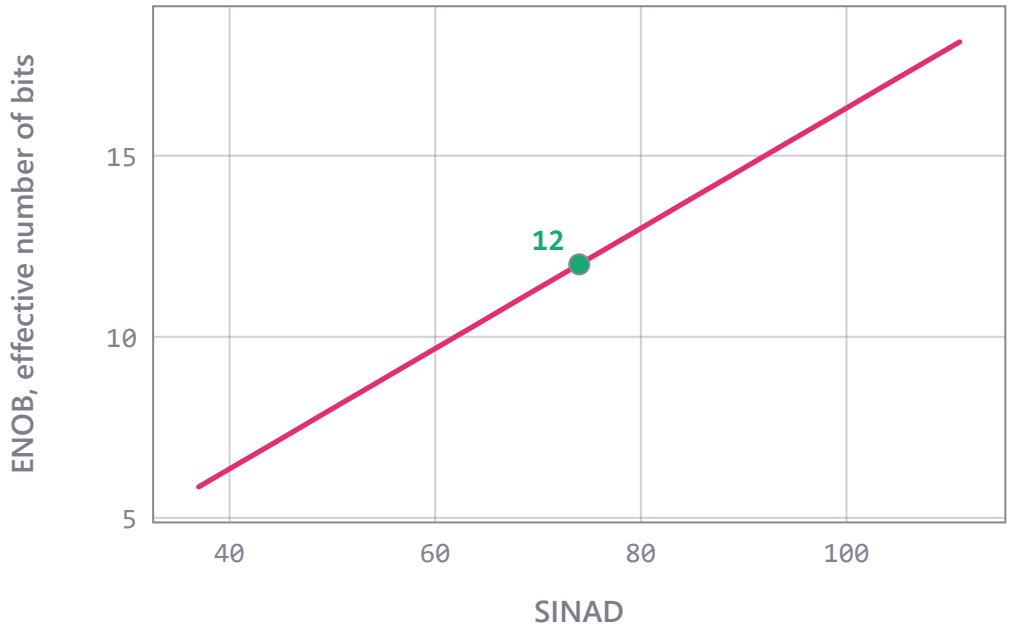
$$\mathrm{ENOB} = \frac{\mathrm{SINAD} - 1.76}{6.02}$$

input	meaning	condition
`SINAD`	signal to noise and distortion ratio in dB	

input	meaning	condition
`n`	nominal resolution in bits	

output	meaning
`ENOB`	effective number of bits
`bits_lost`	n - ENOB

- **Method:** ENOB = (SINAD - 1.76)/6.02
- **Assumes:** full-scale sine test
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ENOB (effective number of bits) against SINAD (signal to noise and distortion ratio in dB) from 37 to 111. Other inputs as in the worked example: n = 14. 31 points computed by electronics-engineering-ai999x; marked: the worked example, SINAD = 74, ENOB = 12.

```
electronics_engineering_ai999x.solve('elx_adc_enob', SINAD=74.0, n=14)
# -> ENOB = 12, bits_lost = 2   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_adc_enob', SINAD=62.5, n=12)
# -> ENOB = 10.0897, bits_lost = 1.9103   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ADC ENOB: SINAD = 74 dB, n = 14	SOLVED	ENOB = 12, bits_lost = 2
ADC ENOB: SINAD = 62.5 dB, n = 12	SOLVED	ENOB = 10.0897, bits_lost = 1.9103
ADC ENOB: n = 14	NEEDS_INPUT	elx_adc_enob needs SINAD (signal to noise and distortion ratio in dB). Example: solve('elx_adc_enob', SINAD=74, n=14).

question	outcome	result
ADC ENOB: SINAD = 74 dB, n = 14, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

SNR limit from sampling-clock (aperture) jitter, `elx_aperture_jitter_snr`

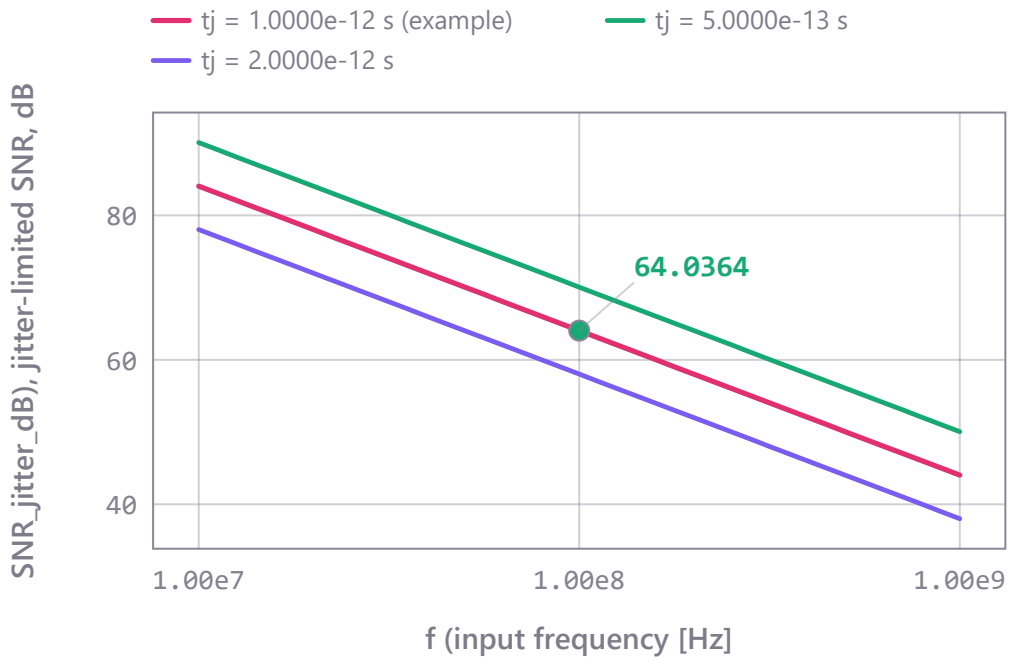
The SNR limit -20 log10(2 pi f tj) set by rms sampling-clock jitter for a full-scale sine, combined with the quantisation SNR of an n-bit converter into a total SNR and effective bits.

$$\mathrm{SNR}_j = -20\log_{10}(2\pi f t_j)$$

input	meaning	condition
`f`	input frequency	
`tj`	rms clock jitter	
`n`	converter resolution in bits	

output	meaning
`SNR_jitter_dB`	jitter-limited SNR, dB
`SNR_quant_dB`	quantisation SNR, dB
`SNR_total_dB`	combined SNR, dB
`ENOB_total`	effective bits of the combination

- **Method:** SNR_j = -20 log10(2 pi f tj); SNR_q = 6.02 $n + 1.76$; noise powers add
- **Assumes:** full-scale sine; jitter and quantisation noise uncorrelated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



SNR_jitter_dB (jitter-limited SNR, dB) against f (input frequency) from 1.0000e+07 Hz to 1.0000e+09 Hz on a logarithmic axis. One curve for each value of tj (rms clock jitter): 1.0000e-12 s, 5.0000e-13 s, 2.0000e-12 s. Other inputs as in the worked example: n = 14. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, f = 1.0000e+08 Hz, SNR_jitter_dB = 64.0364.

```
electronics_engineering_ai999x.solve('elx_aperture_jitter_snr', f=100000000.0, tj=1e-12, n=14)
# -> SNR_jitter_dB = 64.0364, SNR_quant_dB = 86.0493, SNR_total_dB = 64.0092, ENOB_total = 10.3404
(SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_aperture_jitter_snr', f=10000000.0, tj=5e-13, n=16)
# -> SNR_jitter_dB = 90.057, SNR_quant_dB = 98.0905, SNR_total_dB = 89.4227, ENOB_total = 14.5619
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Aperture jitter SNR: f = 100000000 Hz, tj = 1 ps, n = 14	SOLVED	SNR_jitter_dB = 64.0364, SNR_quant_dB = 86.0493, SNR_total_dB = 64.0092, ENOB_total = 10.3404
Aperture jitter SNR: f = 10000000 Hz, tj = 0.5 ps, n = 16	SOLVED	SNR_jitter_dB = 90.057, SNR_quant_dB = 98.0905, SNR_total_dB = 89.4227, ENOB_total = 14.5619
Aperture jitter SNR: tj = 1 ps, n = 14	NEEDS_INPUT	elx_aperture_jitter_snr needs f (input frequency). Example: solve('elx_aperture_jitter_snr', f='1e+08 Hz', tj='1e-12 s',
Aperture jitter SNR: f = 100000000 Hz, tj = 1 ps, n = 14, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

DAC output voltage (R-2R or any binary DAC), `elx_dac_output`

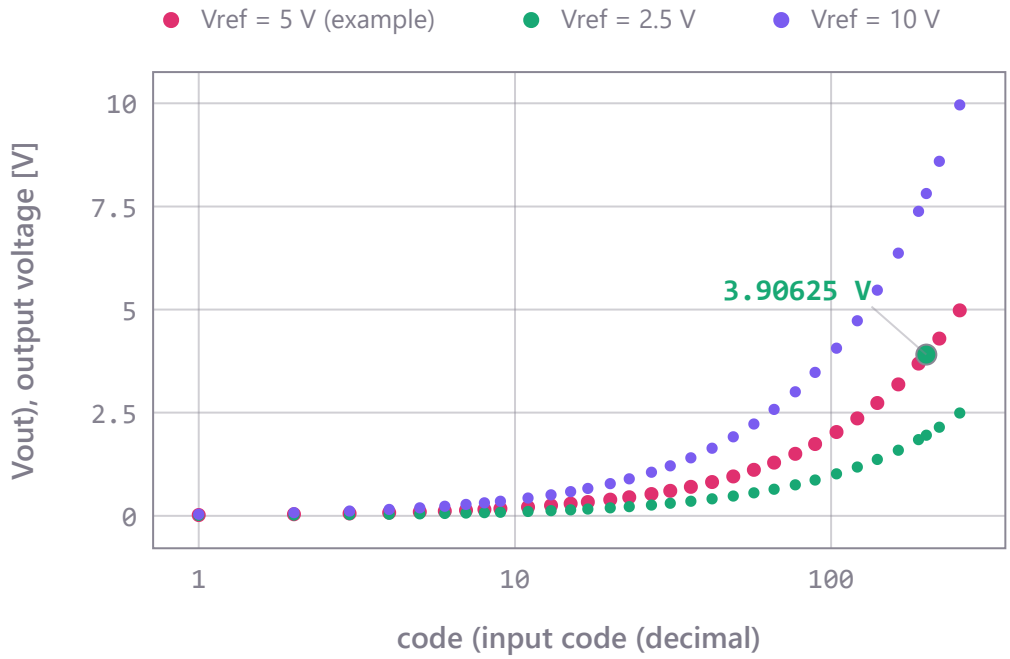
The output voltage of an ideal n-bit unipolar DAC for a code, Vout = Vref $D/2^n$, its step size and its full-scale output.

$$V_{out} = V_{ref}\frac{D}{2^n}$$

input	meaning	condition
`n`	resolution in bits	
`code`	input code (decimal)	
`Vref`	reference voltage	

output	meaning
`Vout`	output voltage
`LSB`	step size Vref/2^n
`V_full_scale`	output for the all-ones code

- **Method:** Vout = Vref $D/2^n$; full scale Vref $(2^n - 1)/2^n$
- **Assumes:** 0 <= code <= 2^n - 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout (output voltage) against code (input code (decimal)) from 1 to 400 on a logarithmic axis. One curve for each value of Vref (reference voltage): 5 V, 2.5 V, 10 V. Other inputs as in the worked example: n = 8. 32 points per curve computed by electronics-engineering-ai999x; marked: the worked example, code = 200, Vout = 3.90625 V.

```
electronics_engineering_ai999x.solve('elx_dac_output', n=8, code=200, Vref=5.0)
# -> Vout = 3.90625 V, LSB = 0.0195312 V, V_full_scale = 4.98047 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_dac_output', n=4, code=11, Vref=10.0)
# -> Vout = 6.875 V, LSB = 0.625 V, V_full_scale = 9.375 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
DAC output voltage: n = 8, code = 200, Vref = 5 V	SOLVED	Vout = 3.90625 V, LSB = 0.0195312 V, V_full_scale = 4.98047 V
DAC output voltage: n = 4, code = 11, Vref = 10 V	SOLVED	Vout = 6.875 V, LSB = 0.625 V, V_full_scale = 9.375 V
DAC output voltage: code = 200, Vref = 5 V	NEEDS_INPUT	elx_dac_output needs n (resolution in bits). Example: solve('elx_dac_output', n=8, code=200, Vref='5 V').
DAC output voltage: n = 8, code = 200, Vref = 5 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Binary-weighted resistor DAC, `elx_dac_binary_weighted`

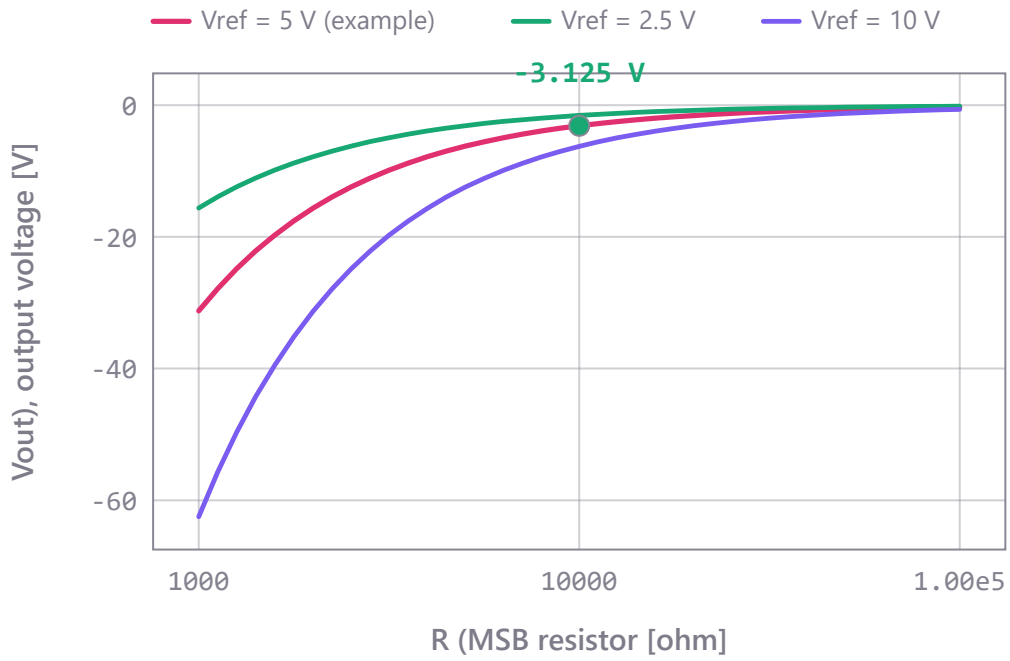
The output voltage and summing current of a binary-weighted resistor DAC with an inverting op-amp (MSB resistor R, each lower bit doubling it) and the largest resistor needed.

$$V_{out} = -V_{ref}\frac{R_f}{R}\sum_{k=0}^{n-1}\frac{b_k}{2^k} = -V_{ref}\frac{R_f}{R}\frac{D}{2^{n-1}}$$

input	meaning	condition
`n`	resolution in bits	
`code`	input code (decimal)	
`Vref`	reference voltage	
`R`	MSB resistor	
`Rf`	feedback resistor	

output	meaning
`Vout`	output voltage
`I_total`	total current into the summing node
`R_LSB`	LSB resistor R 2^{n-1}

- **Method:** I = Vref D/(R 2^{n-1}); Vout = -Rf I
- **Assumes:** ideal switches and op-amp
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout (output voltage) against R (MSB resistor) from 1000 ohm to 100000 ohm on a logarithmic axis. One curve for each value of Vref (reference voltage): 5 V, 2.5 V, 10 V. Other inputs as in the worked example: n = 4, code = 10, Rf = 5000.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R = 10000 ohm, Vout = -3.125 V.

```
electronics_engineering_ai999x.solve('elx_dac_binary_weighted', n=4, code=10, Vref=5.0, R=10000.0,
Rf=5000.0)
# -> Vout = -3.125 V, I_total = 0.000625 A, R_LSB = 80000 ohm   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_dac_binary_weighted', n=8, code=129, Vref=2.5, R=1000.0,
Rf=1000.0)
# -> Vout = -2.51953 V, I_total = 0.00251953 A, R_LSB = 128000 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Binary-weighted DAC: n = 4, code = 10, Vref = 5 V, R = 10000 ohm, Rf = 5000 ohm	SOLVED	Vout = -3.125 V, I_total = 0.000625 A, R_LSB = 80000 ohm
Binary-weighted DAC: n = 8, code = 129, Vref = 2.5 V, R = 1000 ohm, Rf = 1000 ohm	SOLVED	Vout = -2.51953 V, I_total = 0.00251953 A, R_LSB = 128000 ohm
Binary-weighted DAC: code = 10, Vref = 5 V, R = 10000 ohm, Rf = 5000 ohm	NEEDS_INPUT	elx_dac_binary_weighted needs n (resolution in bits). Example: solve('elx_dac_binary_weighted', n=4, code=10, Vref='5 V'
Binary-weighted DAC: n = 4, code = 10, Vref = 5 V, R = 10000 ohm, Rf = 5000 ohm, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Successive-approximation (SAR) ADC conversion, `elx_sar_conversion`

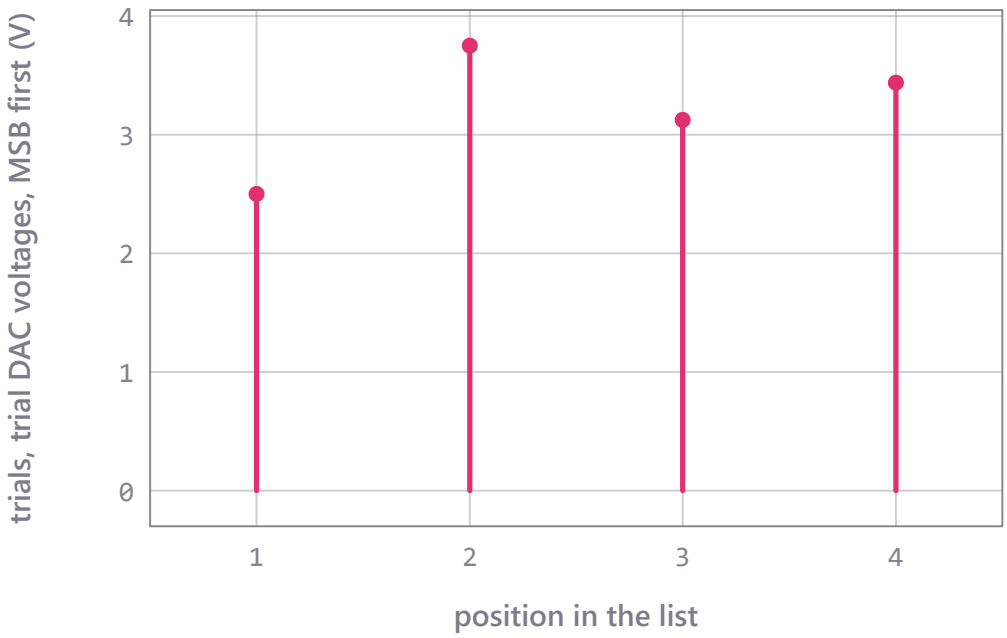
The bit-by-bit trial voltages, the final code (decimal and binary) and the DAC voltage of an ideal n-bit successive-approximation ADC converting an input.

$$b_k = \left[V_{in} \geq V_{DAC}(D_k + 2^k)\right]$$

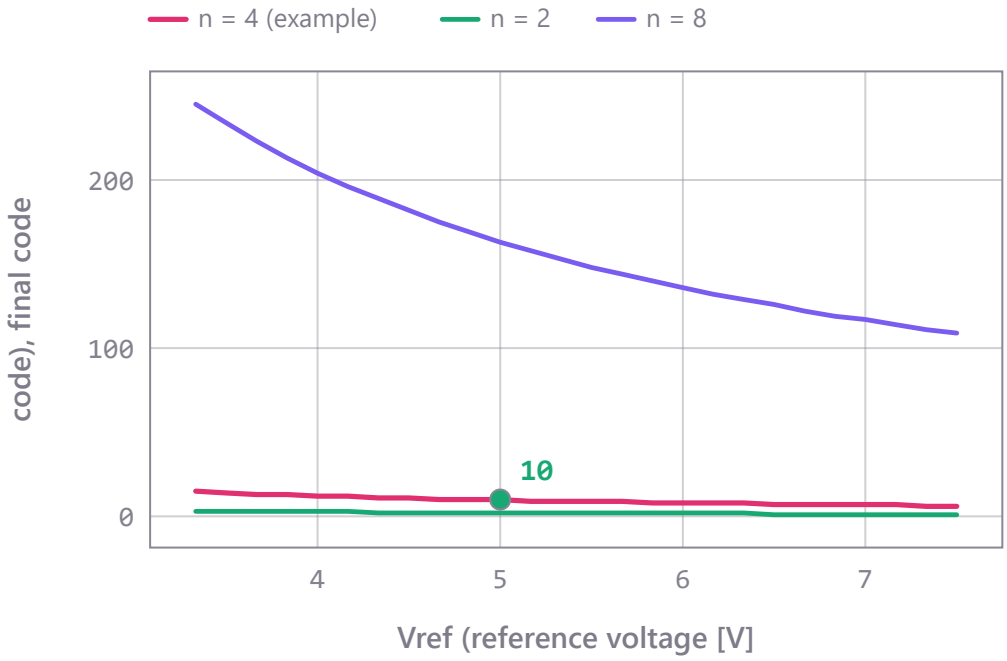
input	meaning	condition
`Vin`	analog input	
`Vref`	reference voltage	
`n`	resolution in bits	

output	meaning
`code`	final code
`bits`	final code in binary
`trials`	trial DAC voltages, MSB first (V)
`V_dac`	DAC voltage of the final code
`cycles`	comparison cycles

- **Method:** for each bit from the MSB: set it, keep it if Vin >= DAC(trial); LSB = Vref/2^n
- **Assumes:** 0 <= Vin < Vref
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



trial DAC voltages, MSB first (V) (trials) returned by electronics-engineering-ai999x for the worked example, one value for each position in the list (4 values, first 2.5, last 3.4375, largest 3.75). Inputs: Vin = 3.2 V, Vref = 5.0 V, n = 4.



code (final code) against Vref (reference voltage) from 2.5 V to 7.5 V. One curve for each value of n (resolution in bits): 4, 2, 8. Other inputs as in the worked example: Vin = 3.2 V. 26 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Vref = 5 V, code = 10.

```
electronics_engineering_ai999x.solve('elx_sar_conversion', Vin=3.2, Vref=5.0, n=4)
# -> code = 10, bits = 1010, trials = [2.5, 3.75, 3.125, 3.4375], V_dac = 3.125 V, cycles = 4
(SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_sar_conversion', Vin=1.7, Vref=3.3, n=8)
# -> code = 131, bits = 10000011, trials = [1.65, 2.475, 2.0625, 1.85625, 1.753125, 1.7015625,
1.67578125, 1.688671875], V_dac = 1.68867 V, cycles = 8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
SAR ADC conversion: Vin = 3.2 V, Vref = 5 V, n = 4	SOLVED	code = 10, bits = 1010, trials = [2.5, 3.75, 3.125, 3.4375], V_dac = 3.125 V, cycles = 4
SAR ADC conversion: Vin = 1.7 V, Vref = 3.3 V, n = 8	SOLVED	code = 131, bits = 10000011, trials = [1.65, 2.475, 2.0625, 1.85625, 1.753125, 1.7015625, 1.67578125, 1.688671875], V_dac = 1.68867 V, cycles = 8
SAR ADC conversion: Vref = 5 V, n = 4	NEEDS_INPUT	elx_sar_conversion needs Vin (analog input). Example: solve('elx_sar_conversion', Vin='3.2 V', Vref='5 V', n=4).
SAR ADC conversion: Vin = 3.2 V, Vref = 5 V, n = 4, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Dual-slope integrating ADC, `elx_dual_slope_adc`

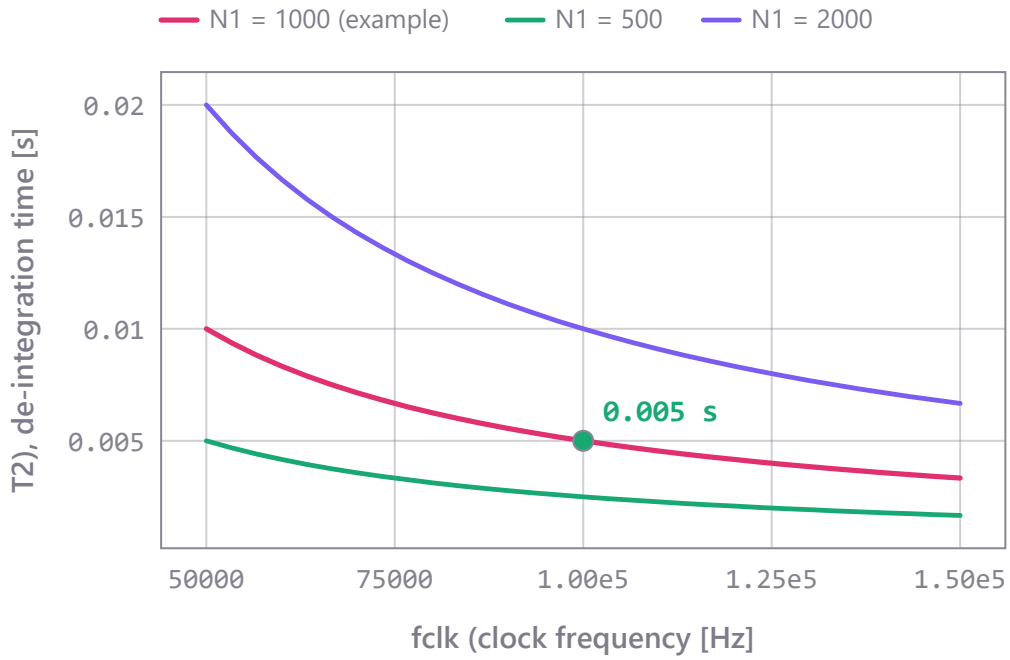
The fixed integration time, the de-integration time and count, and the peak integrator voltage of a dual-slope ADC.

$$T_2 = T_1 \frac{V_{in}}{V_{ref}}, \quad N_2 = N_1 \frac{V_{in}}{V_{ref}}$$

input	meaning	condition
`Vin`	input voltage (magnitude)	
`Vref`	reference voltage (magnitude)	
`N1`	fixed count of the integration phase	
`fclk`	clock frequency	
`R`	integrator resistor	
`C`	integrator capacitor	

output	meaning
`T1`	integration time N1/fclk
`T2`	de-integration time
`N2`	count of the de-integration phase (unrounded)
`V_peak`	peak integrator output

- **Method:** T1 = N1/fclk; $T_2 = T_1$ Vin/Vref; $N_2 = N_1$ Vin/Vref; V_peak = Vin T1/(R C)
- **Assumes:** 0 < Vin <= Vref
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



T2 (de-integration time) against fclk (clock frequency) from 50000 Hz to 150000 Hz. One curve for each value of N1 (fixed count of the integration phase): 1000, 500, 2000. Other inputs as in the worked example: Vin = 0.5 V, Vref = 1.0 V, R = 100000.0 ohm, C = 1e-07 F. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, fclk = 100000 Hz, T2 = 0.005 s.

```
electronics_engineering_ai999x.solve('elx_dual_slope_adc', Vin=0.5, Vref=1.0, N1=1000,
fclk=100000.0, R=100000.0, C=1e-07)
# -> T1 = 0.01 s, T2 = 0.005 s, N2 = 500, V_peak = 0.5 V   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_dual_slope_adc', Vin=1.234, Vref=2.0, N1=2000,
fclk=1000000.0, R=47000.0, C=2.2e-07)
# -> T1 = 0.002 s, T2 = 0.001234 s, N2 = 1234, V_peak = 0.238685 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dual-slope ADC: Vin = 0.5 V, Vref = 1 V, N1 = 1000, fclk = 100000 Hz, R = 100000 ohm, C = 0.1 uF	SOLVED	T1 = 0.01 s, T2 = 0.005 s, N2 = 500, V_peak = 0.5 V
Dual-slope ADC: Vin = 1.234 V, Vref = 2 V, N1 = 2000, fclk = 1000000 Hz, R = 47000 ohm, C = 0.22 uF	SOLVED	T1 = 0.002 s, T2 = 0.001234 s, N2 = 1234, V_peak = 0.238685 V
Dual-slope ADC: Vref = 1 V, N1 = 1000, fclk = 100000 Hz, R = 100000 ohm, C = 0.1 uF	NEEDS_INPUT	elx_dual_slope_adc needs Vin (input voltage (magnitude)). Example: solve('elx_dual_slope_adc', Vin='0.5 V', Vref='1 V',
Dual-slope ADC: Vin = 0.5 V, Vref = 1 V, N1 = 1000, fclk = 100000 Hz, R = 100000 ohm, C = 0.1 uF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Sample-and-hold droop, `elx_sample_hold_droop`

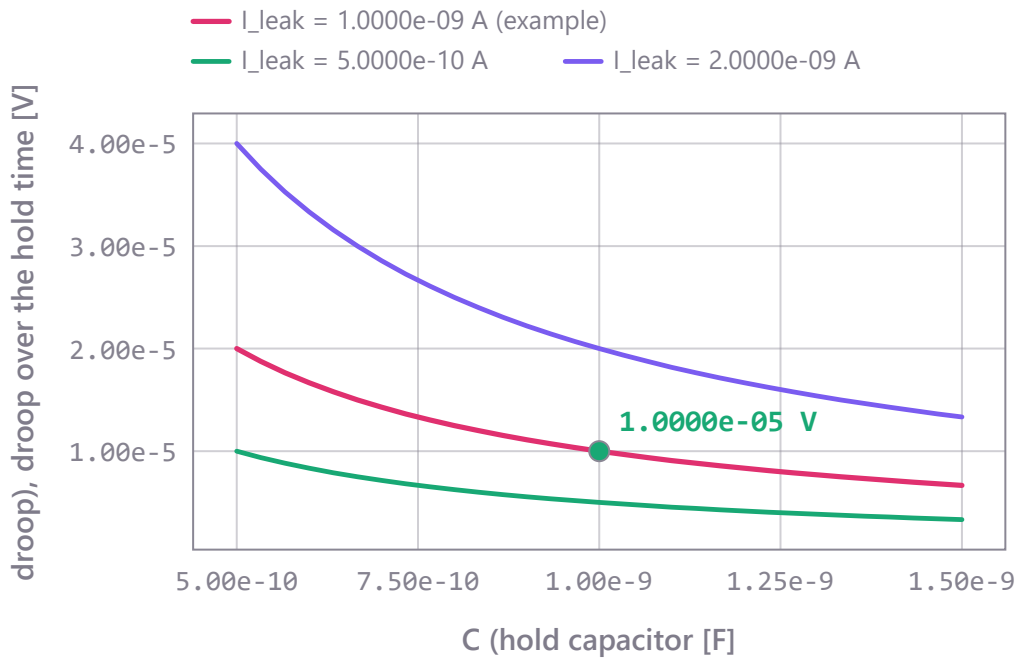
The droop rate I/C of a sample-and-hold capacitor discharged by a leakage current, the droop over a hold time (in volts and LSB) and the longest hold that keeps the droop within half an LSB of an n-bit converter.

$$\frac{dV}{dt} = \frac{I_{leak}}{C}$$

input	meaning	condition
`I_leak`	leakage (bias) current	
`C`	hold capacitor	
`t_hold`	hold time	
`Vref`	converter full scale	
`n`	converter resolution in bits	

output	meaning
`droop_rate`	droop rate I/C
`droop`	droop over the hold time
`droop_lsb`	droop in LSB
`t_max_half_lsb`	longest hold for half an LSB

- **Method:** $dV/dt = I/C$; $dV = I$ t/C; t_max = (Vref/2^(n+1)) C/I
- **Assumes:** constant leakage current
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



droop (droop over the hold time) against C (hold capacitor) from 5.0000e-10 F to 1.5000e-09 F. One curve for each value of I_leak (leakage (bias) current): 1.0000e-09 A, 5.0000e-10 A, 2.0000e-09 A. Other inputs as in the worked example: t_hold = 1e-05 s, Vref = 5.0 V, n = 12. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, C = 1.0000e-09 F, droop = 1.0000e-05 V.

```
electronics_engineering_ai999x.solve('elx_sample_hold_droop', I_leak=1e-09, C=1e-09, t_hold=1e-05, Vref=5.0, n=12)
# -> droop_rate = 1 V/s, droop = 1e-05 V, droop_lsb = 0.008192, t_max_half_lsb = 0.000610352 s (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_sample_hold_droop', I_leak=5e-11, C=1e-10, t_hold=2e-05,
Vref=10.0, n=16)
# -> droop_rate = 0.5 V/s, droop = 1e-05 V, droop_lsb = 0.065536, t_max_half_lsb = 0.000152588 s
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sample-and-hold droop: I_leak = 1 nA, C = 1 nF, t_hold = 10 us, Vref = 5 V, n = 12	SOLVED	droop_rate = 1 V/s, droop = 1e-05 V, droop_lsb = 0.008192, t_max_half_lsb = 0.000610352 s
Sample-and-hold droop: I_leak = 0.05 nA, C = 0.1 nF, t_hold = 20 us, Vref = 10 V, n = 16	SOLVED	droop_rate = 0.5 V/s, droop = 1e-05 V, droop_lsb = 0.065536, t_max_half_lsb = 0.000152588 s
Sample-and-hold droop: C = 1 nF, t_hold = 10 us, Vref = 5 V, n = 12	NEEDS_INPUT	elx_sample_hold_droop needs I_leak (leakage (bias) current). Example: solve('elx_sample_hold_droop', I_leak='1e-09 A', C
Sample-and-hold droop: I_leak = 1 nA, C = 1 nF, t_hold = 10 us, Vref = 5 V, n = 12, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

RC delay, 10-90 % rise time and bandwidth, `elx_rc_delay_rise_time`

The time constant, the 50 % delay (ln 2 RC), the $\mathbf{10-90}$ % rise time (ln 9 $\boldsymbol{RC = 2.2}$ RC), the -3 dB bandwidth and the rise-time-bandwidth product (0.35) of a first-order RC low-pass.

$$t_r = \ln 9\, RC \approx 2.2RC, \quad t_r f_{3dB} \approx 0.35$$

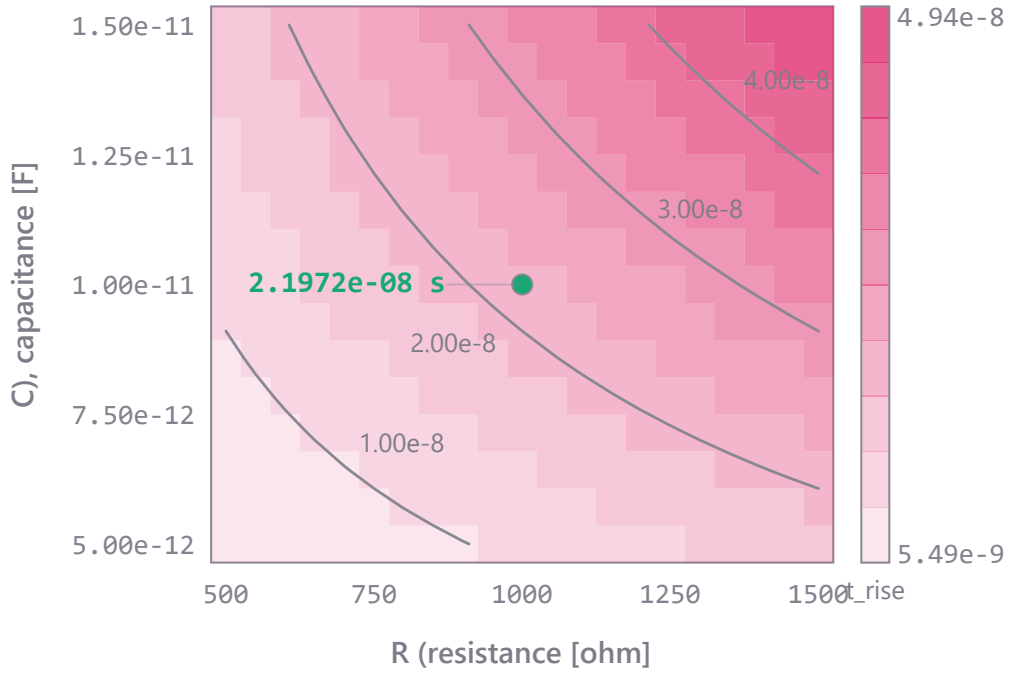
input	meaning	condition
`R`	resistance	
`C`	capacitance	

output	meaning
`tau`	time constant RC
`t50`	50 % delay
`t_rise`	$\mathbf{10-90}$ % rise time
`f_3dB`	-3 dB bandwidth
`tr_times_bw`	rise time x bandwidth

- **Method:** $\boldsymbol{t50 = ln}$ 2 RC; $\boldsymbol{tr = ln}$ 9 RC; f = 1/(2 pi RC); tr $\boldsymbol{f = ln}$ 9/(2 pi)
- **Assumes:** step input, first-order RC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t_rise (10-90 % rise time) against R (resistance) from 500 ohm to 1500 ohm. One curve for each value of C (capacitance): 1.0000e-11 F, 5.0000e-12 F, 2.0000e-11 F. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R = 1000 ohm, t_rise = 2.1972e-08 s.



t_rise (10-90 % rise time) over R (resistance) from 500 ohm to 1500 ohm and C (capacitance) from 5.0000e-12 F to 1.5000e-11 F: each of the 21 × 15 cells is one run of electronics-engineering-ai999x, shaded by its value (light 5.4931e-09 s to dark 4.9438e-08 s); iso-lines at t_rise = 1.00e-8, 2.00e-8, 3.00e-8, 4.00e-8 s. Marked: the worked example, R = 1000 ohm, C = 1.0000e-11 F, t_rise = 2.1972e-08 s.

```
electronics_engineering_ai999x.solve('elx_rc_delay_rise_time', R=1000.0, C=1e-11)
# -> tau = 1e-08 s, t50 = 6.93147e-09 s, t_rise = 2.19722e-08 s, f_3dB = 1.59155e+07 Hz,
tr_times_bw = 0.349699   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_rc_delay_rise_time', R=50.0, C=1e-10)
# -> tau = 5e-09 s, t50 = 3.46574e-09 s, t_rise = 1.09861e-08 s, f_3dB = 3.1831e+07 Hz, tr_times_bw
= 0.349699   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
RC delay and rise time: R = 1000 ohm, C = 10 pF	SOLVED	tau = 1e-08 s, t50 = 6.93147e-09 s, t_rise = 2.19722e-08 s, f_3dB = 1.59155e+07 Hz, tr_times_bw = 0.349699
RC delay and rise time: R = 50 ohm, C = 100 pF	SOLVED	tau = 5e-09 s, t50 = 3.46574e-09 s, t_rise = 1.09861e-08 s, f_3dB = 3.1831e+07 Hz, tr_times_bw = 0.349699
RC delay and rise time: C = 10 pF	NEEDS_INPUT	elx_rc_delay_rise_time needs R (resistance). Example: solve('elx_rc_delay_rise_time', R='1000 ohm', C='1e-11 F').
RC delay and rise time: R = 1000 ohm, C = 10 pF, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Elmore delay of an RC ladder (interconnect), `elx_elmore_delay`

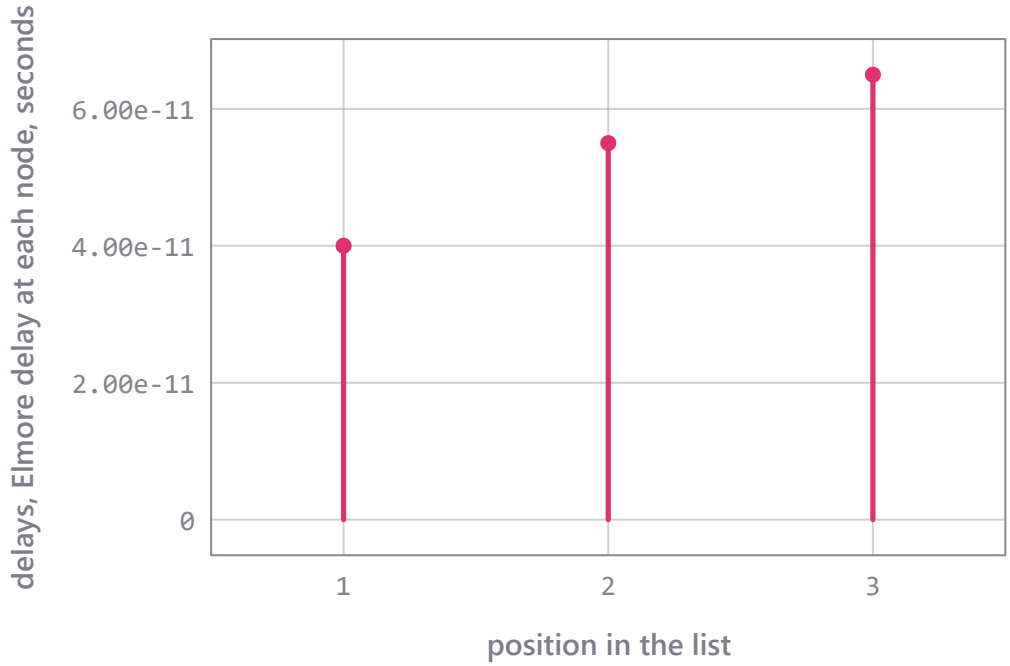
The Elmore delay to every node of an RC ladder (a driver resistance and wire segments, each followed by a capacitance to ground), the delay at the far end and its 50 % estimate ln 2 tau.

$$\tau_D = \sum_i C_i \sum_{j \le i} R_j$$

input	meaning	condition
`R`	series resistances in ohms, from the driver outwards	
`C`	node capacitances in picofarads, one after each resistor	

output	meaning
`tau_D`	Elmore delay at the far end
`t50`	50 % delay estimate ln 2 tau_D
`delays`	Elmore delay at each node, seconds

- **Method:** tau_k = sum_i C_i R(shared path of nodes i and k)
- **Assumes:** one capacitance per resistor; step input
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Elmore delay at each node, seconds (delays) returned by electronics-engineering-ai999x for the worked example, one value for each position in the list (3 values, first 4.0000e-11, last 6.5000e-11, largest 6.5000e-11). Inputs: R = [100.0, 50.0, 50.0], C = [0.1, 0.1, 0.2].

```
electronics_engineering_ai999x.solve('elx_elmore_delay', R=[100.0, 50.0, 50.0], C=[0.1, 0.1, 0.2])
# -> tau_D = 6.5e-11 s, t50 = 4.50546e-11 s, delays = [4e-11, 5.5e-11, 6.5e-11]   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_elmore_delay', R=[1000.0, 200.0], C=[0.05, 0.5])
# -> tau_D = 6.5e-10 s, t50 = 4.50546e-10 s, delays = [5.5e-10, 6.5e-10]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Elmore delay: R = [100, 50, 50], C = [0.1, 0.1, 0.2]	SOLVED	tau_D = 6.5e-11 s, t50 = 4.50546e-11 s, delays = [4e-11, 5.5e-11, 6.5e-11]
Elmore delay: R = [1000, 200], C = [0.05, 0.5]	SOLVED	tau_D = 6.5e-10 s, t50 = 4.50546e-10 s, delays = [5.5e-10, 6.5e-10]
Elmore delay: C = [0.1, 0.1, 0.2]	NEEDS_INPUT	elx_elmore_delay needs R (series resistances in ohms, from the driver outwards). Example: solve('elx_elmore_delay', R=[1
Elmore delay: R = [100, 50, 50], C = [0.1, 0.1, 0.2], zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

CMOS dynamic (switching) power, `elx_cmos_dynamic_power`

The dynamic power alpha C VDD^2 f of CMOS logic, the energy drawn from the supply per 0 -> 1 transition, the energy stored on the load and the average supply current.

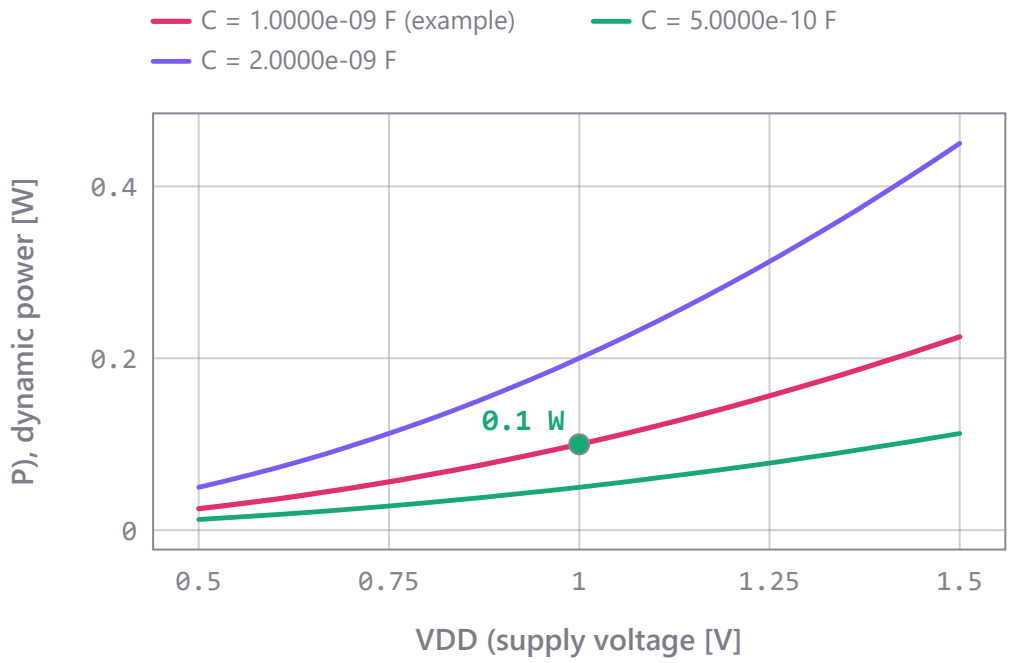
$$P_{dyn} = \alpha C V_{DD}^2 f$$

input	meaning	condition
`alpha`	activity factor: probability of a 0 -> 1 transition per clock cycle	

input	meaning	condition
`C`	switched capacitance	
`VDD`	supply voltage	
`f`	clock frequency	

output	meaning
`P`	dynamic power
`E_supply`	energy from the supply per transition, C VDD^2
`E_stored`	energy stored on C, C VDD^2/2
`I_avg`	average supply current

- **Method:** $P = \alpha$ C VDD^2 f
- **Assumes:** $0 \leq \alpha \leq 1$; full-swing switching, short-circuit and leakage power not included
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P (dynamic power) against VDD (supply voltage) from 0.5 V to 1.5 V. One curve for each value of C (switched capacitance): 1.0000e-09 F, 5.0000e-10 F, 2.0000e-09 F. Other inputs as in the worked example: alpha = 0.1, f = 1000000000.0 Hz. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, VDD = 1 V, P = 0.1 W.

```
electronics_engineering_ai999x.solve('elx_cmos_dynamic_power', alpha=0.1, C=1e-09, VDD=1.0,
f=1000000000.0)
# -> P = 0.1 W, E_supply = 1e-09 J, E_stored = 5e-10 J, I_avg = 0.1 A   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_cmos_dynamic_power', alpha=0.25, C=5e-11, VDD=3.3,
f=100000000.0)
# -> P = 0.0136125 W, E_supply = 5.445e-10 J, E_stored = 2.7225e-10 J, I_avg = 0.004125 A
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
CMOS dynamic power: alpha = 0.1, C = 1000 pF, VDD = 1 V, f = 1000000000 Hz	SOLVED	P = 0.1 W, E_supply = 1e-09 J, E_stored = 5e-10 J, I_avg = 0.1 A
CMOS dynamic power: alpha = 0.25, C = 50 pF, VDD = 3.3 V, f = 100000000 Hz	SOLVED	P = 0.0136125 W, E_supply = 5.445e-10 J, E_stored = 2.7225e-10 J, I_avg = 0.004125 A
CMOS dynamic power: C = 1000 pF, VDD = 1 V, f = 1000000000 Hz	NEEDS_INPUT	elx_cmos_dynamic_power needs alpha (activity factor: probability of a 0 -> 1 transition per clock cycle). Example: solve
CMOS dynamic power: alpha = 0.1, C = 1000 pF, VDD = 1 V, f = 1000000000 Hz, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Flip-flop setup and hold slack, maximum clock frequency, `elx_setup_hold_timing`

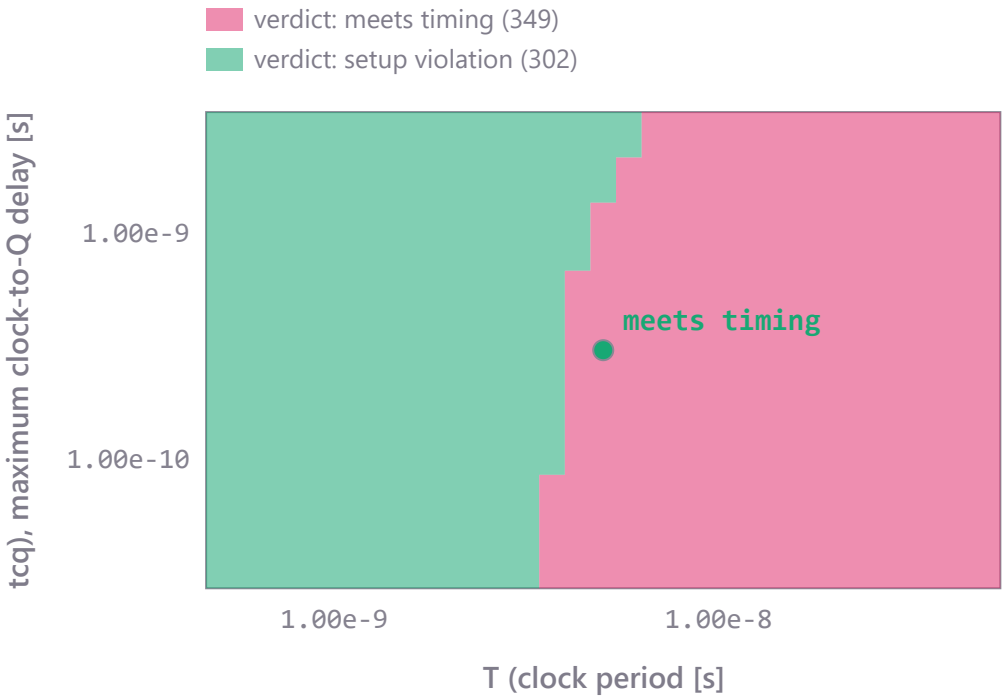
The setup slack, hold slack, maximum clock frequency and a verdict for a register-to-register path from clock-to-Q, logic delays, setup and hold times and clock skew (positive when the capture clock is late).

$$T \geq t_{cq} + t_{pd} + t_{su} - t_{skew}, \quad t_{ccq} + t_{cd} \geq t_h + t_{skew}$$

input	meaning	condition
`T`	clock period	
`tcq`	maximum clock-to-Q delay	
`tpd`	maximum logic delay	
`tsu`	setup time	
`tskew`	clock skew, capture minus launch	
`tccq`	minimum (contamination) clock-to-Q	
`tcd`	minimum (contamination) logic delay	
`th`	hold time	

output	meaning
`setup_slack`	setup slack
`hold_slack`	hold slack
`f_max`	maximum clock frequency
`verdict`	meets timing, setup violation, hold violation or both

- **Method:** setup slack = T + skew - (tcq + tpd + tsu); hold slack = tccq + tcd - th - skew; f_max = 1/(tcq + tpd + tsu - skew)
- **Assumes:** edge-triggered registers, single clock
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



meets timing, setup violation, hold violation or both (verdict) over T (clock period) from 5.0000e-10 s to 5.0000e-08 s and tcq (maximum clock-to-Q delay) from 3.0000e-11 s to 3.0000e-09 s (logarithmic axes): each of the 31 × 21 cells is one run of electronics-engineering-ai999x, coloured by the verdict it returned, meets timing: 349 cells, setup violation: 302 cells. Other inputs as in the worked example: tpd = 3.5e-09 s, tsu = 2e-10 s, tskew = 1e-10 s, tccq = 1e-10 s, tcd = 2e-10 s, th = 1e-10 s. Marked: the worked example, T = 5.0000e-09 s, tcq = 3.0000e-10 s → meets timing.



setup_slack (setup slack) against T (clock period) from 2.5000e-09 s to 7.5000e-09 s. One curve for each value of tpd (maximum logic delay): 3.5000e-09 s, 1.7500e-09 s, 7.0000e-09 s. Other inputs as in the worked example: tcq = 3e-10 s, tsu = 2e-10 s, tskew = 1e-10 s, tccq = 1e-10 s, tcd = 2e-10 s, th = 1e-10 s. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, T = 5.0000e-09 s, setup_slack = 1.1000e-09 s.

```
electronics_engineering_ai999x.solve('elx_setup_hold_timing', T=5e-09, tcq=3e-10, tpd=3.5e-09,
tsu=2e-10, tskew=1e-10, tccq=1e-10, tcd=2e-10, th=1e-10)
# -> setup_slack = 1.1e-09 s, hold_slack = 1e-10 s, f_max = 2.5641e+08 Hz, verdict = meets timing
(SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_setup_hold_timing', T=2e-09, tcq=2e-10, tpd=1.9e-09,
tsu=1e-10, tskew=-5e-11, tccq=5e-11, tcd=2e-11, th=1e-10)
# -> setup_slack = -2.5e-10 s, hold_slack = 2e-11 s, f_max = 4.44444e+08 Hz, verdict = setup
violation   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Setup and hold timing: T = 5 ns, tcq = 0.3 ns, tpd = 3.5 ns, tsu = 0.2 ns, tskew = 0.1 ns, tccq = 0.1 ns, tcd = 0.2 ns, th = 0.1 ns	SOLVED	setup_slack = 1.1e-09 s, hold_slack = 1e-10 s, f_max = 2.5641e+08 Hz, verdict = meets timing
Setup and hold timing: T = 2 ns, tcq = 0.2 ns, tpd = 1.9 ns, tsu = 0.1 ns, tskew = -0.05 ns, tccq = 0.05 ns, tcd = 0.02 ns, th = 0.1 ns	SOLVED	setup_slack = -2.5e-10 s, hold_slack = 2e-11 s, f_max = 4.44444e+08 Hz, verdict = setup violation
Setup and hold timing: tcq = 0.3 ns, tpd = 3.5 ns, tsu = 0.2 ns, tskew = 0.1 ns, tccq = 0.1 ns, tcd = 0.2 ns, th = 0.1 ns	NEEDS_INPUT	elx_setup_hold_timing needs T (clock period). Example: solve('elx_setup_hold_timing', T='5e-09 s', tcq='3e-10 s', tpd='3
Setup and hold timing: T = 5 ns, tcq = 0.3 ns, tpd = 3.5 ns, tsu = 0.2 ns, tskew = 0.1 ns, tccq = 0.1 ns, tcd = 0.2 ns, th = 0.1 ns, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Synchronizer metastability MTBF, `elx_metastability_mtbf`

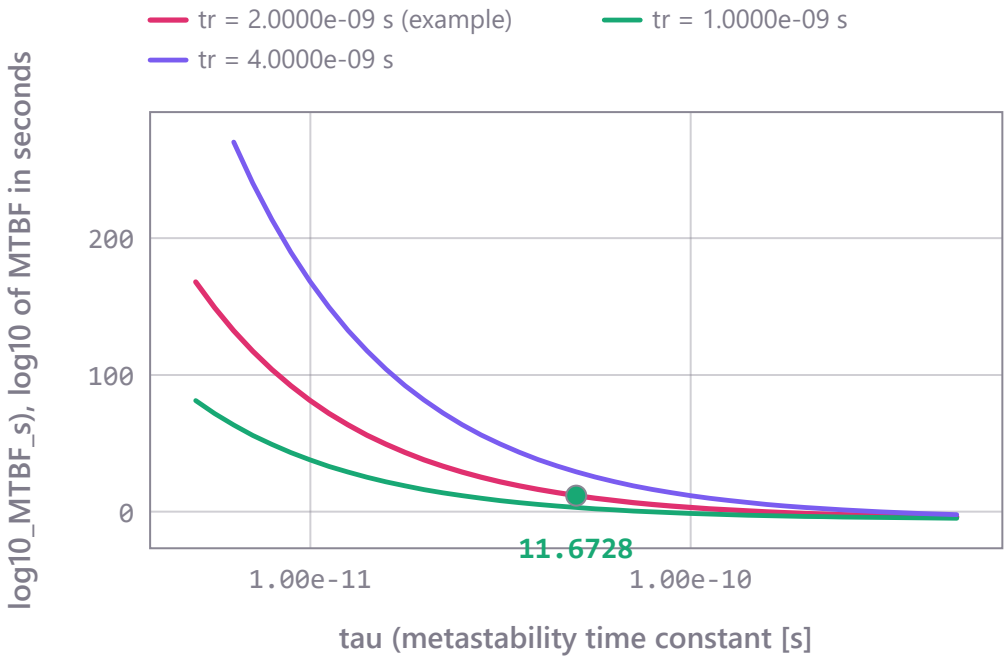
The mean time between synchronizer failures MTBF = exp(tr/τ)/(T_0 fclk fdata) from the resolution time allowed, the flip-flop's time constant and window, and the clock and data rates.

$$\mathrm{MTBF} = \frac{e^{t_r/\tau}}{T_0 f_{clk} f_{data}}$$

input	meaning	condition
`tr`	resolution time available	
`tau`	metastability time constant	
`T0`	metastability window T0	
`fclk`	clock frequency	
`fdata`	rate of asynchronous data changes	

output	meaning
`MTBF_s`	MTBF
`MTBF_years`	MTBF in years (365.25 days)
`log10_MTBF_s`	log10 of MTBF in seconds

- **Method:** MTBF = exp(tr/τ)/(T_0 fclk fdata)
- **Assumes:** $tr/\tau \leq 700$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



log10_MTBF_s (log10 of MTBF in seconds) against tau (metastability time constant) from 5.0000e-12 s to 5.0000e-10 s on a logarithmic axis. One curve for each value of tr (resolution time available): 2.0000e-09 s, 1.0000e-09 s, 4.0000e-09 s. Other inputs as in the worked example: T0 = 1e-10 s, fclk = 500000000.0 Hz, fdata = 10000000.0 Hz. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, tau = 5.0000e-11 s, log10_MTBF_s = 11.6728.

```
electronics_engineering_ai999x.solve('elx_metastability_mtbf', tr=2e-09, tau=5e-11, T0=1e-10,
fclk=500000000.0, fdata=10000000.0)
# -> MTBF_s = 4.70771e+11 s, MTBF_years = 14917.8, log10_MTBF_s = 11.6728   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_metastability_mtbf', tr=1e-09, tau=2e-11, T0=5e-11,
fclk=1000000000.0, fdata=50000000.0)
# -> MTBF_s = 2.07388e+15 s, MTBF_years = 6.57174e+07, log10_MTBF_s = 15.3168   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Metastability MTBF: tr = 2 ns, tau = 50 ps, T0 = 100 ps, fclk = 500000000 Hz, fdata = 10000000 Hz	SOLVED	MTBF_s = 4.70771e+11 s, MTBF_years = 14917.8, log10_MTBF_s = 11.6728
Metastability MTBF: tr = 1 ns, tau = 20 ps, T0 = 50 ps, fclk = 1000000000 Hz, fdata = 50000000 Hz	SOLVED	MTBF_s = 2.07388e+15 s, MTBF_years = 6.57174e+07, log10_MTBF_s = 15.3168
Metastability MTBF: tau = 50 ps, T0 = 100 ps, fclk = 500000000 Hz, fdata = 10000000 Hz	NEEDS_INPUT	elx_metastability_mtbf needs tr (resolution time available). Example: solve('elx_metastability_mtbf', tr='2e-09 s', tau=
Metastability MTBF: tr = 2 ns, tau = 50 ps, T0 = 100 ps, fclk = 500000000 Hz, fdata = 10000000 Hz, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Linear regulator dissipation and efficiency, `elx_linear_regulator_efficiency`

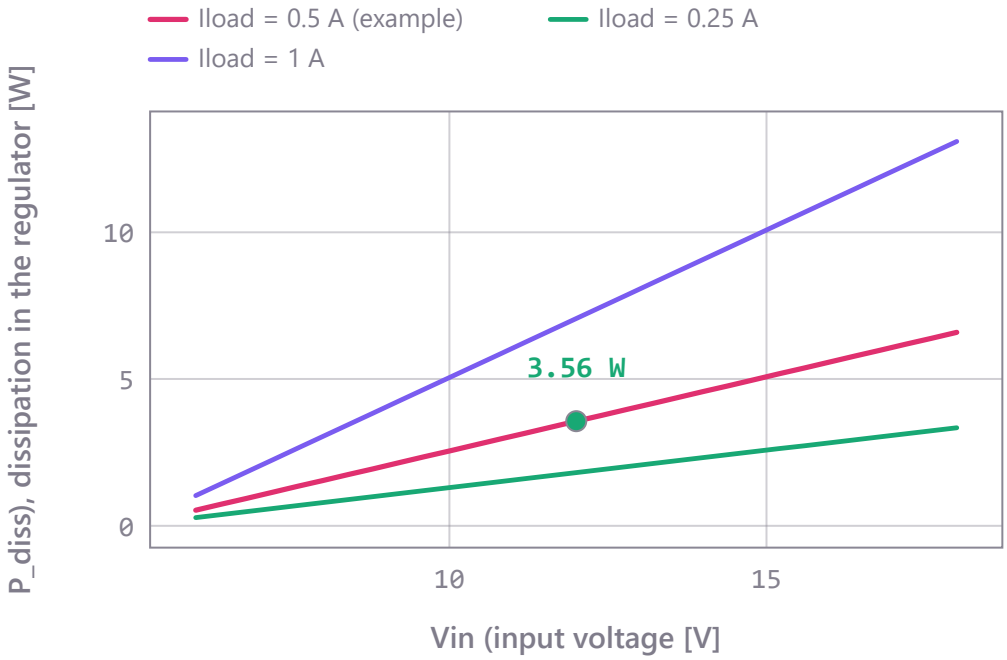
The input and output power, the dissipation in the pass element and the efficiency of a linear (series) regulator including its quiescent current.

$$P_D = (V_{in} - V_{out})I_L + V_{in}I_q, \quad \eta = \frac{V_{out}I_L}{V_{in}(I_L + I_q)}$$

input	meaning	condition
`Vin`	input voltage	
`Vout`	regulated output voltage	
`Iload`	load current	
`Iq`	quiescent (ground-pin) current	

output	meaning
`P_in`	input power
`P_out`	output power
`P_diss`	dissipation in the regulator
`efficiency`	efficiency
`efficiency_pct`	efficiency in per cent

- **Method:** P_in = Vin (IL + Iq); P_out = Vout IL; P_diss = P_in - P_out; eta = P_out/P_in
- **Assumes:** Vin > Vout
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P_diss (dissipation in the regulator) against Vin (input voltage) from 6 V to 18 V. One curve for each value of Iload (load current): 0.5 A, 0.25 A, 1 A. Other inputs as in the worked example: Vout = 5.0 V, Iq = 0.005 A. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Vin = 12 V, P_diss = 3.56 W.

```
electronics_engineering_ai999x.solve('elx_linear_regulator_efficiency', Vin=12.0, Vout=5.0,
Iload=0.5, Iq=0.005)
# -> P_in = 6.06 W, P_out = 2.5 W, P_diss = 3.56 W, efficiency = 0.412541, efficiency_pct = 41.2541
(SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_linear_regulator_efficiency', Vin=5.0, Vout=3.3,
Iload=0.2, Iq=5e-05)
# -> P_in = 1.00025 W, P_out = 0.66 W, P_diss = 0.34025 W, efficiency = 0.659835, efficiency_pct =
65.9835   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Linear regulator efficiency: Vin = 12 V, Vout = 5 V, Iload = 500 mA, Iq = 5 mA	SOLVED	P_in = 6.06 W, P_out = 2.5 W, P_diss = 3.56 W, efficiency = 0.412541, efficiency_pct = 41.2541
Linear regulator efficiency: Vin = 5 V, Vout = 3.3 V, Iload = 200 mA, Iq = 0.05 mA	SOLVED	P_in = 1.00025 W, P_out = 0.66 W, P_diss = 0.34025 W, efficiency = 0.659835, efficiency_pct = 65.9835
Linear regulator efficiency: Vout = 5 V, Iload = 500 mA, Iq = 5 mA	NEEDS_INPUT	elx_linear_regulator_efficiency needs Vin (input voltage). Example: solve('elx_linear_regulator_efficiency', Vin='12 V',
Linear regulator efficiency: Vin = 12 V, Vout = 5 V, Iload = 500 mA, Iq = 5 mA, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Adjustable three-terminal regulator output voltage, `elx_adjustable_regulator_output`

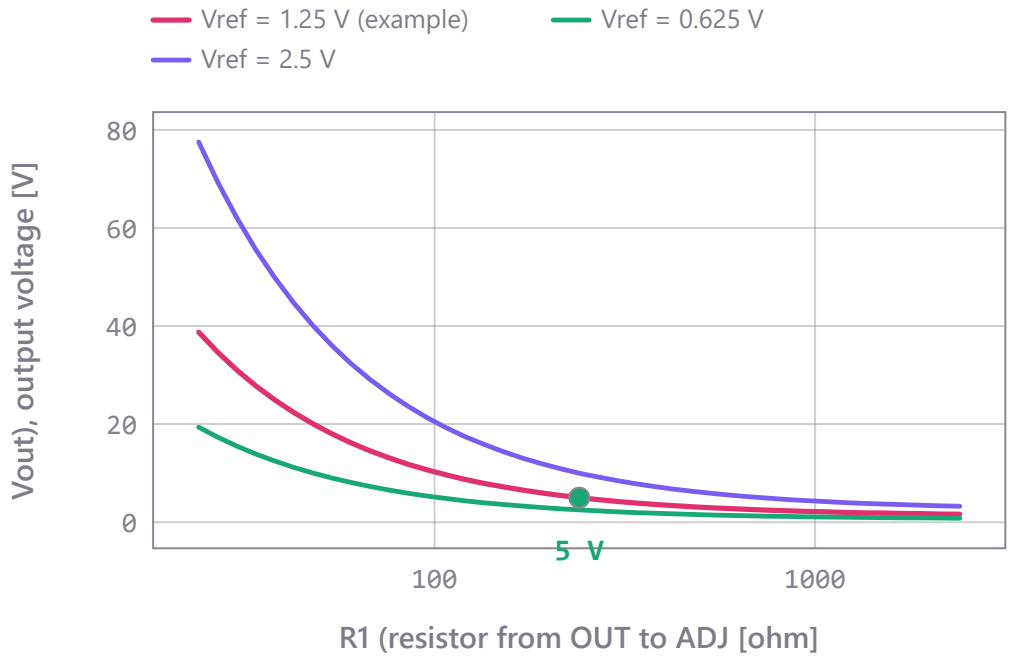
The output voltage of an adjustable three-terminal regulator (LM317 type: reference between OUT and ADJ, R1 from OUT to ADJ, R2 from ADJ to ground), including the adjust-pin current.

$$V_{out} = V_{ref}\left(1 + \frac{R_2}{R_1}\right) + I_{adj}R_2$$

input	meaning	condition
`R1`	resistor from OUT to ADJ	
`R2`	resistor from ADJ to ground	
`Vref`	internal reference (1.25 V for the LM317 type)	
`Iadj`	adjust-pin current (default 0)	

output	meaning
`Vout`	output voltage
`I_R1`	current through R1

- **Method:** Vout = Vref (1 + R2/R1) + Iadj R2
- **Assumes:** regulator in regulation (input high enough)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout (output voltage) against R1 (resistor from OUT to ADJ) from 24 ohm to 2400 ohm on a logarithmic axis. One curve for each value of Vref (internal reference (1.25 V for the LM317 type)): 1.25 V, 0.625 V, 2.5 V. Other inputs as in the worked example: R2 = 720.0 ohm. 41 points per curve computed by electronics-engineering-ai999x; marked: the worked example, R1 = 240 ohm, Vout = 5 V.

```
electronics_engineering_ai999x.solve('elx_adjustable_regulator_output', R1=240.0, R2=720.0,
Vref=1.25)
# -> Vout = 5 V, I_R1 = 0.00520833 A   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_adjustable_regulator_output', R1=240.0, R2=2000.0,
Vref=1.25, Iadj=5e-05)
# -> Vout = 11.7667 V, I_R1 = 0.00520833 A   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Adjustable regulator output: R1 = 240 ohm, R2 = 720 ohm, Vref = 1.25 V	SOLVED	Vout = 5 V, I_R1 = 0.00520833 A
Adjustable regulator output: R1 = 240 ohm, R2 = 2000 ohm, Vref = 1.25 V, Iadj = 50 uA	SOLVED	Vout = 11.7667 V, I_R1 = 0.00520833 A
Adjustable regulator output: R2 = 720 ohm, Vref = 1.25 V	NEEDS_INPUT	elx_adjustable_regulator_output needs R1 (resistor from OUT to ADJ). Example: solve('elx_adjustable_regulator_output', R
Adjustable regulator output: R1 = 240 ohm, R2 = 720 ohm, Vref = 1.25 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Zener-referenced series-pass (emitter-follower) regulator, `elx_series_pass_regulator`

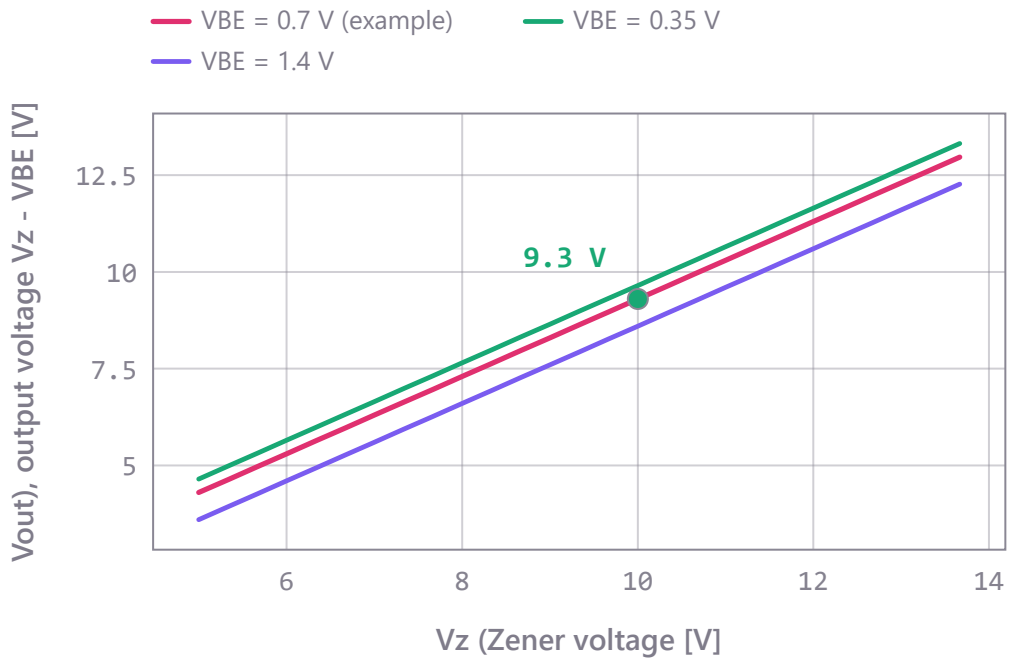
The output voltage, load, base, Zener and resistor currents and the pass-transistor dissipation of a regulator in which a Zener diode biased through Rs sets the base of an emitter-follower pass transistor.

$$V_{out} = V_Z - V_{BE}, \quad I_Z = \frac{V_{in} - V_Z}{R_S} - \frac{I_L}{\beta + 1}$$

input	meaning	condition
`Vin`	unregulated input	
`Vz`	Zener voltage	
`Rs`	Zener bias resistor	
`RL`	load resistance	
`beta`	current gain of the pass transistor	
`VBE`	base-emitter voltage	

output	meaning
`Vout`	output voltage Vz - VBE
`I_L`	load current
`I_B`	base current
`I_z`	Zener current
`I_Rs`	current in Rs
`P_Q`	pass-transistor dissipation

- **Method:** Vout = Vz - VBE; $IB = IL/(\beta + 1)$; Iz = (Vin - Vz)/Rs - IB; P = (Vin - Vout) IC
- **Assumes:** Vin > Vz > VBE; the Zener stays in breakdown $(Iz > 0)$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Vout (output voltage Vz - VBE) against Vz (Zener voltage) from 5 V to 15 V. One curve for each value of VBE (base-emitter voltage): 0.7 V, 0.35 V, 1.4 V. Other inputs as in the worked example: Vin = 15.0 V, Rs = 1000.0 ohm, RL = 100.0 ohm, beta = 100.0. 27 points per curve computed by electronics-engineering-ai999x; marked: the worked example, Vz = 10 V, Vout = 9.3 V.

```
electronics_engineering_ai999x.solve('elx_series_pass_regulator', Vin=15.0, Vz=10.0, Rs=1000.0,
RL=100.0, beta=100.0, VBE=0.7)
# -> Vout = 9.3 V, I_L = 0.093 A, I_B = 0.000920792 A, I_z = 0.00407921 A, I_Rs = 0.005 A, P_Q =
0.524851 W   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_series_pass_regulator', Vin=20.0, Vz=12.7, Rs=470.0,
RL=50.0, beta=60.0, VBE=0.7)
# -> Vout = 12 V, I_L = 0.24 A, I_B = 0.00393443 A, I_z = 0.0115975 A, I_Rs = 0.0155319 A, P_Q =
1.88852 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Series-pass regulator: Vin = 15 V, Vz = 10 V, Rs = 1000 ohm, RL = 100 ohm, beta = 100, VBE = 0.7 V	SOLVED	Vout = 9.3 V, I_L = 0.093 A, I_B = 0.000920792 A, I_z = 0.00407921 A, I_Rs = 0.005 A, P_Q = 0.524851 W
Series-pass regulator: Vin = 20 V, Vz = 12.7 V, Rs = 470 ohm, RL = 50 ohm, beta = 60, VBE = 0.7 V	SOLVED	Vout = 12 V, I_L = 0.24 A, I_B = 0.00393443 A, I_z = 0.0115975 A, I_Rs = 0.0155319 A, P_Q = 1.88852 W
Series-pass regulator: Vz = 10 V, Rs = 1000 ohm, RL = 100 ohm, beta = 100, VBE = 0.7 V	NEEDS_INPUT	elx_series_pass_regulator needs Vin (unregulated input). Example: solve('elx_series_pass_regulator', Vin='15 V', Vz='10
Series-pass regulator: Vin = 15 V, Vz = 10 V, Rs = 1000 ohm, RL = 100 ohm, beta = 100, VBE = 0.7 V, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

Heat-sink thermal path: junction, case and sink temperatures, `elx_heat_sink_temperatures`

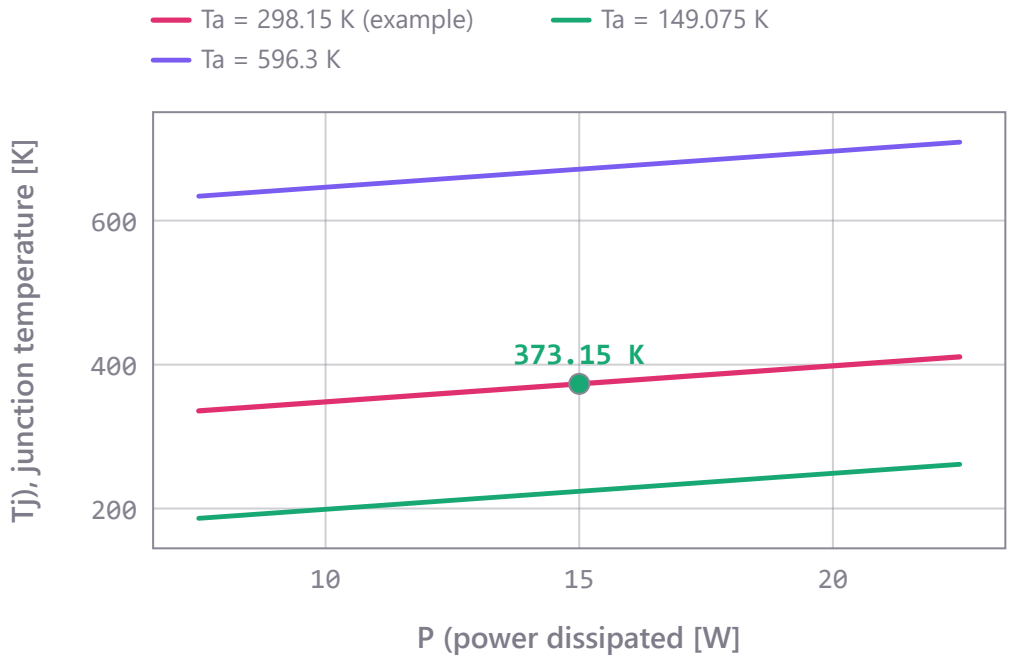
The junction, case and heat-sink temperatures of a power device dissipating P through junction-to-case, case-to-sink (interface) and sink-to-ambient thermal resistances.

$$T_j = T_a + P\,(\theta_{jc} + \theta_{cs} + \theta_{sa})$$

input	meaning	condition
`P`	power dissipated	
`Ta`	ambient temperature	
`theta_jc`	junction-to-case thermal resistance	
`theta_cs`	case-to-sink thermal resistance	
`theta_sa`	sink-to-ambient thermal resistance	

output	meaning
`Tj`	junction temperature
`Tc`	case temperature
`Ts`	sink temperature
`Tj_C`	junction temperature in degC
`theta_ja`	total junction-to-ambient resistance

- **Method:** $Tj = Ta + P$ (theta_jc + theta_cs + theta_sa)
- **Assumes:** steady state, series thermal path
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Tj (junction temperature) against P (power dissipated) from 7.5 W to 22.5 W. One curve for each value of Ta (ambient temperature): 298.15 K, 149.075 K, 596.3 K. Other inputs as in the worked example: theta_jc = 1.5 K/W, theta_cs = 0.5 K/W, theta_sa = 3.0 K/W. 31 points per curve computed by electronics-engineering-ai999x; marked: the worked example, P = 15 W, Tj = 373.15 K.

```
electronics_engineering_ai999x.solve('elx_heat_sink_temperatures', P=15.0, Ta=298.15, theta_jc=1.5, theta_cs=0.5, theta_sa=3.0)
# -> Tj = 373.15 K, Tc = 350.65 K, Ts = 343.15 K, Tj_C = 100, theta_ja = 5 K/W   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_heat_sink_temperatures', P=40.0, Ta=313.15, theta_jc=0.8, theta_cs=0.2, theta_sa=1.2)
# -> Tj = 401.15 K, Tc = 369.15 K, Ts = 361.15 K, Tj_C = 128, theta_ja = 2.2 K/W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Heat sink temperatures: P = 15 W, Ta = 25 degC, theta_jc = 1.5 K/W, theta_cs = 0.5 K/W, theta_sa = 3 K/W	SOLVED	Tj = 373.15 K, Tc = 350.65 K, Ts = 343.15 K, Tj_C = 100, theta_ja = 5 K/W
Heat sink temperatures: P = 40 W, Ta = 40 degC, theta_jc = 0.8 K/W, theta_cs = 0.2 K/W, theta_sa = 1.2 K/W	SOLVED	Tj = 401.15 K, Tc = 369.15 K, Ts = 361.15 K, Tj_C = 128, theta_ja = 2.2 K/W
Heat sink temperatures: Ta = 25 degC, theta_jc = 1.5 K/W, theta_cs = 0.5 K/W, theta_sa = 3 K/W	NEEDS_INPUT	elx_heat_sink_temperatures needs P (power dissipated). Example: solve('elx_heat_sink_temperatures', P='15 W', Ta='298.15
Heat sink temperatures: P = 15 W, Ta = 25 degC, theta_jc = 1.5 K/W, theta_cs = 0.5 K/W, theta_sa = 3 K/W, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te

question	outcome	result
Heat sink temperatures: P = 15 W, Ta = 25 degC, theta_jc = 1.5 degC/W, theta_cs = 0.5 degC/W, theta_sa = 3 degC/W	SOLVED	Tj = 373.15 K, Tc = 350.65 K, Ts = 343.15 K, Tj_C = 100, theta_ja = 5 K/W

Required heat-sink thermal resistance, `elx_heat_sink_required`

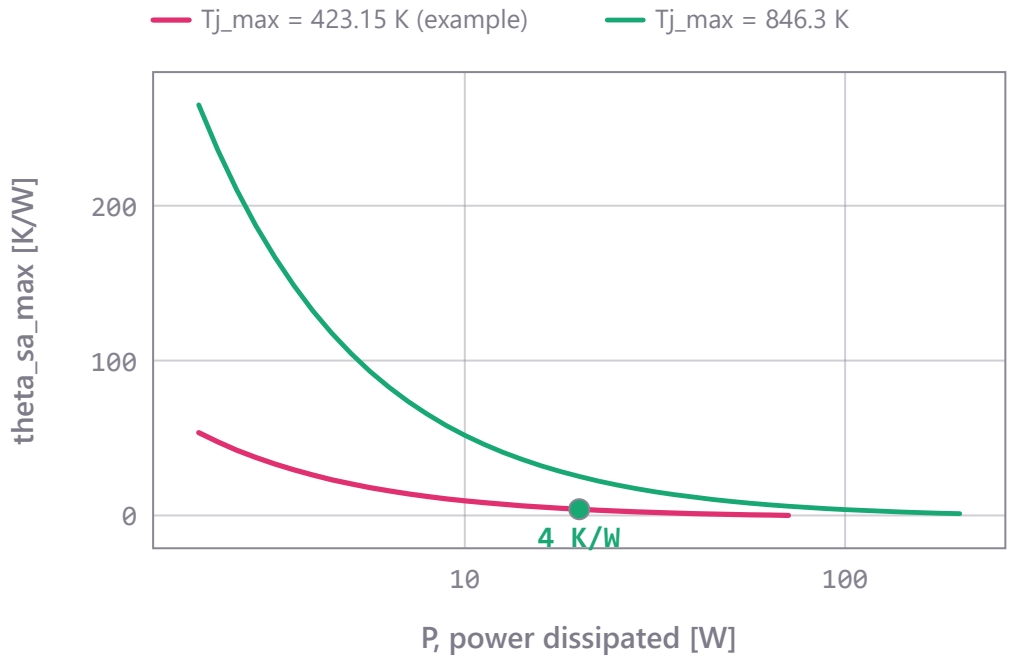
The largest sink-to-ambient thermal resistance (and total junction-to-ambient resistance) that keeps a device dissipating P below its maximum junction temperature.

$$\theta_{sa} \leq \frac{T_{j,max} - T_a}{P} - \theta_{jc} - \theta_{cs}$$

input	meaning	condition
`P`	power dissipated	
`Ta`	ambient temperature	
`Tj_max`	maximum junction temperature	
`theta_jc`	junction-to-case thermal resistance	
`theta_cs`	case-to-sink thermal resistance	

output	meaning
`theta_ja_max`	largest junction-to-ambient resistance
`theta_sa_max`	largest sink-to-ambient resistance

- **Method:** theta_sa = (Tj,max - Ta)/P - theta_jc - theta_cs
- **Assumes:** a positive result (otherwise no heat sink suffices)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



theta_sa_max (largest sink-to-ambient resistance) against P (power dissipated) from 2 W to 200 W on a logarithmic axis. One curve for each value of Tj_max (maximum junction temperature): 423.15 K, 846.3 K. Other inputs as in the worked example: Ta = 313.15 K, theta_jc = 1.0 K/W, theta_cs = 0.5 K/W. 32 points per curve computed by electronics-engineering-ai999x; marked: the worked example, P = 20 W, theta_sa_max = 4 K/W.

```
electronics_engineering_ai999x.solve('elx_heat_sink_required', P=20.0, Ta=313.15, Tj_max=423.15,
theta_jc=1.0, theta_cs=0.5)
# -> theta_ja_max = 5.5 K/W, theta_sa_max = 4 K/W   (SOLVED)
```

```
electronics_engineering_ai999x.solve('elx_heat_sink_required', P=8.0, Ta=323.15, Tj_max=398.15,
theta_jc=2.5, theta_cs=0.3)
# -> theta_ja_max = 9.375 K/W, theta_sa_max = 6.575 K/W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Required heat sink: P = 20 W, Ta = 40 degC, Tj_max = 150 degC, theta_jc = 1 K/W, theta_cs = 0.5 K/W	SOLVED	theta_ja_max = 5.5 K/W, theta_sa_max = 4 K/W
Required heat sink: P = 8 W, Ta = 50 degC, Tj_max = 125 degC, theta_jc = 2.5 K/W, theta_cs = 0.3 K/W	SOLVED	theta_ja_max = 9.375 K/W, theta_sa_max = 6.575 K/W
Required heat sink: Ta = 40 degC, Tj_max = 150 degC, theta_jc = 1 K/W, theta_cs = 0.5 K/W	NEEDS_INPUT	elx_heat_sink_required needs P (power dissipated). Example: solve('elx_heat_sink_required', P='20 W', Ta='313.15 K', Tj_
Required heat sink: P = 20 W, Ta = 40 degC, Tj_max = 150 degC, theta_jc = 1 K/W, theta_cs = 0.5 K/W, zz = 1	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: electronics_engineering_ai999x.te